Tell us about your PDF experience.

Microsoft Power Platform developer documentation

Learn how to extend Microsoft Power Platform using code first components and integrating with back-end services.

Get started

OVERVIEW

Power Platform for developers

HOW-TO GUIDE

Create a developer environment

TRAINING

Introduction to developing with Power Platform

Fusion development

CONCEPT

Fusion development

REFERENCE

Fusion development ebook

TRAINING

Transform business apps with fusion development

ALM for developers

REFERENCE

Power Platform ALM basics

Build Tools for Azure DevOps

GitHub Actions

Power Platform CLI

Create packages using Package Deployer

Work with solutions using SDK

For ISVs

HOW-TO GUIDE

Publish your app on AppSource

Tools for Developers

DOWNLOAD

Power Platform CLI

Power Platform Tools for Visual Studio

ALM developer tools

Dataverse developer tools

Visual Studio connected service

Dataverse

REFERENCE

Dataverse developer docs

Plug-ins

Webhooks

Custom APIs

Virtual tables

Power Apps

REFERENCE

Canvas apps developer docs

Model-driven apps developer docs

Power Apps component framework

Custom connectors

JavaScript and the Client API

QUICKSTART

Create a Canvas App front end for your ASP.NET web API (Preview)

Power Automate

REFERENCE

Custom connectors

Use workflow definition functions

Use APIs with desktop flows

Power BI

REFERENCE

Custom visuals

Use Power BI REST APIs

Analysis Services developer documentation

Power Virtual Agents

REFERENCE

Bot Framework Composer

Power Pages

REFERENCE

Custom page layouts

Work with Liquid templates

Power Pages Web API

Use Visual Studio Code and Power Platform CLI

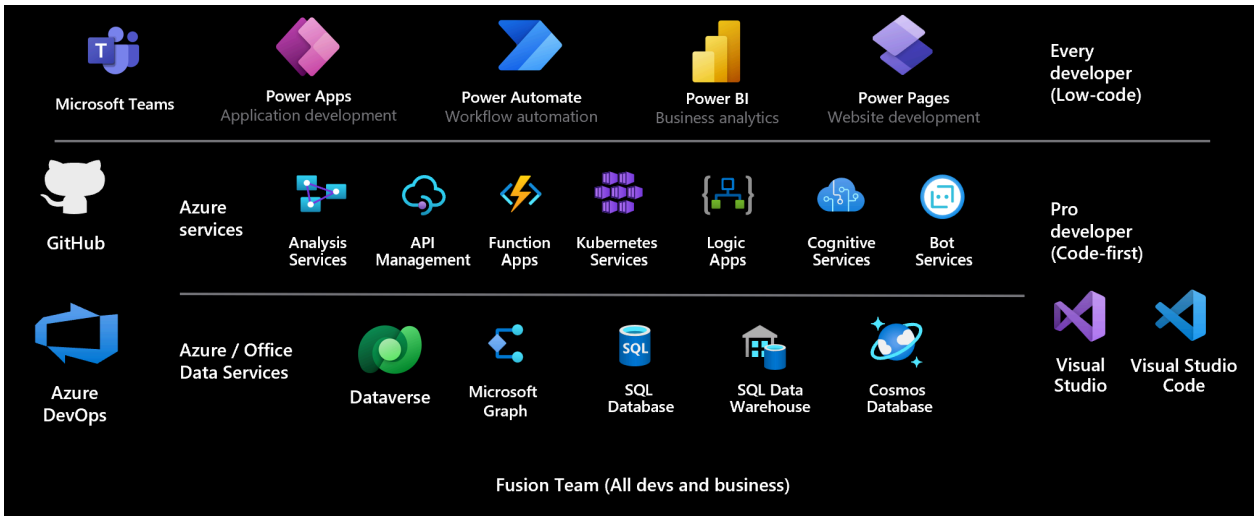
Use code components

Introduction to Microsoft Power Platform for developers

Article • 05/03/2024

Microsoft Power Platform is a low-code platform for rapidly building customized end-to-end business solutions. It consists of five product areas: Power Apps, Power Automate, Power BI, and Power Pages. Each area can be used individually or together. United by the underlying Microsoft Dataverse technology, all areas are designed for a connected experience to achieve your business goals, whether it's creating analytics, process automation, or data-driven productivity applications.

The platform's low-code approach enables everyone in an organization—from frontline workers to professional developers—to participate in the development of business solutions that drive positive business outcomes. Developers can collaborate with citizen developers to enhance business solutions and solve complex requirements by using the platform's various extension capabilities with code. Thanks to a native integration with Azure, developers can use their cloud development skills to extend Microsoft Power Platform with different Azure services. With a low-code integrated development approach, business goals can be achieved faster and frees up time for the developers.



As a developer interacting with the platform, it's important to understand the basics of what the platform offers. In the following, you'll find a high-level overview of each of the core product areas and their key extensibility points.

Power Apps

Apps created with Power Apps can be shared with internal users and can be run in a browser or on a mobile device. You can create following types of apps: canvas and

model-driven.

Canvas apps

Provides the Power Apps maker with complete control over the pixel-perfect screen contents and navigation between multiple screens. Canvas applications use different connectors to work with data and services. Canvas applications can be embedded into SharePoint, Teams, Power BI, and Dynamics 365 applications.

Key developer extensibility points

- Build custom UI components using Power Apps component framework
- Create custom connectors to communicate to your external data sources and services. A custom connector is a wrapper around a REST API and can be created using tools like Azure Functions and Azure API Management

Model-driven apps

Data-driven applications that are built on top of Microsoft Dataverse. Other data sources and services can be used by model-driven applications by embedding a canvas application.

Key developer extensibility points

- Build custom UI components using Power Apps component framework
- Implement client business logic using JavaScript and the client API
- Build HTML web resources

Power Automate

Power Automate is used to automate tasks and orchestrate activities across various services that use integrated or custom connectors. Using Power Automate you can create cloud flows or desktop flows.

Expand table

Type	Description
Cloud flows	Can be triggered manually, configured to run when specific events occur, such as when a record is created, or scheduled to run at a specific time. Power Automate is

Type	Description
	built on top of Azure Logic Apps but offers unique integration with Microsoft Power Platform.
Desktop flows	Used to automate repetitive interactive tasks on the web or the desktop.

> ### Key developer extensibility points
>
> - Create custom connectors to communicate to your external data sources and services. A custom connector is a wrapper around a REST API and can be created using tools like Azure Functions and Azure API Management
> - Use workflow definition functions to build complex expressions

Power BI

Power BI is Microsoft's business analytics solution that provides interactive data visualization BI tools to help users visualize and share data and insights across their organization.

> ### Key developer extensibility points
>
> - Embed Power BI in apps, websites, and portals
> - Create custom visuals using the open-source SDK

Power Pages

Power Pages is a secure, enterprise-grade, low-code software as a service (SaaS) platform for creating, hosting, and administering modern external-facing business websites. Low-code makers can rapidly design, configure, and publish websites that seamlessly work across web browsers and devices. Professional developers can extend these capabilities to address advanced business requirements.

> ### Key developer extensibility points
>
> - Custom web page templates built with HTML, CSS, and Liquid
> - Extend site integration with Dataverse with the portals Web API
> - Use Visual Studio Code and the Power Platform CLI to enable metadata editing and CI/CD (Continuous Integration/Continuous Deployment) of site configurations

Dataverse

Dataverse is a cloud scale data store that builds on Microsoft's data technologies and abstracts away data management complexity from the app maker. It lets you securely store and manage data that is used by business applications. Business data within Dataverse is stored within a set of data tables. Role-based security allows businesses to control access to the data tables and even individual records for different users within your organization. Developers can extend or customize Dataverse functionality by adding custom business logic. You can define calculated table columns, business rules, workflows, and business process flows to ensure data quality and drive business processes.

Key developer extensibility points

- Create plug-ins, similar to event handlers, that customize or extend Dataverse data processing with custom business logic (code)
- Use webhooks and Azure Service Bus to integrate with external systems
- Extend the Dataverse API with your own custom API that implements your business logic
- Use virtual tables to integrate data stored in external systems into Dataverse without replicating the data

See also

Create a developer environment
Fusion Development

Feedback

Was this page helpful? Yes No

Provide product feedback

About the Power Apps Developer Plan

Article • 04/17/2024

The Power Apps Developer Plan gives you a free development environment to build and test with Power Apps, Power Automate, and Microsoft Dataverse. It enables you to:

- Create apps and flows without writing code, with full-featured Power Apps and Power Automate development tools. Easily share and collaborate on these solutions with others.
- Connect to any data source by using Power Platform connectors or by creating your own custom connectors.
- Use a fully managed, scalable data platform with Dataverse, including support for common business application actions. Use out-of-the-box common tables or build your own data schema.
- Export the solutions you create in your developer environment, and publish them on Microsoft AppSource so your customers can test-drive them.

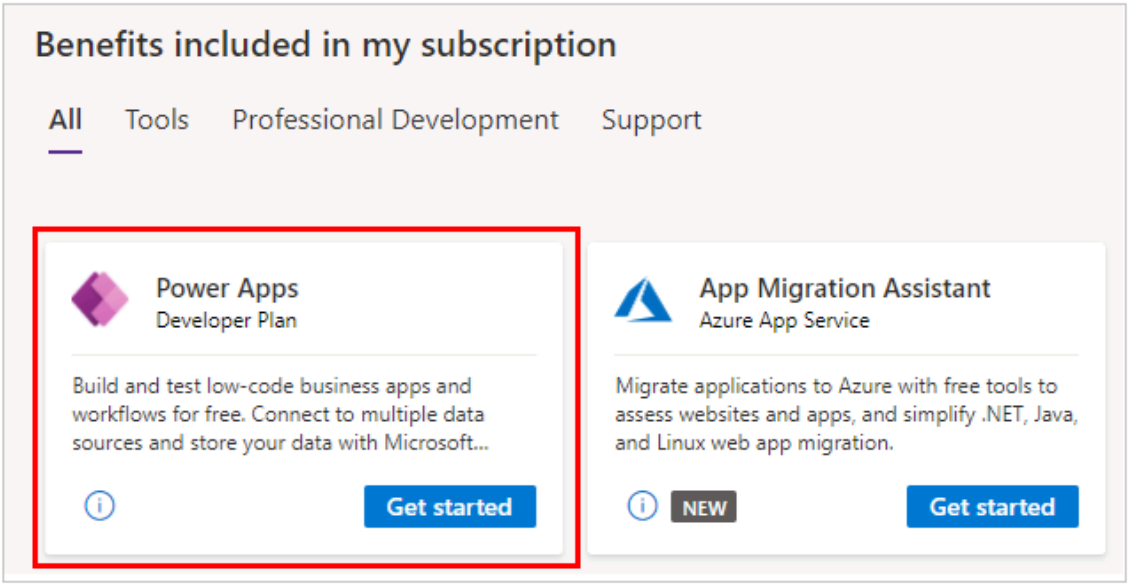
Who can sign up for the Power Apps Developer Plan?

Anyone with a work or school email address backed by Microsoft Entra ID can sign up for the Power Apps Developer Plan.

How can I get the Power Apps Developer Plan?

You can get the plan using one of the following ways:

- Sign up on the Power Apps Developer Plan website.
- Get the Power Apps Developer Plan with Visual Studio Dev Essentials. If you're a Visual Studio Dev Essentials user, the Power Apps Developer Plan is included in your benefits. Visit My benefits and select the Power Apps tile to sign up for the Power Apps Developer Plan.



Which features are included in the Power Apps Developer Plan?

With the developer environment, you get the following functionality:

Expand table

Functionalities	Developer environment
Key features	
Create and test apps	Yes. You can create unlimited apps for development and test purposes.
Share apps	Yes
Use Dataverse	Yes
Model your data using Dataverse	Yes
Enterprise-grade administration of the environment and user policies	Yes
Connectivity	
Connect to Office 365, Dynamics 365, and other connectors	Yes
Connect to cloud-based services like Azure SQL, Dropbox, Twitter, and many more	Yes
Use premium connectors like Salesforce, DB2 and many more	Yes

Functionalities	Developer environment
Access on-premises data using an on-premises gateway	Yes
Create custom connectors to connect to your own systems	Yes
Dataverse	
Create and run applications In Power Platform, Dynamics 365, Teams, or Standalone using the .Net SDK or OData API	Yes
Model your data in Dataverse	Yes
Create a database in Dataverse	Yes
Create and use dataflows	No
Management	
Add coworkers as environment makers and admins	Yes
Add coworkers to the database roles	Yes
Supports data policies established by the Office 365 administrator	Yes
Establish data policies for the developer environment	Yes

What are the capacity limits for the developer environment?

The following capacity limits apply to the developer environments:

Expand table

Capacity	Limits
Flow runs/month	750
Database size	2 GB

You can't increase capacity by applying add-ons to the quantities we include. If you hit capacity limits, we recommend purchasing the Power Apps Per User Plan. Learn more from the Power Apps pricing page.

> **Note**
>
> The capacity entitlements of the developer environment, whether used or not, doesn't contribute to your company's overall quota.

Publishing to Microsoft AppSource

Do you have an app that you'd like to share with customers? We now support Power Apps solutions on Microsoft AppSource as a way for you to share apps and flows with customers, and generate leads for your business. For more information, see Publish your app on AppSource.

FAQ

The following section lists frequently asked questions relevant to the Power Apps Developer Plan. For more information about different types of environments including the developer environment, see Types of environments in Power Platform.

What should I do if I reach the capacity limits of the environment?

There's a limited capacity provided because this environment is meant for development and test purposes, not for production use. The capacity limits are:

Expand table

Capacity	Limits
Flow runs/month	750
Database size	2 GB

If you reach one or more capacity limits, we recommend you purchase a plan that supports production use. Learn more about our plans on the Power Apps pricing page.

Can I transfer the apps, flows, and other resources created in the developer environment to another environment?

Yes, you should be able to export the resources from this environment to other environments. For more information, see Solutions.

Will my Power Apps Developer Plan subscription ever expire?

You can continue using your Power Apps Developer Plan for free as long as there's active usage and no abuse of the plan (for example, apps aren't used for production purposes and the capacity limits aren't exceeded). Environments created using Power Apps Developer Plan that are inactive for the last 90 days will be deleted after notifying the environment owners.

Can I get or create multiple developer environments?

Yes. You have one developer environment, which is created for you by Power Platform when you sign up for the Developer Plan. You can create up to three environments in the Power Platform admin center.

What's the difference between the Power Apps Plan Trial and Power Apps Developer Plan? Which one should I sign up for?

Both Power Apps Plan Trial and Power Apps Developer Plan are free, but are created for different purposes:

- **Power Apps Plan Trial** gives you the Power Apps per user plan for 30 days. This duration is meant for trying out production Power Apps applications. Once your trial expires, you can purchase a plan. If you're already using Power Apps with Office 365 or Dynamics 365, this is the right plan to test the premium functionalities of Power Apps, which are available with paid Power Apps plans.
- **Power Apps Developer Plan** gives you access to the same Power Apps functionality available in paid plans (aligned with Trials), including Dataverse and Power Automate, but in a developer environment meant for development and test use only. A paid plan is required to deploy or run solutions in a production environment for production use.

Can I sign up with my personal account?

No, we currently don't support signing up with a personal account. You can only sign up with your work or school account.

Which kind of license do I need to get Developer Environment?

You need a Power Apps Developer plan to get a developer environment. This Developer plan is a viral/internal license that your admin needs to enable so that it can be assigned in the tenant. The license is assigned and verified during signup. The tenant must allow for viral or internal licenses for developer environment to be provisioned.

- To verify if the respective policy is already allowed, admin can use the Get-AllowedConsentPlans (Microsoft.PowerApps.Administration.PowerShell) PowerShell cmdlet.
- To modify the policy, admin can use the Add-AllowedConsentPlans (Microsoft.PowerApps.Administration.PowerShell) PowerShell cmdlet.

Can I reset or delete my developer environment?

Yes. The owner of the developer environment, or users with the permission to delete an environment can delete a developer environment.

Makers can also reset the developer environment through the Power Platform admin center.

Can I prevent users from creating developer environments?

Yes. To prevent users from creating developer environments, go to Control environment creation.

Does the developer environment get auto-created with a developer plan?

Yes. A developer environment is created when the user signs up for the developer plan using any of the experiences in the Power Apps Developer Plan website, Power Apps, or Power Platform admin center. However, when you delete a developer environment, or you need more developer environments, you need to create them manually.

Does the Power Apps Developer plan include Power Automate RPA use rights?

No. However, users may start a Power Automate trial including RPA use rights to try out RPA capabilities. Power Apps Developer plan includes cloud flow use rights since these can be connected to an act as an extension of Power Apps apps.

Does the Power Apps Developer plan include AI Builder use rights?

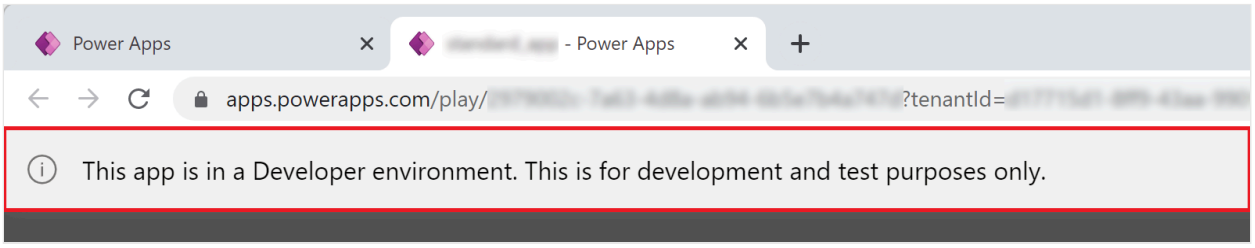
No. However, users may start an AI Builder trial to try out the AI Builder capabilities.

Can a developer environment be a managed environment?

The developer environment can become a managed environment. However, managed environment use rights aren't included in the developer plan. As a result, end users require premium licenses in order to run their assets in a managed developer environment.

Why does a banner appear with apps running in the developer environment?

Since developer environments aren't intended to support production scenarios, apps display a temporary banner to remind makers about them running in a developer environment. This banner will no longer appear if you move the app from a developer environment to a non-developer environment.



Can I launch a premium app with a Power Apps Developer Plan in non-Developer environments?

No. You can't use the Power Apps Developer Plan to launch premium apps that aren't in a Developer environment. You need a valid Power Apps premium entitlement. Learn more about Power Apps plans.

Are Managed Environment use rights included in the Developer Plan?

Managed Environment isn't included as an entitlement in the Developer Plan when users run their assets. In other words, users—including makers running their own apps or flows—need a premium license in addition to the developer plan to run their assets in a managed, developer environment.

Are Dynamics 365 apps available in a developer environment?

No. Dynamics 365 apps can't be installed in a developer environment. You'll see the following message when you try to install them:

"Installing Dynamics 365 apps such as Dynamics 365 Sales and Dynamics 365 Customer Service is only supported on environments that have been created with a database with the option to Enable Dynamics 365 apps set to "Yes". Try installing this application in an environment with a database and Enable Dynamics 365 apps enabled on it."

If you want to explore Dynamics 365 apps, refer to the app-specific documentation for free trials, or to install them on existing environments.

Are users automatically added to a developer environment?

No. Users are not automatically added to the developer environment, they need to be added manually.

Improvements from the previous Power Apps Community Plan

In May 2021, we extended the capabilities of the Power Apps Community Plan to meet the needs of developers and rebranded it as the **Power Apps Developer Plan**.

Key changes include:

- The ability to share apps with team members for development and testing purposes
- Increased capacity allocations
- The ability to create up to three environments

> **Note**
>
> Existing Power Apps Community Plan users will also get all of these updates.

Next step

Create a developer environment with the Power Apps Developer Plan

See also

Power Platform for developers
Fusion Development

Feedback

Was this page helpful? Yes No

Provide product feedback

Create a developer environment with the Power Apps Developer Plan

Article • 02/23/2024

To fully use the Power Apps Developer Plan as a developer, you need an Azure account and a work account. This article guides you through the process for creating a Power Platform environment and a test tenant if needed.

Where do I start?

If you have a work account, and want to use it to learn Power Platform, go to the next section.

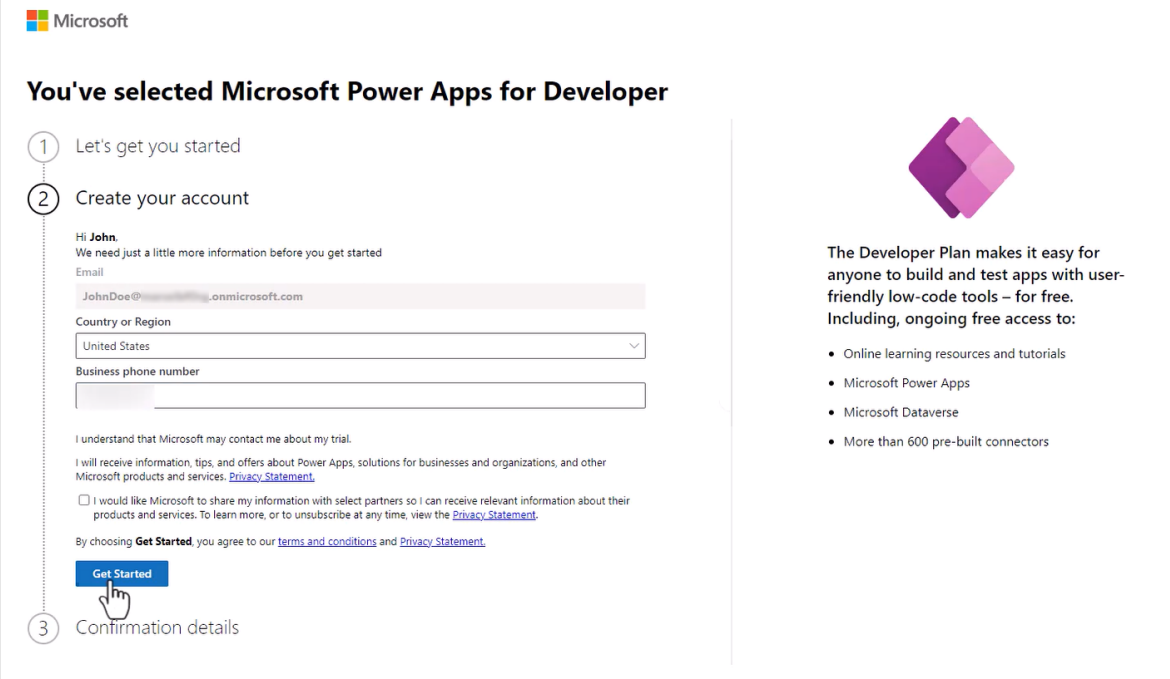
If you don't have a work account or prefer a Sandbox tenant to learn Power Platform, read information in the create a test tenant section later in this article before signing up for the developer environment.

Sign up for the Power Apps Developer Plan

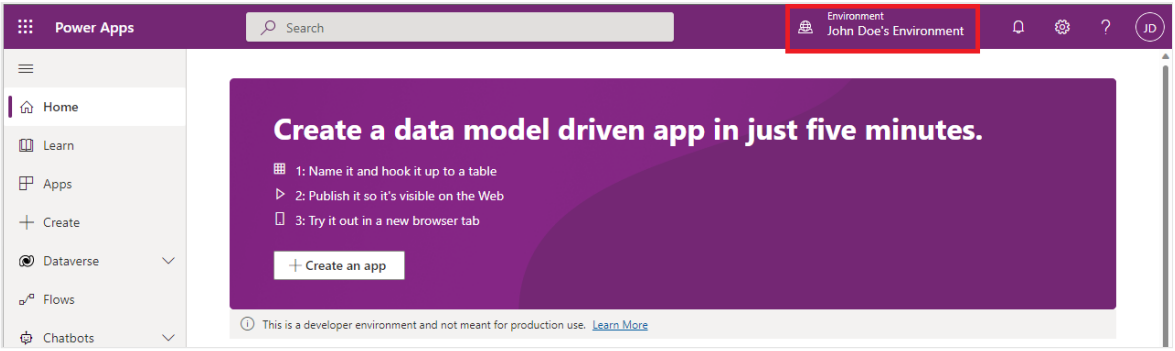
The Power Apps Developer Plan gives you a free development environment to build and test with Power Apps, Power Automate, and Microsoft Dataverse.

It's simple to sign up for the Power Apps Developer Plan:

1. Ensure that you have a work account. If you don't, create a test tenant first.
2. Sign up on the Power Apps Developer Plan website.

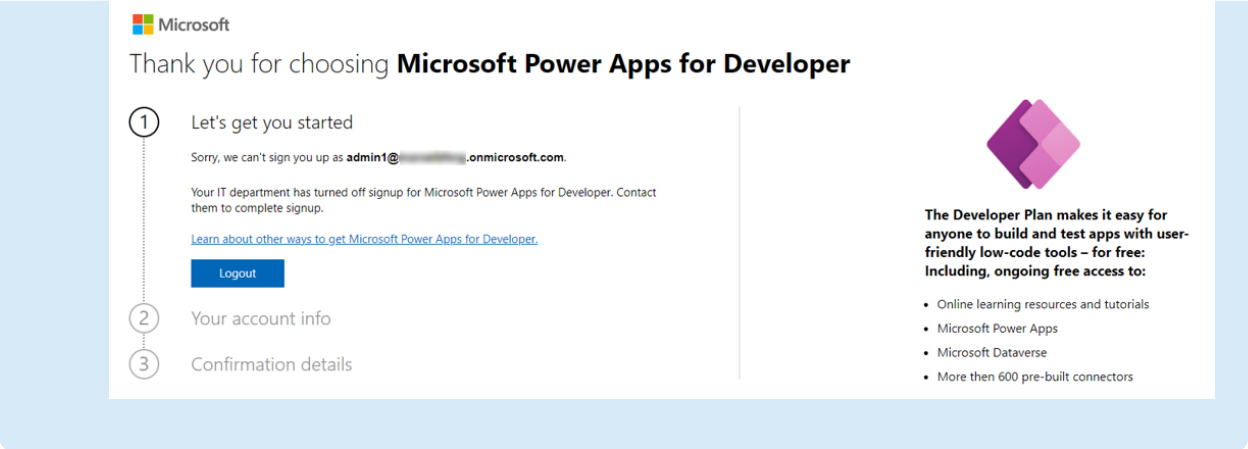


After signing up for the Developer Plan, you'll be redirected to Power Apps. The environment uses your name, for example **John Doe's environment**. If there's already an environment with that name, the developer new environment is named **John Doe's (1) environment**.



ⓘ Important

- Use the developer environment instead of your tenant's default environment to work with certain capabilities such as premium and custom connectors.
- You might need to select your developer environment from the top-right corner of the screen in Power Apps.
- It might take a couple of minutes for the new environment to be provisioned and become available in the list of the environments. You can see the progress of the environment creation in the **Power Platform admin center**.
- In some situations, your admin might have turned off the sign up process. In this case, please contact your administrator, or create a test tenant.



For detailed information about the developer plan, go to Sign up for the Power Apps Developer Plan.

How to create a test tenant?

If you don't already have a dedicated test tenant, you might qualify for one through the Microsoft 365 Developer Program; for details, see the FAQ. Alternatively, you can sign up for a one-month free trial or purchase a Microsoft 365 plan.

You can also manually create a test tenant.

Now that you have your test tenant, sign up for the Power Apps Developer Plan as explained earlier in this article.

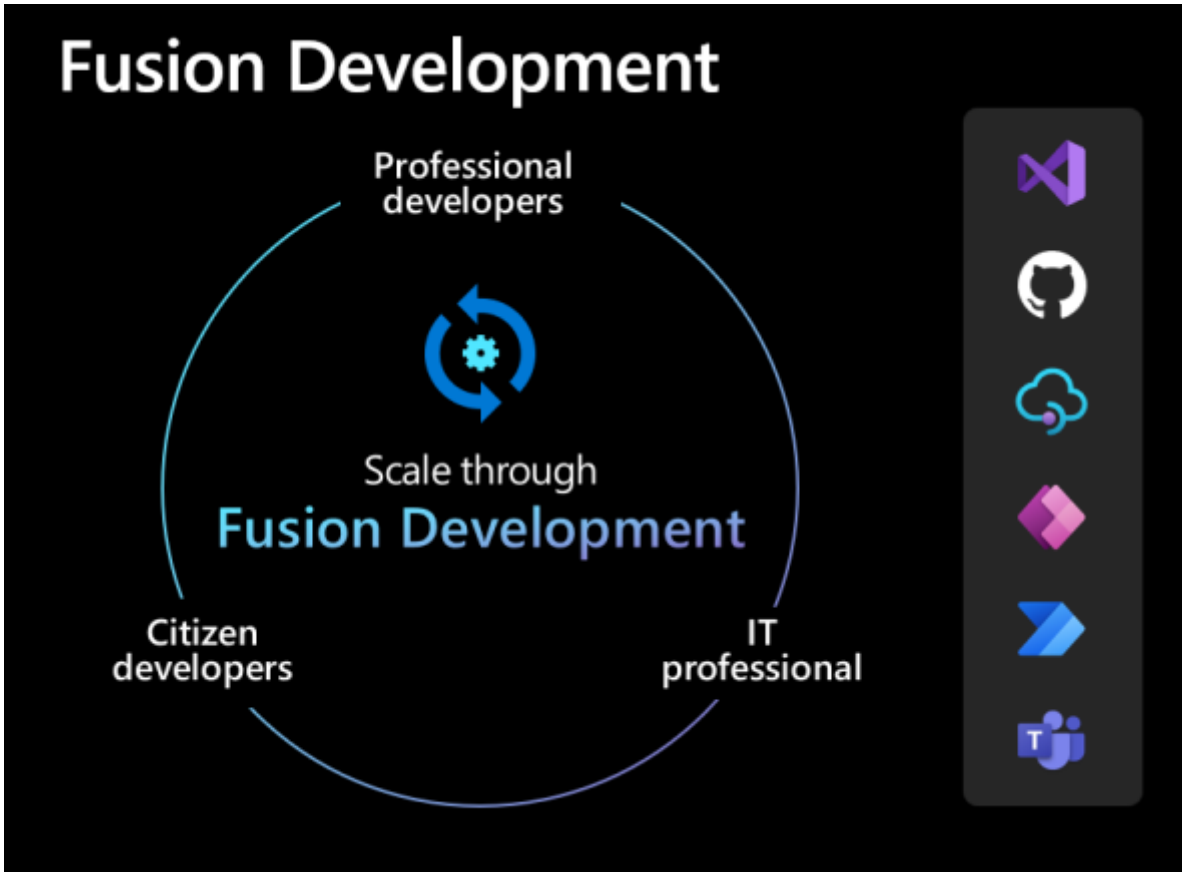
See also

Power Platform for developers
Fusion Development

Fusion development in Power Platform

Article • 11/09/2022

Fusion development or fusion teams development, is defined by Gartner as "distributed and multidisciplinary digital business teams that blend technology and other types of domain expertise." (Source: 2019 Gartner Digital Business Teams Survey)



Fusion Development is an approach with the main goal of scaling application development through low-code. Fusion development allows your business to build better applications, faster by bringing together professional developers with citizen, or low-code, developers. It can also mean using low-code capabilities and combine them with code-first components to meet business needs and create Fusion applications.

Personas for fusion development

Below you can find the personas and the definition in a fusion team:

Persona	Description
Citizen developer or maker	A business user with business knowledge looking for ways to improve a business process. Use visual, drag and drop and low code tools. Usually don't have a technical background.

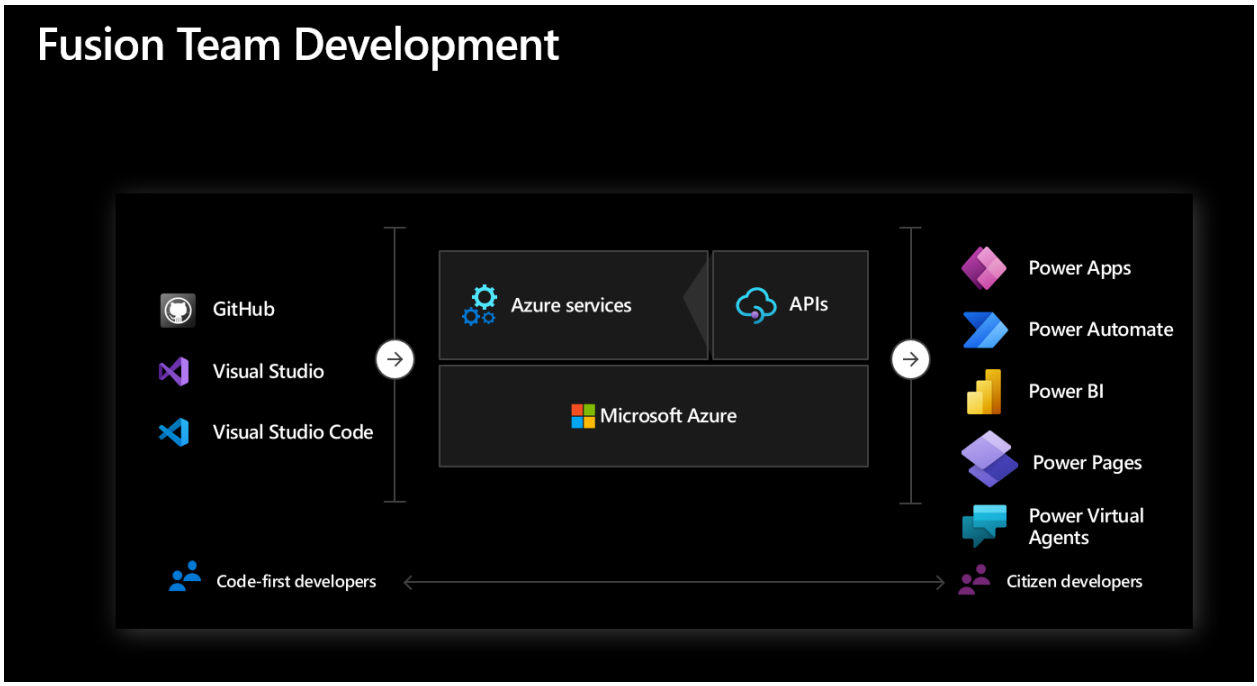
Persona	Description
Professional developer or code-first developer	Pro devs use pro dev tools to accomplish pro dev tasks. For example, write code in Visual Studio or Visual Studio Code using programming languages like C# or JavaScript.
IT professional or DevOps engineer	An IT professional responsible for the application lifecycle management (ALM) of business applications.

> **ⓘ Important**
>
> Our goal is to empower diverse groups of people with different skill sets and boost velocity by scaling solution development through low-code. The Microsoft Power Platform is a tool in your tool set. We want to help you work more efficiently and focus on tasks that match your desired skill level.

Empowering citizen developer can enable pro developers to focus on specialized problems, avoiding simple and repetitive tasks that can be performed by the citizen developer.

Collaboration

Power Platform is designed to empower fusion teams allowing three key audiences to come together.



> **Tip**

Low-code and pro-dev tools should integrate smoothly to enable different groups of people to work together effectively.

The role of the professional developer and IT professional is to create a secure and governed environment where citizen developer can be empowered to achieve more with minimum friction. Everyone should use citizen maker tools to accomplish citizen maker tasks. For example, pro devs use same visual tooling as citizen makers when creating apps on Power Platform.

See Also

Create a developer environment
Fusion Development ebook
Power Platform ALM for developers
Power Platform for developers

ALM for developers

Article • 02/17/2023

The articles in this section describe how you as a developer can use available APIs, tools, and other resources to implement application lifecycle management (ALM) using Microsoft Power Platform.

We'll start off talking a little about team development and then dive into Azure DevOps and available build tools.

To learn more about key ALM concepts and working with solutions, see Overview of application lifecycle management and watch the following video.

https://learn-video.azurefd.net/vod/player?ev=powerful-devs-conference&session=how-to-do-alm-with-power-platform&locale=en-us&embedUrl=%2Fpower-platform%2Falm%2Falm-for-developers

Team development

When we refer to *team development*, we're talking about multiple developers collaborating in the same environment versus multiple developers working on the same application and sharing source code. For team development, it's a good practice to use tooling or processes to achieve developer isolation.

More information: Scenario 5: Supporting team development

Conflict management

For team development, the goal is to avoid conflict when making changes to shared code. With a source control system, branching and merging help to avoid change conflicts and keep team members from affecting each other with partially completed work in a shared repo.

Another method is to use strategies to avoid conflict. For example, you can have only one person at a time work on a complex component, to avoid a merge conflict that a source control system might not automatically be able to resolve.

Working with complex components

What are complex components? Examples include forms, canvas apps, flows, and workflows.

Coordinate efforts with other team members to avoid having more than one developer work on the same form or component at a time. If you do have multiple developers working on the same canvas app, have them work on separate components to avoid conflict.

See also

Microsoft Power Platform Build Tools for Azure DevOps
Power Platform developer docs

Publish your app on AppSource

Article • 04/04/2023

Publish your app on Microsoft AppSource so that app subscribers can find it, try it, and buy it. The apps you build and publish with Power Apps are discovered and used in Dynamics 365 on the web. More information: Discover apps via AppSource.

Prepare for submitting your app on AppSource

Use the following steps to prepare for submitting your app to be listed on Microsoft AppSource:

> Step 1: Create a Microsoft Partner Center account
>
> Step 2: Create a managed solution for your app
>
> Step 3: Create an AppSource package for your app
>
> Step 4: Store your package on Azure Storage

Submit your app on Partner Center

Once your app is ready for submission, you use the Partner Center to submit your app to be listed on Microsoft AppSource. Access to the Partner Center and related documentation is available only to Microsoft Partner Network members.

Checklists

- App design best practices: Use this checklist to evaluate your app design to provide the best and consistent user experience.
- App certification: Provides information about the checks done during the app certification process after you submit your app for publishing on AppSource.

See also

Video: Microsoft AppSource: find the right app for your business

Step 1: Create a Microsoft Partner Center account

Article • 04/04/2023

If you are not already a member of the Microsoft Partner Center, you must become a member by enrolling at https://aka.ms/joinmarketplace.

You must be a member of Partner Center to publish apps on AppSource.

Step 2: Create a managed solution for your app

See also

Microsoft commercial marketplace
Create a commercial marketplace account in Partner Center

Step 2: Create a managed solution for your app

Article • 04/04/2023

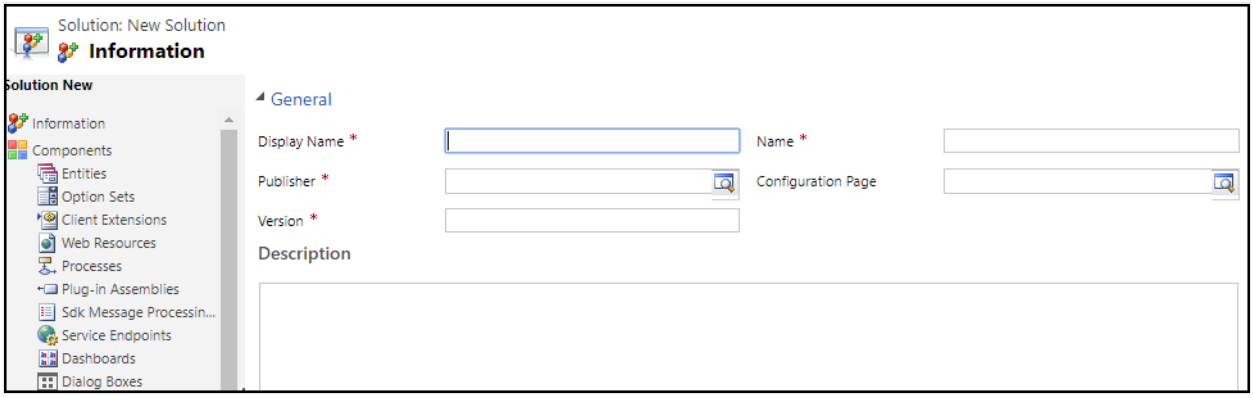
Create a managed solution to include all the components for your app. You might find these topics helpful as you plan and create a managed solution to package your app components:

- Solution concepts
- Create a solution

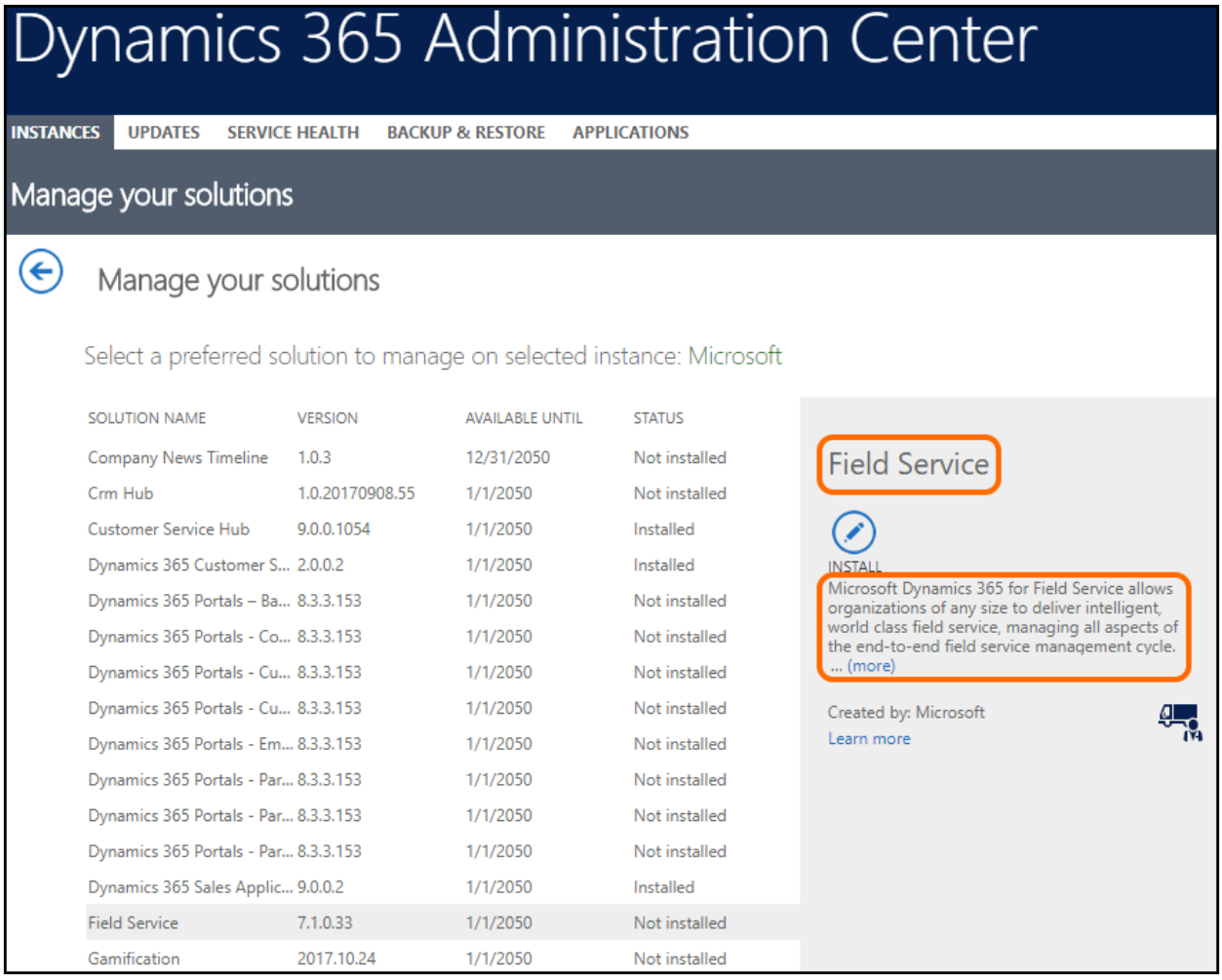
You can also work with solutions using code: Create, export, or import an unmanaged solution

Display name and description of your solution

While creating a solution to package your app components, make sure you provide appropriate values in the **Display Name** and **Description** columns for your new solution that you want to be displayed to your customers.



The **Display Name** and **Description** values for a solution are displayed to the customers in the **Dynamics 365 Administration Center** portal.



Export the solution

Proceed with exporting the solution from the environment. When exporting the solution, choose to export as a managed solution. More information: Export solutions

Supporting data for your solution

If your solution requires supporting or demo data:

1. Create supporting/demo data in your test environment.
2. Use the Configuration Migration tool to create a schema to include your supporting/demo data.
3. Save the schema file so that you can reuse it later to update the data in case your demo data changes.
4. Use the schema to export the data. Ensure that you provide a meaningful name to your export file. The data is exported and as a .zip file.

For detailed information about using the Configuration Migration tool to create a schema and export your data, see Create a schema to export configuration data

At the end of this step

You will now have a managed solution file (example: *SampleSolution.zip*) and optionally a demo data file (example: *SampleData.zip*) for your app.

Optional licensing information: If you wish to add licensing information to your solution, see Appendix: Add licensing information to your solution

Step 3: Create an AppSource package for your app

Step 3: Create an AppSource package for your app

Article • 05/22/2023

You must create an AppSource package (.zip file) to include your solution and demo data files along with other required files. An AppSource package consists of the following files:

Expand table

File	Description
Package file	A package file used by Package Deployer tool to deploy your solutions and demo configuration data into multiple languages.
[Content_Types].xml	File that provides MIME type information of the file type extensions included in the AppSource package. Typically, these are .config, .dll, .exe, .xml, and .zip file types, but you can add almost any file type that is supported by Windows.
Icon file	An image file for the appsource package icon; size should be 32x32 pixels. Valid image formats are PNG and JPG.
HTML file	File containing your License terms.
Input.xml	Files that describes the assets in your AppSource package.

Create a Package file

A package lets you bundle and deploy multiple files related to your app at once.

1. Create a Dynamics 365 package to include the solution and configuration data files that you created in Step 2: Create a managed solution for your app. A package can also contain custom code that can run before, while, or after the package is deployed to the Microsoft Dataverse instance. For more information about creating a package file, see Create packages for the Package Deployer.

   After you have created a package, your package will consist of the following things:

   - **<PackageName> folder**: This folder contains all solutions, configuration data, flat files, and the contents for your package. For example: **PkgFolder**.
   - **<PackageName>.dll**: The assembly contains the custom code for your package. For example: **SamplePackage.dll**.

2. Next, create a **[Content_Types].xml** file that provides MIME type information of the file type extensions that are included in your package. This is separate from the one that will be included again in the AppSource package. Here is the sample contents of a [Content_Types].xml file with file types listed:

   XML

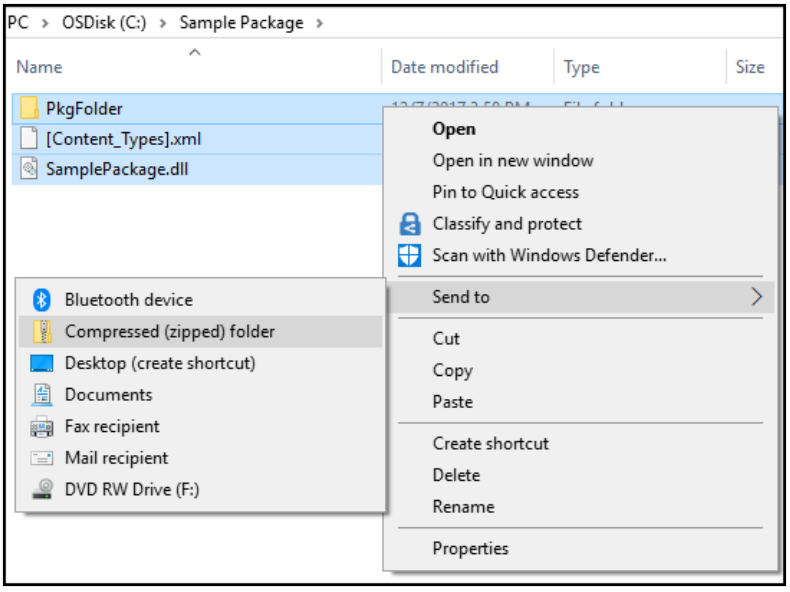
   ```xml
   <?xml version="1.0" encoding="utf-8"?>
   <Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">
     <Default Extension="xml" ContentType="application/octet-stream" />
     <Default Extension="xaml" ContentType="application/octet-stream" />
     <Default Extension="dll" ContentType="application/octet-stream" />
     <Default Extension="zip" ContentType="application/octet-stream" />
     <Default Extension="jpg" ContentType="application/octet-stream" />
     <Default Extension="gif" ContentType="application/octet-stream" />
     <Default Extension="png" ContentType="application/octet-stream" />
     <Default Extension="htm" ContentType="application/octet-stream" />
     <Default Extension="html" ContentType="application/octet-stream" />
     <Default Extension="db" ContentType="application/octet-stream" />
     <Default Extension="css" ContentType="application/octet-stream" />
     <Default Extension="json" ContentType="application/octet-stream" />
     <Default Extension="msapp" ContentType="application/octet-stream" />
   </Types>
   ```

3. Compress (zip) the following files into a file called *package.zip*:

   - package folder (PkgFolder)

- package dll (SamplePackage.dll)
- [Content_Types].xml

  To compress these files, browse to the folder where these files are present, select them all, right-click and select **Send to** > **Compressed (zipped) folder**.



4. Rename the .zip file to **package.zip**.

Create [Content_Types].xml

You can reuse the **[Content_Types].xml** that you created in the previous section under step 2.

Create an icon for your AppSource package

Create an icon file of size 32x32 to display along with the preferred solution name and description in the Dynamics 365 Administration Center portal. Valid file formats are PNG and JPG.

Create an HTML file for license terms

Create an HTML file containing your license terms. You can have an HTML file per language to display the license terms in the user selected language if your app supports multiple languages.

Create Input.xml file

Create an *Input.xml* file that provides information about your package and the contents of the package. Here is the contents of a sample **Input.xml** file; each element is explained later in the table.

XML

```
<?xml version="1.0" encoding="utf-8"?>
<PvsPackageData>
  <ProviderName>Microsoft</ProviderName>
  <PackageFile>package.zip</PackageFile>
  <SolutionAnchorName>SampleSolution.zip</SolutionAnchorName>
  <StartDate>12/01/2017</StartDate>
  <EndDate>01/01/2031</EndDate>
  <SupportedCountries>US,CA</SupportedCountries>
  <LearnMoreLink>https://www.microsoft.com</LearnMoreLink>
  <Locales>
    <PackageLocale Code="1033" IsDefault="true">
      <Logo>logo32x32.png</Logo>
      <Terms>
```

```
        <PackageTerm File="TermsOfUse.html" />
      </Terms>
    </PackageLocale>
  </Locales>
</PvsPackageData>
```

> **Important**
>
> All attributes of XML elements are case sensitive.

Here is a description of the elements in the **Input.xml** file.

Expand table

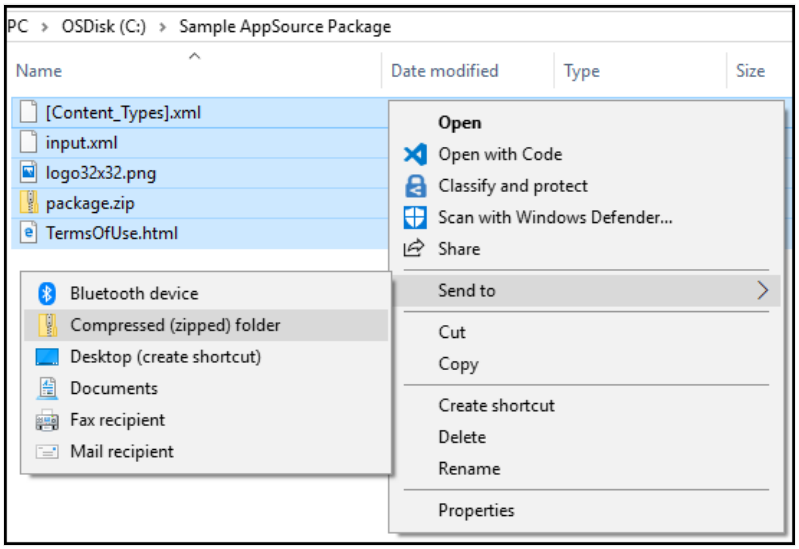
Element	Description
ProviderName	Name of the solution provider. If created by a Microsoft internal team, specify **Microsoft**.
PackageFile	Name of package (.zip file) for the Package Deployer tool. This zip file should contain the package assembly, the package folder with your app assets, and the Content_Types.xml file. For example, the package.zip file created under the Create a Package file section.
SolutionAnchorName	Name of the solution zip file in the package that is used for the display name and description of solution assets.
StartDate	Date on which the app becomes available on AppSource. The format is MM/DD/YYYY.
EndDate	Date on which the app stops being available on AppSource. The format is MM/DD/YYYY.
SupportedCountries	This is a comma-separated list of countries or regions where the app should be available. At the time of writing this article, the supported countries list is the following: AD,AE,AF,AG,AI,AL,AM,AO,AQ,AR,AS,AT,AU,AW,AX,AZ,BA,BB,BD,BE,BF,BG,BH,BI,BJ,BL,BM,BN,BO,BR,BS,BT,BV,BW,BY, BZ,CA,CC,CD,CF,CG,CH,CI,CK,CL,CM,CN,CO,CR,CV,CW,CX,CY,CZ,DE,DJ,DK,DM,DO,DZ,EC,EE,EG,ER,ES,ET,FI,FJ,FK,FM, FO,FR,GA,GB,GD,GE,GF,GG,GH,GI,GL,GM,GN,GP,GQ,GR,GS,GT,GU,GW,GY,HK,HM,HN,HR,HT,HU,ID,IE,IL,IM,IN,IO,IQ,IS, IT,JE,JM,JO,JP,KE,KG,KH,KI,KM,KN,KR,KW,KY,KZ,LA,LB,LC,LI,LK,LR,LS,LT,LU,LV,LY,MA,MC,MD,ME,MF,MG,MH,MK,ML, MM,MN,MO,MP,MQ,MR,MS,MT,MU,MV,MW,MX,MY,MZ,NA,NC,NE,NF,NG,NI,NL,NO,NP,NR,NU,NZ,OM,PA,PE,PF,PG,PH,PK,PL,PM, PN,PR,PS,PT,PW,PY,QA,RE,RO,RS,RU,RW,SA,SB,SC,SE,SG,SH,SI,SJ,SK,SL,SM,SN,SO,SR,ST,SV,SZ,TC,TD,TF,TG,TH,TJ, TK,TL,TM,TN,TO,TR,TT,TV,TW,TZ,UA,UG,UM,US,UY,UZ,VA,VC,VE,VG,VI,VN,VU,WF,WS,YE,YT,ZA,ZM,ZW
LearnMoreLink	URL to the detailed information page for this package.
Locales	An instance of this node for each language you want to support in the Preferred solution UI. This node contains the following children elements: - **PackageLocale.Code**: LCID of the language for this node. Example: US English is 1033 **Note**: Please make sure to have same locale codes in both solution.xml and input.xml files. - **PackageLocale.IsDefault**: Indicates the default language. This is used as the fallback language if the language chosen by the customer is not available. - **Logo**: Logo for your app package. Size of the image must be 32x32. Valid image formats are PNG and JPG. - **Terms**: Name of the HTML file that contains your license terms for each language.

> **Important**
>
> For the `SupportedCountries` element, ensure not to have any spaces and new line keywords between the country codes.

Add the items to an AppSource package

The final step is to add all the components that you created earlier into a single compressed (zip) file, which will be your app source package.

1. Navigate to the folder that contains the package file, [Content_Types].xml, icon, license terms file (HTML), select them all, right-click and then select **Send to** > **Compressed (zipped) folder**.



> **Important**
>
> You must follow the content structure precisely for your package as described here. Otherwise, your package will fail during certification. Some common issues that lead to certification failure are incorrect file names or a nested file structure.

2. Rename the file appropriately as per your app. We recommend that you include your company name and app name. For example: **Microsoft_SamplePackage.zip**.

Step 4: Store your AppSource Package on Azure Storage and generate a URL with SAS key

Step 4: Store your AppSource Package on Azure Storage and generate a URL with SAS key

Article • 04/04/2023

Microsoft Azure Storage is a Microsoft-managed cloud service that provides storage that is highly available, secure, durable, scalable, and redundant. More information: Introduction to Microsoft Azure Storage.

To maintain security of your files, all app developers must store their AppSource package file in a Microsoft Azure Blob storage account, and use a Shared Access Signature (SAS) key to share the package file. Your package file is retrieved from your Azure Storage location for certification, and then for AppSource trials.

Before you upload your package

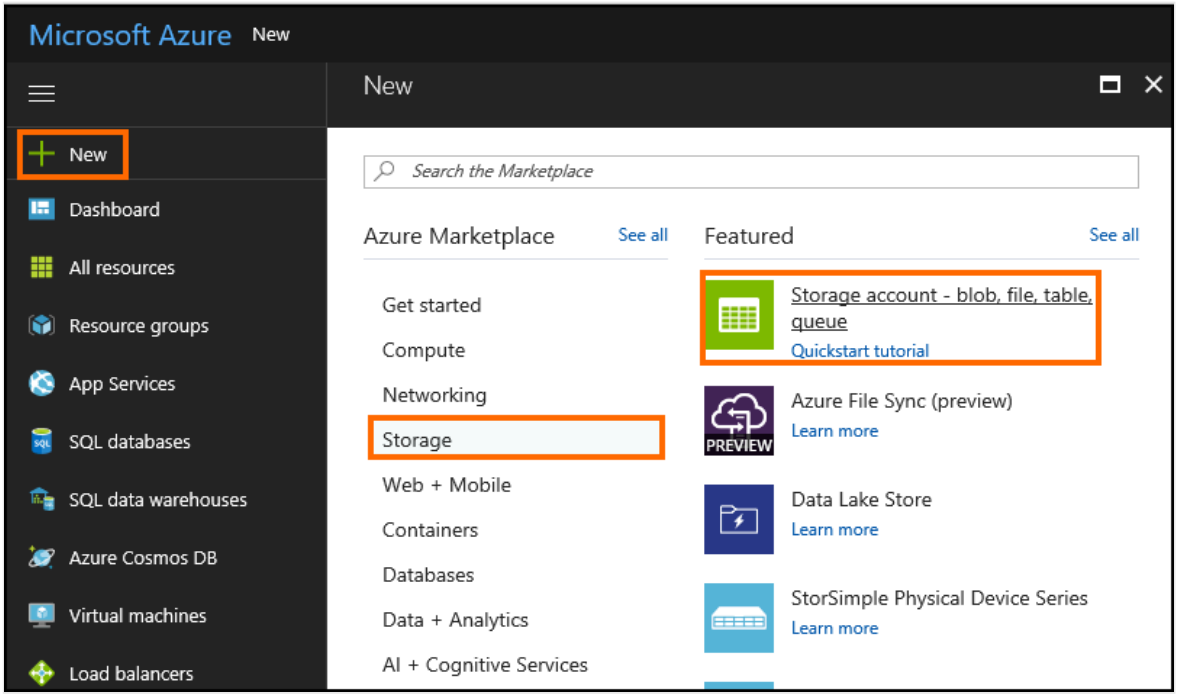
Download and install the Microsoft Azure Storage Explorer from https://storageexplorer.com.

Azure Storage Explorer lets you easily manage the contents of your storage account.

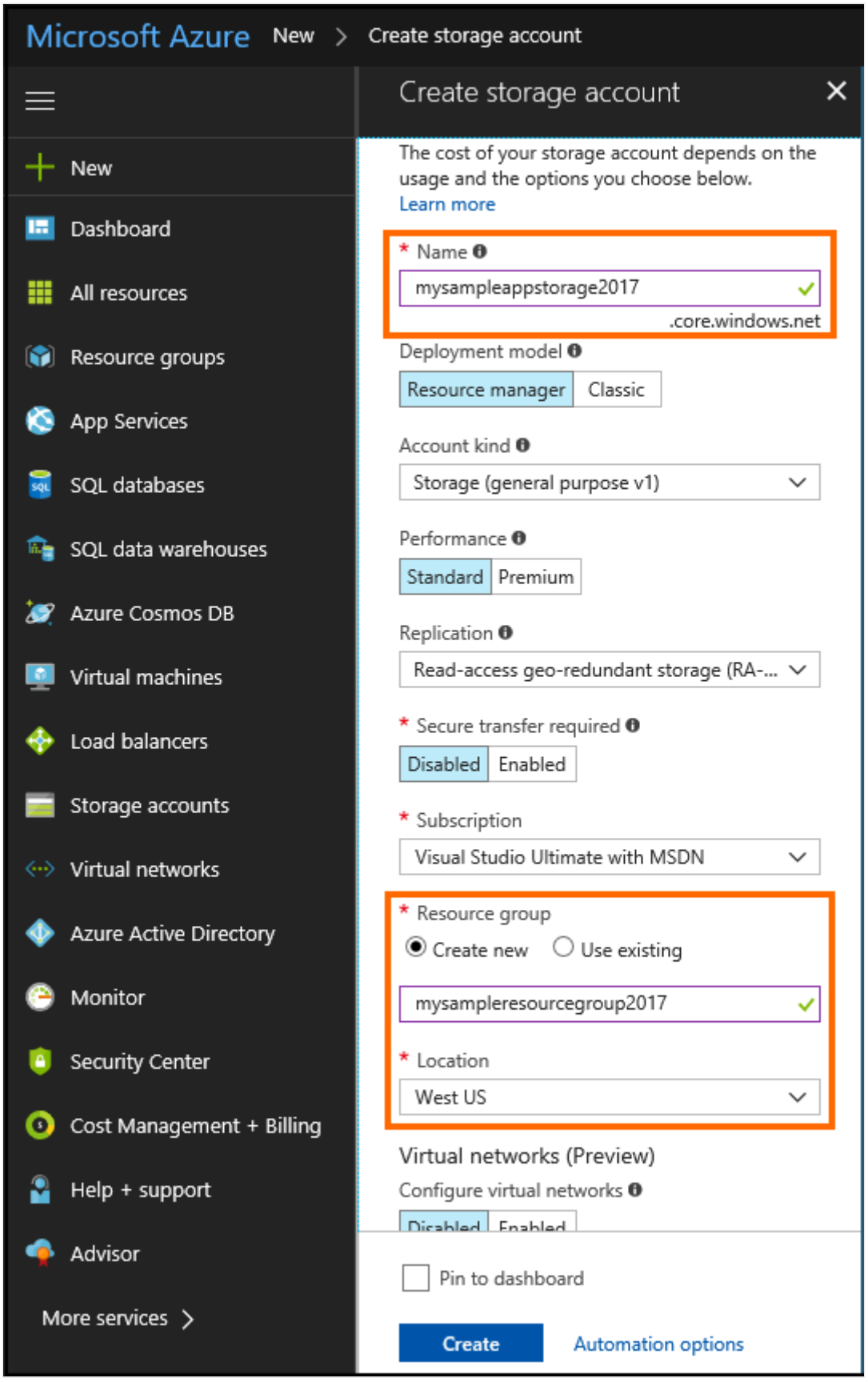
Upload your package and generate a URL with SAS key

To upload your package to Azure Blob storage:

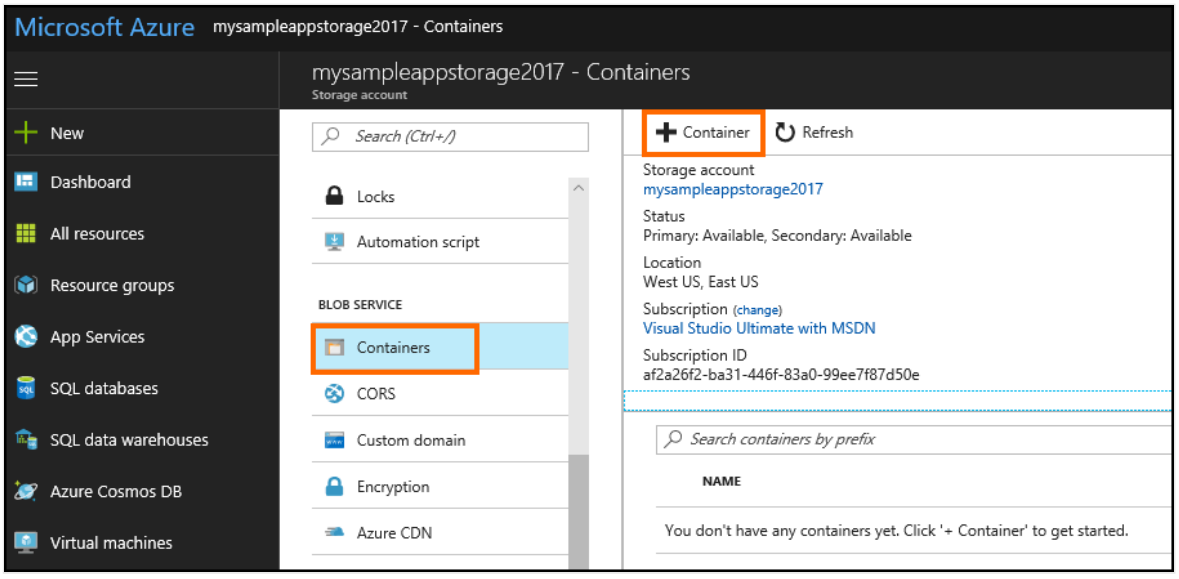
1. Create a free trial or pay as you go Azure account at https://azure.microsoft.com.
2. Sign in to Azure Management portal at https://portal.azure.com.
3. Create a new Storage account by clicking > **Storage** > **Storage account - blob, file, table, queue**.



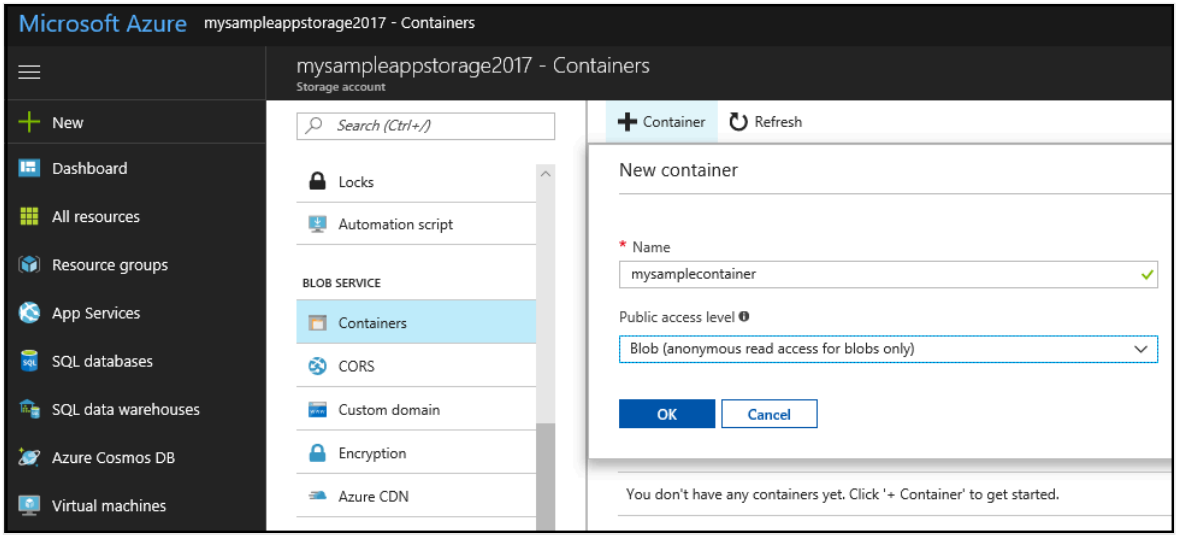
4. On the **Create storage account** page, specify **Name**, **Resource group**, and **Location** for your storage account. Leave the rest of the columns with the default options. Click **Create**.



5. After your storage account is created, navigate to the newly created resource group, and create a new Blob container. Under **Blob Service**, select **Containers**, and then + **Container**.

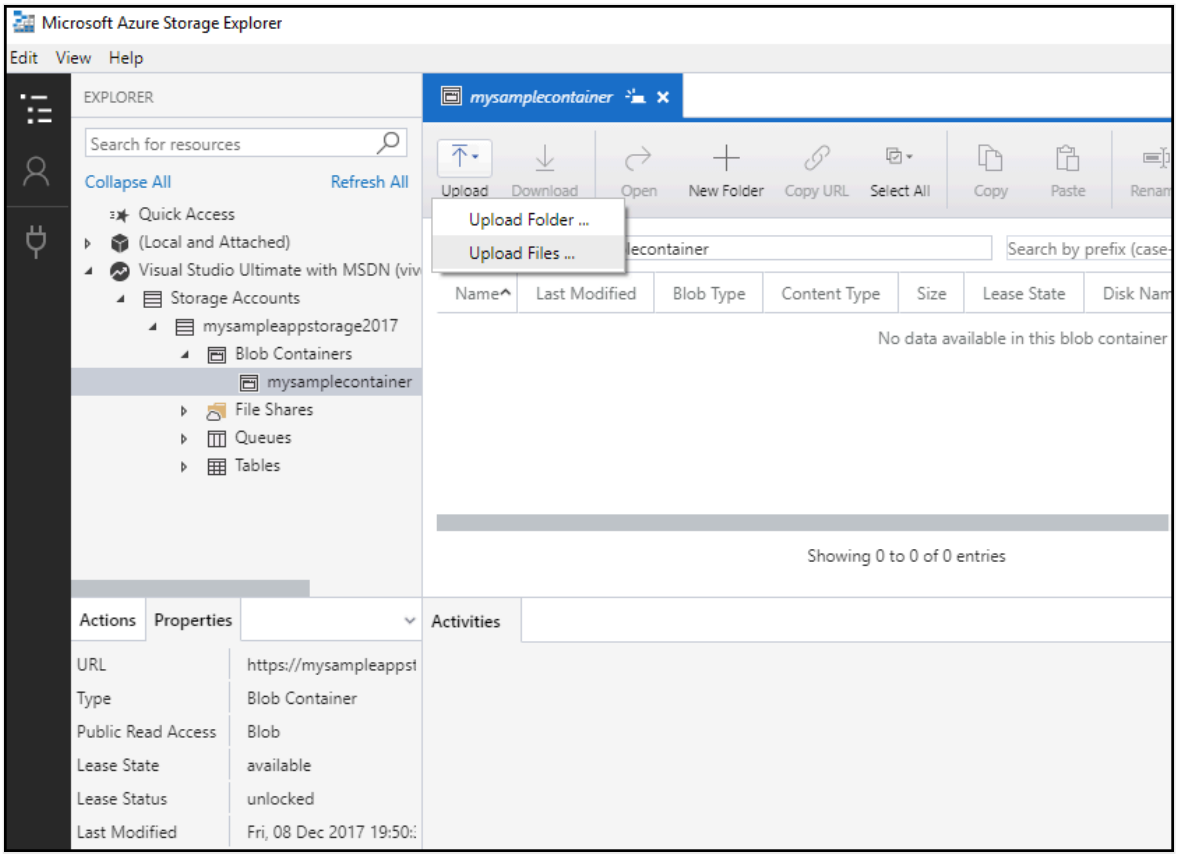


6. Specify a name for your container, and select the **Public access level** as **Blob**. Click **OK**.



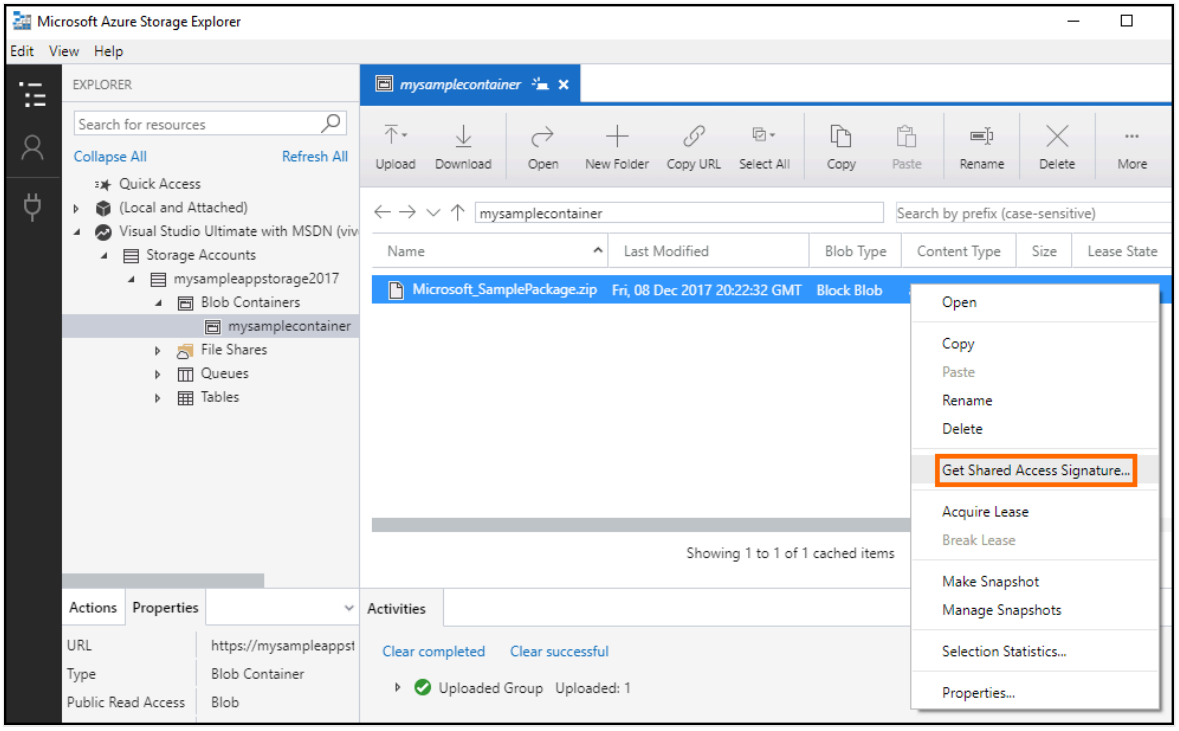
7. Start Azure Storage Explorer on your computer, and connect to your Azure Storage account by signing in using the same account with which you created your Azure Storage account.

8. In Azure Storage Explorer, select the newly created container, and then select **Upload** > **Upload Files** to upload the app source package that you created in Step 3: Create an AppSource package for your app.

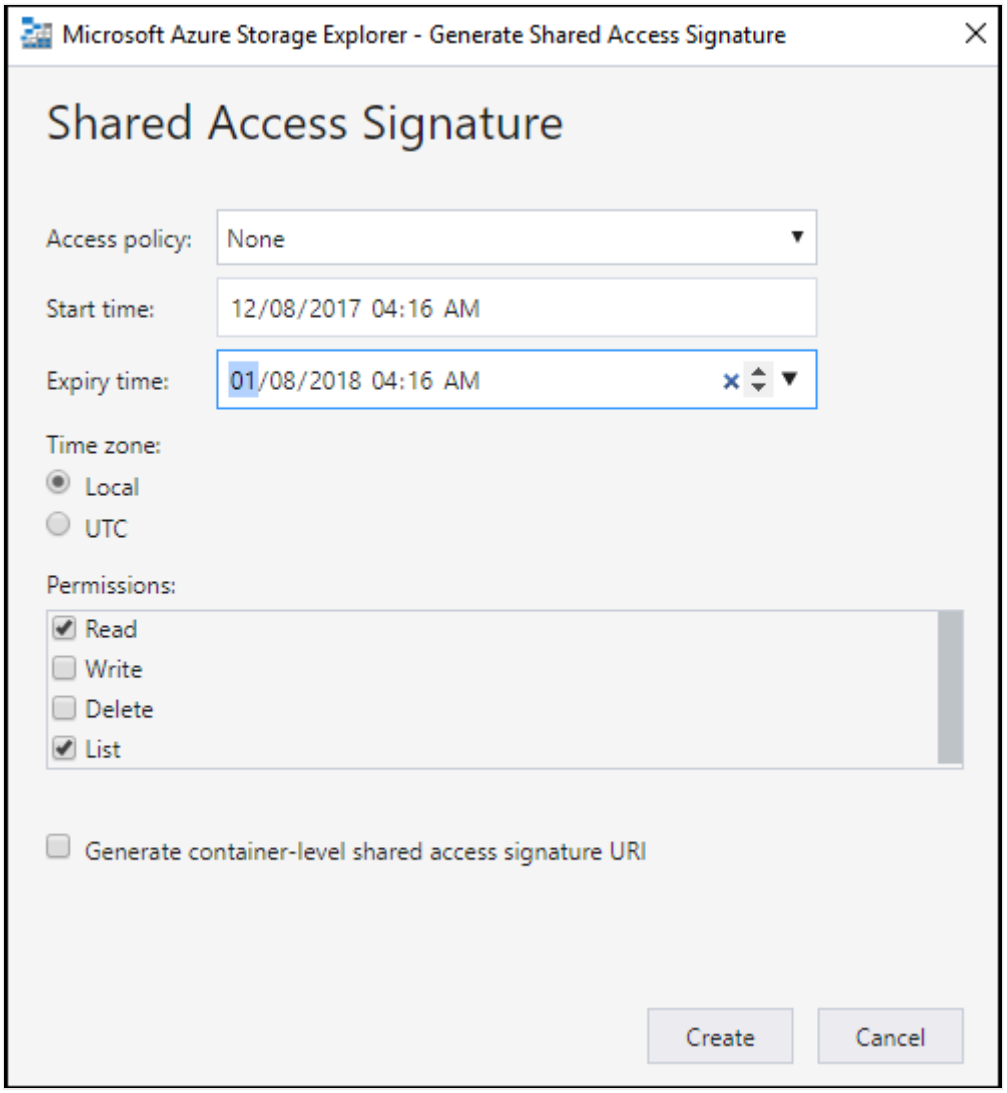


9. Browse to the AppSource package file on your computer, and select to upload it.

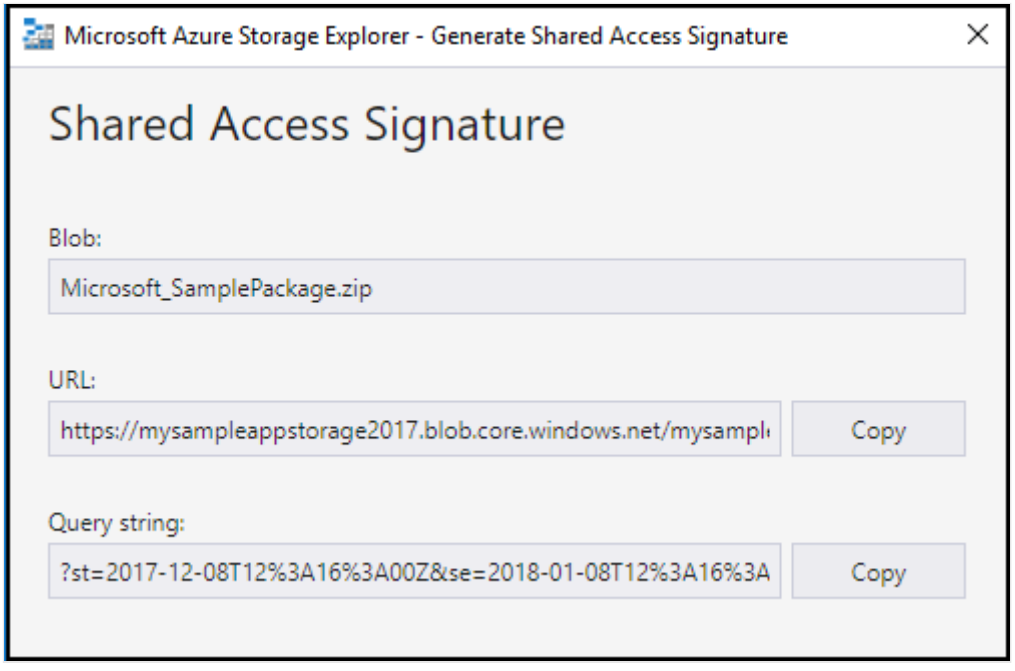
10. Right-click on the uploaded AppSource package file, and select **Get Shared Access Signature**.



11. On the **Shared Access Signature** page, modify the **Expiry time** value to make the Shared Access Signature (SAS) active for a month from the **Start time**. Click **Create**.



12. The next page displays information about the generated SAS information. Copy the **URL** value and save it for later. You will need to specify this URL while creating an offer in Partner Center.



Next steps: Submit your app on Partner Center

Next steps: Submit your app on Partner Center

Article • 04/04/2023

Your app is ready for submission after completing steps 1-4.

Sign in to Partner Center to create an offer and submit your app.

All apps that are submitted to be listed on AppSource go through a certification process. For information about the checks done in the certification process, see Appendix: App certification checklist

See also

Overview of the Commercial Marketplace portal in Partner Center
Publish your app on AppSource
Microsoft AppSource

Appendix: App design best practices checklist

Article • 04/04/2023

Use the following checklist to evaluate your app design.

Expand table

S.No	Artifact	Success Criteria
1	Font uniformity	• Font size is uniform across the solution • Font color is uniform across the solution • Fonts are readable in all the intended device
2	Color Accessibility	Colors used in this solution are accessible by all user groups.
3	Uniformity of color and design scheme	The color and UI design are uniform and presentable
4	Ease of use for users	The solution is intuitive and easy to use with minimal directions
5	Minimum number of screens to achieve a use-case	All use-cases could be achieved in minimum number of screens and steps
6	Performance	• Caching is used where needed • Only columns that are needed are pulled from database • Minimum number of calls to server is made to achieve any use-case.
7	Presentation of information	• Information is presented is most accessible format • UI layout is suitable for providing the information in the best way • UI layout provides seamless experience in all the devices that solution could be run
8	Responsiveness	The solution is tested in all the intended devices for the solution
9	Accessible content	• Content provided in the solution could be rendered in all the intended devices.

		Example: Do not have videos that could not render in certain browsers. • Does not have unnecessary redirects • Does not have interstitials that interrupts the user's flow • Design is suitable for touch screen. If your intended device is mobile or tablet, then your solution should be designed for a touch screen
10	Accuracy of information	The information provided in the UI should help the user achieve the use case with minimal interruption in the intended flow of user actions to achieve that scenario

For information on best practices for creating Canvas apps, see Canvas App Coding Standard and Guidelines.

Appendix: App certification checklist

Article • 04/04/2023

The following checklist provides the list of validations performed by Microsoft during the certification process after you submit your app.

Expand table

Apps	Validation Type	Certification checklist
Model-driven apps, canvas apps, and flows that connect to Microsoft Dataverse	Sanity Check	• Check for app registration type: Free, Trial or Contact me. • Verify the submitted package contains all the artifacts required to publish on AppSource. • Download the end-to-end (E2E) functional document from Partner Center and validate if document is updated with functional scenarios and user/admin journey.
	Code Validation	• Code validation for canvas apps will be done through Accessibility Checker tool in Power Apps to check for the following: ◦ Static formula errors and warnings: If any issues found then certification team will share the feedback to resolve and resubmit to AppSource. ◦ Runtime errors: May occur once the app is opened in Run mode to view. Any issues found will be reported through email. ◦ Accessibility errors and warnings: All Accessibility errors should be resolved as per Solution Checker guidelines. • Code Validation for Dataverse solution will be done utilizing Power Apps Checker. • Issues reported from Power Apps Checker will be manually validated for correctness and false positive issues will be reduced to low severity. • The quality of the solution and packages are validated against the AppSource certification ruleset. • Generated report is shared with the publisher through email.
	Deployment Validation	• Solution will be installed to a Power Apps studio using Package Deployer. Installed canvas apps will be manually located in the solution as well as on Apps section after installation and will make sure the app is

	opened in edit and run mode. Canvas App will be manually deleted from Power Apps studio to validate successful uninstallation • Check the canvas app successfully connects through the connectors provided by the publishers. For example, Dataverse or any other connections. • Check all Dataverse components (tables, web resources, plug-ins and other components) are available in the solution. • Manually uninstall the solution and check if all the components associated to the managed solution is removed.
Functionality Validation	• Validate the functionality of the app based on the functional document shared by the publisher. All the features that are implemented in the app should pass. • Publisher should submit E2E functional document through Partner Center or can share video links through emails. • If app requires any license configuration, certification team will share the instance details for publisher to update the required license.
Security Validation	• Check whether canvas app connects to any external data source or connections that require access, and proper connection details to be shared in E2E document. • Check canvas app connects to any external connections out of Power Apps connectors. • Check any custom code provided inside Package Deployer. Validate the code before approving the app to AppSource. • Manually validate the code to see if the custom code is retrieving any customer data from target environment. • Publisher can run code in their package as part of deployment. • Publisher can access anything that the deploying customer can access inside the Dataverse instance whwre the deployment is happening. • Publisher can add service accounts to Dataverse with a consent flow disclosing that the service account is being added. This generally requires Single Page Application, and in case of first parties, the consent is automatic. • Any service account added to the customers tenant should not be granted Maker, System Admin, or System customizer security roles. Publisher should

		have a custom security role that provides least privilege access for the action.
Canvas apps and flows that connect to data sources *other* than Dataverse	Sanity Check	• Check canvas app contains a valid .msapp file. • Check the package folder has all required components like manifest, Jason and other image components.
	Code Validation	• Same as explained earlier for model-driven apps, canvas apps, and flows that connect to Dataverse
	Deployment Validation	• Canvas app will be manually installed to a Power Apps studio using import Apps feature. Installed canvas apps will be manually located in the Apps section after installation and will make sure the app is opened in edit and run mode. Canvas app will be manually deleted from Power Apps studio to validate successful uninstallation. • Check that the canvas app successfully connects the connectors provided by the publishers.
	Functionality Validation	• Same as explained earlier for model-driven apps, canvas apps, and flows that connect to Dataverse
	Security Validation	• Same as explained earlier for model-driven apps, canvas apps, and flows that connect to Dataverse

For information on best practices for creating:

- Canvas apps, see Canvas App Coding Standard and Guidelines
- Model-driven apps, see Understand model-driven app components

See also

Partner Center documentation

Appendix: Add licensing information to your solution

Article • 04/04/2023

This article is related to the license management capability that Microsoft provides for ISVs to manage and enforce licenses for their products. More information: ISV app license management

This article outlines how to add licensing information to a Microsoft Dataverse solution, package it, and upload the solution zip file to Microsoft AppSource. More information: Step 3: Create an AppSource package for your app.

The following steps are required to add license information to your solution:

- Clone existing solution.
- Use Microsoft Power Platform CLI to add license information to the solution.
- Build the solution, package it, and upload the zip file to AppSource.

Download the latest version of Microsoft Power Platform CLI

You can learn how to download the Power Platform CLI in this article:What is Microsoft Power Platform CLI?.

Microsoft Power Platform CLI tool includes the pac solution add-license command for adding licensing information to the solution.

Clone a solution

Use Microsoft Power Platform CLI commands to clone the solution. To clone a solution:

1. Create an authentication profile for the environment where you want to add your solution by using the pac auth create command:

   CLI

   ```
   pac auth create --name <name of your choice> --kind Dataverse --url
   <your dataverse url> --username <your username> --password <your
   password>`
   ```

2. Clone the existing solution by using the pac solution clone command:

```
CLI

pac solution clone --name <your solution name> --outputDirectory <your
chosen output directory>
```

You should now have a solution project in the specified output folder.

Create licensing files

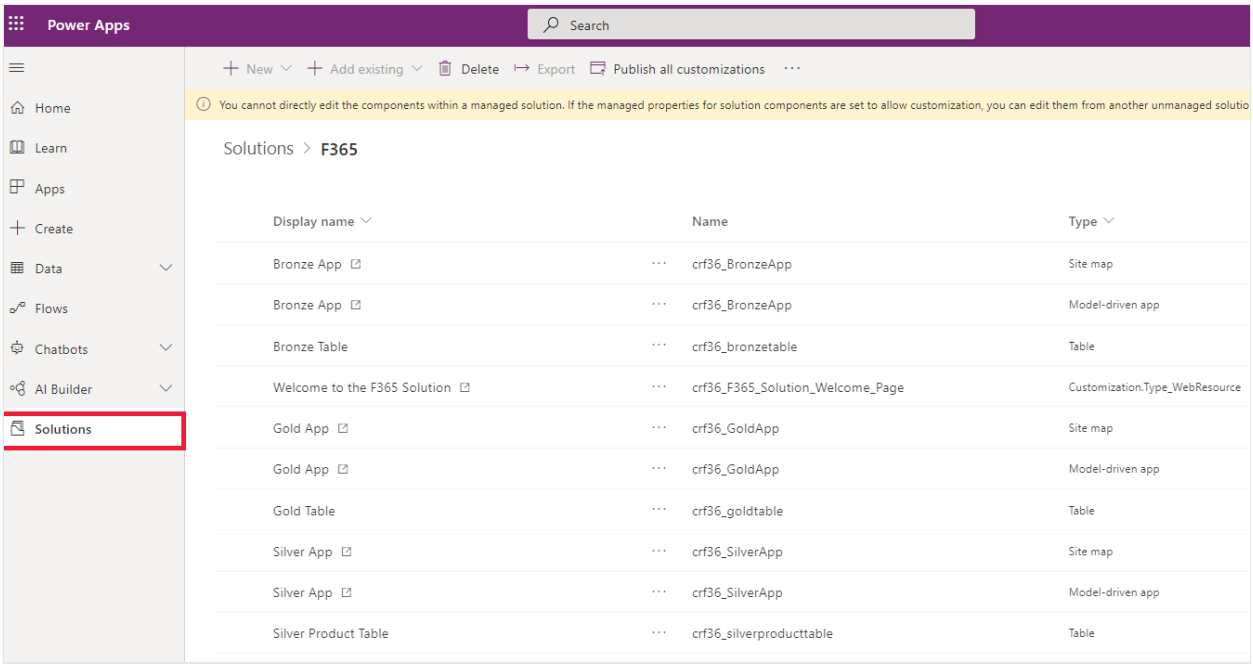
To add licensing information to your solution, you need to create two `.CSV` files using the text editor of your choice. The following are the two files that you need to create:

- **Plan definition file**: Define the details of the plans you created in the Partner Center.
- **Plan mapping file**: Maps those plans to the components of your solution.

In this article, we will create these two CSV files for an example solution called F365. This solution contains several model-driven apps for which we want to restrict access to via license management. We will follow a **Russian doll** licensing model for this solution as shown in the following table:

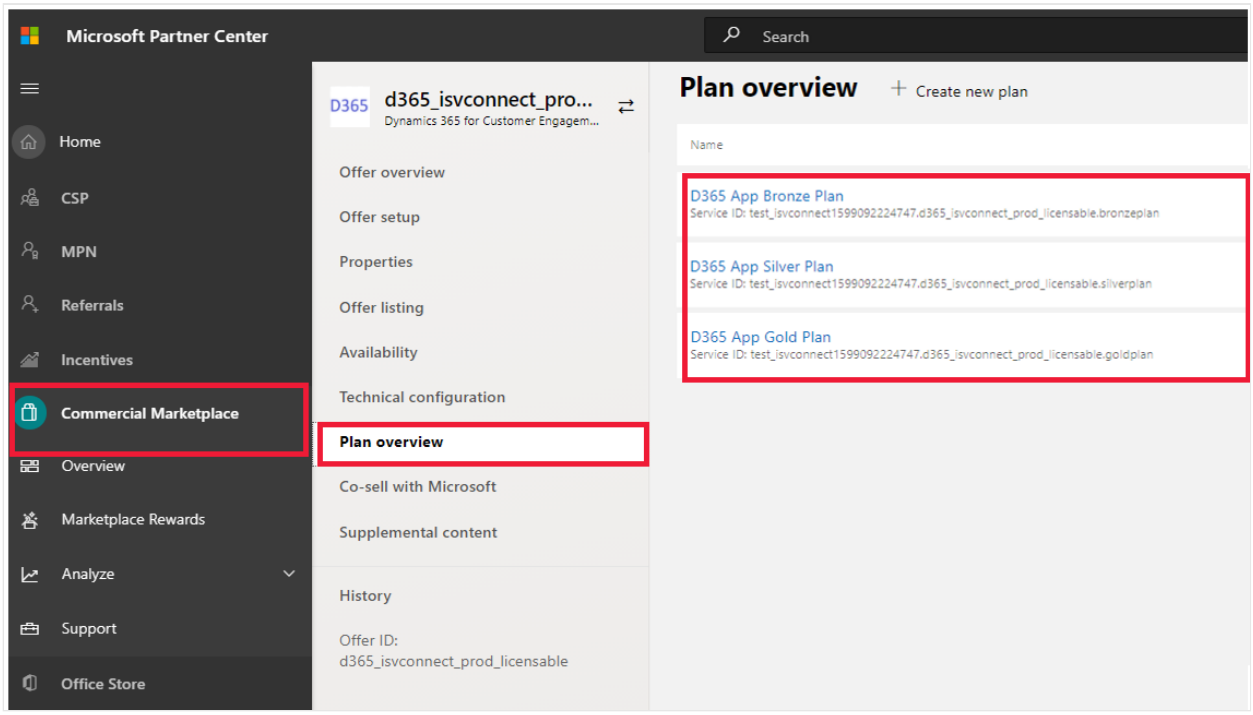
Expand table

Plan	Gets access to
Gold plan	Gold App, Silver App, Bronze App
Silver plan	Silver App, Bronze App
Bronze plan	Bronze App



To add licensing information to your solution, you need to create plans in the Partner Center. More information: Create plans in Partner Center.

Once you're finished creating the plans, each plan creates a **Service ID** value which is required while creating Plan definition file.



> **Note**
>
> Currently model-driven apps are the only types of solution components that can be mapped to licenses. In future releases, we plan to add support for other component types such as canvas apps and tables.

Plan definition file

The plan definition file must include the following columns laid out in the format shown in the example below:



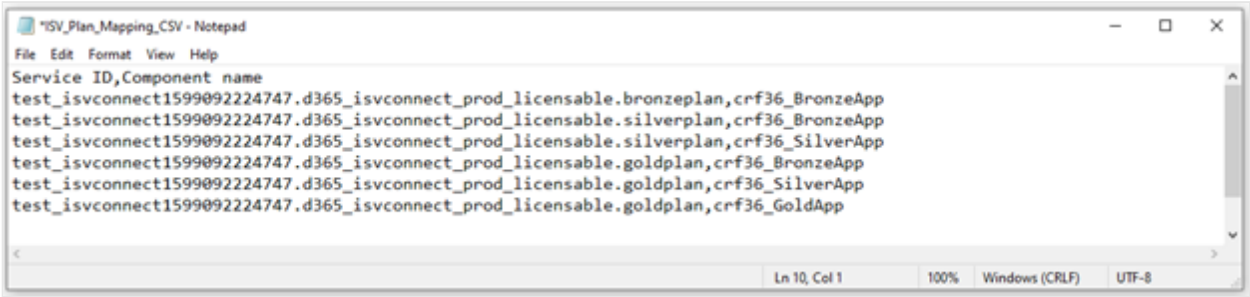
Service ID: This is created automatically when you create a plan in the Partner Center as part of the offer creation. Copy it from partner center into the plan definition file. More information: Create plans in Partner Center.

Display Name: This is a descriptive name for your plan which you can choose. It will be show to users on license check error screens so it should be easily recognizable to your customers.

More info URL: This is a URL where you would like to direct users to get more information about your solution, in the event they run into a license check error.

Plan mapping file

The plan mapping file must include the following columns laid out in the format shown in the example below:



Service ID: This is created automatically when you create a plan in the Partner Center as part of the offer creation. Copy it from partner center into the plan definition file. More information: Create plans in Partner Center.

Component name: The solution component that you would like to restrict access to using license management. This name must match the name of the component in the **Name** column in the Power Apps solution viewer (image above).

Once you have created your own licensing CSV files, the next step is to add the information from these files to your solution.

Add licensing information

To add the license information from the CSV files (created above) to the solution:

1. Using a command window, navigate to the location where you have cloned solution. Use the pac solution add-license command:

   CLI

   ```
   pac solution add-license -pd <path to plan definition file> -pm <path
   to plan mapping file>
   ```

2. To check whether the licensing information is added, navigate to the location where you have cloned your solution zip file, you should see a new folder **ServicePlans** is created with two XML files.

3. Optional: To determine how the license information from the CSV files was captured, open the XML files in your favorite editor.

   - The **ServicePlans.xml** file contains the plan definition information from the plan definition file.
   - The **ServicePlansAppModules.xml** file contains the plan mapping information from the plan mapping file.

Build the solution and create an AppSource package

1. To create the solution zip file, execute the MSBuild command shown below from the project folder. MSBuild is included in Visual Studio installations.

   msbuild

   ```
   msbuild /t:build /restore -p:Configuration=Release
   ```

   The resulting solution zip file can be found in the bin/Release folder of the project.

2. To validate whether the licensing information is included, after the solution is built, look for the licensing information in your solution `customizations.xml` file.

3. Import the unmanaged solution you just built into an environment, and then export it as a managed solution.

You may delete the imported solution from the environment afterwards since it is no longer needed. More information: Import solutions, Export solutions

4. To create the AppSource package, see Step 3: Create an AppSource package for your app.

See also

Step 2: Create a managed solution for your app
Appendix: App certification checklist

Using the catalog with Power Platform CLI (preview)

Article • 05/31/2024

[This article is pre-release documentation and is subject to change.]

The catalog in Power Platform enables developers and makers to:

- Crowd-source and find templates and components within their environment easily.
- Find and install the latest and authoritative version of a component.
- Get started with templates and components that provide immediate value.

Before you can work with catalog items, you must install and configure it. More information: Administer the catalog (preview)

Application makers and developers can submit items to the catalog so that they can help their colleagues solve business problems.

> **Important**
>
> - This is a preview feature.
> - Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.
> - While not required for preview, when this feature is generally available, it'll require the use of Managed Environment capabilities. More information: Managed Environments overview

Get started

1. Install Power Platform CLI
2. Get access to catalog as submitter or reader

Install Power Platform CLI

Follow the steps to install Power Platform CLI: Install Microsoft Power Platform CLI

Get access to catalog as submitter or reader

Contact your administrator to grant access to the catalog. You need the **Catalog Submitter** security role to be associated with your user account or a team that you belong to. More information: Edit user access

Use the catalog

After you have installed the Power Platform CLI, you must create an authentication profile using the pac auth create command. Using the authentication profile for the environment where the catalog is installed, you can perform the following tasks using PAC CLI:

1. View available catalogs in your tenant
2. View items in the catalog
3. Install items from the catalog
4. Submit items to the catalog
5. Check status of catalog submissions

View available catalogs in your tenant

Use the pac admin list command to view the catalogs available in your tenant.

> ⓘ **Note**
>
> This command requires an administrator authentication profile.

When using this command, you must use the following `--application` parameter with the value `83a35943-cb41-4266-b7d2-81d60f383695`.

PowerShell

```
pac admin list --application 83a35943-cb41-4266-b7d2-81d60f383695
```

`83a35943-cb41-4266-b7d2-81d60f383695` is the application ID associated with the catalog.

View items in the catalog

Use the pac catalog list command to view items in the catalog.

PowerShell

```
pac catalog list
Connected to... TestCatalog
Connected as user@domain
Listing all published catalog items from the current Dataverse
Organization...
Catalog Item Name                    Publisher Name            Catalog Item
Id                        Revision Id                          Version Status
Contoso Conference Custom Connector Catalog Conferences Team
ContosoConferencesCustomConnector    4e882dd6-74f3-ed11-8849-000d3a0a286b
1.0.0.1 Published
Contoso Themed Components           ContosoPublisher
ContosoThemedComponents              efbc469d-f1b2-ed11-83fd-000d3a0a2d9d
1.0.0.1 Published
```

Install items from the catalog

Use the pac catalog install command to install items from the catalog.

PowerShell

```
pac catalog install -tu https://<your org>.crm.dynamics.com/ -cid
ContosoConferencesCustomConnector
Connected to... TestCatalog
Connected as user@domain
ContosoConferencesCustomConnector
Tracking id for this installation is 202012ec-80f3-ed11-8849-000d3a0a2d9d
```

Submit items to the catalog

To submit an item to the catalog, you need:

- A solution or package deployer package containing the item you want to submit
- A submission metadata JSON document

Use the pac catalog create-submission command to get an example submission metadata JSON document. You must edit this document and more submission attributes can be added. More information: Submission attributes

After your submission metadata JSON document is ready, use the pac catalog submit command to submit it.

PowerShell

```
pac catalog submit -p "BuildDemoSubmission.json" -sz
"ContosoConference_1_0_0_1_managed.zip"
```

```
Creating package for catalog submit request...
Connected to... TestCatalog
Connected as user@domain
Tracking id for this submission is 0e6b119d-80f3-ed11-8849-000d3a0a2d9d
```

Check status of catalog submissions

Use the pac catalog status command to check the status of catalog submissions.

PowerShell

```
pac catalog status --tracking-id 0e6b119d-80f3-ed11-8849-000d3a0a2d9d --type
submit
Connected to... TestCatalog
Connected as user@domain
Status of the Submit request: Submitted
```

Submission attributes

Before you can submit items to a catalog, you must prepare a JSON document that describes the items you want to submit. More information: Submit items to the catalog

To help you, the pac catalog create-submission command generates a sample `submission.json` file.

> **Note**
>
> More attributes are supported in the submission file and those are covered in the subsequent section.

You need to edit this file to submit an item. Here's an example submission created from the JSON above.

JSON

```
{
  "modelVersion": "1.0.0.0",
  "operation": "CreateOrUpdate",
  "sourcePortal": 526430005,
  "businessJustification": "Power Platform custom connector for Conference
API",
  "publisherDetails": {
    "publisherId": "ContosoConferencesTeam",
    "publisherDisplayName": "Catalog Conferences Team"
```

```
  },
  "catalogItemDefinition": {
    "id": "ContosoConferencesCustomConnector",
    "displayName": "Contoso Conference Custom Connector",
    "description": "Demo Custom connector to query Conference Speakers &
Sessions",
    "offer": {
      "type": "Component_Collection",
      "deploymentType": "Normal",
      "engineeringName": {
        "firstName": "Jennifer",
        "lastName": "Wilkins",
        "email": "jwilkins@contoso.com",
        "phoneNumber": "555-111-1234"
      },
      "supportName": {
        "firstName": "Aidan",
        "lastName": "Hunt",
        "email": "ahunt@mouse.com",
        "phoneNumber": "555-111-1234"
      }
    }
  }
}
```

Required data

The following items are required for all submissions:

- Publisher
- Catalog item
- Engineering contact
- Support contact

Publisher

Creates a publisher record that can be associated to one or more Microsoft Entra ID groups containing authorized users who can make updates to the catalog item going forward.

Don't confuse this term with *Solution Publisher*. Catalog item publisher isn't used with solutions.

You need to provide the ID publisher `DisplayName` at a minimum. ID can be any string value. The system checks if that publisher exists, else creates it and assigns an ID.

For example, HR IT team of developers can create a publisher and ID the developers using a Microsoft Entra ID group.

Here's a sample of a publisher record:

JSON

```
"publisherDetails": {
   "publisherId": "MyPublisherId",
   "publisherDisplayName": "Contoso Publishing",
   "publisherUpnList": [
      {
         "action": "Add",
         "upn": "john.doe@contoso.com"
      }
   ],
   "publisherAADGroupOIDS": [
      {
         "action": "Add",
         "groupName": "PowerCatalogSubmitters",
         "groupOID": "2ded6de9-ab44-4478-9bd4-e609947daa2e",
         "groupType": "Security"
      }
   ]
}
```

Catalog item

The actual Power Platform solution. A package is what is stored in the catalog.

`id` and `displayName` are mandatory. `description` and `businessCategories` are optional.

Seeded business categories list of value is in the appendix. `ComponentUsedInApplication` is for future use only.

Here's an example of a catalog item including its `engineeringName` and `supportName` contacts that are mandatory.

JSON

```
"catalogItemDefinition": {
   "id": "ContosoTravelsApp",
   "displayName": "Contoso Travel Expense App",
   "description": "Submit expenses on the go!",
   "offer": {
   "businessCategories": [526430000, 526430001, 526430002],
   "type": "Component_Collection",
   "deploymentType": "Normal",
   "small48x48Image": {
```

```
      "name": "Small48Image",
      "fileSasLink": "https://www.contoso.com/icon48.jpg"
   },
   "large216x216Image": {
      "name": "Large216image2",
      "fileSasLink": "https://www.contoso.com/icon216.jpg"
   },
   "documents": [
      {
         "name": "ImaNewfile",
         "fileSasLink": "https://www.contoso.com/screenshot1.jpg"
      },
      {
         "name": "ImaNewfile2",
         "fileSasLink": "https://www.contoso.com/screenshot2.jpg"
      }
   ],
   "helpLink": "https://www.contoso.com/help.html",
   "privacyPolicyLink": "https://www.contoso.com/privacy.html",
   "legalTerms": "https://www.contoso.com/legal.html",
   "engineeringName": {
      "firstName": "John",
      "lastName": "Roon",
      "email": "john@contoso.com",
      "phoneNumber": "999-111-1234"
   },
   "supportName": {
      "firstName": "Jane",
      "lastName": "Drill",
      "email": "jane@contoso.com",
      "phoneNumber": "999-111-1234"
   }
   }
}
```

Engineering contact

Developer product contact.

Contact that the consumer of catalog item can reach out to if they have technical questions to extend or reuse the app for another purpose.

JSON

```
"engineeringName": {
   "firstName": "String"
   "lastName": "String (required)"
   "email": "String (required)"
   "phoneNumber": "String"
}
```

Example

JSON

```
   "engineeringName": {
      "firstName": "John",
      "lastName": "Roon",
      "email": "john@contoso.com",
      "phoneNumber": "999-111-1234"
   },
```

Support contact

IT support specialist.

Can be same as the engineering contact that the consumer of a catalog item can reach out to if they run into issues with the application.

JSON

```
"supportName": {
   "firstName": "String"
   "lastName": "String (required)"
   "email": "String (required)"
   "phoneNumber": "String"
}
```

Recommended data

The following items are highly recommended, but not required.

- small48x48image
- Documents
- Help Link

small48x48image

Provide information of the icon to be displayed in maker or other discovery UI. If you don't provide this image a system default icon is used.

JSON

```
"small48x48image":{
   "name": "String",
```

```
    "fileSasLink": "Fully qualified publicly accessible url"
}
```

Documents

You can pass screenshots here of your application to help makers identify whether this app provides the functionality or capability they're looking for.

The most common thing people do before downloading an app or template is look at its images and screenshots to understand the capabilities instead of reading the description.

JSON

```
{
    "name": "String",
    "fileSasLink": "Fully qualified publicly accessible url"
}
```

Help link

URL link to help documentation.

Fully qualified URLs your users are able to access. These URLs can be links to internal docs or sharepoint resources.

JSON

```
 "helpLink": "https://www.yourcompany.com"
```

The following items are optional

- Source Portal
- Operation
- Business Justification
- large216x216Image
- Privacy policy
- Legal terms

Source portal

Source of submission.

When submitting from the Power Platform CLI, use `526430005` as the **Source Portal** value. For other sources, use `526430000`.

Business categories

One or more of the business categories can be included in the submission from the list below.

Expand table

Code	Description
526430000	Customer Service
526430001	Project Management
526430002	Calendar Management & Scheduling
526430003	Email Management
526430004	Files & Documentation
526430005	Notification & Reminders
526430006	Analytics
526430007	Collaboration
526430008	Commerce
526430009	Finance
526430010	Compliance & Legal
526430011	Sales
526430012	IT Tools
526430013	Marketing
526430014	Operations & Supply Chain
526430015	Internet of Things
526430016	AI Machine Learning
526430017	Geolocation
526430018	Human Resources

Currently, new business categories can't be configured.

Operation

Possible values:

- `CreateOrUpdate`
- `Create`
- `Update`
- `Disabled`

Use `CreateOrUpdate` for new submissions.

JSON

```
  "operation": "CreateOrUpdate"
```

Business justification

Explains the business value created by this catalog item. This information is visible to approvers and appears in the planned consumption experience in the maker discovery user experience.

Users can say their submission helps with cost reduction or productivity, for example. Some organizations require certain classes of value and those values can be entered here.

String can contain HTML, or Rich Text (RTF).

JSON

```
"businessJustification": "Submit your travel expenses!"
```

large216x216Image

Provide information of the icon to be displayed in the maker or other discovery UI.

Some user experiences uses larger icons to render cards for discovery.

JSON

```
name: String
fileSasLink: fully qualified publicly accessible url
```

```
LogicalName: string, for future use, ignored
recId: guid, for future use, ignored
attributeName: string, for future use, ignored
```

An example of the image metadata.

JSON

```
      "large216x216Image": {
        "name": "Large216image2",
        "fileSasLink": "https://mywebsite/largeicon.jpeg"
      },
```

Privacy policy

URL link to pertinent policies.

Policies regarding careful usage of the component or application being submitted.

JSON

```
"privacyPolicyLink": "https://www.mycompany.com/privacy.html"
```

Legal terms

URL link to pertinent terms and conditions.

Any legal terms or internal guidance for evaluation of the consumer of the catalog item.

JSON

```
"legalTerms": "https://www.mycompany.com/legalterms.html"
```

See also

Administer the catalog (preview)

Feedback

Was this page helpful? Yes No

Provide product feedback

Unified developer experience for finance and operations apps (preview)

Article • 10/25/2023

[This article is pre-release documentation and is subject to change.]

The unified developer experience consolidates the disparate developer tools and environments across finance and operations apps and Power Platform to reduce friction and simplify working across these apps. Finance and operations apps provide a rich ecosystem for professional developers by using a metadata and code-based development environment for mission critical scenarios. Power Platform brings the ability to author solutions quickly and seamlessly using low-code development paradigms and leverages Microsoft Dataverse as the relational data store. Power Platform also adds the ability to build and deploy your solutions using a continuous integration and deployment (CI/CD) pipeline.

> **Important**
>
> - This is a preview feature.
> - Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.

Typical scenario

Consider a typical scenario where a developer would create a simple app for capturing orders in Microsoft Power Apps and persisting the gathered data in Dataverse. By means of the data synchronization engines mentioned below, the data would be available for finance and operations apps to do the heavy lifting of determining whether the requested items are available and pricing, etc. The results would be available near real-time in Power Apps.

Finance and operations app development

Development for finance and operations apps is accomplished in Visual Studio using the Visual Studio add-in. This development environment provides everything a professional developer would expect. The unified environment runs in the cloud, and because of that

we have made several changes and improvements to the way a developer works with finance and operations apps. However, existing finance and operations app developers will find that the large majority of their workflows will not be different from what they are used to. The article Write, deploy, and debug X++ code (preview) walks you through how to build X++ code, deploy it to the cloud, execute and debug it there. All other content that you are likely to find about finance and operations app development, like how to extend existing code, is likely to still hold true.

More information: Install and configure development tools (preview)

Power Platform

Power Platform is supported by a rich development ecosystem. There is a rich set of tools in Power Platform that you can leverage for development. In particular, the Dataverse relational database management system is utilized by the unified developer experience. Many unified solutions work by synchronizing data between tables in the finance and operations app database and tables (entities) in Dataverse. There are two technologies for that: dual-write, and virtual entities.

More information: Microsoft Power Platform developer documentation

Dual Write

Dual-write provides a tightly coupled near real-time and bi-directional integration between the finance and operations apps and Dataverse. Once an entity is enabled for dual-write, any create, update, or delete change in finance and operations apps results in writes to Dataverse and vice versa. For example, a change in the Customer entity in finance and operations apps is reflected in the Account entity in Dataverse and vice-versa. While all this happens with minimal setup, Power Platform does provide an advanced user interface for all your customization needs.

More information: DualWrite .

Virtual tables

Virtual tables, also known as virtual entities, enable the integration of data residing in external systems (including finance and operations). The virtual table capability seamlessly represents external data in Dataverse tables without replication of data and often without custom coding.

More information: Virtual Tables.

Next steps

Set up Visual Studio on your local development computer.

Install and configure development tools

See also

Install and configure development tools (preview)
Write, deploy, and debug X++ code (preview)
Frequently asked questions (preview)
Tutorial: Provision a new environment with an ERP-based template (preview)
Develop and customize home page (Dynamics 365 legacy information)
Unified admin experience for finance and operations apps (preview)

Install and configure development tools (preview)

Article • 05/21/2024

[This article is pre-release documentation and is subject to change.]

This article describes how to install and configure required Visual Studio finance and operations tools into your local development environment. The instructions in each section should be followed in the order presented in this article.

> **Important**
>
> As a prerequisite, you need access to a **provisioned developer-focused sandbox environment**.
>
> The user account you'll be using for development in the sandbox environment must be assigned the System Administrator role.
>
> The development machine running Microsoft Windows must have Visual Studio 2022 installed with at least the .NET desktop development workload and the Modeling SDK. This SDK can be selected under individual components in the Visual Studio installer. Refer to **required Visual Studio components**. The development machine also must have Microsoft SQL Server or SQL Server Express LocalDB installed.

Install the Power Platform extension for Visual Studio

Follow the instructions in this article to install the extension from the marketplace: What is Power Platform Tools for Visual Studio. You don't need to install the plug-in profiler as directed in that article.

Connect to your online environment

After installing the Power Platform Tools extension for Visual Studio, follow these instructions to connect Visual Studio to your unified developer-focused environment.

1. Launch Visual Studio if not already running.

2. In the **Tools** menu, select **Connect to Dataverse** or **Download Dynamics 365 FinOps assets**.

    - **Connect to Dataverse** only downloads assets if not downloaded earlier.
    - **Download Dynamics 365 FinOps assets** always downloads.

3. Select the desired options in the dialog and select **Login**.

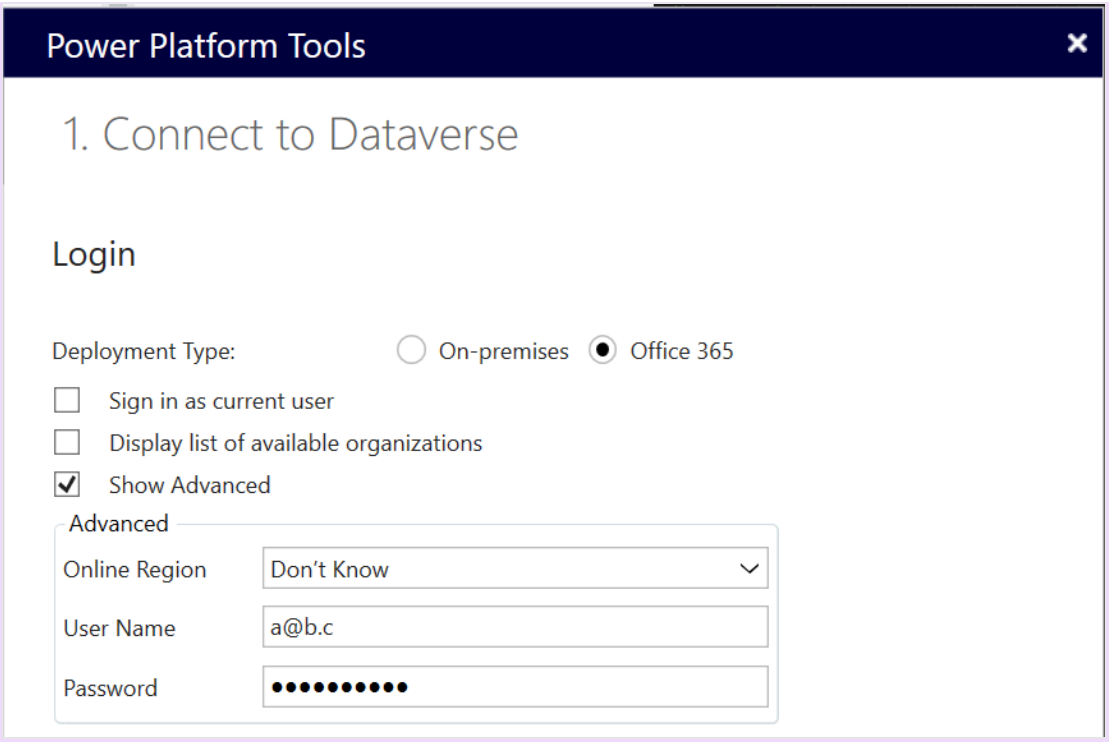
    If you need to use another account or a different online environment, don't choose to use the signed-in user. Enter the credentials for your developer user account with access to the online environment.

4. Select an available solution in the dialog, and then choose **Done**.

5. Find the sandbox environment created for online development in the list of Dataverse environments and connect to it.

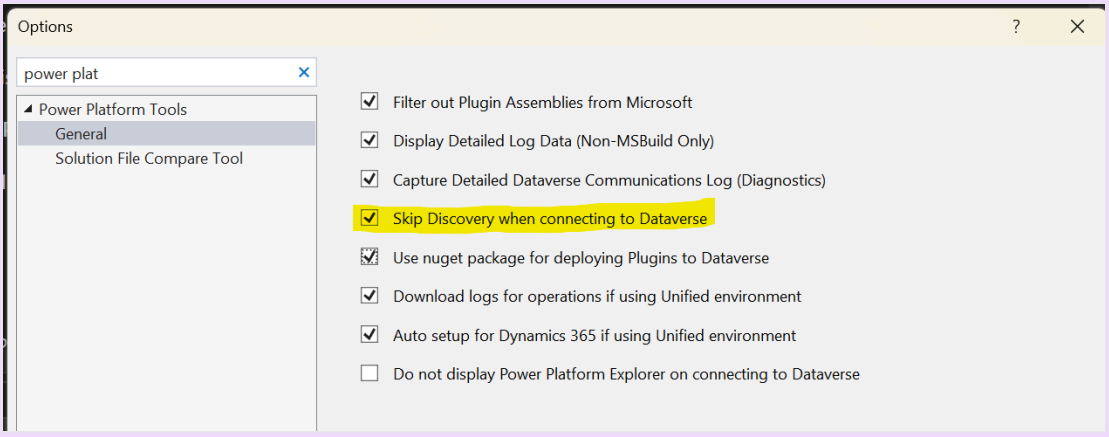
    > ⓘ **Note**
    >
    > If you have only one online development environment, the organization list dialog may not be shown as you would automatically connect to the environment.

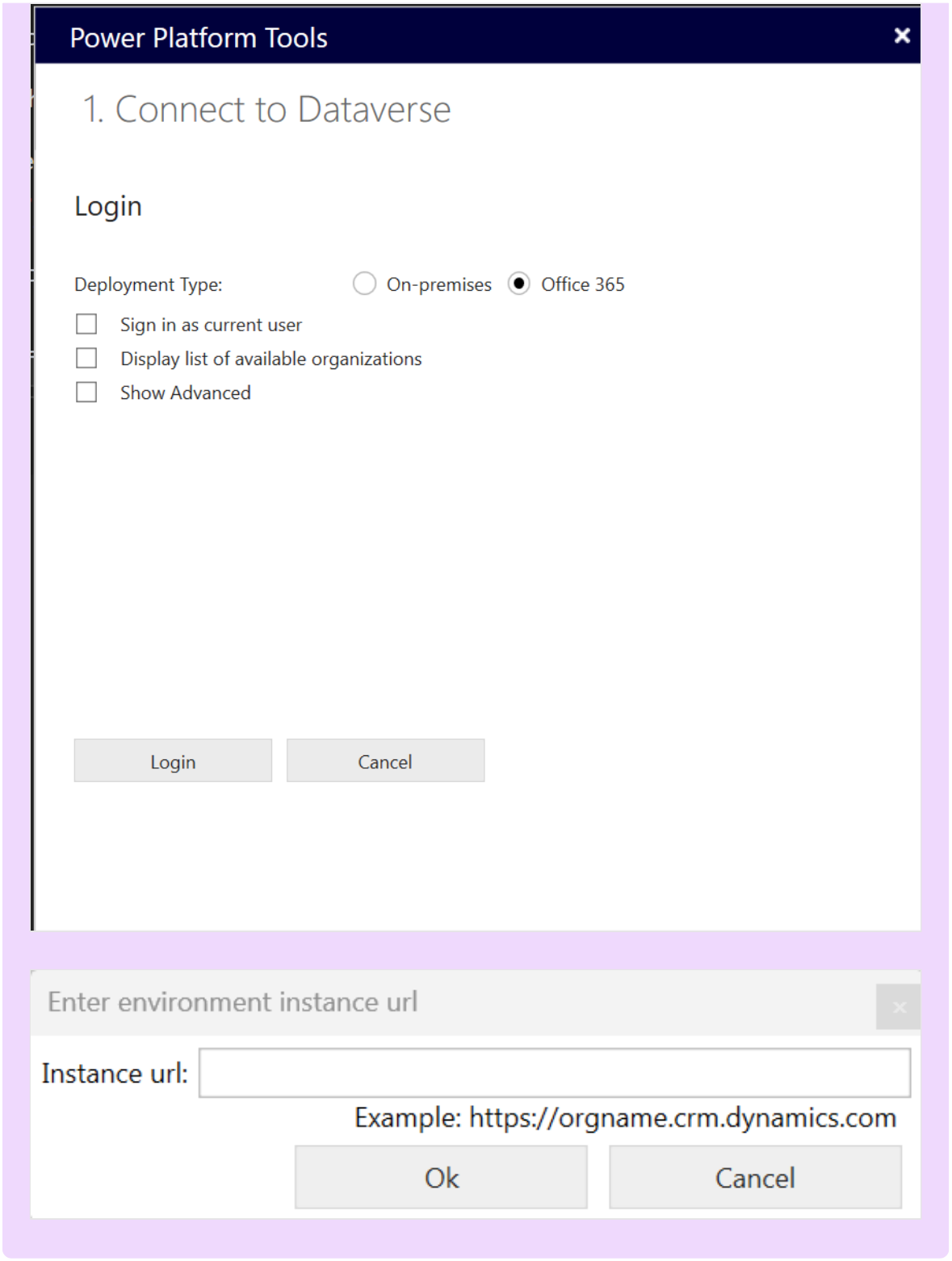
    



If your account requires multi factor authentication, uncheck all checkboxes on login screen.

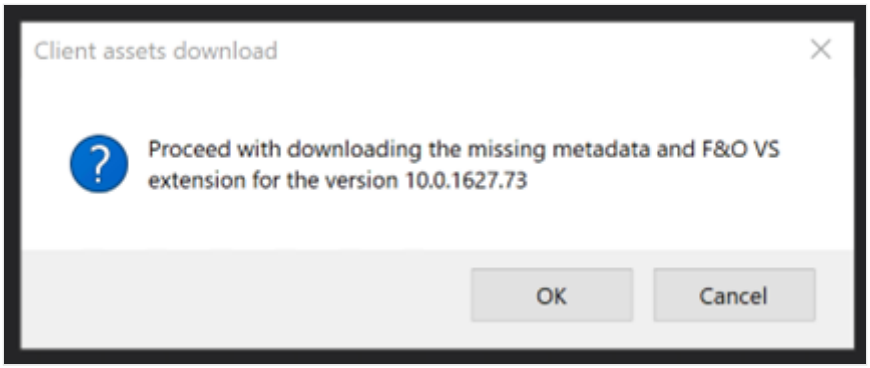
If you are logging in to an online development environment in a tenant different from your own, where your user principal name (UPN) is added as a guest, update **Visual Studio** > **Tools** > **Options** > **Power Platform Tools** and select **Skip Discovery when connecting to Dataverse** and uncheck all the boxes on the login pane. You will be prompted to provide the Dataverse uri.





Download and install the finance and operations extension and metadata

After installing the Power Platform Tools extension and connecting to the online Dataverse sandbox environment, you'll be presented with a dialog to download the finance and operations Visual Studio extension and metadata.

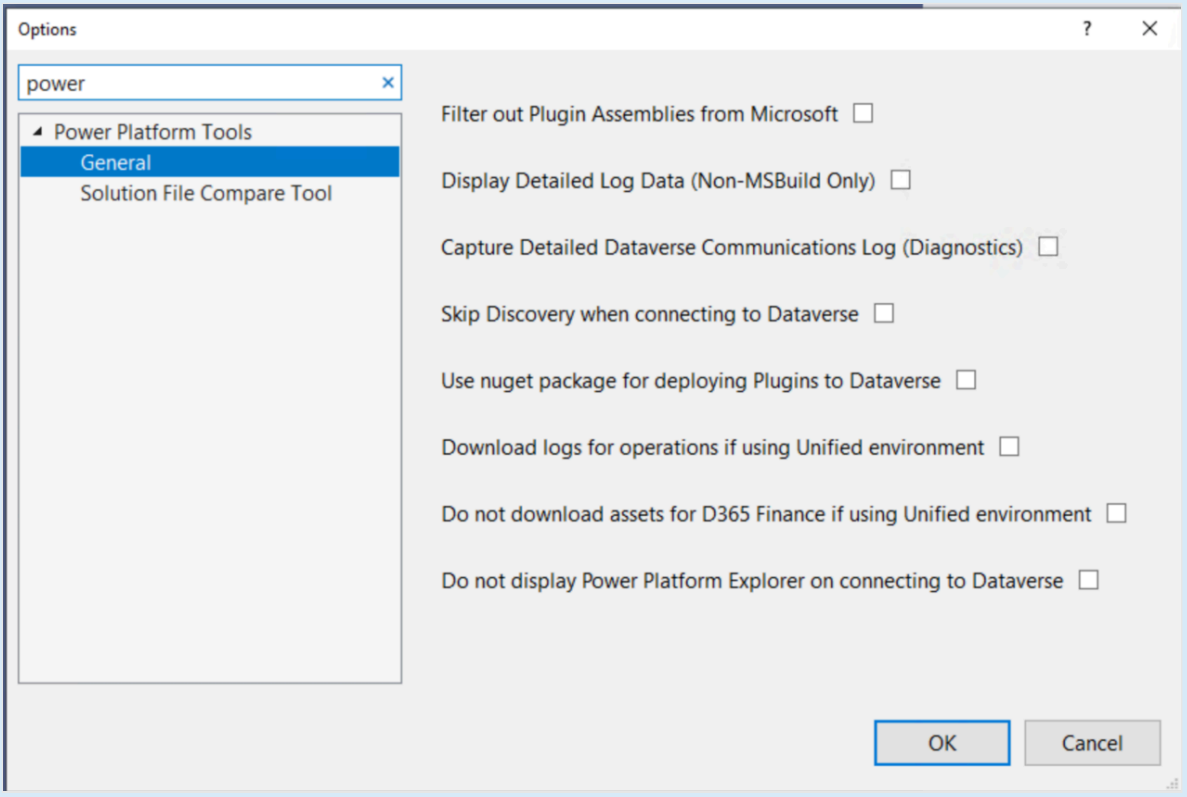


ⓘ Important

You can configure settings using the Visual Studio setting **Tools** > **Options** > **Power Platform Tools**.

The option **Auto setup for Dynamics 365...** is enabled by default. This ensures you do not have to manually extract system metadata or create the metadata configuration. The tool will do this for you. Do look into the Visual Studio output pane for any user action needed.

Consider selecting the option **Do not display Power Platform Explorer...** to speed up connecting to your environment and **Download logs...** to get logs for deployment and DBSync operations.



You need at least 12 GB of free space on the local system drive to download the extension and metadata.

Choose **OK** in the dialog to start the download. Visual Studio keeps you updated on the progress in the Visual Studio output pane.

Extract system metadata

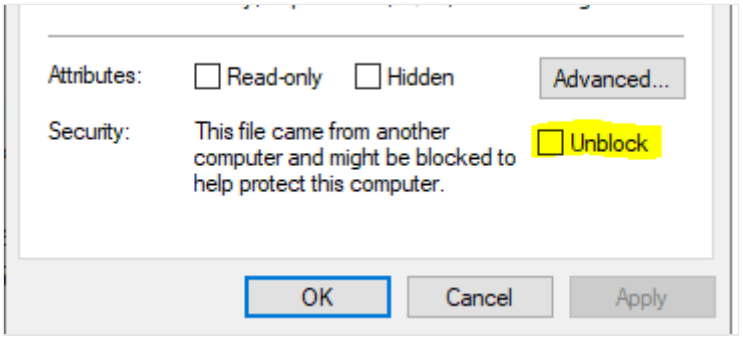
> **Note**
>
> This is not required when auto setup is enabled

Downloaded assets can be found in the following location:
`C:\Users\<User>\AppData\Local\Microsoft\Dynamics365\<ApplicationVersion>`

The following files are downloaded:

- `DYNAMICSXREFDB.bak` is a backup of the cross reference database.
- `Microsoft.Dynamics.Framework.Tools.Installer.vsix` is the Finance and operations extension.
- `PackagesLocalDirectory.zip` is the System metadata.
- `TraceParser.msi` Learn more about Trace Parser.

Unblock any of the three files that need unblocking.



Unpack the `PackagesLocalDirectory.zip` file as `PackagesLocalDirectory`. We recommend using the 7zip utility to speed up extraction.

Install the finance and operations extension

> **Note**
>
> This is only required when the Visual Studio output pane indicates it is.

Using Windows File Explorer, from the folder specified below, open the `Microsoft.Dynamics.Framework.Tools.Installer.vsix` file to install the extension.

`C:\Users\<User>\AppData\Local\Microsoft\Dynamics365\<ApplicationVersion>`

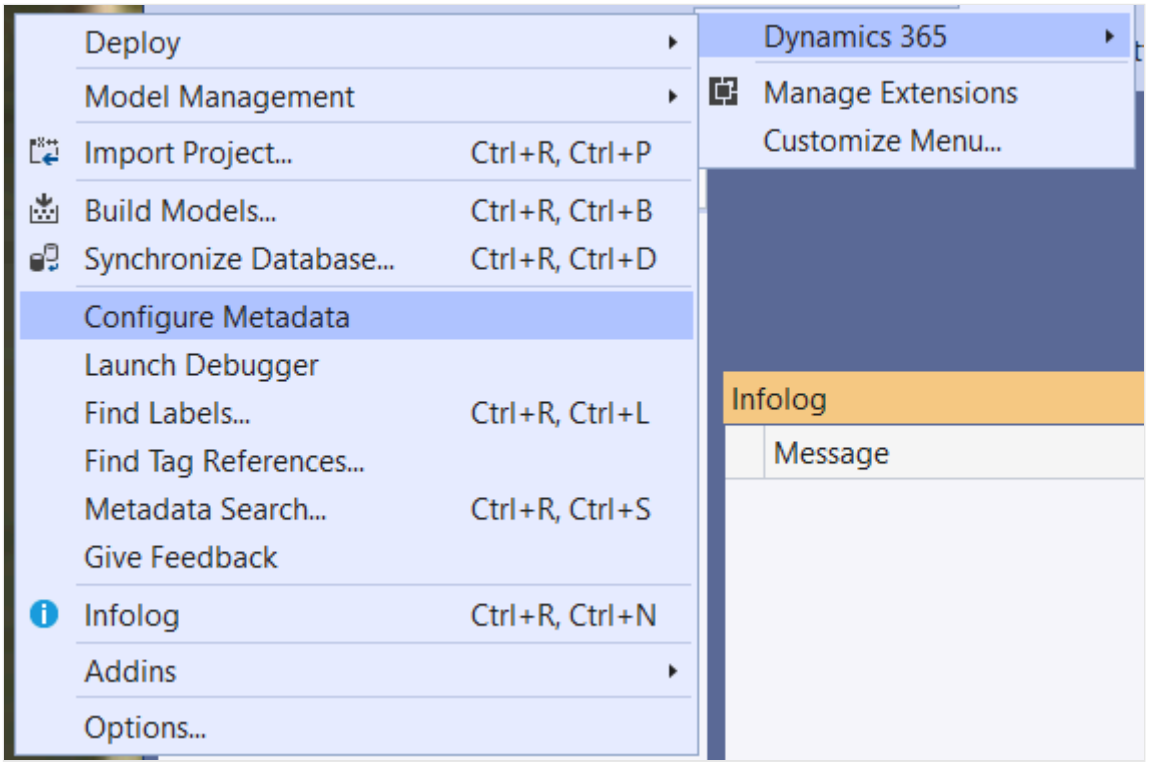
Configure the finance and operations extension

> **Note**
>
> This is not required when auto setup is enabled.

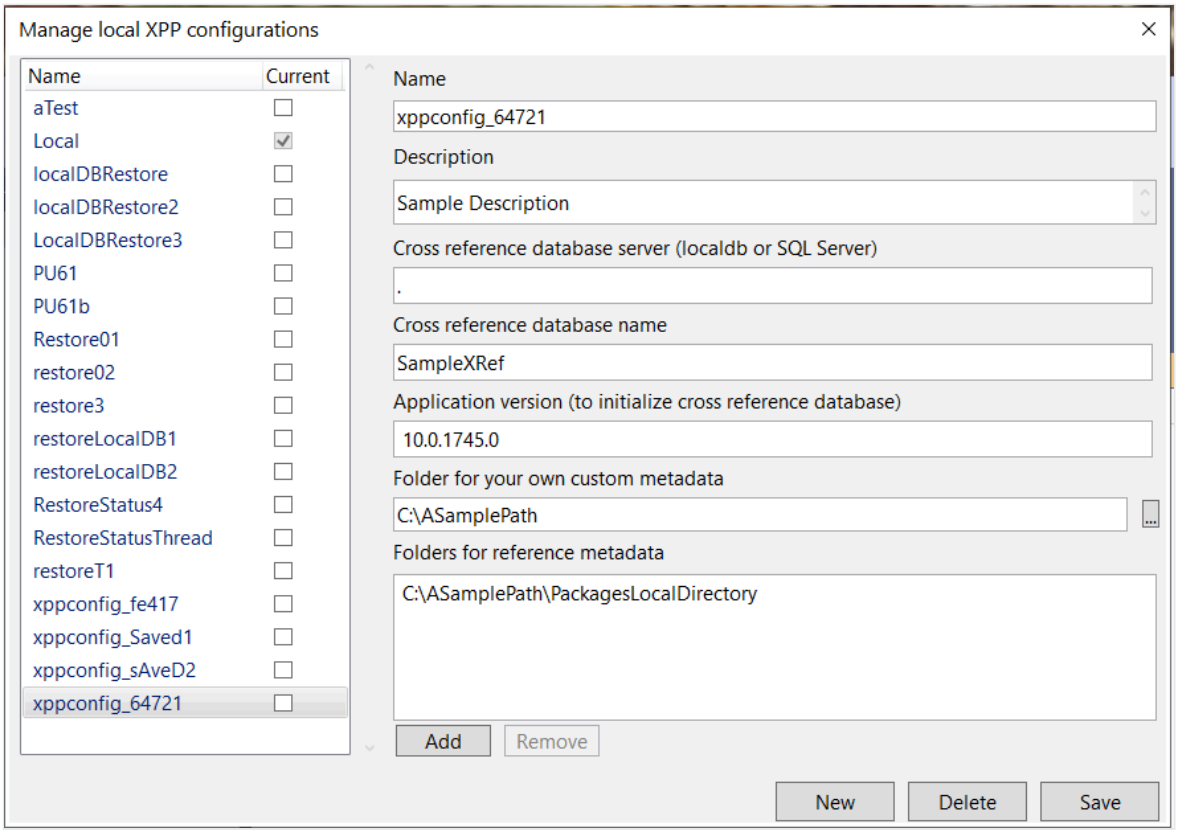
To configure the extension, follow these instructions.

1. Start Visual Studio, and then select **Continue without code** from the dialog.
2. Navigate to **Extensions** > **Dynamics 365** > **Configure Metadata** for the first time.

   If you don't see the **Configure Metadata** option, open **Extensions** > **Dynamics 365** > **Infolog** and retry.



3. On the configuration form, select **New** and then create a new configuration. Refer to the field descriptions provided in the Configuration form field descriptions table that follows this procedure.



4. Select **Save**.

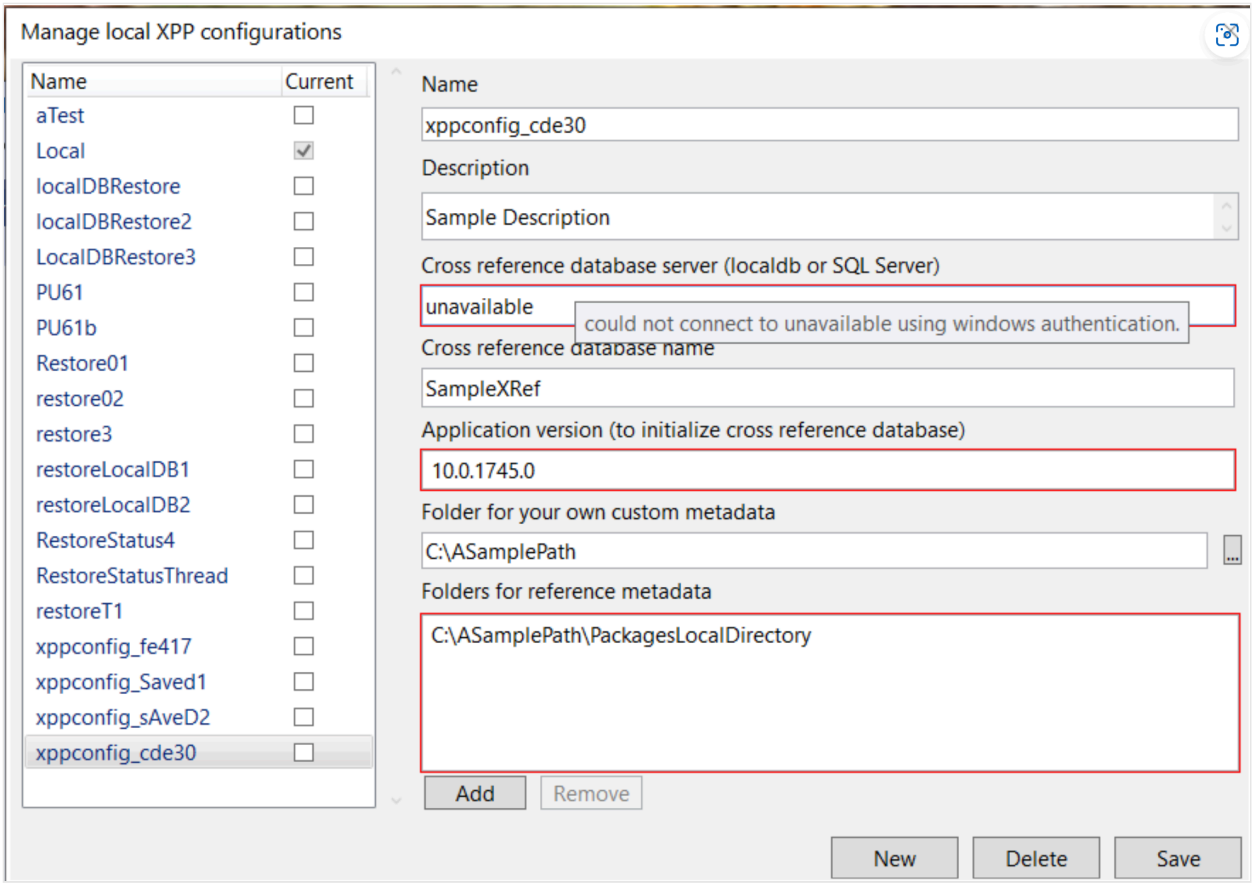
Configuration form field descriptions

Expand table

Form field	Description
Name	Name of the configuration you're creating now.
Description	Description of the configuration you're creating now.
Cross reference database server	Set to `(localdb)\.` if you're using LocalDB, or `localhost` if you're using Microsoft SQL Server.
Cross reference database name	The name for the cross reference database. The Visual Studio extension creates one if it doesn't exist.
Application version to restore cross reference database from	Select this. It's populated from the assets downloaded when you connected to the unified developer environment.
Folder for your own custom metadata	Folder where your own code is (or will be).
Folders for reference metadata	Should contain at least the path where you have the extracted the PackagesLocalDirectory folder mentioned in Extract system metadata.

Troubleshooting configuration

If the **Save** button is still grayed out after filling in all the fields, you'll see error messages with a red border in tooltips on fields with invalid values.



Ensure that you entered the correct values for fields with errors. For example, use `(localdb)\.` if you're using LocalDB. Also, if you're using LocalDB, you might need to first configure and test it. Consider issuing the following command from a Command prompt:

`sqllocaldb create MSSQLLocalDB -s`.

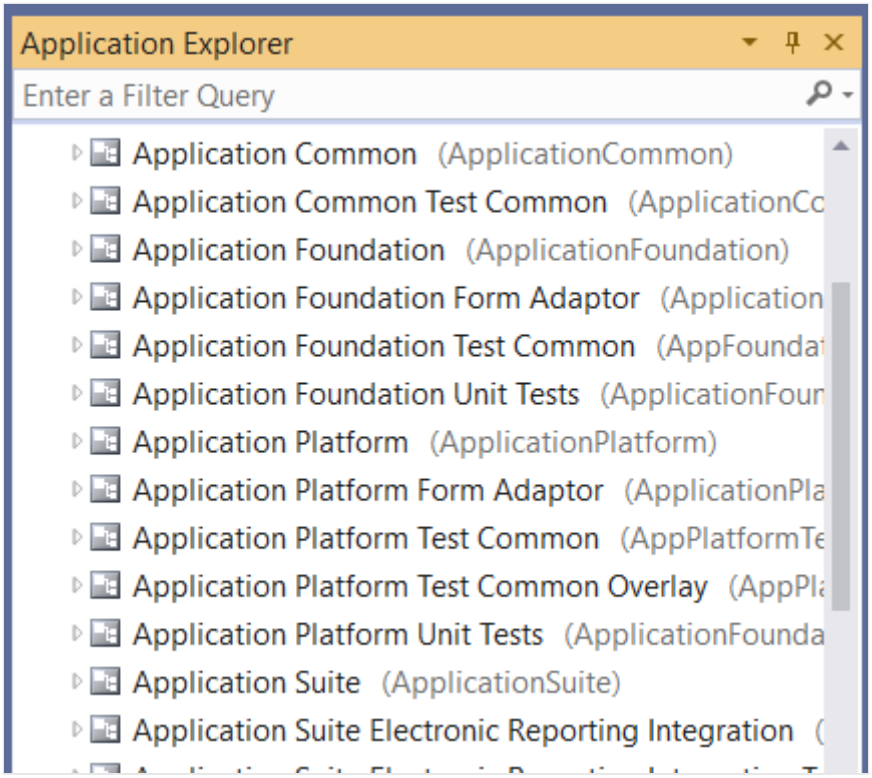
More information: SQL Server Express LocalDB.

> **Note**
>
> If the cross reference database does not already exist, the tool will restore the database from the backup downloaded to C:\Users\`<User>`\AppData\Local\Microsoft\Dynamics365\`<ApplicationVersion>`. You'll see notifications in the Visual Studio **Output** pane. If you need to restore it again by yourself you can do so. For instructions to do a restore refer to Restore to SQL Server.

Completeing configuration

After the configuration is saved, you might get a prompt for elevation as administrator to register the protocol handler and to extract compiler files. Go ahead and accept these prompts.

Once the configuration completes, navigate to **View** > **Application Explorer** . You should see the **Application Explorer** window open. You might have to restart Visual Studio first.



You can create several configurations, but must select one to be current. The chosen configuration takes effect on new instances of Visual Studio.

Summary

Congratulations! You're now ready to build, deploy, debug, and test your X++ modules working with the Power Platform online development environment. Explore the possibilities with the power of finance and operations integrated with Power Platform.

Next steps

Start writing code, deploy, and debug.

Write, deploy, and debug X++ code

See also

Feedback

Was this page helpful? Yes No

Provide product feedback

Write, deploy, and debug X++ code (preview)

Article • 09/14/2023

[This article is pre-release documentation and is subject to change.]

The unified developer experience for finance and operation apps enables you to write code on your local development computer and run it inside a cloud service. There exists a separation of functionality between these two tiers (local and cloud).

- Development tier - contains the finance and operations metadata and any X++ source code on your local development computer
- Execution tier - cloud hosted site that stores the business data and executes X++ code

This article shows you how to:

1. Write and compile a runnable X++ class
2. Deploy the class to the cloud runtime
3. Debug the deployed X++ code

Let's begin by developing a runnable X++ class to use as an example. The class's only task is to log a message into the Infolog. The important thing here isn't what the runnable class does, rather it's how we compile, run, and debug it. If you want to learn about the X++ language, see the programming reference.

As a prerequisite, you need to have completed setting up your local Visual Studio developer environment. More information: Install and configure development tools (preview)

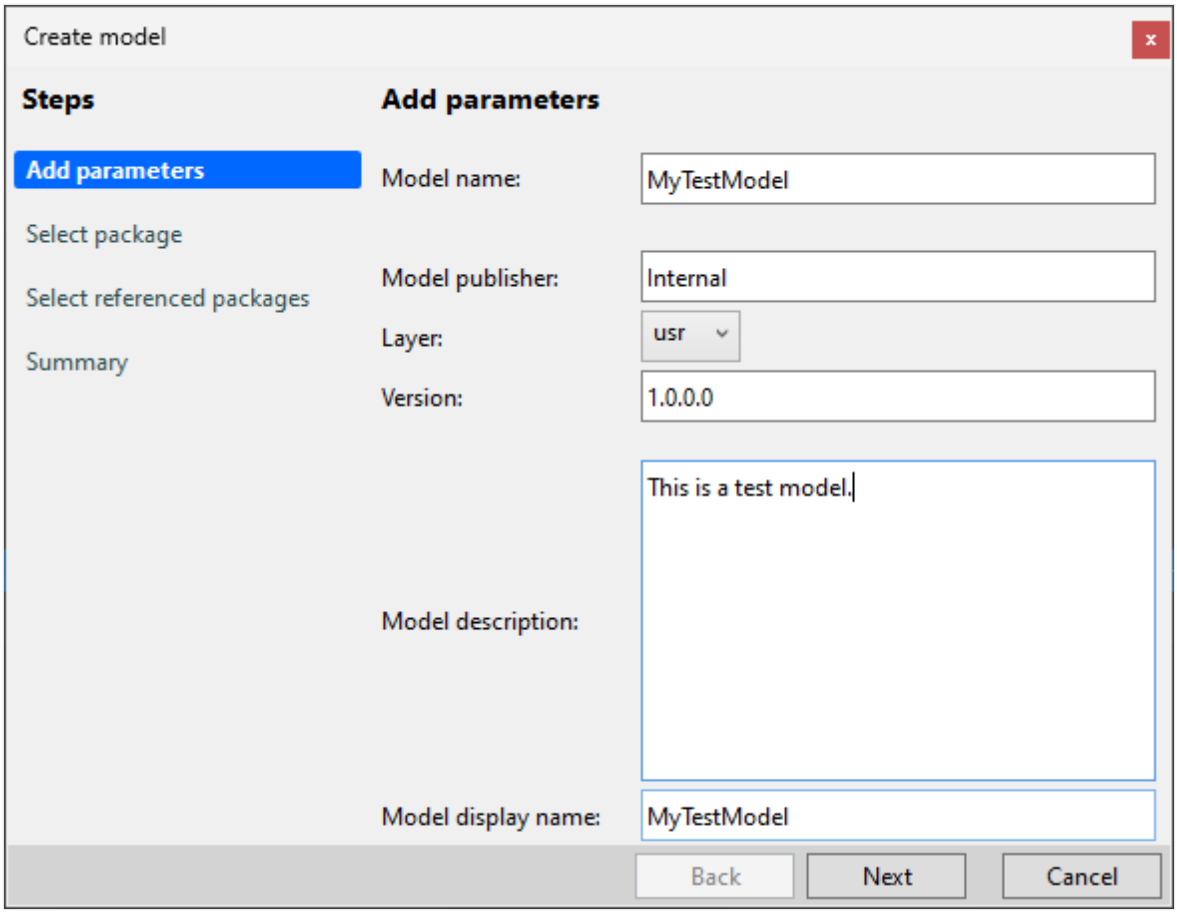
Create a model

We start by creating a model in Visual Studio.

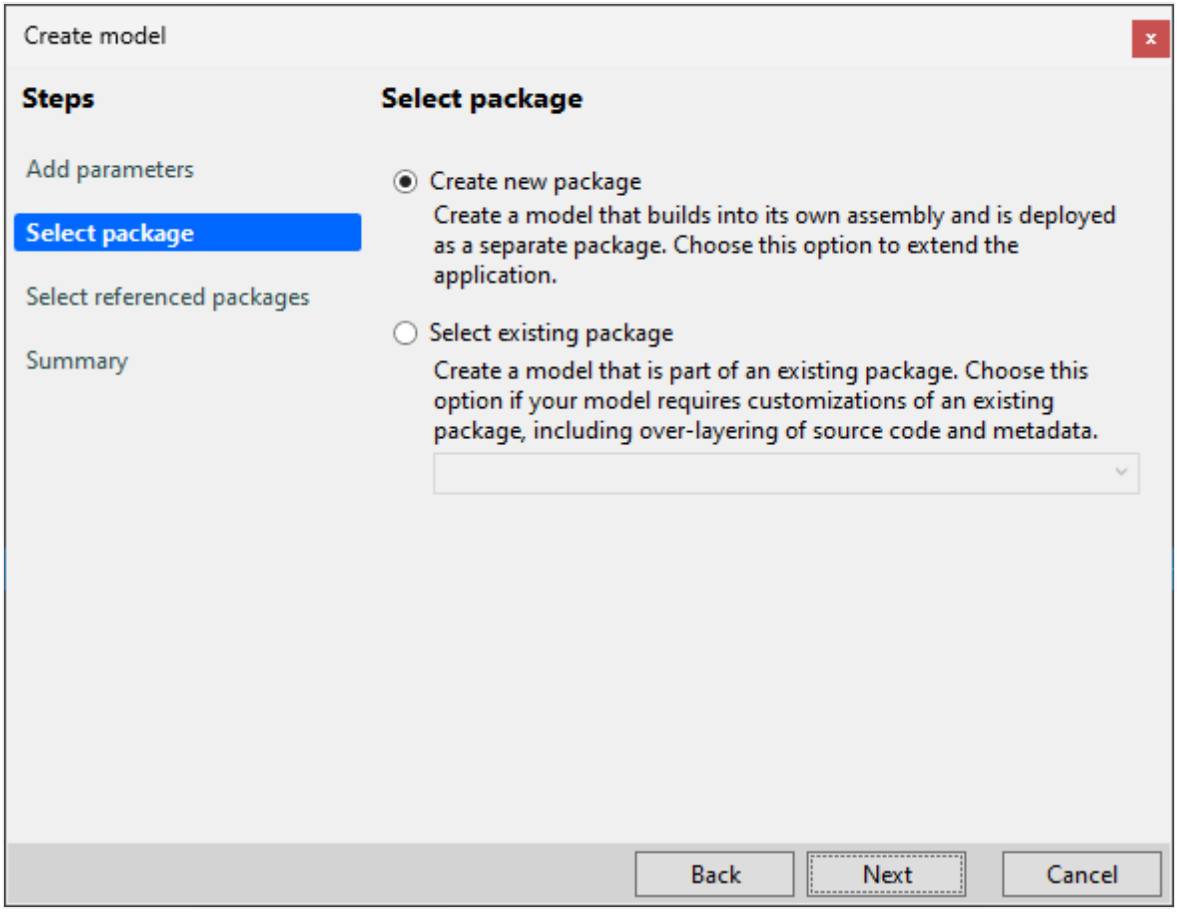
1. Launch Visual Studio and choose **Continue without code**.

2. Choose the menu option **Extensions** > **Dynamics 365** > **Model management** > **Create model...**. If you don't see this menu option, you don't have the required Visual Studio extension installed.

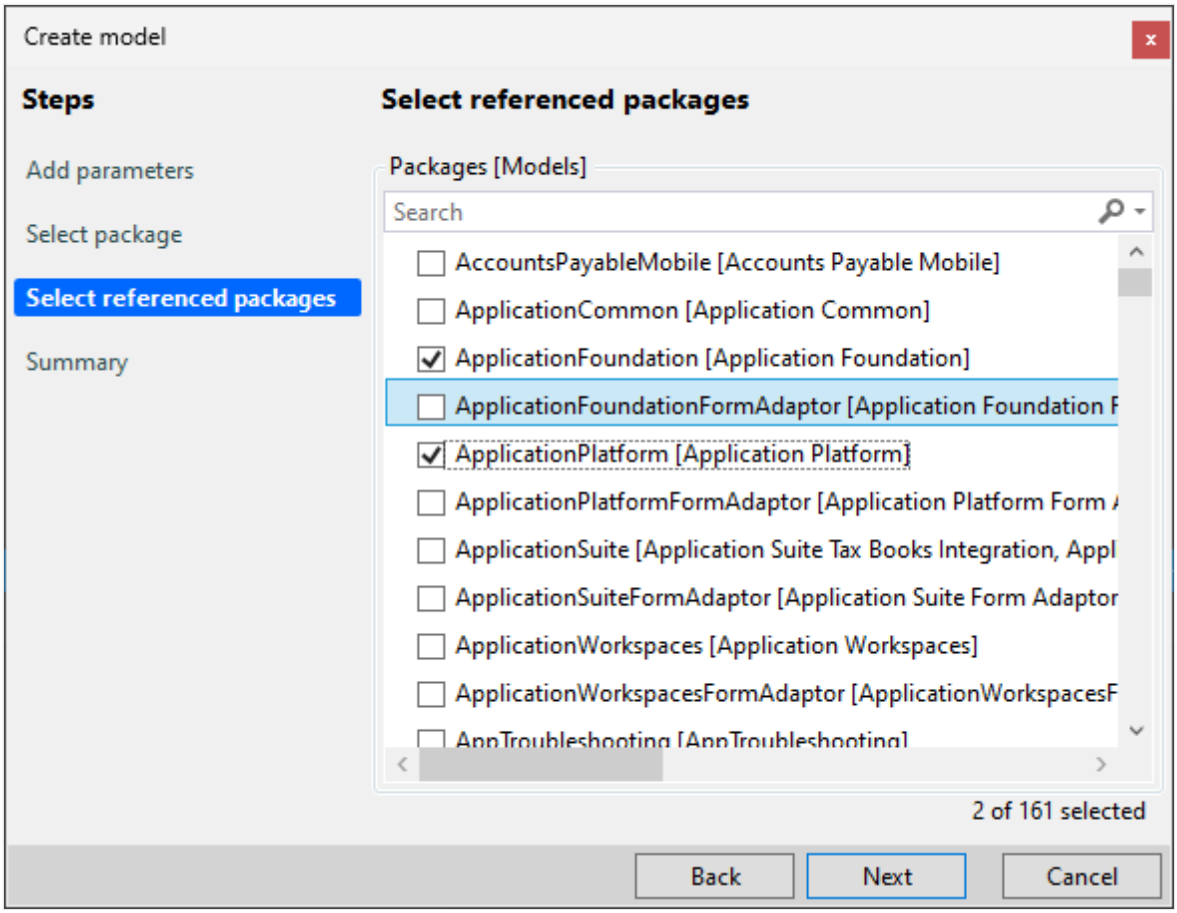
3. Enter a model name of "MyTestModel" when prompted by the form, providing relevant information for the rest of the required information.



4. On the next step of the form, choose **Create new package**.



5. Choose all referenced models. You only need to specify dependencies on the Application Platform and Application Foundation packages in this trivial example.



6. Select **Next** multiple times until the package has been created.

Add and compile the runnable X++ class

Create a project and X++ class in Visual Studio by following these steps.

1. Create a project in Visual Studio that contains the runnable class. The name of the class isn't important here. For this exercise, keep the default name that is FinanceOperations3.
2. Add a runnable class to the project in **Solution Explorer** by right-clicking the project and choosing **Add** > **New item**.
3. Select a **Runnable Class** in the **Code** menu under **Dynamics 365 items**, and call it "MyRunnableClass". Visual Studio opens a window with the template for a runnable class providing a static `main` method that is called as the runnable class runs.
4. Add the following X++ code to the `main` method. Since we're only providing a token implementation here, add a call to log a string message in the Infolog:

```
X++

internal final class MyRunnableClass
{
    public static void main(Args _args)
```

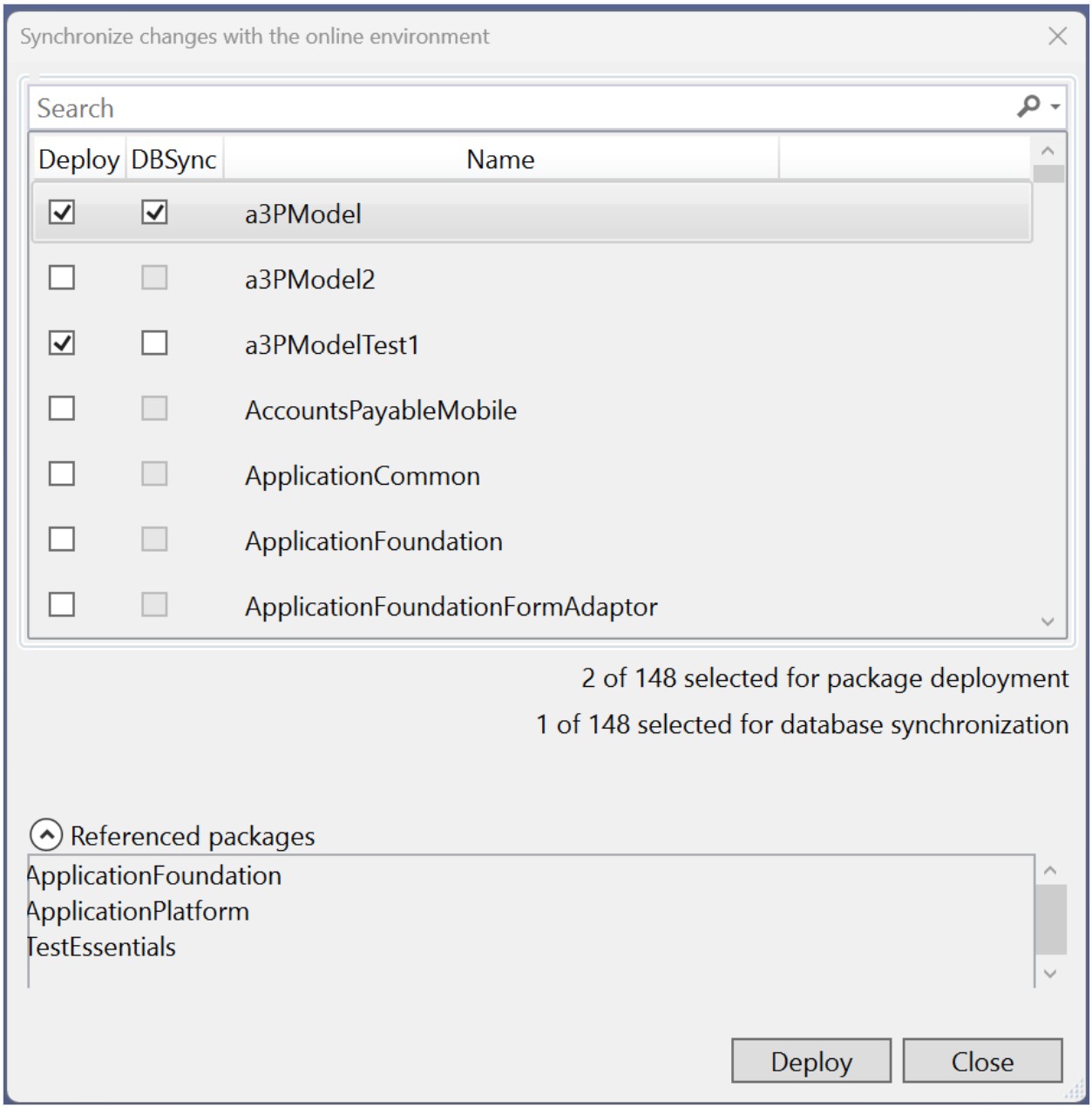
```
    {
        Info('Hello World from the Server Side');
    }
}
```

Compile the class to make sure you didn't introduce any errors to this code. The compilation is done by selecting **Build** from the context menu on the project.

Deploy the class

All the work that we have done up until now has happened on the client side. The X++ compiler and the related tools are all installed locally, along with the metadata that defines the application you're developing against. In Visual Studio, you can use the **Configure Metadata** menu item to manage the configuration that you're working with.

Now the time has come to execute the code, and for code execution you need to deploy the code to the server (cloud) tier. For this operation, use the **Extension** > **Dynamics 365** > **Deploy** > **Deploy Models to Online Environment** menu item. The dialog that appears allows you to select the packages to deploy. In this case, you need to select "MyTestModel". You can easily find that model by using the search field at the top of the list of packages. At the bottom of the dialog, see the packages that are referenced. Since these packages are part of the code that Microsoft ships, you don't need to deploy them as they're already installed on the server tier.



At this point you're accessing the endpoint in the cloud, so unless you are already logged in you likely need to authenticate and connect to the environment. Use your Microsoft Dataverse credentials to connect to Dataverse and select the solution to work against. After a short wait, the authentication will complete.

> **ⓘ Important**
>
> There are different ways to deploy your changes to the unified development environment.
>
> 1. Deploy the full model via the **Dynamics 365** > **Deploy** > **Deploy models ...** dialog. You can choose to synchronize the database if so desired.
> 2. As part of a full build, deploy the full model. This deployment is initiated via the **Dynamics 365** > **Build models** dialog if the option to **Deploy to**

connected online environment is turned on. The tool will synchronize database or not based on the **synchronize database** setting on the same **Options** tab.

3. As part of an incremental build from Solution Explorer, setting **Deploy changes to online environment** to true will only deploy changes since the last successful deployment or those made as part of this build. It will honor the project settings for **Synchronize database**.

4. Right-click a project and choose to **Deploy model for project ...**. This will only deploy changes since last successful deployment but will not synchronize the database for the module.

5. Synchronize the database for all models without deploying anything new via **Dynamics 365** > **Synchronize database...**.

You can follow the progress of the deployment by navigating to the "FinOps Cloud Runtime" channel in the Visual Studio output window. Once that deployment is complete, you can see log information by following the link displayed in the output window.

Debug the deployed code

After you have compiled the project code and uploaded it to the cloud endpoint, the binary is ready to be executed. To verify that all is well with the compiled class code, debug it.

1. In Visual Studio, set the debugging options to load the symbols for your package and also for ApplicationPlatform and ApplicationFoundation packages.

   To set the options, start by opening the options from the **Extensions** menu, and go to the debugging tab. The **Debugging** page shows options for loading symbols. Make sure you indicate that you want to load the items in your solution and also the ApplicationFoundation and ApplicationPlatform packages.

2. Set a breakpoint in the line where we call the `Info` method and then press the green arrow in the toolbar, or use the F5 keyboard shortcut, to start debugging.

   A browser window opens with a URL that designates the runnable class. After a little while the browser loads the form that shows that the class runner is active, and is executing the MyRunnableClass class. Next, the debugger loads the symbols for your project and execution stops at the breakpoint.

3. Press F5 to continue execution and then see that the Infolog pane in Visual Studio shows the message we entered into the Infolog using the call to the `Info` method.

4. Refresh the browser window, which reloads the URL.

   You'll see that the breakpoint loads immediately. The debugger is still running, and the output message appears a second time in the Infolog without restarting the debugger. This technique is especially useful when running forms. You can always refresh the browser, and the form is loaded again without any delay caused by loading the symbols.

Let's now try that. Instead of tediously writing our own form all over again, use a form that already exists. Start by using the SysUserSetup form that allows you to set the colorization (theme) used to render forms. We'll change the URL to open the SysUserSetup form instead of running the class by changing the menuitem (mi) - that part that designates the menu item to start the class runner. The URL will end up with something like this:

```
https://<environment>?prt=initial&debug=vs&activityid=\
<unchanged>&cmp=DAT&mi=SysUserSetup
```

You'll now see the form that allows you to set the colors in Dynamic's open.

Now let's set a breakpoint in the form.

1. Open the **Application Explorer** from the **View** menu.
2. Enter the string `SysUserSetup type:"form"` in the search bar in the Application Explorer window.
3. Select the form, and select **View code** to open the editor with the source code for this form.
4. In that form, set a breakpoint in the `selectionChange` method in the ThemeChooser control. This code is triggered when the user selects a new color theme.
5. In the browser window, select one of the colored squares, let's say the red one. You'll hit the breakpoint.
6. Continue the execution by pressing F5. Another colored square is hit and the breakpoint is triggered again.

> **ⓘ Important**
>
> If you want to attach the debugger to the running AOS process in the unified environment use **Dynamics 365** > **Launch debugger**.

To detach the debugger from the runtime without restarting the AOS, use **Detach in place of Stop**.

Use Application Insights to monitor and diagnose the application executing in the unified development environment. Learn more about observability: **Monitoring and telemetry using Application Insights**

Summary

As a recap, here's a summary and the points to take away from this exercise.

We started with using Visual Studio to create a simple runnable class. After the compilation, the package needed to be pushed to the cloud endpoint for execution. X++ code can only be executed in the cloud, so we performed a deployment operation. Once the deployment was complete, we executed the code by setting a breakpoint and running the debugger. We then started a form from the browser, and noticed that breakpoints were hit correctly as the form was used, and we did'nt have to restart the debugger for that to happen. In fact, the debugger can be started by using the **Launch debugger** option on the **Extension** menu. You can then enter a URL in the browser and the breakpoints are hit.

See also

Unit testing in the unified developer experience
Create and manage environments in the Power Platform admin center
Manage Dynamics 365 apps

Unit testing (preview)

Article • 09/14/2023

[This article is pre-release documentation and is subject to change.]

This how-to article shows you how to create and run reliable test cases.

Prerequisites

You need to have a Power Platform unified developer experience installed and configured on your local machine and a Power Platform developer-focused sandbox environment linked to it. It's important to keep the cloud and local representation of the code the same (synchronized).

More information: Install and configure development tools (preview)

Key concepts

- Use SysTest Framework to author unit/component test code
- Test isolation

Use SysTest Framework to author unit/component test code

You can create new test cases to test the functionality in an application.

1. Open Visual Studio.
2. On the **File** menu, select **Open** > **Project/Solution**, and then select **FleetManagement solution** from the desktop folder. If the solution file isn't on your computer, the steps to create it are listed in End-to-end scenario for the Fleet Management sample application.
3. In **Solution Explorer**, right-click the **Fleet Management** solution, select **Add**, and then select **New Project**.
4. Choose **finance and operations** as the project type to create.
5. Name this new project *FleetManagementUnitTestSample*, specify the FleetManagement folder on the desktop (C:\Users\Public\Desktop\FleetManagement) as the location, and then select **OK**.
6. In **Solution Explorer**, right-click the new project, and then select **Properties**.
7. Set the **Model** property to **FleetManagementUnitTests**, and then select **OK**.

8. Now add a test class containing the tests against the fleet management code. Right-click the FleetManagementUnitTestSample project, select **Add**, and then select **New Item**.
9. In the **Add New Item** window, select **Test Class** as the type of element to add. Name the new class FMUnitTestSample, and then select **Add**.

The template class contains information that should get you started with writing your test. In this example, the test class only consist of two tests, each marked with the SysTestMethod attribute. Since the methods aren't subject to extensibility, they have been adorned with a `Hookable(false)` attribute. In addition, the test class contains a method called `setup()` that is called prior to any tests being run. This method is often useful for setting up state shared across all tests.

X++

```
class FMUnitTestSample extends SysTestCase
{
    public void setup()
    {
        // Reset the test data to be sure things are clean
        FMDataHelper::main(null);
    }

    [SysTestMethod, Hookable(false)]
    public void testFMTotalsEngine()
    {
        FMRental rental;
        FMTotalsEngine fmTotals;
        FMRentalTotal fmRentalTotal;
        FMRentalCharge rentalCharge;
        FMRentalTotal expectedtotal;
        str rentalID = '000022';

        // Find a known rental
        rental = FMRental::find(rentalID);

        // Get the rental charges associated with the rental
        // Data is seeded randomly, so this will change for each run
        select sum(ExtendedAmount) from rentalCharge
                where rentalCharge.RentalId == rental.RentalId;

        fmTotals = FMTotalsEngine::construct();
        fmTotals.calculateRentalVehicleRate(rental);

        // Get the totals from the engine
        fmRentalTotal = fmTotals.totals(rental);

        // Set the expected amount
        expectedTotal = rental.VehicleRateTotal +
rentalCharge.ExtendedAmount;
```

```
        this.assertEquals(expectedTotal,fmRentalTotal);
    }

    [SysTestMethod, Hookable(false)]
    public void testFMCarValidateField()
    {
        FMCarClass fmCar;

        fmCar.NumberOfDoors = -1;
        this.assertFalse(fmCar.validateField(Fieldnum("FMCarClass",
"NumberOfDoors")));

        fmCar.NumberOfDoors = 4;
        this.assertTrue(fmCar.validateField(Fieldnum("FMCarClass",
"NumberOfDoors")));
    }
}
```

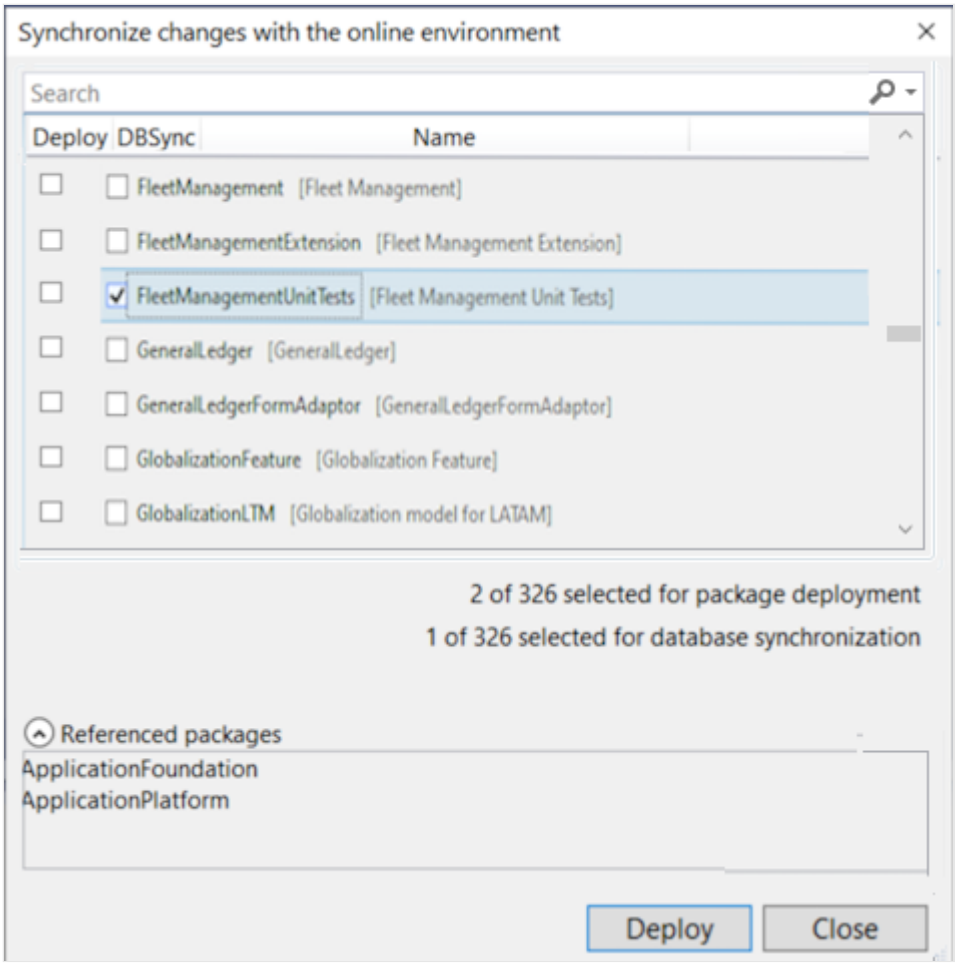
Continue by following these remaining instructions.

1. Save the test class.
2. Build the project by right-clicking on the FleetManagementUnitTestSample project in **Solution Explorer**, and then select **Build.**

After you have fixed any problems diagnosed by the X++ compiler, you're ready to run your unit tests using the Visual Studio test explorer. In the unified developer experience, the execution takes place in the cloud against the data that resides in the running SQL Server. So you need to deploy the results of your compilation to the cloud by following these steps.

1. Select the **Deploy model for project FleetManagementUnitTests** menu item from the context menu on the project in Solution Explorer. You're prompted to connect to Power Platform if you aren't already connected.

2. The deployment dialog is displayed. Since we haven't added any new tables that need to be synced to the database, we now perform the deployment step. Fill out the dialog and choose **Deploy**.



3. On the **Test** menu of Visual Studio, open **Test Explorer**.

4. Select **Run selected test** to execute in the cloud the two test methods of the test case. Note that we do not currently support the debug menu in Test Explorer.

Test Explorer shows the results of each test after it completes. Hopefully, all your tests succeed. If not, you can use Test Explorer to navigate to the offending test code.

Test isolation

For a test to be of high value it must be reliable. A test must pass or fail consistently independent of other factors such as other tests. One typical cause of unreliable tests is leaking state, such as data left behind in the database that influences downstream tests. Another cause of unreliable tests is relying on tests to be called in a particular order. To prevent this type of issue, you can use the `SysTestTransaction` attribute.

TestTransactionMode	**Description**
AutoRollback	Default. Provides the best isolation. All transactions are rolled back using SQL save points, and

TestTransactionMode	Description
	all database statements are routed to the main connection including user connections. No data is persisted.
LegacyRollback	All insert statements are tracked and deleted during clean-up. All insert statements are downgraded to row-by-row. One typical use case is when testing user connections or concurrency scenarios. This isolation level cleans up setup data, and the recommendation is to wrap each test method in a ttsBegin and ttsAbort.
LegacyRollbackWithUpdateTracking	All update, delete, and insert statements are tracked and reverted during cleanup. All insert, update, and delete statements are tracked and downgraded to row-by-row. This mode is the slowest isolation level.
None	Only use this mode for debugging. Provides no isolation. This setting can be useful to temporarily debug a test as it allows you to use the regular user interface to navigate the data that the test created.

Example:

```
X++
```

```
    [SysTestTransaction(TestTransactionMode::LegacyRollback)]
    class MyTestSample extends SysTestCase
```

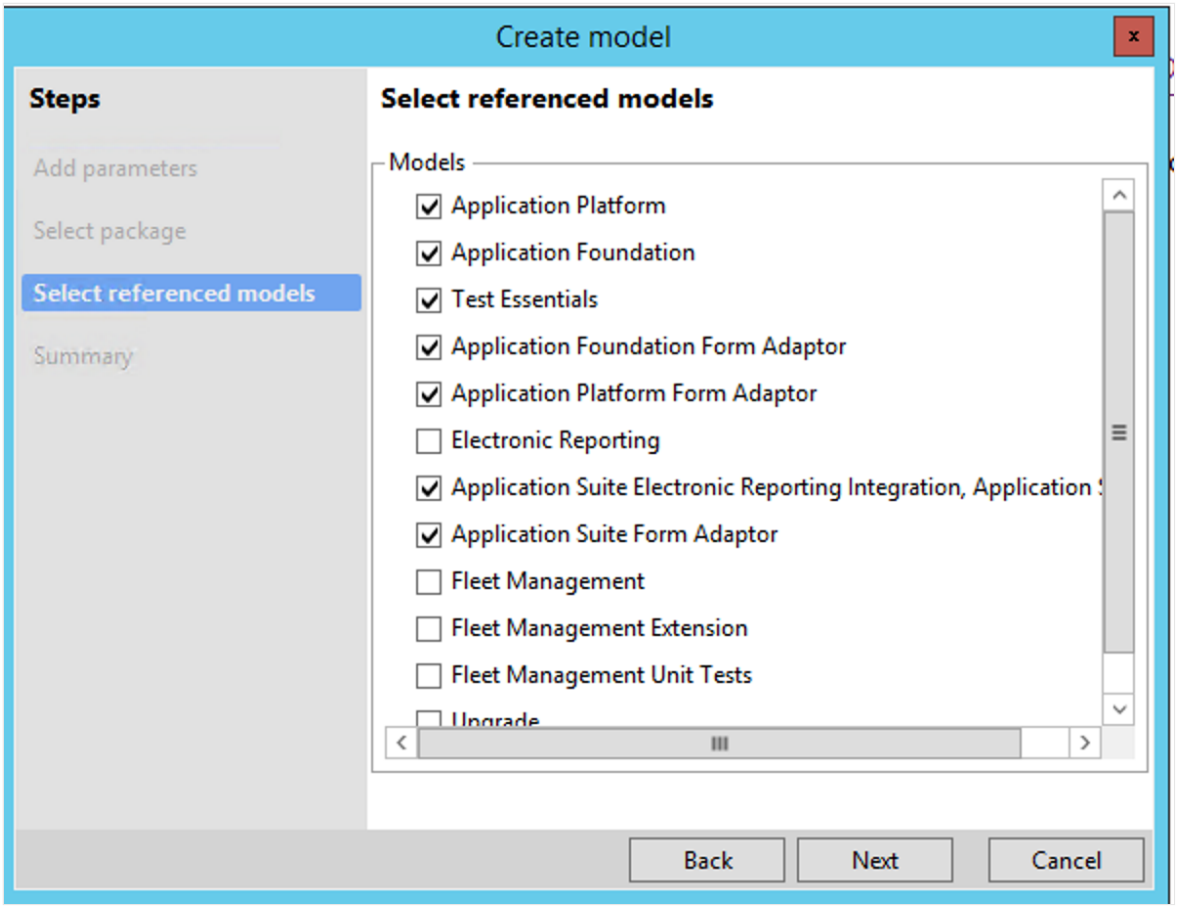
Test module creation to manage test code and FormAdaptors

Creating a test specific module helps to keep test code together and manageable.

1. Open **Visual Studio** and go to **Dynamics 365** > **Model Management** > **Create model**.

2. Enter the model name, select the layer, and then enter any more details. It's a good idea to include the word **Test** in the name of the test module. The default build definition is configured to discover all test modules that contain the word **Test**.

3. Because this model holds forms from the Application Platform/Foundation, add references to models selected (checked) in the following figure.



See also

Write, deploy, and debug X++ code (preview)

Continuous integration and deployment (preview)

Article • 09/14/2023

[This article is pre-release documentation and is subject to change.]

This article explains how to implement continuous integration and deployment (CI/CD) of Power Platform unified developer experience assets using Azure Pipelines. Specifically, how to:

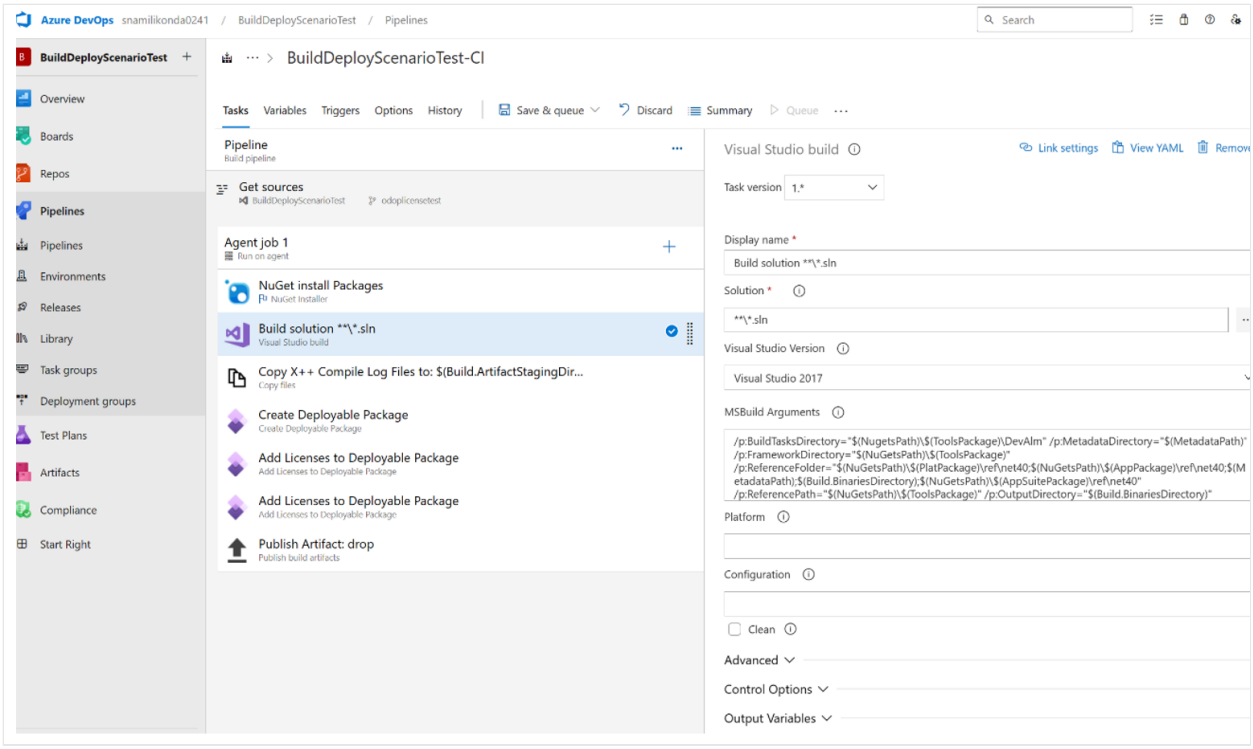
- Get the latest Dynamics 365 Finance and Operations Tools and Power Platform Build Tools
- Create a build pipeline to generate a package in Power Platform unified package format
- Upload your solution and relevant artifacts, and link the generated package to your deploy pipeline

Install or update the latest build tools

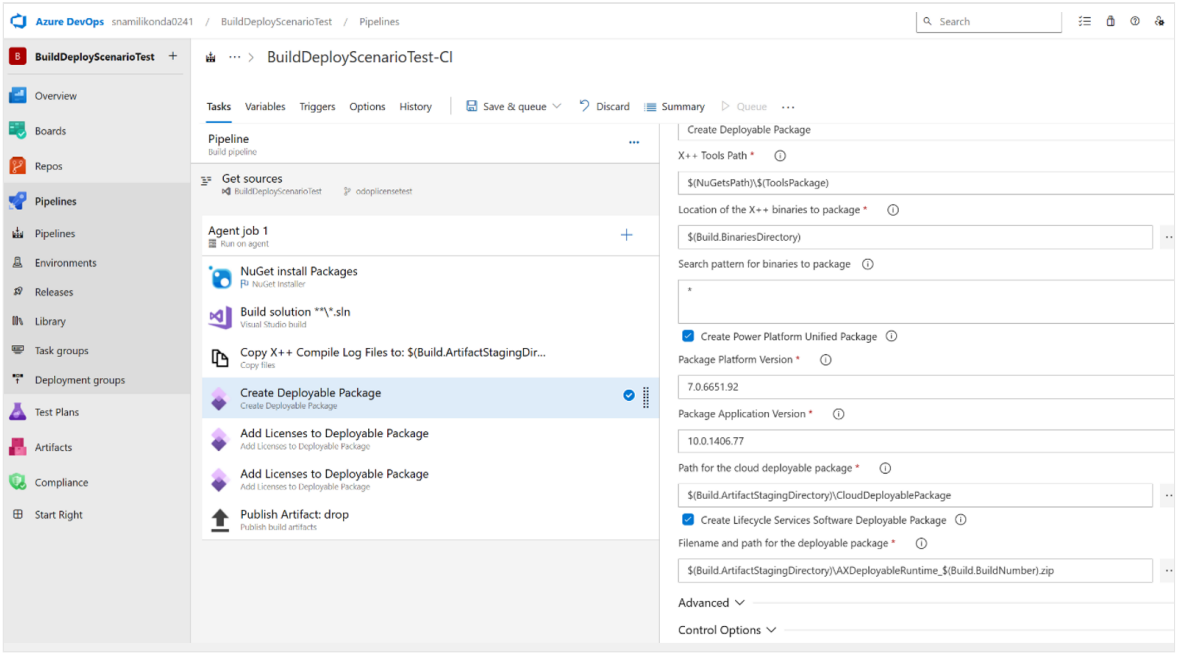
1. Sign into the Power Platform admin center and select the environment you want to work in. In the **Organization Settings** > **Extensions** menu, locate and install the "Dynamics 365 Finance and Operations Tools" shared extension.
2. Add a **Create Deployable Package** task in your build pipeline, and ensure it contains task version 1. Ensure **Add Licenses to Deployable Package** has task version 1. If you have tasks with same name, then select "View Yaml" for the task to ensure it's a preview task.
3. Locate and install the Power Platform Build Tools extension from **Organization Settings** > **Extensions** > **Browse Marketplace**.
4. Download the X++ NuGet packages with the required version and add them to your artifact feed. For details on this step refer to Build automation that uses Microsoft-hosted agents and Azure Pipelines - Finance & Operations.

Create a pipeline to generate a Power Platform unified format package

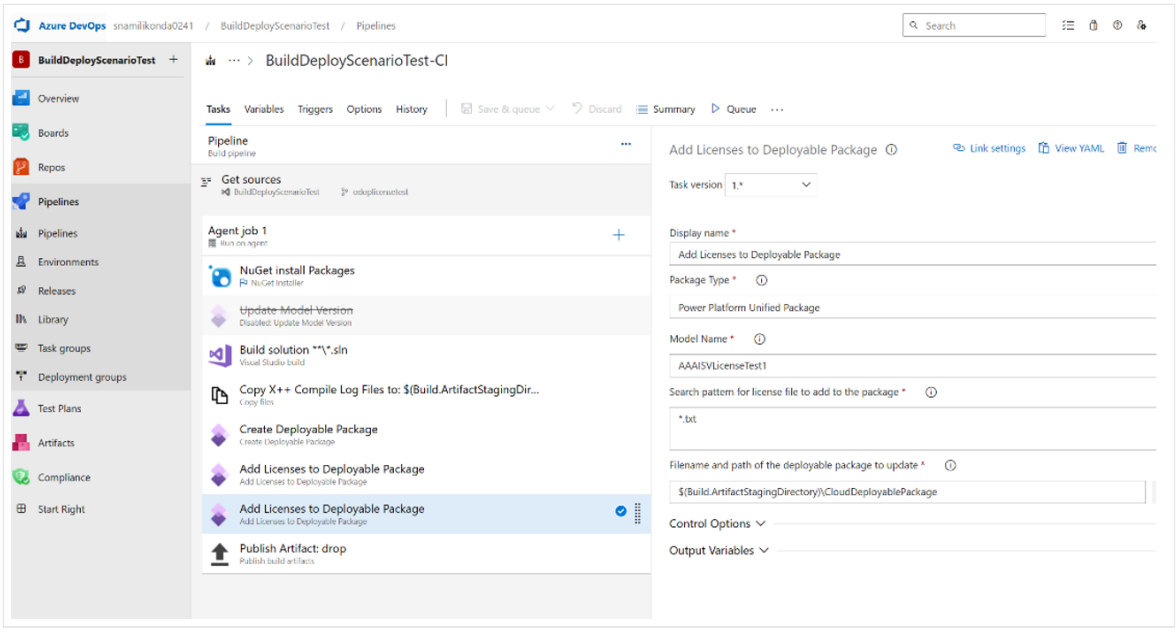
The process for building your solution remains same, and can be based on preexisting build pipelines. Follow the instructions that begin below this next image.



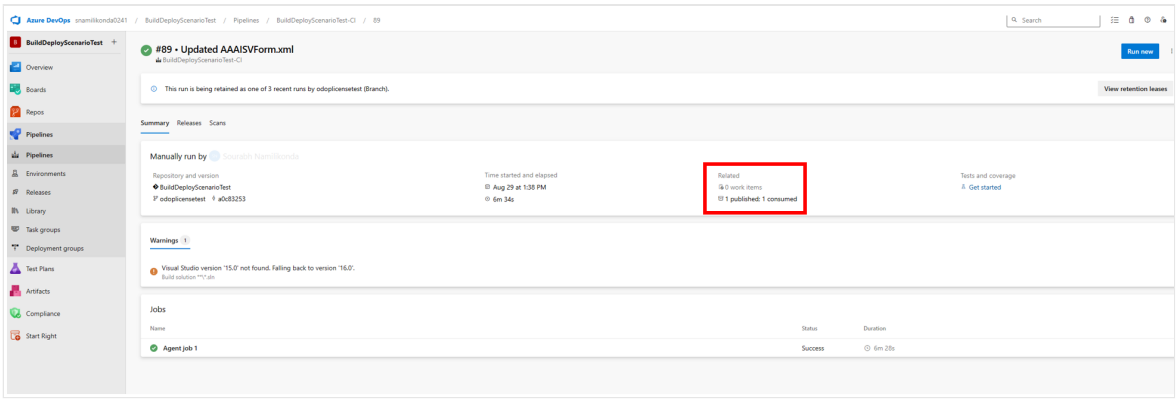
1. To generate a Power Platform unified package, add your version 1 task to **Create Deployable Package**. Check the box for **Create Power Platform Unified Package**. Add the package platform and application versions used to build the solution, and provide the path for the deployable package location. Optionally, you can choose to generate a separate Lifecycle Services format package.



2. If licensing is required, select the **Add Licenses to Deployable Packages** for your version 1 task and choose **Power Platform Unified Package** from the drop down menu. Provide the **Model Name** to include a license. Add a task for each model that the license is needed. Modify the path to the deployable package created in previous step.



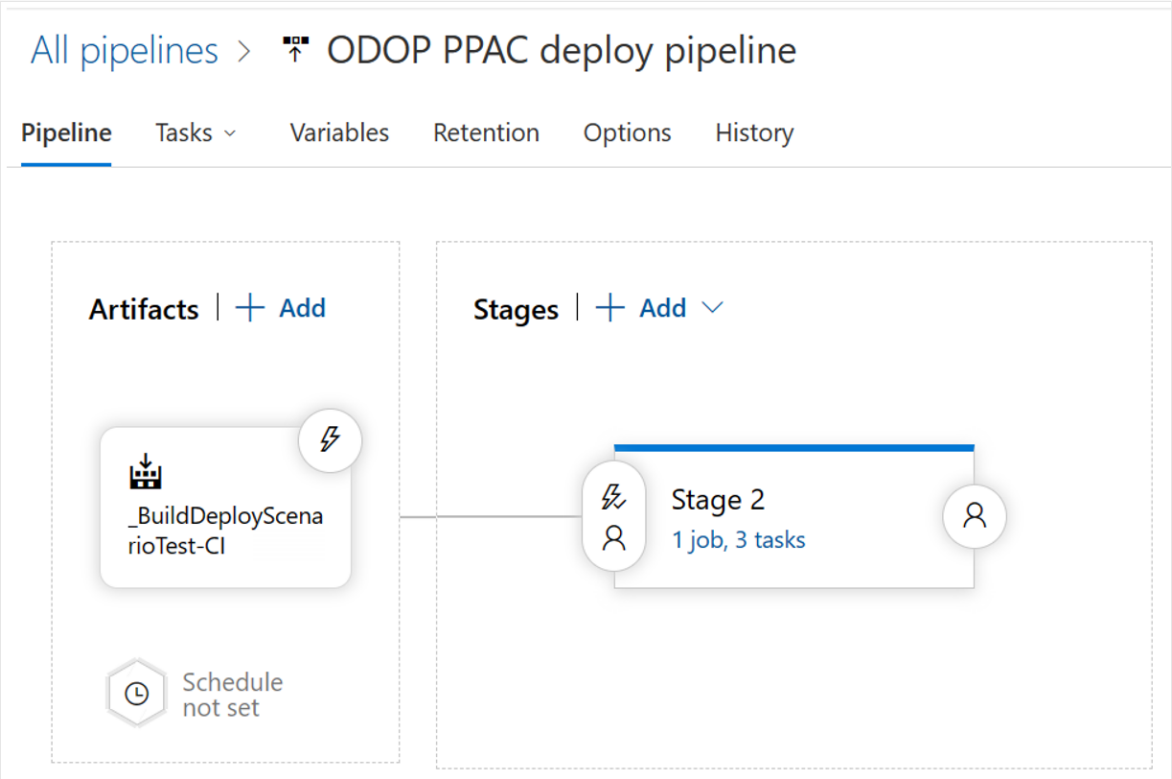
3. Once the pipeline is built, locate the published artifacts under **Related** in the **Summary** tab.



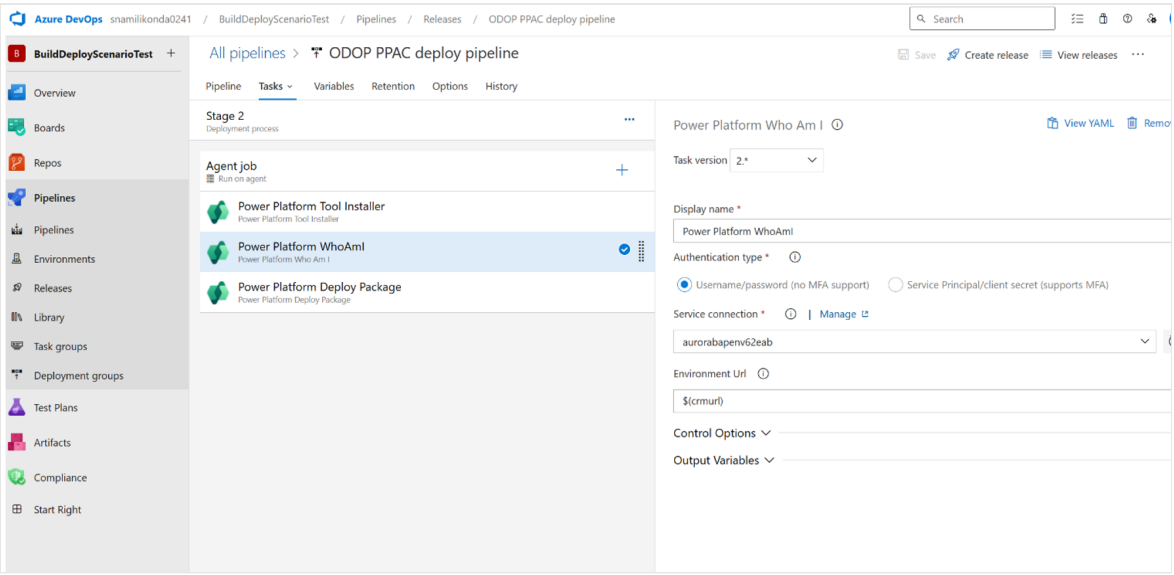
Link to your deploy pipeline

Follow these next instructions to link to your deployment pipeline.

1. Create a release pipeline, choosing the build pipeline from the preceding section.

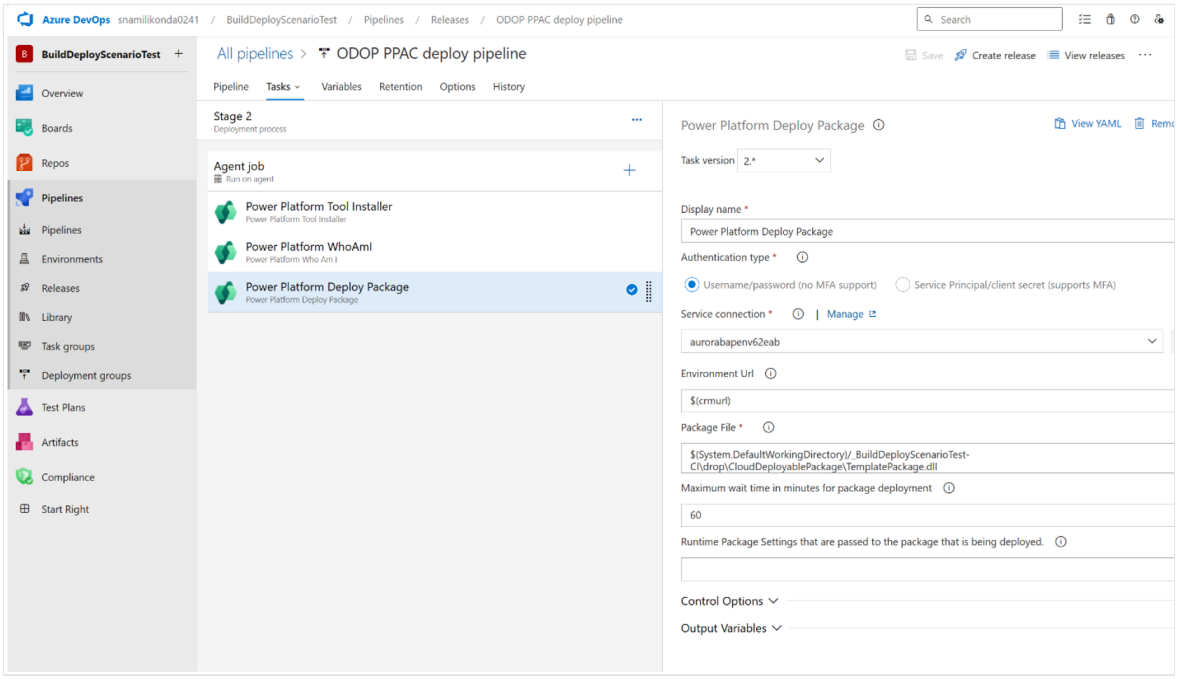


2. Add the following tasks to stage 2 as shown in the following image.
   a. Power Platform Tool Installer
   b. Power Platform WhoAmI
   c. Power Platform Deploy Package



   For the WhoAmI task, add a generic service connection for your environment. You can get the connection details from the **Your Organization** > **Details**.

3. To deploy the task, provide the path of the generated package from the build pipeline.


Azure DevOps snamilikonda0241 / BuildDeployScenarioTest / Pipelines / Releases / ODOP PPAC deploy pipeline
Search
BuildDeployScenarioTest
Overview
Boards
Repos
Pipelines
Pipelines
Environments
Releases
Library
Task groups
Deployment groups
Test Plans
Artifacts
Compliance
Start Right
All pipelines > ODOP PPAC deploy pipeline
Save
Create release
View releases
Pipeline
Tasks
Variables
Retention
Options
History
Stage 2
Deployment process
Agent job
Run on agent
Power Platform Tool Installer
Power Platform Tool Installer
Power Platform WhoAmI
Power Platform Who Am I
Power Platform Deploy Package
Power Platform Deploy Package
Power Platform Deploy Package
View YAML
Task version 2.*
Display name *
Power Platform Deploy Package
Authentication type *
Username/password (no MFA support)
Service Principal/client secret (supports MFA)
Service connection *
Manage
aurorabapenv62eab
Environment Url
$(crmurl)
Package File *
$(System.DefaultWorkingDirectory)/_BuildDeployScenarioTest-CI\drop\CloudDeployablePackage\TemplatePackage.dll
Maximum wait time in minutes for package deployment
60
Runtime Package Settings that are passed to the package that is being deployed.
Control Options
Output Variables

Diagnose issues and analyze performance using the Trace Parser

Article • 05/29/2024

[This article is pre-release documentation and is subject to change.]

This article explains how you can use the Trace Parser tool to consume traces and analyze performance in your unified developer experience deployment. The tool can visualize the execution of X++ methods and show execution call trees including the time spent in each method and the time spent for each database operation.

> **ⓘ Important**
>
> - This is a preview feature.
> - Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.

Finding the Trace Parser

The Trace Parser tool is a desktop application running on the development (client) machine. When you download the metadata, cross reference database, or more for a particular application version, an installation program for the Trace Parser (named TraceParser.msi) is also downloaded. Use this MSI file to install the Trace Parser tool. When the tool is started the first time, it asks for a name for the database that holds the traces. Provide a name that allows you to distinguish the trace database from other databases on your server (for example "Traces").

How does it work?

The Trace Parser captures events that occur in running application code, when instructed to do so, as described here. The events are stored in an Event Trace Log file (with a .ETL file extension) that can be downloaded to the client tier.

The Trace Parser needs to query the data captured in the ETL file in advanced ways, so the information in the ETL file must be extracted from the file and injected into a SQL database on the client side. This work is done from the tool, where the ETL file can be

loaded and the database selected. Any SQL database can be used for this purpose. We suggest using the same database as is used for the cross reference data.

Caveats

The data injected into the SQL database, that is subject to querying with the Trace Parser, contains queries, data access statements, and call stacks. The user can optionally include extended data into the logs to make diagnosing problems easier. Setting the **Include SQL parameter values** option when collecting the trace may leak information in the application to the trace. The recorded information should be kept as safe as the production environment itself. You're advised to delete any of these databases as soon as you're done using them, to not store ETL files in unsafe places, and to delete them after use. The traces can be deleted in the Trace Parser tool. When the tool starts, it shows an overview of the traces, and individual traces can be deleted from here.

Capturing a trace

When you wish to diagnose a problem in the running application, you start the tracing, perform the application steps that demonstrate the problem, and then stop the trace. The trace is then analyzed in the Trace Parser. You can provide a meaningful name to the trace when you choose to start the trace.

In this section, we explain how to capture a trace from the application's web page.

1. On the browser navigation bar, select the question mark **?**, and then select **Trace**.
2. In the **Trace name** field, enter a name for the trace that you're about to capture. It's good practice to choose a name that reflects the problem you want to investigate.
3. As required, set the **Include SQL parameter value** option to **Yes**. Refer to the caveats in the previous section.
4. Select the **Start trace** button.
5. Perform actions that must be analyzed, such as navigation to **Accounts payable** > **Vendors** > **All vendors**.
6. When done, select **Stop trace**.
7. After the trace stops, select one of the following options. For this tutorial, select the first option.

- **Download trace** – Store the captured trace on the local machine. You can then analyze a downloaded trace by using the desktop version of Trace Parser.

  > ⓘ Note
  >
  > If you download a trace, it won't be available for later upload.

- **Upload trace** – Store the trace in the cloud for later download (for example by the administrator). Uploaded traces are automatically deleted after seven (7) days. They can also be manually deleted from the **Captured traces** page.

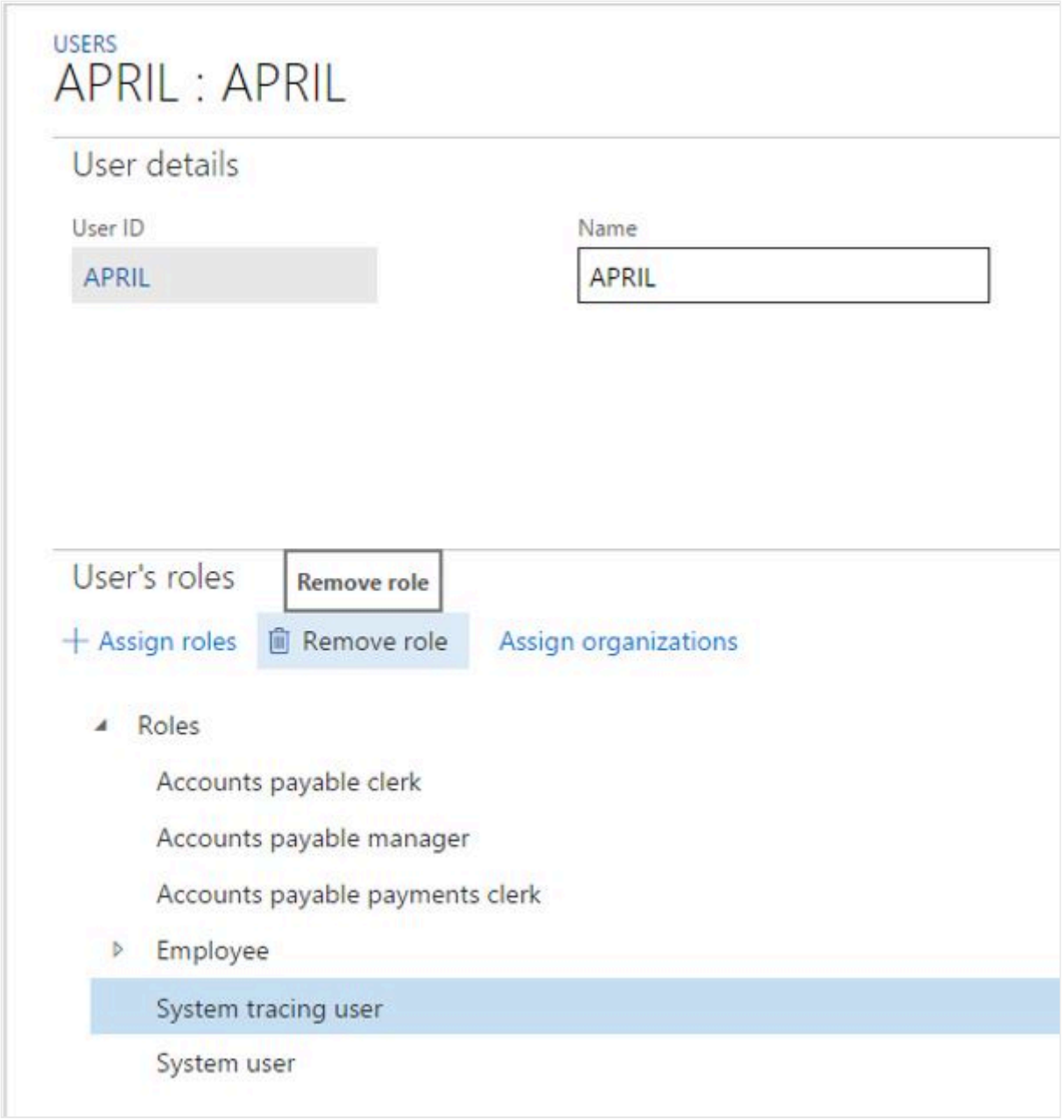
- **Return to main menu** – Return to the main tracing menu to start another trace.

> ⓘ Note
>
> If your scenario takes more than a few minutes, you should consider splitting it into multiple traces. If a trace is too large, it can't easily be analyzed.

Assign trace rights to a user

Each system user account that is to capture and view traces must be assigned the "System tracing user" role.

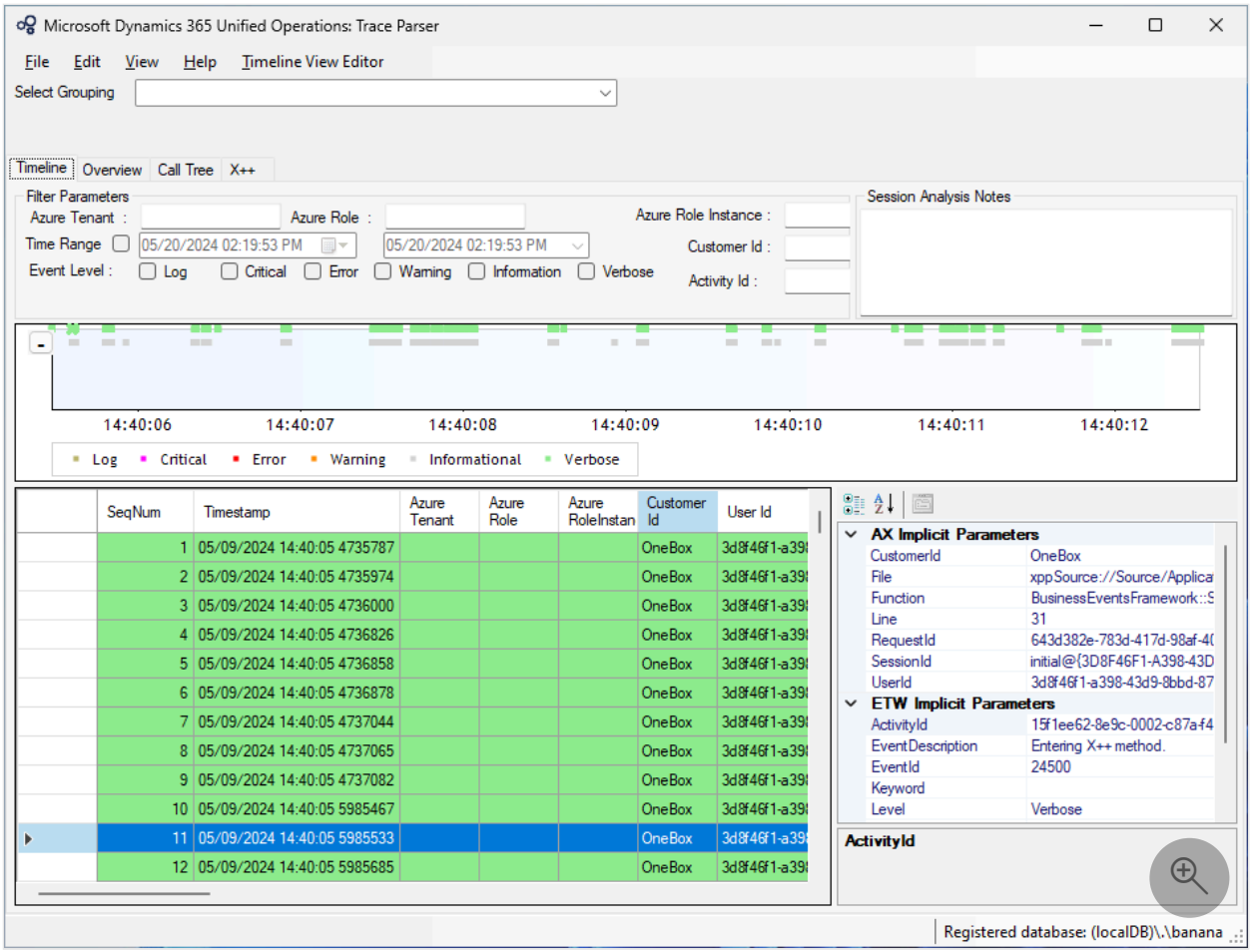
1. To give a user rights to capture a trace, go to **System administration** > **Users** > **Users**.
2. Select the user and assign that person the **System tracing user** role.



> ⓘ **Note**
>
> To help prevent unwanted tracing, remove the user role after the user has finished tracing.

Viewing traces

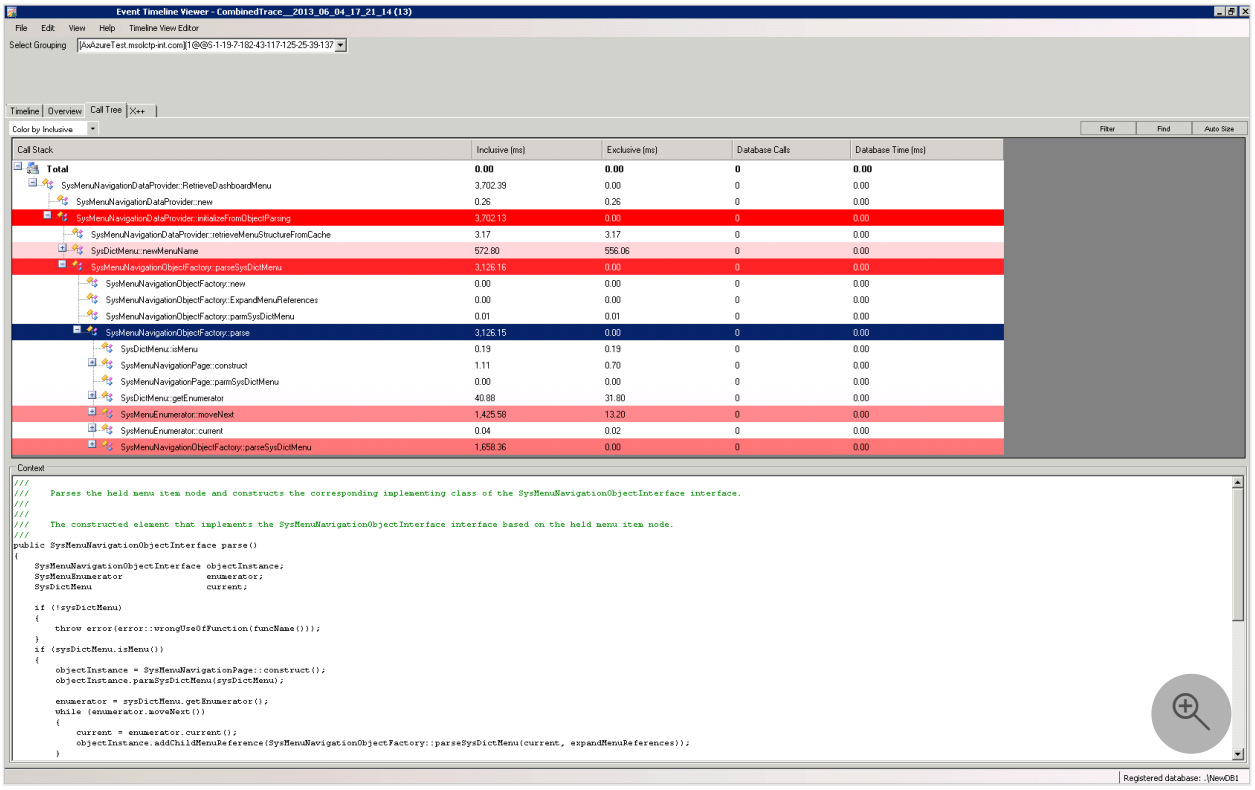
The **Timeline view** tab is the first tab that you see after you import a trace into the Trace Parser. This tab is shown in the following illustration.



The **Timeline** tab has the following major components:

- The **Select Grouping** drop-down allows you to group based on various categories, such as Customer ID, Username, Session Name, and more. Groupings display maximum and minimum timestamp of events, total number of events, and lowest event level within the grouping.
- List of all events in a threaded or unthreaded view.
- Property grid displayed for the selected event.
- Timeline chart for all the selected events.
- Filtering of events.
- Session analysis notes.

The **Call tree view** is accessed by selecting the **Call Tree** tab. You can see the call tree for all X++ methods.



Similarly, you can display the **X++** tab to view a list of all the X++ methods. They are sorted by fields such as inclusive/exclusive durations, RPC, or database calls. These methods are similar to the corresponding tabs in Trace Parser and have the same behavior.

See also

X++ language reference

Feedback

Was this page helpful? Yes No

Provide product feedback

Execute unit tests in Azure Pipelines (preview)

Article • 05/11/2024

[This article is pre-release documentation and is subject to change.]

An Azure Pipeline build task is available that lets users execute unit tests on connected cloud runtime environments using a continuous integration/continuous delivery (CI/CD) pipeline. The "Execute Unit Tests in Unified Environment" task is included in the Dynamics 365 Finance and Operations Tools extension.

More information: Continuous integration and deployment (preview), Build automation that uses Microsoft-hosted agents and Azure Pipelines

> **Important**
>
> - This is a preview feature.
> - Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.

Prerequisites

To be successful in using this unit test capability, you must have:

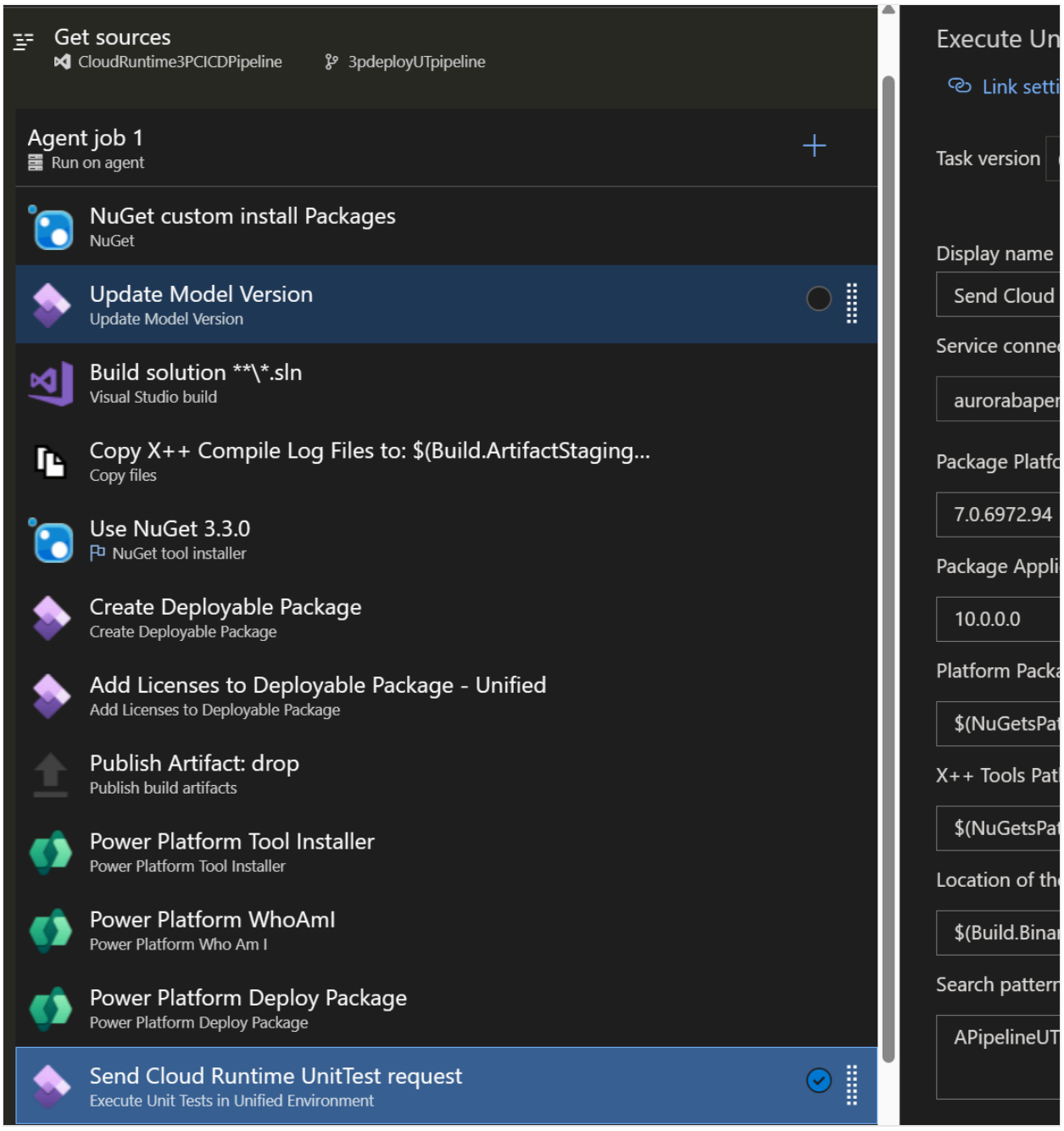
- Working knowledge of building and deploying X++ code using Azure Pipelines. For more information, see the linked articles provided in the See also section.
- Understanding of X++ unit test execution and general development.
- Availability of this unit test execution capability in your organization.

Add a unit test task to a pipeline

Adding the ability to execute unit tests requires a build pipeline that can build and deploy a solution to the unified developer experience environment. The tests are obtained from the test models included in the repository, based on the assembly output of the Build Solution pipeline task. The same tests must be pre-deployed on the unified developer experience environment for the test execution to be possible. This deployment can be accomplished in the same run of the pipeline or separately.

The deployment task is only required if there are any changes of the test cases that aren't deployed on the environment. However, the build task is necessary to identify which tests must be run. This unit test task must be included after the optional build and deploy tasks.

In this next figure, the "Execute Unit Tests in Unified Environment" task is included in en example build pipeline.



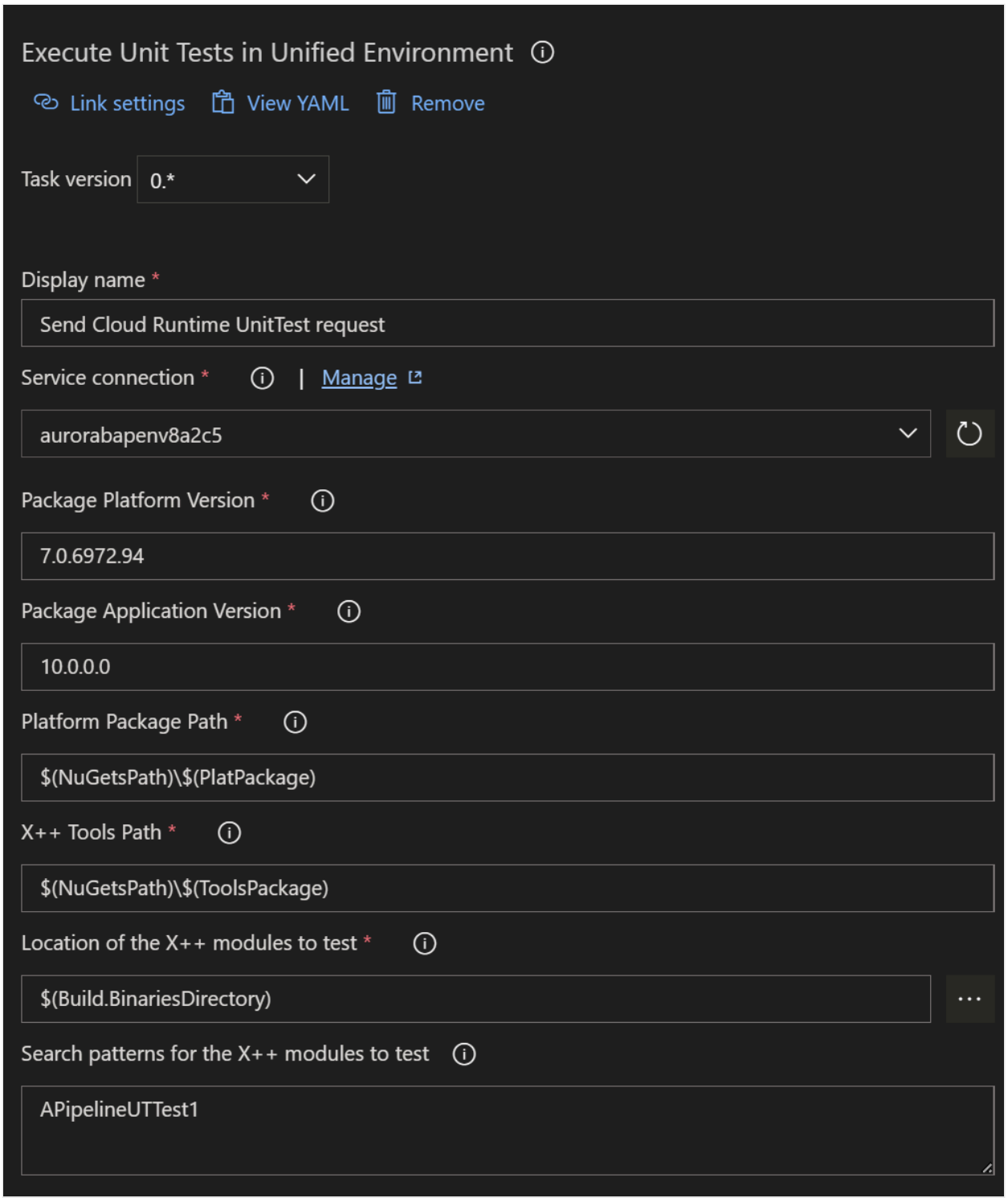
The output logs of the task contains information of each test executed and its output. The task fails if any of the tests that are run fail.

Creating the unit test task

> **Important**

- A new version (1.*) of the task has been released. See Update: Version 1 for more information.

In this next image is an example unit test being defined. A description of each form field follows.



Expand table

Form field	Description
Display name	The name of the test execution task.

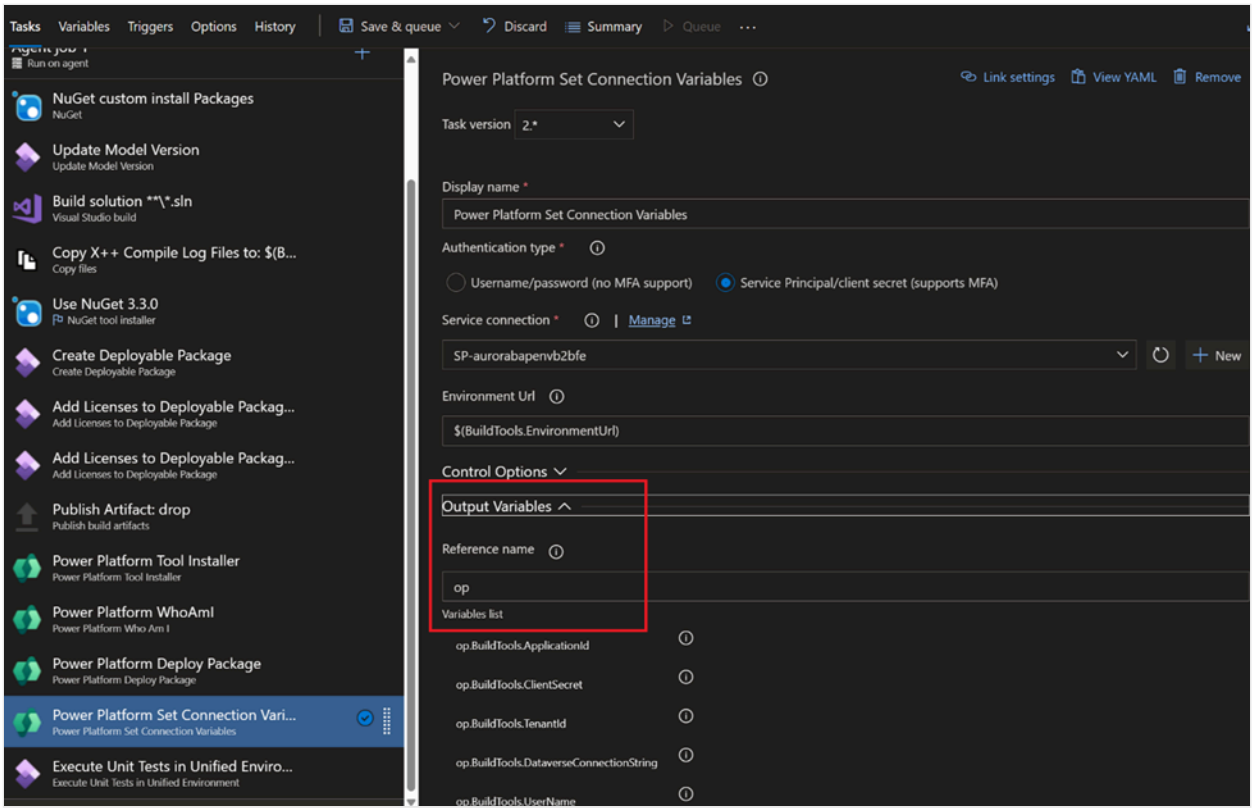
Form field	Description
Service connection	Service connection to the unified developer experience environment on which the deployment has been made.
Package Platform Version Package Application Version	Versions of the deployed package and connected environment.
Platform Package Path	The location of the Microsoft.Dynamics.AX.Platform.DevALM.BuildXpp NuGet package installed on the agent.
X++ Tools Path	The location of the Microsoft.Dynamics.AX.Platform.CompilerPackage NuGet package installed on the agent.
Location of the X++ modules to test	The storage location where the build task produced the assemblies of the models included. This value is often set to $(Build.BinariesDirectory).
Search patterns for the X++ modules to test	Include the pattern for the modules that the tests must be run for. Provide a name matching pattern for X++ package (module) names inside the path that is specified in the location of the X++ modules to test option. You can also specify a list of names instead of search patterns, or exclusion filters. More information: File matching patterns reference.

Update: Version 1

A new version (1.*) of the **Execute Unit Tests in Unified Environments** task is available. This version replaces the **Service Connection** authentication field. Now you can use multiple authentication mechanisms, like username/password and appId/client Secret to execute the test requests.

You can use the existing **Power Platform Set Connection Variables** task to generate the connection string in the same pipeline before using the unit test execution task. Learn more about the details for this task and about connection types. You can retrieve the connection string and use it in subsequent tasks by creating a reference name in the **Output Variables** section for classic pipelines.

The following image shows how to use the task to generate an output variable named `op` that is referenced in the subsequent unit test execution task:



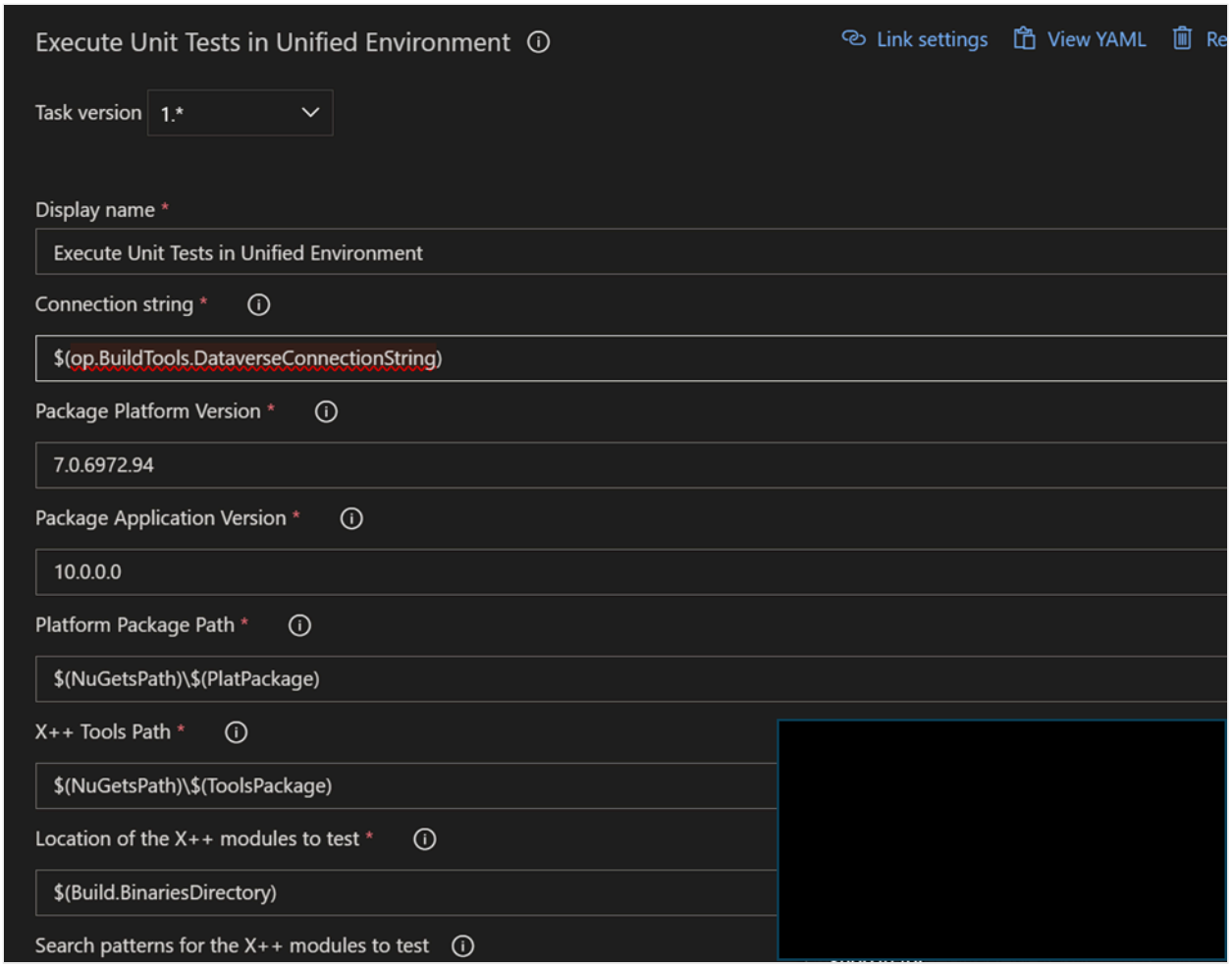
You can also use this in a YAML pipeline:

Sample **Power Platform Set Connection Variables** task – Use either of the connection types and replace the URL and the service connection name, as shown in Power Platform Set Connection Variables. Use the `name` field to specify the output variable.

yml

```yml
task: microsoft-IsvExpTools.PowerPlatform-BuildTools.set-connection-variables.PowerPlatformSetConnectionVariables@2
  displayName: 'Power Platform Set Connection Variables '
  inputs:
    authenticationType: PowerPlatformSPN
    PowerPlatformSPN: <Service Connection>
    Environment: '<Power platform environment URL>'
  name: 'op'
```

The following screenshot shows the same reference name used in a unit test task:



See also

Unit testing (preview)
Continuous integration and deployment (preview)
Build automation that uses Microsoft-hosted agents and Azure Pipelines
Unit Test Framework

Feedback

Was this page helpful? Yes No

Provide product feedback

Frequently asked questions (preview)

Article • 05/22/2024

[This article is pre-release documentation and is subject to change.]

This FAQ answers common questions regarding the unified developer experience for finance and operations apps.

Licensing, capacity, and environment provisioning

My environment was showing as preparing in Power Platform admin center, but has now disappeared. Why has my environment deployment failed?

This missing environment situation happens when provisioning fails. Contact Microsoft Support with the environment name and tenant ID.

I can't provision an environment with PowerShell 7

Use Windows PowerShell instead of PowerShell 7

My provisioning fails with an error about missing licenses

When you get the following error, ensure that you have licenses for the template you're deploying.

Error message:

"You don't have the required Dynamics 365 licenses to create a database with `<app name>` Dynamics app."

How can I provision or update my unified environment to a specific finance and operations version

We currently don't have the capability to provision or update the environment to a specific runtime update or version. Support for this is planned, but until this support is enabled, we suggest using the environment copy feature to get your unified

development environment on the same version as that of sandbox or production environments. Copy a Lifecycle Services environment to a unified environments

Developer setup (metadata download, configuration, connecting to an environment)

I can't access SQL Server using my SQL Server Management Studio client running in a OneBox environment

We don't currently have a solution for secure connection to SQL Server running behind a cloud endpoint. A TDS endpoint (like the one Dataverse has) is planned to provide read-only queries to a database from SQL Server Management Studio. There's also a JIT access that will allow you to access the data after providing authentication.

I connected to the environment from Visual Studio but didn't get any pop-up to download metadata

Ensure that the unified development environment you created is of type **Sandbox**. A **Trial** environment won't work.

The Configure Metadata option doesn't appear on the menu

When this option is missing from the menu, open the Infolog from the menu and retry.

How do I download again the client assets

When you connect to the unified development environment, the tool does a few checks and doesn't initiate download if the required assets were previously downloaded. In case you need to download again, simply rename/delete the child folder with the application version of the environment under %LOCALAPPDATA%\Microsoft\Dynamics365.

PackagesLocalDirectory extraction doesn't complete

Use 7-Zip to extract the metadata.

The message "Please wait for update to complete" pop up keeps appearing

When a configuration is created, saved, and the dialog closed, the necessary files are updated and this popup will stop. If the dialog keeps recurring even after you open a new Visual Studio instance, the find the file PackagesLocalDirectory/bin/InstalledVersion.json in reference metadata folder and update the version to match the platform version in your environment.

Cross reference database validation keeps failing

Ensure that your Microsoft Windows user has access to the specified SQL Server or LocalDB. The following instructions describe a local DB setup.

1. Download the LocalDB 2019 installer by using the SQL Server Express installer.
2. Run the installer and select **Download Media**
3. Select "LocalDB" and **Download**
4. Before running the SqlLocalDB.msi installer, delete your current MSSQLLocalDB instance:

PowerShell

```
sqllocaldb stop MSSQLLocalDB
sqllocaldb delete MSSQLLocalDB
```

5. Run the new SqlLocalDB.msi (2019) installer. It creates a new MSSQLLocalDB instance.

I get a missing licenses error when running unified developer experience operations

When you get the following error, contact Microsoft Support with environment details to fix the issue.

Error message:

"Principal user (Id=`<Redacted>`, type=8, rollCount=2, privilegeCount=<>, accessMode=1, MetadataCachePrivilegesCount=7371, businessUnitId=`<Redacted>` (Setup/Stub unlicensed user with filtered privileges from associated roles. Consider assigning license.)), is missing prvCreatemsprov_fnopackage privilege"

Unified developer experience operations (apply package, debugging, copy, pipeline setup, package conversion)

My Operations fails with error EnvironmentNotInReadyState

This result happens when the the unified developer environment isn't ready to service the request. Retry after a few minutes. You can check if the environment is in the ready state by opening the finance and operations environment link.

What is the earliest version of source environment I can copy from?

Unified developer experience functionality is available for version 10.0.35 and later. Since during copy we also copy compute, if the source environment is earlier than 10.0.35 the target loses unified developer experience functionality.

I provisioned an environment from UI, but am unable to run unified developer experience operations

The UI doesn't provision online environments. Only sandbox and production environments are provisioned via the UI, and unified developer experience operations aren't supported on these environments. Provision OnlineDev environments via the command line, and then run unified developer experience operations.

Is package deployment of the unit test code required for executing tests in the cloud runtime?

Yes. Any new tests or modifications must be deployed to the environment for the test execution to succeed.

What dependencies are needed for running unit tests on a new model?

Application Foundation and Test Essentials are required for unit test execution.

How can I set up a pipeline that automates creation and application of a deployable package to a new environment?

Use the latest version of the Azure DevOps extension from the marketplace. It's now capable of package creation, ISV license addition, and application of the package. Refer to Dynamics 365 Finance and Operations Tools.

How do I include a license and directly deploy from Visual Studio?

Licenses can be directly included and deployed by placing them in the __License (with *two* underscores) folder in the model, at the same level as your bin folder. This license is verified and applied when you deploy the package.

Additionally, a **Full DB Sync** from the Dynamics 365 menu in **Extensions** is required for the applied license to take effect.

How can I convert a fully deployable package (Lifecycle Services legacy package) into the new format to be compatible for deployment to environments?

Locate ModelUtil.exe inside the bin folder and run it from the command line to see usage. Choose the `-convertToUnifiedPackage` option and provide the package zip and output location as parameters.

Stopping debugging restarts the runtime

Use **Detach All** to end debugging in place of **Stop Debugging**.

Checking logs and history

I applied a package, but it failed. How can I do further debugging to determine the failure?

A link to download operation logs is available in the Visual Studio output pane. It's downloaded if there were failures, or if you enable the setting in **Tools**> **Options**> **Power Platform Tools**.

You can also download the logs from your Dataverse organization:

- Login to the Dataverse organization
- Find **Finance and Operation Package Manager App** on the main page
- Select the app and then from left pane, select **Operation History**
- Open the respective record by selecting the **Operation Name** and download the operation logs (`operationlogs.zip` file)

How can I find out what packages are applied to my environment?

You can check the package applied to your Finance and Operations environment with the following steps:

- Login to the Dataverse organization
- Find **Finance and Operation Package Manager App** on the main page
- Select the app and then select **Package** in the left pane to see packages applied to your Finance and Operations environment
- Select **Operation History** to see the history of operations performed on your Finance and Operations environment

My operation failed, and wasn't able to find sufficient information in the logs. What information do I provide to Microsoft Support?

The correlation ID, client machine name, and timestamp are required for Microsoft to investigate. The correlation ID is in the Visual Studio output pane. Additionally, a log file named VisualStudioD365Extension*.log is generated at a path similar to C:\Users\AppData\Roaming\Microsoft Corporation\Microsoft® Visual Studio®\16.11.32802.440\VisualStudioD365Extension-2022-08-12.log.

How do I monitor and diagnose deployed code running in the unified environment?

Use Application Insights to monitor and diagnose the application executing in the unified developer environment.

Learn more about observability: Monitoring and telemetry using Application Insights

See also

Feedback

Was this page helpful? Yes No

Provide product feedback

Power Platform developer tools

Article • 05/14/2024

Microsoft Power Platform provides various tools for developers to extend platform features and capabilities using code.

Expand table

Tool	Release notes	Description
Power Platform CLI	link	Microsoft Power Platform CLI is a simple, one-stop developer CLI that empowers developers and ISVs to perform various operations in Microsoft Power Platform related to environment lifecycle, authentication, and work with Microsoft Dataverse environments, solution packages, portals, code components, and more.
Power Platform Tools for Visual Studio Code	link	The Power Platform extension for Visual Studio Code makes it easy to manage Power Platform environments and allows the developer to create, build and deploy Power Platform solutions, packages and portals. Installing this extension will also make the latest Power Platform CLI (aka pac) available in your VSCode terminal.
Power Platform Tools for Visual Studio	link	Power Platform Tools for Visual Studio supports the rapid creation, debugging, and deployment of plug-ins. Other capabilities include development of custom workflow activities, web resources, integration technologies like Azure Service endpoints and webhooks, and more.
Visual Studio connected service		Add Power Platform as a connected service in Visual Studio to create a custom connector.
ALM developer tools		ALM developer tools for Power Platform enables developers to create and manage projects related to testing, deployment, maintenance, and governance of solutions.
Dataverse developer tools		Dataverse developer tools enable developers to extend data platform capabilities in Power Platform.
Power Apps Test Engine (preview)	link	Power Apps Test Engine is an open source project that provides developers with an automated testing platform for canvas apps.

Feedback

Was this page helpful? Yes No

Provide product feedback

What is Microsoft Power Platform CLI?

Article • 05/15/2024

Microsoft Power Platform CLI is a simple, one-stop developer CLI that empowers developers and ISVs to perform various operations in Microsoft Power Platform related to:

- Environment lifecycle
- Authentication
- Microsoft Dataverse environments
- Solution packages
- Power Pages
- Code components
- and more...

Provide feedback

Use github.com/microsoft/powerplatform-build-tools/discussions to view and submit feedback for PAC CLI.

Install Microsoft Power Platform CLI

There are three ways to install the Power Platform CLI. You can use multiple installation methods on the same computer.

Expand table

Method	OS Support	Description
Install the Visual Studio Code extension	Windows, Linux, macOS	Enables use of commands within a PowerShell terminal within Visual Studio Code on Windows 10, Windows 11, Linux, and macOS. When you install using ONLY this method, by default PAC CLI will only be available within a Visual Studio Code terminal unless you enable PAC CLI in Command Prompt (CMD) and PowerShell terminals for Windows
Install with .NET Tool	Windows, Linux, macOS	Enables use of commands within a PowerShell, CMD, or Bash shell on Windows 10, Windows 11, Linux, and macOS. This installation method doesn't enable use of pac data or

Method	OS Support	Description
		certain pac package commands (deploy and show) that are only available for Windows.
Install with Windows MSI	Windows only	Enables use of commands within a PowerShell terminal within Visual Studio Code on Windows only. You can manage installed versions with this installation method.

> **Note**
>
> The following commands are only available on Windows:
>
> - pac data
> - pac package deploy
> - pac package show
>
> To use these commands on Windows, install Power Platform CLI using either (or both) of these installation methods:
>
> - Visual Studio Code extension
> - Windows MSI
>
> These commands aren't available on Windows if you only install using the .NET Tool.

Check whether Power Platform CLI is already installed

How to determine whether the Power Platform CLI is installed depends on your operating system.

Windows

1. Open a PowerShell terminal:
   - If you ONLY installed the Visual Studio Code extension, open a Visual Studio Code Powershell terminal window.
   - If you installed using the .NET Tool, open a Visual Studio Code Powershell terminal OR a Powershell terminal.
2. Type `Get-Command pac | Format-List` at the prompt, and press **Enter**.

The results should look something like this:

```
Name            : pac.exe
CommandType     : Application
Definition      : C:\Users\you\.dotnet\tools\pac.exe
Extension       : .exe
Path            : C:\Users\you\.dotnet\tools\pac.exe
FileVersionInfo : File:             C:\Users\you\.dotnet\tools\pac.exe
                  InternalName:     pac.dll
                  OriginalFilename: pac.dll
                  FileVersion:      1.29.11
                  FileDescription:  Microsoft Power Platform CLI
                  Product:          Microsoft Power Platform&copy;
                  ProductVersion:   1.29.11+g9e2b163
                  Debug:            False
                  Patched:          False
                  PreRelease:       False
                  PrivateBuild:     False
                  SpecialBuild:     False
                  Language:         Language Neutral
```

When Power Platform CLI isn't installed, you'll get this error:

PowerShell

```
Get-Command: The term 'pac' is not recognized as a name of a cmdlet,
function, script file, or executable program.
Check the spelling of the name, or if a path was included, verify that
the path is correct and try again.
```

Check which version of Power Platform CLI is installed

Open the command prompt (on Microsoft Windows), or a Terminal session (in Linux), type `pac` and press **Enter**.

You should see something like this where the version is on the second line.

```
Microsoft PowerPlatform CLI
Version: 1.30.3+g0f0e0b9
```

Manage auth profiles

Most PAC CLI commands require authenticated access to resources. You'll need to create and use auth profiles.

> **Note**
>
> With the Visual Studio Code extension, you can view and manage authentication profiles.

Connect to your tenant

Use the pac auth create command to connect to your tenant. Running `pac auth create` creates an authentication profile on your machine. You can have multiple authentication profiles available. Having multiple authentication profiles is interesting when you work with multiple tenants. For instance, when your company uses multiple tenants (for development, production, and test) or when you're a consultant that works for multiple customers.

The `pac auth create` command has a couple of helpful parameters:

- The `--environment` parameter enables you to automatically connect to the right environment. If you omit the `--environment` parameter, you're connected to the default environment, but with the `--environment` parameter added to it, you can connect to a different one. You can use the environment ID, url, unique name, or partial name as the value.
- In environments where you don't have the ability to use an interactive experience, adding the `--deviceCode` parameter makes sure you can still connect. For example, in GitHub Codespaces, the `--deviceCode` parameter automatically gets added when you run `pac auth create`.

Example

To connect to the `HR-Dev` environment, you can use the following example:

PowerShell

```
pac auth create --environment "HR-Dev"
```

Switch to another authentication profile

When you have multiple authentication profiles, you can easily switch using the pac auth list and pac auth select commands.

The `pac auth list` command lists all authentication profiles on your machine. The `pac auth select` command selects a different command.

`pac auth list` Example

To list all authentication profiles, use the following example:

PowerShell

```
pac auth list
```

Running the `pac auth list` command returns all authentication profiles:

PowerShell

```
Index Active Kind      Name Friendly Name                  Url
User                                      Cloud  Type
[1]   *      UNIVERSAL      Personal Productivity (Default)
https://x.crm.dynamics.com/         user@contoso.onmicrosoft.com
Public User
```

Select a different authentication profile using the index value.

PowerShell

```
pac auth select --index 2
```

Enable tab completion in PowerShell

Power Platform CLI has a `pac complete` command similar to the .NET CLI complete command.

When configured using the instructions below, you can type just the beginning of a command and then use the `Tab` key to complete the input entered.

To add tab completion to **PowerShell** for the PAC CLI, create or edit the profile stored in the variable `$PROFILE`. For more information, see How to create your profile and Profiles and execution policy.

Add the following code to your profile:

PowerShell

```
$scriptblock = {
    param($wordToComplete, $commandAst, $cursorPosition)

    &pac complete -s "$($commandAst.ToString())" | ForEach-Object {
        [System.Management.Automation.CompletionResult]::new($_, $_,
'ParameterValue', $_)
    }
}

Register-ArgumentCompleter -Native -CommandName pac -ScriptBlock
$scriptblock
```

Other Shells

You can also use this with bash, zsh, fish, and nushell. Instead of `dotnet complete` use `pac complete`

US Sovereign cloud availability

Microsoft Power Platform CLI is available for use in the GCC and GCC High (US Sovereign cloud) regions. See the --cloud parameter for the pac auth create command to find out about supported US Sovereign cloud environments.

Next steps

Learn about the commands. For the complete list of supported commands, run the `pac help` command or `pac <subcommand> help` - for example: `pac solution help`.

Or view the list of commands in the documentation:

Microsoft Power Platform CLI Command Groups

See also

Microsoft Power Platform CLI release notes
Microsoft Power Platform CLI Command Groups
Power Apps component framework

Feedback

Was this page helpful? Yes No

Provide product feedback

Install the Power Platform Tools Visual Studio Code extension

Article • 02/14/2024

Applies to: Windows Linux macOS

The Visual Studio Code extension is the recommended way to install Power Platform CLI. This extension upgrades automatically when new versions are released. You also have the options to install a previous version or side-load the extension.

Prerequisites

Download and install Visual Studio Code.

Install

You can install the Microsoft Power Platform CLI using the Visual Studio Code extension from the Visual Studio Marketplace or from Visual Studio Extensions.

Install from the Visual Studio Marketplace

1. Go to Power Platform Tools
2. Select **Install**.
3. Select **Continue** if the **Visual Studio Code is required to install this extension** dialog appears
4. Select **Open** if the **This site is trying to open Visual Studio Code dialog** appears
5. In Visual Studio Code, select **Install**

Install from Visual Studio Code extensions

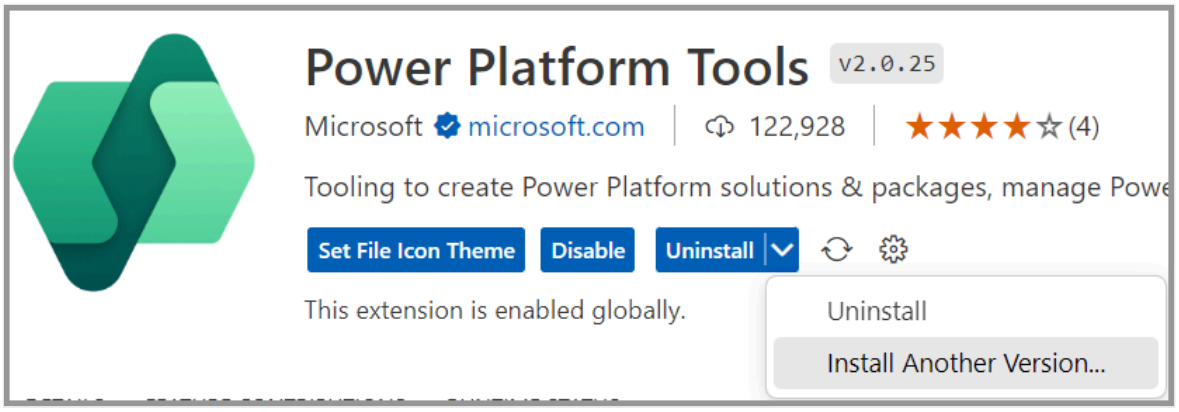
1. In Visual Studio Code, select the **Extensions** icon in the navigation bar.
2. Search for 'power platform tools'
3. Select the **Install** button in the search results.

Install a previous version

If there's an issue with the current version, you can revert to an earlier version.

1. In Visual Studio Code, select **Extensions**

2. Search for 'power platform tools'
3. In the main window, select the drop-down arrow on the **Uninstall** button
4. Select **Install Another version...**



5. Select the previous version you want to install
6. After the installation is completed, select **Reload Required**

When you select this option, the extension gives you the option to **Update** and ignores automatic updates. To resume automatic updates:

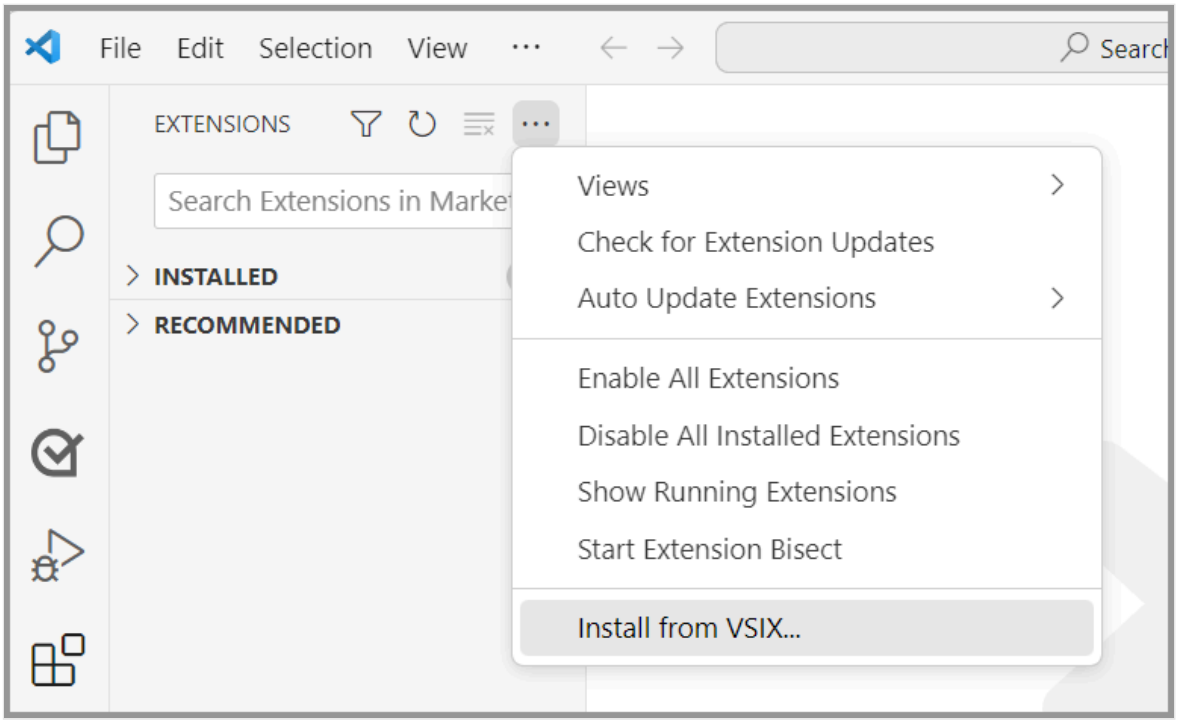
1. While viewing the Power Platform Tools extension in Visual Studio Code, select the drop-down arrow on the **Update** button.
2. Clear the **Ignore Updates** check box
3. Close and reopen Visual Studio Code to trigger the update.

Side-load install

In some organizations, downloading or initiating an install over the web is prohibited. Most cases, the organization downloads the installation media and stores it in a secure location and verify that it's working according to their standards, before it's distributed within the organization. Use the following instructions to support this type of installation.

1. Go to Power Platform Tools in the Visual Studio Marketplace
2. On the right side, in the **Resources** group select the **Download Extension** link.

   This downloads a file named `microsoft-IsvExpTools.powerplatform-vscode-x.x.xx.vsix` where `x.x.xx` is the current version.
3. In Visual Studio Code select **Extensions**
4. Select the ellipsis on the **Extensions** side bar, and then select **Install from VSIX...**

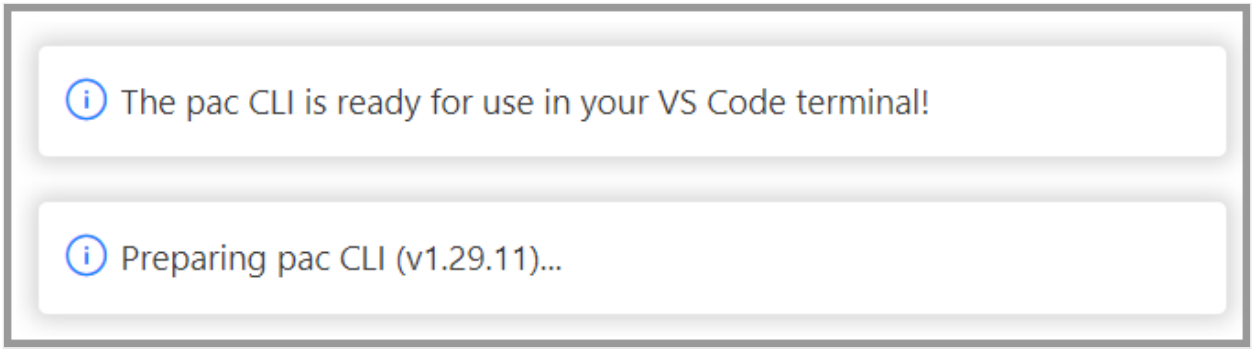


5. Select the downloaded file.

6. When installation is complete, you'll see a notification saying **Completed installing Power Platform Tools extension from VSIX**.

Update

There's usually no need to update. The extension updates automatically as new versions are released. When you open Visual Studio Code, you'll see a notification like this:



If you choose to install a previous version, you'll need to re-enable automatic updates.

Uninstall

1. In Visual Studio Code, select **Extensions**
2. Search for 'power platform tools'.
3. In the main window, select **Uninstall**.

Enable PAC CLI in Command Prompt (CMD) and PowerShell terminals for Windows

By default, the Visual Studio Code extension only enables commands using the Visual Studio Code PowerShell terminal.

To enable using PAC CLI in in Command Prompt (CMD) and PowerShell terminals for Windows, you can do any of the following:

- Install Power Platform CLI with .NET Tool
- Install Power Platform CLI using Windows MSI
- Add the PAC CLI executable to the Windows environment PATH.

Authentication profile management

When you install the Visual Studio Code extension, you can view and manage authentication profiles.

See also

What is Microsoft Power Platform CLI?
Install Power Platform CLI with .NET Tool
Install Power Platform Tools for Windows

Install Power Platform CLI with .NET Tool

Article • 02/14/2024

Applies to: Windows Linux macOS

You can install, update, and uninstall the Power Platform CLI using .NET tool commands.

> **Note**
>
> An internet connection is required for install and update. Power Platform CLI with .NET Tool is installed and updated from this NuGet package: Microsoft.PowerApps.CLI.Tool

Prerequisites

You must have .NET installed. .NET 6.0 is recommended.

Windows

Install .NET on Windows

Install

Install Power Platform CLI by using the dotnet tool install command:

.NET CLI

```
dotnet tool install --global Microsoft.PowerApps.CLI.Tool
```

Update

Update Power Platform CLI by using the dotnet tool update command:

.NET CLI

```
dotnet tool update --global Microsoft.PowerApps.CLI.Tool
```

> **Note**
>
> All updates for Power Platform CLI require internet connection to access the latest Nuget package: Microsoft.PowerApps.CLI

Uninstall

Uninstall Power Platform CLI by using the dotnet tool uninstall command:

.NET CLI

```
dotnet tool uninstall --global Microsoft.PowerApps.CLI.Tool
```

File locations

The default location for a tool's executable file depends on the operating system:

Expand table

OS	Path
Linux/macOS	`$HOME/.dotnet/tools`
Windows	`%USERPROFILE%\.dotnet\tools`

This location is added to the user's path when the SDK is first run. So global tools can be invoked from any directory without specifying the tool location.

Tool access is user-specific, not machine global. A global tool is only available to the user that installed the tool.

See also

What is Microsoft Power Platform CLI?
Install the Power Platform Tools Visual Studio Code extension
Install Power Platform Tools for Windows
How to manage .NET tools

Install Power Platform CLI using Windows MSI

Article • 02/27/2024

Applies to: Windows only

The pac data and certain pac package commands are only available for Windows because they currently have dependencies on .NET Framework, which isn't cross-platform compatible.

> **Note**
>
> An internet connection is required for install and update. Power Platform CLI for Windows is installed and updated from this NuGet package: Microsoft.PowerApps.CLI

Install

1. Download the powerapps-cli-1.0.msi file.
2. Open the `powerapps-cli-1.0.msi` file and accept the terms in the License Agreement.



3. Select **Install**

4. When the Power Apps CLI Setup wizard is completed, select **Finish**.

Update

To get the latest updates for Power Platform Tools for Windows, run this command:

PowerShell

```
pac install latest
```

Uninstall

1. Download the powerapps-cli-1.0.msi file
2. Open the `powerapps-cli-1.0.msi` file
3. Select **Next**
4. In the **Change, repair, or remove installation** screen, select **Remove**
5. In the **Ready to remove PowerApps CLI** screen, select **Remove** to confirm
6. When the Power Apps CLI Setup wizard is completed, select **Finish**

Manage versions

Each time you install Power Platform CLI using Windows MSI, the old installations aren't removed. You can revert back to an earlier installed version if there's an issue with a newer version.

You can type `pac use` and press `Enter` to list installed versions. If you have multiple versions installed on your system, then you can type `pac use <CLI version number>` and press `Enter` to use a specific version.

```
C:\> pac use
  1.29.11 (In Use)
  1.30.3
  1.30.6 (Latest)
C:\> pac use 1.30.6
Now using version 1.30.6 of the Microsoft PowerApps CLI.
C:\>
C:\> pac use
  1.29.11
  1.30.3
```

```
  1.30.6 (In Use) (Latest)
C:\>
```

Known issue with `pac power-fx repl`

When you install using the Windows MSI, the shell can stop responding when you try to use pac power-fx repl. To work around this issue, install and use the Visual Studio Code extension or install Power Platform CLI with .NET Tool.

See also

What is Microsoft Power Platform CLI?
Install the Power Platform Tools Visual Studio Code extension
Install Power Platform CLI with .NET Tool

Create and register a plug-in package using PAC CLI

Article • 02/26/2024

In this article you'll learn how to create a plug-in package, upload and register it on the Dataverse server, and perform some other package management tasks. You'll be using PAC CLI to create the plug-in package and the Plug-in Registration tool to upload and register the package.

This article helps you:

- ✓ Create a Visual Studio plug-in project
- ✓ Add a dependent assembly to the project and build
- ✓ Use the Plug-in Registration tool to upload and register your package
- ✓ Use the Plug-in Registration tool to update and delete a package

More information: Build and package plug-in code.

Prerequisites

- Access to a Dataverse environment
- A system user account, with System Administrator or System Customizer role, in the target environment
- The tools described in the next table

Expand table

Tool/App	Instructions
Microsoft Power Platform CLI	Use version 1.17 or higher. The preferred installation method is using Visual Studio Code. See Power Platform Tools. You can also download and install the Windows version here: https://aka.ms/PowerAppsCLI. If you have already installed the Windows version, make sure you run `pac install latest` to get the latest version. More information: What is Microsoft Power Platform CLI?
Plug-in Registration tool (PRT)	Use version 9.1.0.184 or higher.

Tool/App	Instructions
	Use these instructions to install the latest version: Dataverse development tools.
Visual Studio	Microsoft Visual Studio 2019 or newer.

Create a Visual Studio project

Use the PAC CLI pac plugin init command to create a Visual Studio project that streamlines your development process with dependent assemblies.

1. Create a folder for your plug-in project. The name of this folder determines the name of the Visual Studio .NET Framework Class library project for your plug-in.

2. Open a PowerShell terminal window in Visual Studio Code to navigate to the folder and run the command pac plugin init. For plug-in packages, we recommend that you use the `--skip-signing` parameter so that your plug-in assemblies aren't signed.

   Example:

   ```
   PowerShell
   ```

   ```
   PS E:\projects\mypluginproject> pac plugin init --skip-signing
   ```

> **Note**
>
> You are not required to sign the assemblies when using dependent assemblies. If you sign your assembly, all dependent assemblies must also be signed. More information: **Signed assemblies are not required**
>
> The **pac plugin init** command has a number of optional parameters. You must use the **--skip-signing** parameter if you do not want to sign your plug-in assembly.

These commands create a Visual Studio .NET Framework class library project based on the name of the folder it was created in.

Depending on your Visual Studio solution configuration, when you open the Visual Studio project in Visual Studio and build it, a NuGet package is generated for the project in the `bin\Debug` or `bin\Release` folder. Each time you build your project, this NuGet package is updated. The NuGet package is the file you upload using the Plug-in Registration tool.

Add a dependent assembly to your project

You can add a NuGet package for satisfying required references to your Visual Studio plug-in project as you normally do. After you build the project, you should find your plug-in assembly in the generated NuGet package located in the `bin\outputPackages` folder.

You can use NuGet Package Explorer to examine the generated NuGet package.

Add a dependent assembly without using NuGet

If you have an assembly that isn't distributed as a NuGet package, you can add it to your project as you normally do. In **Solution Explorer**, right-click **Dependencies** and choose **Add Assembly Reference...**. Select the assembly you want to add.

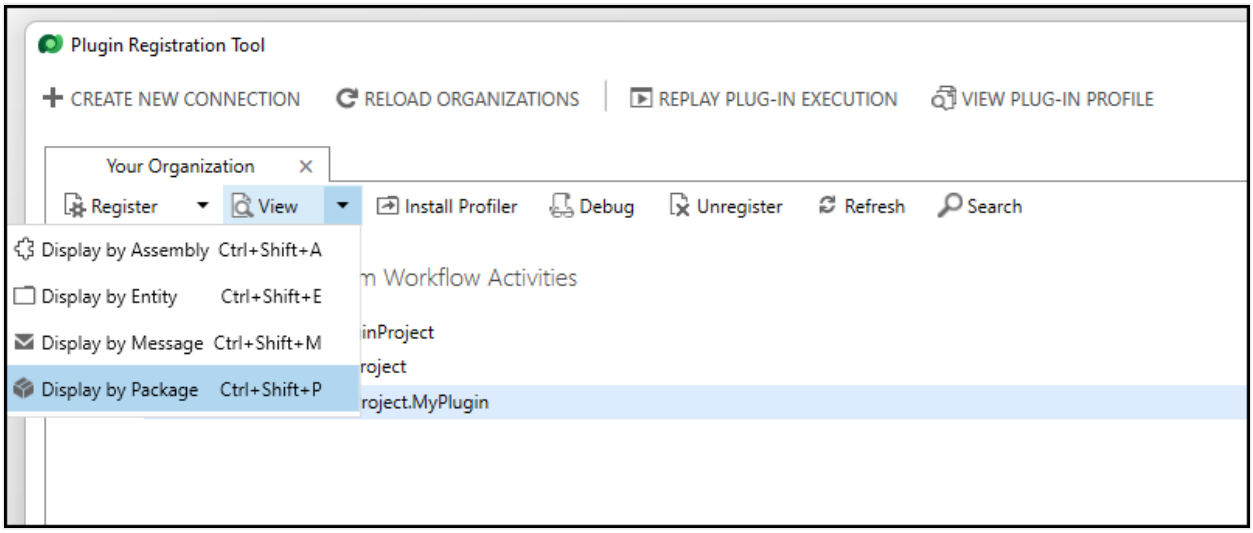
Use the Plug-in Registration tool

You can use the Plug-in Registration tool (PRT) to perform the following tasks:

1. View list of available plug-in packages
2. Register a NuGet package as a plug-in package

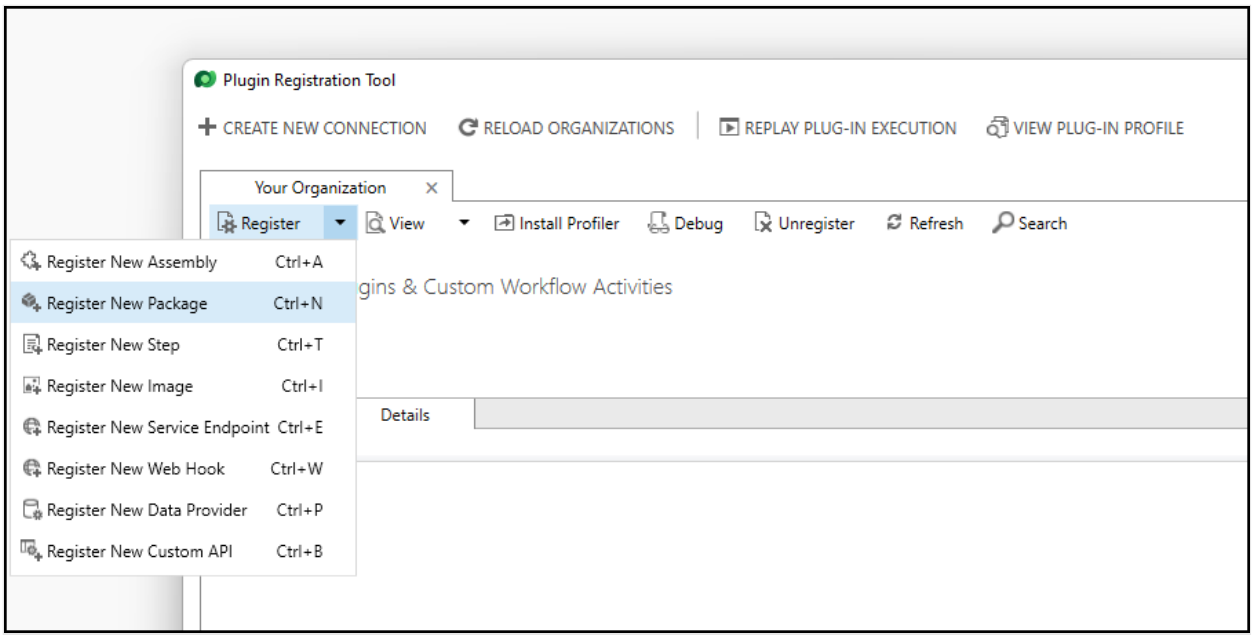
View list of available plug-in packages

The PRT has a new **Display by Package** view to list any plug-in packages previously imported:



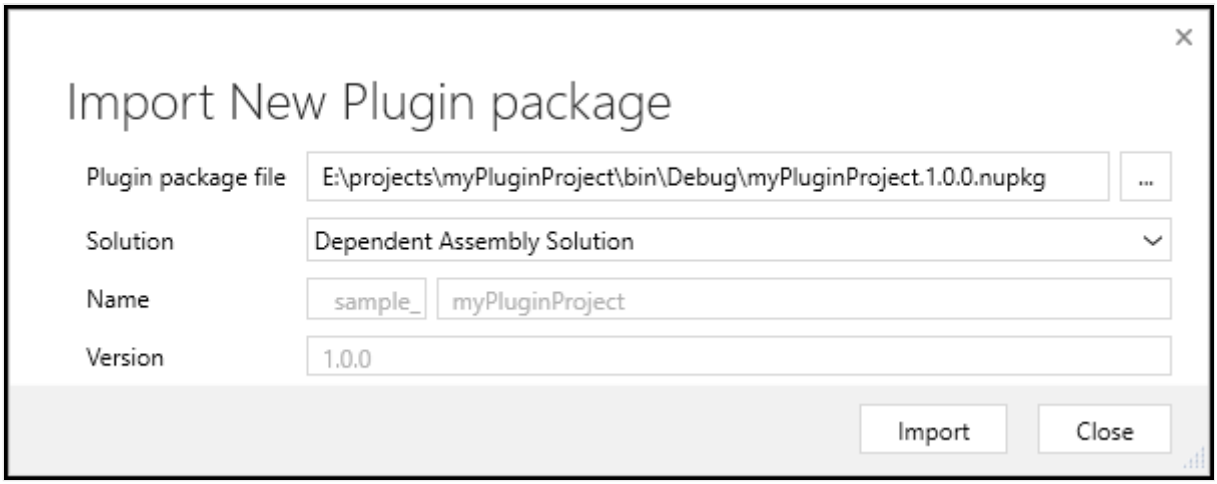
Register a NuGet package as a plug-in package

The PRT has a command to select a NuGet package to import/register as a plug-in package.

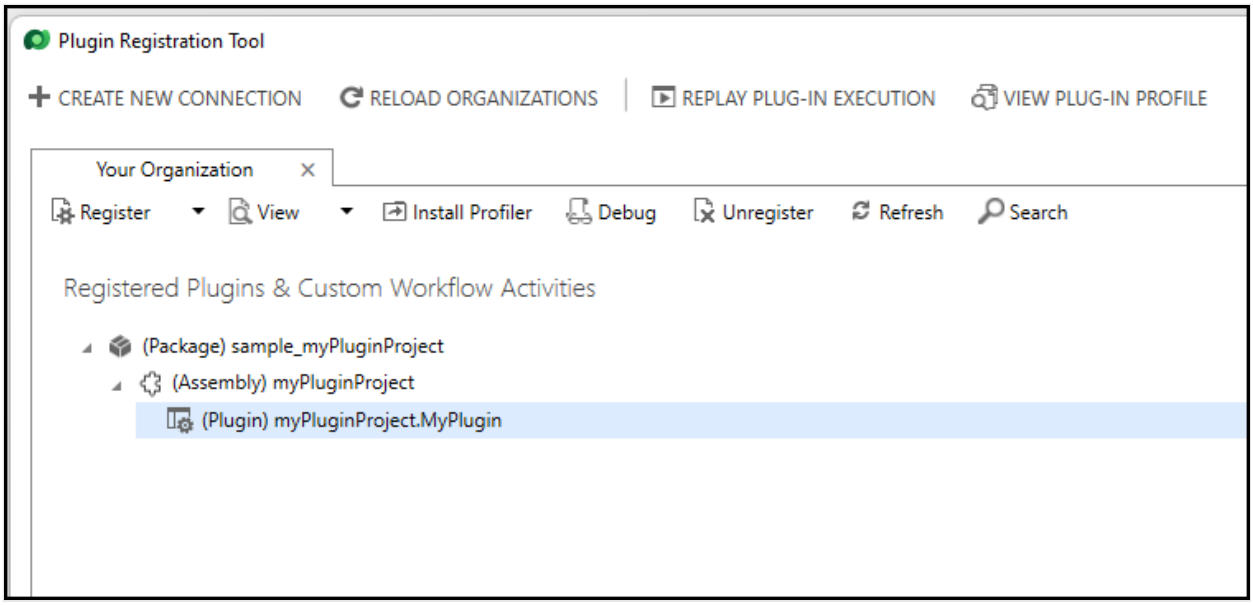


This command opens a dialog to select the plug-in package.

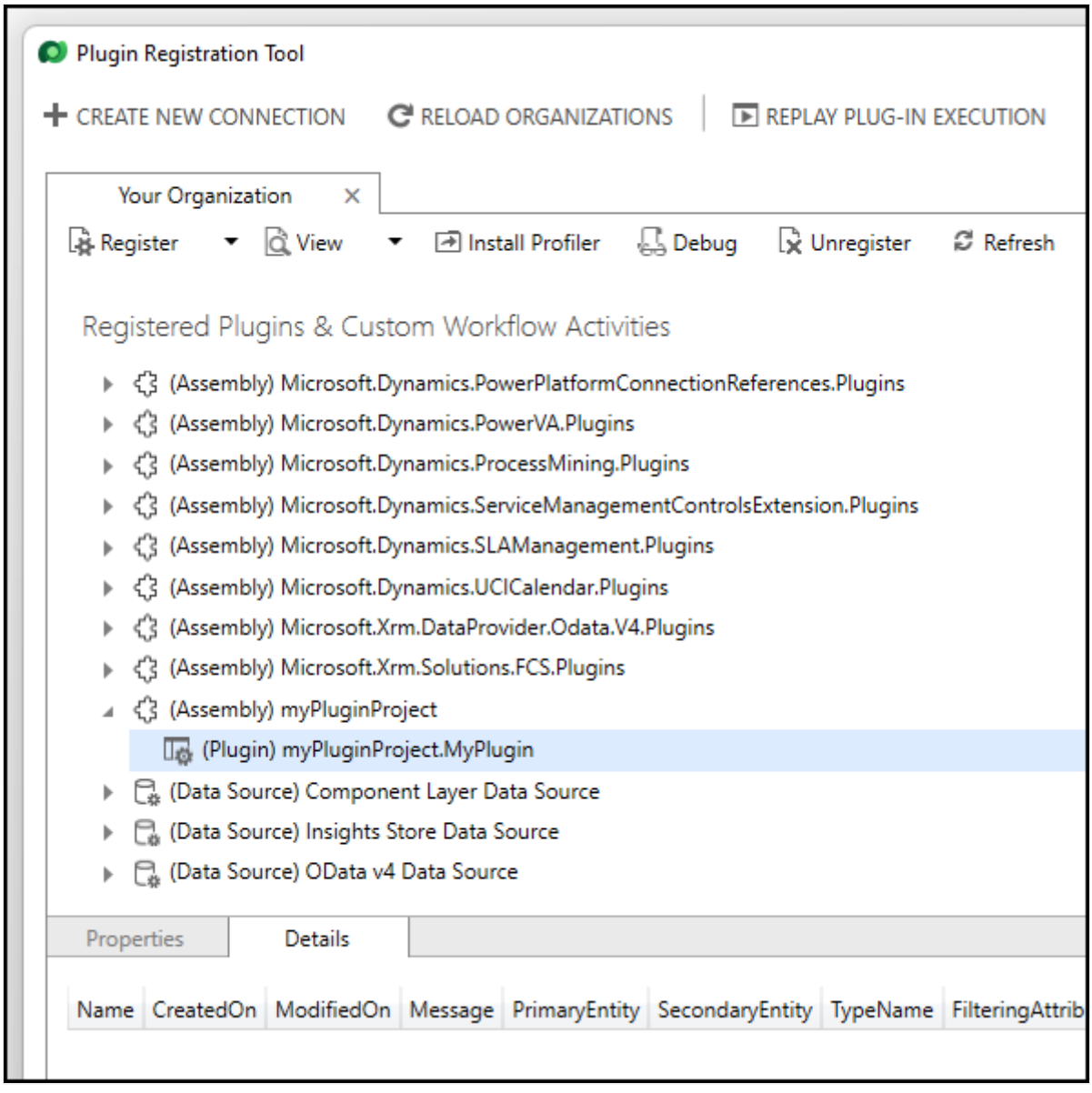
You can select an existing solution or create a new one.



From the **Display by Package** view, you can select the assembly and register steps.



The assembly is also available within the **Display by Assembly** view.

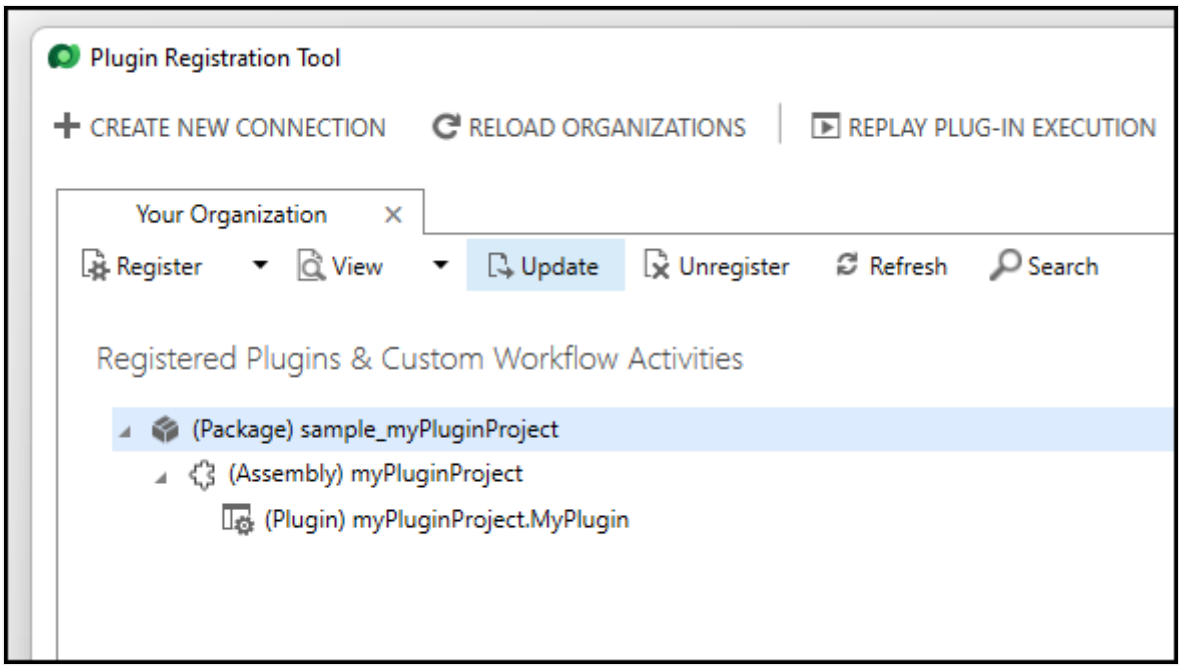


Plug-in package management

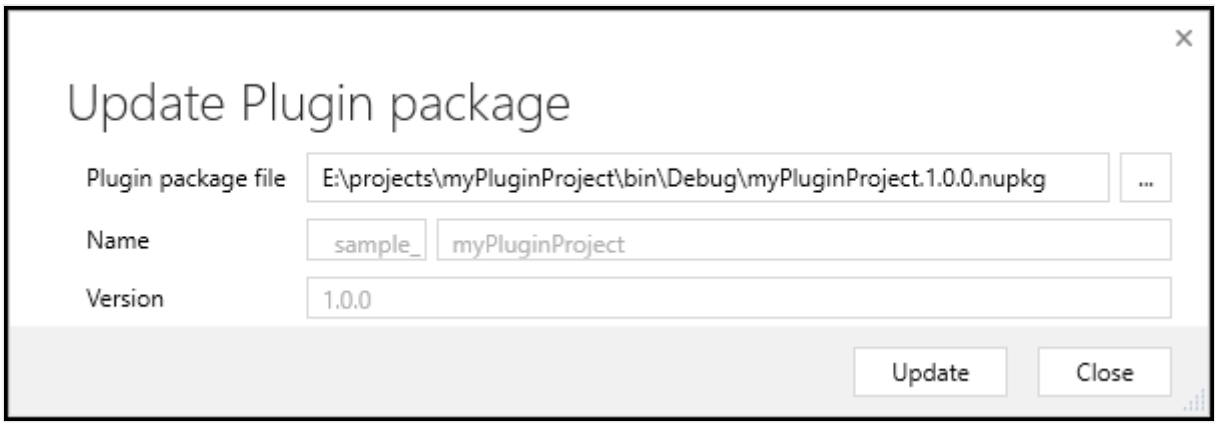
Within the Plug-in Registration tool, you can perform additional operations on plug-in packages as described in the next sections.

Update a plug-in package

While viewing the list of plug-in packages using the **Display by Package** view, select the plug-in package and select the **Update** command.



This command opens a dialog so you can select the NuGet Package with changes.



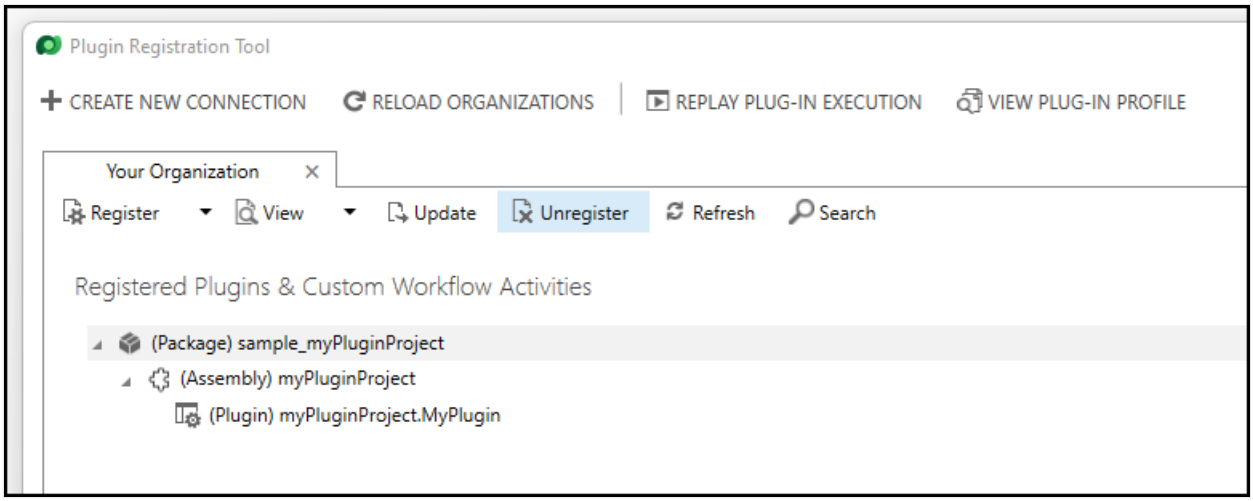
> **ⓘ Important**
>
> If your update removes any plug-in assemblies, or types which are used in plug-in step registrations, the update will be rejected. You must manually remove any step registrations that use plug-in assemblies or plug-in types that you want to remove with your update.

The version of the plug-in package or plug-in assembly is not a factor in any upgrade behaviors. You can update the version of the plug-in assembly as you need.

The name and version of the plug-in package cannot be changed once created. Attempting to do so, for example in an API call, results in an error.

Delete plug-in packages

While viewing the list of plug-in packages using the **Display by Package** view, select the plug-in package and select the **Unregister** command.



ⓘ Important

You cannot unregister a package that has any plug-in step registrations for any plug-in assemblies in the package. You must first unregister all step registrations for the assemblies in the package before you can delete the package.

See also

About the Plug-in Registration tool

Microsoft Power Platform CLI Command Groups

Article • 05/14/2024

Expand table

Command Group	Description
pac admin	Work with your Power Platform Admin Account
pac application	Commands for listing and installing available Dataverse applications from AppSource
pac auth	Manage how you authenticate to various services
pac canvas	Operating with Power Apps .msapp files
pac catalog	Commands for working with Catalog in Power Platform
pac connection	Commands for working with Dataverse connection.
pac connector	Commands for working with Power Platform Connectors
pac copilot	Tools and utilities for copilot management
pac data	Import and export data from Dataverse.
pac env	Work with your Dataverse organization.
pac help	Show help for the Microsoft Power Platform CLI.
pac modelbuilder	Code Generator for Dataverse APIs and Tables
pac package	Commands for working with Dataverse package projects
pac pages	Commands for working with Power Pages website.
pac pcf	Commands for working with Power Apps component framework projects
pac pipeline	Work with Pipelines
pac plugin	Commands for working with Dataverse plug-in class library
pac power-fx	(Preview) Commands for working with Power Fx
pac solution	Commands for working with Dataverse solution projects
pac telemetry	Manage telemetry settings.

Command Group	Description
pac test	(Preview) Execution of automated tests for a Power App
pac tool	Power Platform tools that can be installed and launched.

See also

Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac admin

Article • 05/13/2024

Work with your Power Platform Admin Account

Commands

Expand table

Command	Description
pac admin application list	List Microsoft Entra ID applications registered under your tenant.
pac admin application register	Register Microsoft Entra ID application with your tenant.
pac admin application unregister	Unregister Microsoft Entra ID application from your tenant.
pac admin assign-group	Assign group to target Dataverse environment with specified security role.
pac admin assign-user	Assign a user to a target Dataverse environment with specified security role.
pac admin backup	Takes a manual backup of your environment.
pac admin copy	Copy Source Environment to Destination Environment.
pac admin create	Creates a Dataverse instance in your tenant.
pac admin create-service-principal	Add Microsoft Entra ID application and associated application user to the Dataverse environment.
pac admin delete	Deletes the environment from your tenant.
pac admin list	List all environments from your tenant.
pac admin list-app-templates	Lists all supported Dataverse templates of model-driven apps in Dynamics 365.
pac admin list-backups	Lists all backups of your environment.
pac admin list-service-principal	List Microsoft Entra ID applications which have access to Dataverse.
pac admin list-tenant-settings	List tenant settings.

Command	Description
pac admin reset	Reset the environment from your tenant.
pac admin restore	Restores an environment to a given backup.
pac admin set-backup-retention-period	Takes a manual backup of your environment.
pac admin set-governance-config	Enable, disable, and edit managed environments.
pac admin set-runtime-state	Update environment administration mode.
pac admin status	This command lists the status of all the operations in progress.
pac admin update-tenant-settings	Update tenant settings.

pac admin application list

List Microsoft Entra ID applications registered under your tenant.

pac admin application register

Register Microsoft Entra ID application with your tenant.

Required Parameters for admin application register

`--application-id` `-id`

Application Id

pac admin application unregister

Unregister Microsoft Entra ID application from your tenant.

Required Parameters for admin application unregister

`--application-id` `-id`

Application Id

pac admin assign-group

Assign group to target Dataverse environment with specified security role.

Required Parameters for admin assign-group

`--group` `-g`

Microsoft Entra ID object ID of group to assign to target Dataverse environment.

`--group-name` `-gn`

Name of the group or team to create in Dataverse.

`--membership-type` `-mt`

Team membership type.

Use one of these values:

- `MembersAndGuests`
- `Members`
- `Owners`
- `Guests`

`--role` `-r`

Name or ID of security role to be applied to user

`--team-type` `-tt`

Type of team.

Use one of these values:

- `Owner`
- `Access`
- `AadSecurityGroup`
- `AadOfficeGroup`

Optional Parameters for admin assign-group

`--business-unit` `-bu`

ID of business unit to associate application user with.

`--environment` `-env`

ID or URL of the environment to assign a user to.

pac admin assign-user

Assign a user to a target Dataverse environment with specified security role.

Examples

The following examples show the use of the `pac admin assign-user` command.

- Assign a user by email to an environment with the basic user role
- Assign a user by guid to an environment with the system administrator role and add the user to a specified business unit
- Assign an application user to an environment with the system administrator role

Assign a user by email to an environment with the basic user role

PowerShell

```
pac admin assign-user `
  --environment 00000000-0000-0000-0000-000000000000 `
  --user "user@company.com" `
  --role "Basic User"
```

Assign a user by guid to an environment with the system administrator role and add the user to a specified business unit

PowerShell

```
pac admin assign-user `
  --environment 00000000-0000-0000-0000-000000000000 `
  --user 00000000-0000-0000-0000-000000000000 `
```

```
  --business-unit` 00000000-0000-0000-0000-000000000000 `
  --role "System Administrator"
```

Assign an application user to an environment with the system administrator role

PowerShell

```
pac admin assign-user `
  --environment 00000000-0000-0000-0000-000000000000 `
  --user 00000000-0000-0000-0000-000000000000 `
  --role "System Administrator"
  --application-user
```

Required Parameters for admin assign-user

`--role` `-r`

Name or ID of security role to be applied to user

`--user` `-u`

Object ID or user principal name (UPN) of Microsoft Entra ID user to be assigned to the environment or Application ID if assigning an Application User.

Optional Parameters for admin assign-user

`--application-user` `-au`

Specifies whether the input user is an application user. If a business unit isn't specified, the application user is added to the authenticated users business unit.

This parameter requires no value. It's a switch.

`--async` `-a`

Deprecated: This parameter is ignored.

`--business-unit` `-bu`

ID of business unit to associate application user with.

`--environment` `-env`

ID or URL of the environment to assign a user to.

pac admin backup

Takes a manual backup of your environment.

Example

Create a manual backup for an environment

PowerShell

```
pac admin backup `
  --environment 00000000-0000-0000-0000-000000000000 `
  --label "Manual Backup October 2022"
```

Required Parameters for admin backup

`--label` `-l`

Sets the backup label as provided.

Optional Parameters for admin backup

`--environment` `-env`

Environment URL or ID of the Environment that requires backup.

`--notes` `-n`

Deprecated: This parameter is ignored.

pac admin copy

Copy Source Environment to Destination Environment.

Examples

The following examples show the use of the `pac admin copy` command.

- Copy an environment from a source environment to a target environment using a minimal copy
- Copy an environment from a source environment to a target environment using a full copy

Copy an environment from a source environment to a target environment using a minimal copy

This example copies over customizations and schemas only from a source environment to a target environment.

PowerShell

```
pac admin copy `
  --name "Environment Name" `
  --source-env 00000000-0000-0000-0000-000000000000 `
  --target-env 00000000-0000-0000-0000-000000000000 `
  --type MinimalCopy
```

Copy an environment from a source environment to a target environment using a full copy

This example copies the whole environment over from a source environment to a target environment.

PowerShell

```
pac admin copy `
  --name "Environment Name" `
  --source-env 00000000-0000-0000-0000-000000000000 `
  --target-env 00000000-0000-0000-0000-000000000000 `
  --type FullCopy
```

Optional Parameters for admin copy

`--async` **`-a`**

Optional boolean argument to run pac verbs asynchronously, defaults to false.

This parameter requires no value. It's a switch.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--name` `-n`

Name of the target environment.

`--skip-audit-data` `-sa`

Switch indicating whether audit data should be skipped

This parameter requires no value. It's a switch.

`--source-env` `-se`

Environment URL or ID of the source environment that is being copied

`--target-env` `-te`

Environment URL or ID of the target environment.

`--type` `-t`

Use one of these values:

- `MinimalCopy`
- `FullCopy`

`--json`

Returns the output of the command as a JSON formatted string.

pac admin create

Creates a Dataverse instance in your tenant.

Examples

The following examples show the use of the `pac admin create` command.

- Simple creation of an environment
- Advanced creation of an environment
- Create using --input-file

Simple creation of an environment

In this example, we'll create a sandbox environment with the default settings:

- Currency: USD
- Language: English
- Region: unitedstates

We don't have to add those properties, because those values are the default values.

PowerShell

```
pac admin create `
  --name "Contoso Test" `
  --type Sandbox `
  --domain ContosoTest
```

Advanced creation of an environment

In this example, we'll create a production environment in the region europe, with the currency set to Euro and the language to English.

We don't have to add the language property, because it's the default value.

PowerShell

```
pac admin create `
  --name "Contoso Marketing" `
  --currency EUR `
  --region europe `
  --type Production `
  --domain ContosoMarketing
```

Create using --input-file

In this example, we create the same environment described in Advanced creation of an environment, except refer to data in a JSON file named `config.json` where config.json contains this data:

JSON

```
{
  "name": "Contoso Marketing",
  "currency": "EUR",
  "region": "europe",
  "type": "Production",
  "domain": "ContosoMarketing"
}
```

Then, refer to the file when using the `pac admin create` command:

PowerShell

```
pac admin create --input-file C:\config.json
```

Required Parameters for admin create

`--type` `-t`

Sets the environment Type.

Use one of these values:

- `Trial`
- `Sandbox`
- `Production`
- `Developer`
- `Teams`
- `SubscriptionBasedTrial`

Optional Parameters for admin create

`--async` `-a`

Optional boolean argument to run pac verbs asynchronously, defaults to false.

This parameter requires no value. It's a switch.

`--currency` `-c`

Sets the currency used for your environment. [defaults to USD]

`--domain` `-d`

The domain name is part of the environment URL. If domain name is already in use, a numeric value is appended to the domain name. For example: If 'contoso' is already in use, then the environment URL is updated to https://{contoso}0.crm.dynamics.com.

Note: Only characters within the ranges [A - Z], [a - z], [0 - 9], or '-' are allowed. The first and last character can't be the '-' character. Consecutive '-' characters aren't allowed.

`--input-file` `-if`

The verb arguments to be passed in a .json input file. For example: {"name" : "contoso"}. The arguments passed through command-line will take precedence over arguments from the .json input file.

`--language` `-l`

Sets the language used for your environment. [defaults to English]

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--name` `-n`

Sets the name of the environment.

`--region` `-r`

Sets the environment's region name. [defaults to unitedstates]

`--security-group-id` `-sgid`

Microsoft Entra ID Security Group Id or Microsoft 365 Group Id (required for Teams environment).

Note: The Security Group Id must be a valid Guid.

`--templates` `-tm`

Sets the Dynamics 365 app that needs to be deployed, passed as comma separated values. For example: -tm "D365_Sample, D365_Sales"

`--user` `-u`

Object ID or user principal name (UPN) of Microsoft Entra ID user to be assigned to the environment.

`--json`

Returns the output of the command as a JSON formatted string.

pac admin create-service-principal

Add Microsoft Entra ID application and associated application user to the Dataverse environment.

Example

PowerShell

```
pac admin create-service-principal  --environment <environment id>
```

More information: Configure service connections using a service principal

Optional Parameters for admin create-service-principal

`--environment` `-env`

Environment (ID, org ID, url, unique name, or partial name).

`--name` `-n`

Application name to create in Entra ID.

`--role` `-r`

Name or ID of security role to be applied to application user. The default value is: 'System Administrator'.

Remarks

When successful, four columns are displayed:

- Power Platform TenantId
- Application ID
- Client Secret (in clear text)
- Expiration

For example:

```PowerShell
PS C:\>pac admin create-service-principal --environment d3fcc479-0122-e7af-
9965-bde57f69ee1d
Connected as admin@M365x57236226.onmicrosoft.com
Successfully assigned user adde6d52-9582-4932-a43a-beca5d182301 to
environment d3fcc479-0122-e7af-9965-bde57f69eeld with security role System
Administrator
Tenant ID                              Application ID
Client Secret                             Expiration
2b0463ed-efd7-419d-927d-a9dca49d899c adde6d52-9582-4932-a43a-beca5d182301
beY8Q~JBZ~CBDgIKKBjbZ3g6BofKzoZkYj23Hbf 7/31/2024 4:27:03 PM
```

pac admin delete

Deletes the environment from your tenant.

Examples

The following examples show the use of the `pac admin delete` command.

- Delete environment by GUID
- Delete environment by URL

Delete environment by GUID

```PowerShell
pac admin delete --environment 00000000-0000-0000-0000-000000000000
```

Delete environment by URL

PowerShell

```
pac admin delete --environment https://contosomarketing.crm4.dynamics.com
```

Optional Parameters for admin delete

`--async` `-a`

Optional boolean argument to run pac verbs asynchronously, defaults to false.

This parameter requires no value. It's a switch.

`--environment` `-env`

Environment URL or ID of the Environment that needs to be deleted from your tenant.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

pac admin list

List all environments from your tenant.

Examples

The following examples show the use of the `pac admin list` command.

- List all environments in the tenant
- List all sandbox environments in the tenant

List all environments in the tenant

This example will list all environments in the tenant.

PowerShell

```
pac admin list
```

An example of the output is displayed below.

```
PowerShell

Environment   Environment Id                         Environment Url
Type        Organization Id
Contoso       00000000-0000-0000-0000-000000000000
https://contoso.crm.dynamics.com/      Production  00000000-0000-0000-0000-
000000000000
Contoso Dev   00000000-0000-0000-0000-000000000000  https://contoso-
dev.crm.dynamics.com/  Sandbox     00000000-0000-0000-0000-000000000000
Contoso Test  00000000-0000-0000-0000-000000000000
https://commdemos.crm.dynamics.com/    Sandbox     00000000-0000-0000-0000-
000000000000
```

List all sandbox environments in the tenant

This example will list all environments in the tenant of the type `Sandbox`.

```
PowerShell

pac admin list --type Sandbox
```

An example of the output is displayed below.

```
PowerShell

Environment  Environment Id                         Environment Url
Type        Organization Id
Contoso Dev  00000000-0000-0000-0000-000000000000  https://contoso-
dev.crm.dynamics.com/  Sandbox     00000000-0000-0000-0000-000000000000
Contoso Test 00000000-0000-0000-0000-000000000000
https://commdemos.crm.dynamics.com/    Sandbox     00000000-0000-0000-0000-
000000000000
```

Optional Parameters for admin list

`--application` `-a`

List all environments that have specified application installed. To specify application, use unique name or ID.

`--environment` `-env`

List all environments that contain given string in their name or ID.

`--name` `-n`

List all environments that contain given string in their name.

`--type` `-t`

List all environments with the given type.

Use one of these values:

- `Trial`
- `Sandbox`
- `Production`
- `Developer`
- `Teams`
- `SubscriptionBasedTrial`

`--json`

Returns the output of the command as a JSON formatted string.

pac admin list-app-templates

Lists all supported Dataverse templates of model-driven apps in Dynamics 365.

Examples

The following examples use the `pac admin list-app-templates` command:

- List all supported Dataverse database templates
- List all supported Dataverse database templates of model-driven apps in Dynamics 365 for region Europe

List all supported Dataverse database templates

This example lists all supported Dataverse database templates of model-driven apps in Dynamics 365.

PowerShell

```
pac admin list-app-templates
```

An example of the output is displayed below.

PowerShell

```
Template Name                Template Location Template Display Name Is
Disabled
D365_CDSSampleApp            unitedstates      Sample App            False
D365_CustomerService         unitedstates      Customer Service      True
D365_CustomerServicePro      unitedstates      Customer Service Pro  True
D365_DeveloperEdition        unitedstates      Developer Edition     False
D365_FieldService            unitedstates      Field Service         True
D365_Guides                  unitedstates      Guides                True
D365_PowerFrameworkTemplate unitedstates      Power Framework       False
D365_ProjectOperations       unitedstates      Project Operations    True
D365_RemoteAssist            unitedstates      Remote Assist         True
D365_Sales                   unitedstates      Sales Enterprise      True
D365_SalesPro                unitedstates      Sales Pro             True
D365_SmbMarketing            unitedstates      SMB Marketing         True
```

List all supported Dataverse database templates of model-driven apps in Dynamics 365 for region Europe

This example lists all supported Dataverse database templates of model-driven apps in Dynamics 365 for the region Europe.

PowerShell

```
pac admin list-app-templates --region Europe
```

An example of the output is displayed below.

PowerShell

```
Template Name                Template Location Template Display Name Is
Disabled
D365_CDSSampleApp            Europe            Sample App            False
D365_CustomerService         Europe            Customer Service      True
D365_CustomerServicePro      Europe            Customer Service Pro  True
D365_DeveloperEdition        Europe            Developer Edition     True
D365_FieldService            Europe            Field Service         True
D365_Guides                  Europe            Guides                True
D365_PowerFrameworkTemplate Europe            Power Framework       True
D365_ProjectOperations       Europe            Project Operations    True
D365_RemoteAssist            Europe            Remote Assist         True
D365_Sales                   Europe            Sales Enterprise      True
D365_SalesPro                Europe            Sales Pro             True
D365_SmbMarketing            Europe            SMB Marketing         True
```

Optional Parameters for admin list-app-templates

`--region` `-r`

Sets the environment's region name. [defaults to unitedstates]

pac admin list-backups

Lists all backups of your environment.

Examples

The following examples show the use of the `pac admin list-backups` command.

- List backups from an environment by ID
- List backups from an environment by URL

List backups from an environment by ID

This example lists the backups of an environment based on the environment ID.

PowerShell

```
pac admin list-backups --environment 00000000-0000-0000-0000-000000000000
```

An example of the output is displayed below.

PowerShell

```
 Index   Id                                   Label        Expiry
Point Date
 1       00000000-0000-0000-0000-000000000000    Backup       14/12/2022
12:50:38   07/12/2022 12:50:38
```

List backups from an environment by URL

This example lists the backups of an environment based on the environment URL.

PowerShell

```
pac admin list-backups --environment https://contoso.crm.dynamics.com
```

Optional Parameters for admin list-backups

`--environment` `-env`

List all environments that contain a given string in their Environment ID or URL.

pac admin list-service-principal

List Microsoft Entra ID applications which have access to Dataverse.

Optional Parameters for admin list-service-principal

`--filter` `-f`

Application name 'starts with' filter.

`--max` `-m`

Max number of applications to show. Default: 20

pac admin list-tenant-settings

List tenant settings.

Optional Parameters for admin list-tenant-settings

`--settings-file` `-s`

The .json file to output tenant settings.

pac admin reset

Reset the environment from your tenant.

Examples

The following examples show the use of the `pac admin reset` command:

- Reset an environment in your tenant
- Reset an environment in your tenant and set the currency to EUR

Reset an environment in your tenant

PowerShell

```
pac admin reset --environment 00000000-0000-0000-0000-000000000000
```

Reset an environment in your tenant and set the currency to EUR

PowerShell

```
pac admin reset --environment 00000000-0000-0000-0000-000000000000 --
currency EUR
```

Optional Parameters for admin reset

`--async` **`-a`**

Optional boolean argument to run pac verbs asynchronously, defaults to false.

This parameter requires no value. It's a switch.

`--currency` **`-c`**

Sets the currency used for your environment. [defaults to USD]

`--domain` **`-d`**

The domain name is part of the environment URL. If domain name is already in use, a numeric value is appended to the domain name. For example: If 'contoso' is already in use, then the environment URL is updated to https://{contoso}0.crm.dynamics.com.

`--environment` **`-env`**

URL or ID of the Environment that needs to be reset.

`--input-file` **`-if`**

The verb arguments to be passed in a .json input file. For example: {"name" : "contoso"}. The arguments passed through command-line will take precedence over arguments from the .json input file.

`--language` `-l`

Sets the language used for your environment. [defaults to English]

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--name` `-n`

Sets the name of the environment.

`--purpose` `-p`

Sets the description used to associate the environment with a specific intent.

`--templates` `-tm`

Sets the Dynamics 365 app that needs to be deployed, passed as comma separated values. For example: -tm "D365_Sample, D365_Sales"

`--json`

Returns the output of the command as a JSON formatted string.

pac admin restore

Restores an environment to a given backup.

Examples

The following examples show the use of the `pac admin restore` command:

- Restore the latest backup
- Restore a backup based on a timestamp

Restore the latest backup

This example restores the latest backup from the source environment with ID `00000000-0000-0000-0000-000000000000` to the environment that's currently selected in the currently active auth profile.

PowerShell

```
pac admin restore --selected-backup latest --source-env 00000000-0000-0000-0000-000000000000
```

Restore a backup based on a timestamp

This example restores the backup that has been created on `12/01/2022 09:00` from the source environment with ID `00000000-0000-0000-0000-000000000000` to the environment that's currently selected in the currently active auth profile.

PowerShell

```
pac admin restore --selected-backup '12/01/2022 09:00' --source-env 00000000-0000-0000-0000-000000000000
```

Required Parameters for admin restore

`--selected-backup` **`-sb`**

DateTime of the backup in 'mm/dd/yyyy hh:mm' format OR string 'latest'.

Optional Parameters for admin restore

`--async` **`-a`**

Optional boolean argument to run pac verbs asynchronously, defaults to false.

This parameter requires no value. It's a switch.

`--max-async-wait-time` **`-wt`**

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--name` `-n`

Optional name of the restored environment.

`--skip-audit-data` `-sa`

Switch indicating whether audit data should be skipped

This parameter requires no value. It's a switch.

`--source-env` `-se`

Environment URL or ID of the source environment required for restore.

`--target-env` `-te`

Environment URL or ID of the target environment required for restore. This would default to source URL/ID if not provided.

`--json`

Returns the output of the command as a JSON formatted string.

pac admin set-backup-retention-period

Takes a manual backup of your environment.

Required Parameters for admin set-backup-retention-period

`--backup-retention-period` `-br`

Sets the backup retention period in days as provided. Valid values are: 7, 14, 21, 28.

Optional Parameters for admin set-backup-retention-period

`--async` `-a`

Optional boolean argument to run pac verbs asynchronously, defaults to false.

This parameter requires no value. It's a switch.

`--environment` `-env`

Environment URL or ID of the Environment that requires backup.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

pac admin set-governance-config

Enable, disable, and edit managed environments.

Required Parameters for admin set-governance-config

`--environment` `-env`

URL or ID of the environment for which managed environments need to be enabled, disabled or edited.

`--protection-level` `-pl`

Set protection level : 'Standard' to enable managed environments, 'Basic' to disable managed environments.

Use one of these values:

- `Basic`
- `Standard`

Optional Parameters for admin set-governance-config

`--checker-rule-overrides` `-cro`

Solution checker rule overrides

`--cloud-flows-limit` `-cfl`

Number of people that makers can share solution cloud flows with

`--cloud-flows-mode` **`-cfm`**

Solution cloud flows limit sharing mode

`--disable-group-sharing` **`-dgs`**

Disable group sharing.

This parameter requires no value. It's a switch.

`--exclude-analysis` **`-ea`**

Exclude usage insights for the environment from the weekly digest email.

This parameter requires no value. It's a switch.

`--include-insights` **`-ii`**

Include insights in the Power Platform Admin Center homepage cards.

This parameter requires no value. It's a switch.

`--limit-sharing-mode` **`-lsm`**

Limit sharing mode.

`--maker-onboarding-markdown` **`-mom`**

Maker onboarding markdown

`--maker-onboarding-url` **`-mou`**

Maker onboarding URL

`--max-limit-user-sharing` **`-ml`**

If group sharing is disabled, specify the number of people that makers can share canvas apps with.

`--solution-checker-mode` `-scm`

Solution checker validation mode.

Use one of these values:

- `none`
- `warn`
- `block`

`--suppress-validation-emails` `-sve`

Suppress validation emails

This parameter requires no value. It's a switch.

pac admin set-runtime-state

Update environment administration mode.

Required Parameters for admin set-runtime-state

`--environment` `-env`

URL or ID of the environment for which administration mode needs to be updated.

`--runtime-state` `-rs`

The environment runtime state

Use one of these values:

- `AdminMode`
- `Enabled`

Optional Parameters for admin set-runtime-state

`--async` `-a`

Optional boolean argument to run pac verbs asynchronously, defaults to false.

This parameter requires no value. It's a switch.

`--background-operations` `-bo`

The environment background operations state

Use one of these values:

- `Enabled`
- `Disabled`

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

pac admin status

This command lists the status of all the operations in progress.

pac admin update-tenant-settings

Update tenant settings.

Required Parameters for admin update-tenant-settings

`--settings-file` `-s`

The .json file with tenant settings.

Remarks

You can find an example using this command in Enable the environment routing setting.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac application

Article • 05/13/2024

Commands for listing and installing available Dataverse applications from AppSource

Commands

Expand table

Command	Description
pac application install	Installs Dataverse application to given environment.
pac application list	List available Dataverse applications from AppSource

pac application install

Installs Dataverse application to given environment.

Examples

The following examples show the use of the `pac application install` command.

- Install an application in a specified environment
- Install a list of applications in a JSON file in a specified environment

Install an application in a specified environment

This example installs the `MSFT_EmployeeIdeas` application in the specified environment.

PowerShell

```
pac application install --environment-id 00000000-0000-0000-0000-
000000000000 --application-name MSFT_EmployeeIdeas
```

Install a list of applications in a JSON file in a specified environment

This example installs the applications in the JSON file `c:\home\applicationslist.json` in the specified environment.

PowerShell

```
pac application install --environment-id 00000000-0000-0000-0000-
000000000000 --application-list c:\home\applicationslist.json
```

Optional Parameters for application install

`--application-list` `-al`

Location of the JSON file with list of the Dataverse applications from AppSource to be installed

`--application-name` `-an`

Unique name of the application to install to the target environment.

`--environment` `-env`

List available Dataverse applications for given environment (by ID or url); if not specified, list all applications in the tenant

pac application list

List available Dataverse applications from AppSource

Examples

The following examples show the use of the `pac application list` command.

- Application List
- Application List JSON Export

Application List

This example lists all available applications in the specified environment.

PowerShell

```
pac application list --environment-id 00000000-0000-0000-0000-000000000000
```

Application List JSON Export

This example lists all available applications for the tenant you're connected to and saves it as a JSON file in the directory `c:\home`.

PowerShell

```
pac application list --output c:\home\applicationslist.json
```

Optional Parameters for application list

`--environment` `-env`

List available Dataverse applications for given environment (by ID or url); if not specified, list all applications in the tenant

`--installState` `-s`

Filter by application install state

Use one of these values:

- `NotInstalled`
- `Installed`
- `All`

`--output` `-o`

Location of the JSON file to be created with list of the Dataverse applications from AppSource

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac auth

Article • 05/14/2024

Manage how you authenticate to various services

Connect to your tenant

Use the pac auth create command to connect to your tenant. Running `pac auth create` creates an authentication profile on your machine. You can have multiple authentication profiles available. Having multiple authentication profiles is interesting when you work with multiple tenants. For instance, when your company uses multiple tenants (for development, production, and test) or when you're a consultant that works for multiple customers.

The `pac auth create` command has a couple of helpful parameters:

- The `--environment` parameter enables you to automatically connect to the right environment. If you omit the `--environment` parameter, you're connected to the default environment, but with the `--environment` parameter added to it, you can connect to a different one. You can use the environment ID, url, unique name, or partial name as the value.
- In environments where you don't have the ability to use an interactive experience, adding the `--deviceCode` parameter makes sure you can still connect. For example, in GitHub Codespaces, the `--deviceCode` parameter automatically gets added when you run `pac auth create`.

Example

To connect to the `HR-Dev` environment, you can use the following example:

PowerShell

```
pac auth create --environment "HR-Dev"
```

Switch to another authentication profile

When you have multiple authentication profiles, you can easily switch using the pac auth list and pac auth select commands.

The `pac auth list` command lists all authentication profiles on your machine. The `pac auth select` command selects a different command.

`pac auth list` Example

To list all authentication profiles, use the following example:

PowerShell

```
pac auth list
```

Running the `pac auth list` command returns all authentication profiles:

PowerShell

```
Index Active Kind      Name Friendly Name                  Url
User                                      Cloud  Type
[1]   *      UNIVERSAL      Personal Productivity (Default)
https://x.crm.dynamics.com/         user@contoso.onmicrosoft.com
Public User
```

Select a different authentication profile using the index value.

PowerShell

```
pac auth select --index 2
```

Commands

Expand table

Command	Description
pac auth clear	Clear all authentication profiles stored on this computer
pac auth create	Create and store authentication profiles on this computer
pac auth delete	Delete a particular authentication profile
pac auth list	List the authentication profiles stored on this computer
pac auth name	Name or rename an existing authentication profile
pac auth select	Select which authentication profile should be active

Command	Description
pac auth update	Update name or target environment of an existing authentication profile
pac auth who	Display information about currently selected authentication profile

pac auth clear

Clear all authentication profiles stored on this computer

pac auth create

Create and store authentication profiles on this computer

Examples

The following examples show the use of the `pac auth create` command.

- Basic Create
- Environment Create
- Named Create
- Named Create with Service Principal

Basic Create

This example will prompt you for credentials and connects to the tenant the credentials belong to.

PowerShell

```
pac auth create
```

> ⓘ **Note**
>
> When running Power Platform CLI in Codespaces this will trigger a device code login.

Environment Create

This example will prompt you for credentials and connects to the Power Platform environment with environment id you specify.

```
PowerShell

pac auth create --environment < Your environment ID >
```

> **Tip**
>
> To find an environment id:
>
> 1. Open Power Platform admin center and select an the **Environment** you want to connect to. In the **Details** section, look for **Environment ID**

Named Create

This example authenticates with your tenant and gives it the name `MyOrg`.

```
PowerShell

pac auth create --name MyOrg
```

Named Create with Service Principal

This example authenticates with your tenant, gives it the name `MyOrg-SPN` and uses a service principal to do so. Don't forget to add the service principal to your environment.

```
PowerShell

pac auth create --name MyOrg-SPN --applicationId 00000000-0000-0000-0000-
000000000000 --clientSecret $clientSecret --tenant 00000000-0000-0000-0000-
000000000000
```

Optional Parameters for auth create

`--applicationId` `-id`

Optional: The application ID to authenticate with.

`--azureDevOpsFederated` `-adof`

(Preview) Use Azure DevOps Federation for Service Principal Auth; requires --tenant and --applicationId arguments

This parameter requires no value. It's a switch.

`--certificateDiskPath` `-cdp`

Optional: The certificate disk path to authenticate with

`--certificatePassword` `-cp`

Optional: The certificate password to authenticate with

`--clientSecret` `-cs`

Optional: The client secret to authenticate with

`--cloud` `-ci`

Optional: The cloud instance to authenticate with

Use one of these values:

- `Public`
- `UsGov`
- `UsGovHigh`
- `UsGovDod`
- `China`

`--deviceCode` `-dc`

Use the Microsoft Entra ID Device Code flow for interactive sign-in.

This parameter requires no value. It's a switch.

`--environment` `-env`

Default environment (ID, url, unique name, or partial name).

`--githubFederated` `-ghf`

(Preview) Use GitHub Federation for Service Principal Auth; requires --tenant and --applicationId arguments

This parameter requires no value. It's a switch.

`--kind` `-k`

Deprecated: This parameter is ignored.

`--managedIdentity` `-mi`

Use Azure Managed Identity.

This parameter requires no value. It's a switch.

`--name` `-n`

The name you want to give to this authentication profile (maximum 30 characters).

Note: The length of the name should be between 1 and 30

`--password` `-p`

Optional: The password to authenticate with

`--tenant` `-t`

Tenant ID if using application ID/client secret or application ID/client certificate.

`--url` `-u`

Deprecated: Use `--environment` instead.

`--username` `-un`

Optional: The username to authenticate with; shows Microsoft Entra ID dialog if not specified.

Remarks

For the `cloud` parameter, the values to use map to these other common acronyms or names:

- GCC = `UsGov`
- GCC High = `UsGovHigh`
- DoD = `UsGovDod`

pac auth delete

Delete a particular authentication profile

Example

PowerShell

```
pac auth delete --index 2
```

Optional Parameters for auth delete

`--index` `-i`

The index of the profile to be deleted

Note: The value must be an integer with minimum value of 1.

`--name` `-n`

The name of the profile to be active

Note: The length of the name should be between 1 and 30

pac auth list

List the authentication profiles stored on this computer

pac auth name

Name or rename an existing authentication profile

Example

The following example shows how to rename an auth profile. In this case, the profile with index 1 will be renamed to 'Contoso Dev'.

```
PowerShell
```

```
pac auth name --index 1 --name "Contoso Dev"
```

Required Parameters for auth name

`--index` `-i`

The index of the profile to be named/renamed

Note: The value must be an integer with minimum value of 1.

`--name` `-n`

The name you want to give to this authentication profile (maximum 30 characters).

Note: The length of the name should be between 1 and 30

pac auth select

Select which authentication profile should be active

Example

```
PowerShell
```

```
pac auth select --index 2
```

Optional Parameters for auth select

`--index` `-i`

The index of the profile to be active.

Note: The value must be an integer with minimum value of 1.

`--name` `-n`

The name of the profile to be active

Note: The length of the name should be between 1 and 30

pac auth update

Update name or target environment of an existing authentication profile

Examples

The following examples show the use of the pac auth update command.

- Update auth profile with environment URL
- Update auth profile with environment ID

Update auth profile with environment URL

This example shows how to update the auth profile with index 1 with the name `Contoso Dev` and the environment URL `https://contosodev.crm.dynamics.com`.

PowerShell

```
pac auth update --index 1 --name "Contoso Dev" --environment
"https://contosodev.crm.dynamics.com"
```

Update auth profile with environment ID

This example shows how to update the auth profile with index 1 with the environment ID `00000000-0000-0000-0000-000000000000`.

PowerShell

```
pac auth update --index 1 --environment 00000000-0000-0000-0000-000000000000
```

Required Parameters for auth update

`--index` `-i`

The index of the profile to be named/renamed

Note: The value must be an integer with minimum value of 1.

Optional Parameters for auth update

`--environment` `-env`

Default environment (ID, url, unique name, or partial name).

`--name` `-n`

The name you want to give to this authentication profile (maximum 30 characters).

pac auth who

Display information about currently selected authentication profile

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac canvas

Article • 05/13/2024

Operating with Power Apps .msapp files

[This article is pre-release documentation and is subject to change.]

> **Important**
>
> - The `pack` and `unpack` commands are in public preview.
> - Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.
> - The `create` command is generally available.

Commands

Expand table

Command	Description
pac canvas create	Generate a canvas app from a custom connector
pac canvas download	Download canvas app as .msapp file
pac canvas list	List canvas apps
pac canvas pack	(Preview) Pack sources into an msapp file
pac canvas unpack	(Preview) Extract an msapp file into sources

pac canvas create

Generate a canvas app from a custom connector

This command will generate an `.msapp` canvas app based on the OpenAPI definition of an existing custom connector. The app will contain screens, control layout, and Power Fx code generated based on how the actions of the custom connector are defined, with a predefined look and feel. The app can be used to directly exercise the actions defined in the custom connector.

Makers can import the generated `.msapp` into Power Apps Studio to play the app, or to do edits and make further customizations. Import the generated app to Power Apps Studio by navigating to **File** > **Open** > **Browse** within Studio.

Examples

The following examples show the use of the `pac canvas create` command.

- Create a canvas app based on an Open API Definition by display name
- Create a canvas app based on an Open API Definition by id

Create a canvas app based on an Open API Definition by display name

This example creates a canvas app (as a *.msapp file) based on an Open API Definition by using the display name of the connector.

PowerShell

```
pac canvas create --msapp HelloWorld.msapp --connector-display-name "My
Custom Connector"
```

Create a canvas app based on an Open API Definition by id

This example creates a canvas app (as a *.msapp file) based on an Open API Definition by using the ID of the connector.

PowerShell

```
pac canvas create --msapp HelloWorld.msapp --connector-id 00000000-0000-
0000-0000-000000000000
```

Either `connector-display-name` or `connector-id` are required.

Required Parameters for canvas create

`--msapp`

Path to .msapp file to be generated

Optional Parameters for canvas create

`--connector-display-name`

The display name of the Connector to generate the Power App from.

`--connector-id`

The ID of the Connector to generate the Power App from.

`--environment`

The target Environment ID or URL. The default value is the environment of your currently active Dataverse Auth Profile.

Remarks

Considerations and limitations:

- The custom connector referenced with the `connector-display-name` or `connector-id` parameters must be part of a Solution.
- After importing the app to Studio, there will be no association to the custom connector. The maker must manually add a reference to the custom connector from which the app was generated before the app can be played. See Add data connections to canvas apps for steps. Until this is done, you may see formula errors in the app which will resolve once the connector reference is added. The name of the custom connector you add to the app must match the display name of the custom connector referenced in the `canvas create` command.
- The app will contain one screen for each `POST` and `GET` action that the connector defines.
  - Other verbs available in custom connectors are not supported.
- The controls rendered on each action's screen are dynamically generated based on how the input and output types of each action are defined. Certain types of input and output defined in the custom connector are not supported by `canvas create`. The command will generate warnings in the console when these are encountered. Specific cases:
  - Arrays of objects and arrays of arrays are not supported as inputs
  - Controls are not generated for objects or nested arrays beyond a depth of 20
  - "File" and "binary format" types are not supported for input or output.

pac canvas download

Download canvas app as .msapp file

Required Parameters for canvas download

`--name` `-n`

Canvas app exact, partial name, or App ID

Optional Parameters for canvas download

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--extract-to-directory` `-d`

Directory name to extract canvas app to

`--file-name` `-f`

File name (usually with .msapp extension). If not provided file is saved in current directory with 'appname'.msapp

`--overwrite` `-o`

Allow file overwrite

This parameter requires no value. It's a switch.

pac canvas list

List canvas apps

Optional Parameters for canvas list

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac canvas pack

(Preview) Pack sources into an msapp file

Example

PowerShell

```
pac canvas pack --sources MyHelloWorldFiles --msapp HelloWorld.msapp
```

Required Parameters for canvas pack

`--msapp`

Path to .msapp file

`--sources`

Directory to sources to be packed

Remarks

Creates an `.msapp` file from the previously unpacked source files. The result can be opened in Power Apps Studio by navigating to **File** > **Open** > **Browse**.

After being unpacked, the source files can be edited and managed with external tools such as Visual Studio Code and GitHub.

pac canvas unpack

(Preview) Extract an msapp file into sources

Example

PowerShell

```
pac canvas unpack --msapp HelloWorld.msapp --sources MyHelloWorldFiles
```

PowerShell

```
pac canvas unpack --msapp HelloWorld.msapp
```

Unpacks to default `HelloWorld_src` directory.

Required Parameters for canvas unpack

`--msapp`

Path to .msapp file

Optional Parameters for canvas unpack

`--sources`

Directory to sources to be unpacked

Remarks

Unpacks the `.msapp` source file.

Download the `.msapp` file from Power Apps Studio by navigating to **File** > **Save as** > **This computer**.

If the **sources** parameter is not specified, a directory with the same name and location as the `.msapp` file is used with `_src` suffix.

Folder structure

Unpack and pack properties use the following folder structure:

- **\src** - Control and component files. This contains the sources.
  - ***.fx.yaml*** - The formulas extracted from the `control.json` file.

> ⓘ Note

> This is the place to edit your formulas.

  - ***CanvasManifest.json*** - A manifest file that contains the information normally present in the header, properties, and publishInfo.
  - ****.json*** - The raw `control.json` file.
  - ***\EditorState*.editorstate.json*** - Cached information for Power Apps Studio to use.
- **\DataSources** - All the data sources used by the app.
- **\Connections** - Connection instances saved with the app and used when reloading into Power Apps Studio.
- **\Assets** - Media files embedded in the app.
- **\pkgs** - A downloaded copy of external references, such as templates, API definition files, and component libraries. These are similar to NuGet/NPM references.
- **\other** - All miscellaneous files needed to re-create the `.msapp`.
  - ***entropy.json*** - Volatile elements (like timestamps) are extracted to this file. This helps reduce noisy differences in other files while ensuring that we can still round-trip.
  - Holds other files from the msapp, such as what's in \references.

File format

The `.fx.yaml` files use a subset of YAML. Similar to Excel, all expressions should begin with an equal sign `=`. More information: Power Fx YAML Formula Grammar

Merging changes with Power Apps Studio

When merging changes that are made in two different Power Apps Studio sessions:

- Ensure that all the control names are unique. For example, inserting a button in two different sessions can result in two `Button1` controls. We recommend that you name the controls soon after you create them. The tool doesn't accept two controls with the same name.
- For these files, merge them as you normally do:
  - \src*.fx.yaml
- If there are conflicts or errors, you can delete these files:
  - \src\editorstate*.json - These files contain optional information in Power Apps Studio.
  - \other\entropy.json
- For any conflicts in these files, it's ok to accept the latest version:
  - \checksum.json

- If there are any merge conflicts under these paths, it isn't safe to merge. Let us know if this happens often; we'll work on restructuring the file format to avoid conflicts.
  - \Connections*
  - \DataSources*
  - \pkgs*
  - CanvasManifest.json

Open source

The canvas commands in Microsoft Power Platform CLI are open source. Discuss improvements, raise issues, and access the code from Power Apps language tooling repository.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac catalog

Article • 05/14/2024

Commands for working with Catalog in Power Platform

Commands

Expand table

Command	Description
pac catalog create-submission	Create catalog submission document. Note: This command will be removed in a future release.
pac catalog install	Install a catalog item to the target environment.
pac catalog list	List all published catalog items from the current Dataverse organization.
pac catalog status	Get status of the catalog install/submit request.
pac catalog submit	Submit catalog approval request.
pac catalog update	Update settings for the catalog.

pac catalog create-submission

Create catalog submission document. Note: This command will be removed in a future release.

Optional Parameters for catalog create-submission

`--path` `-p`

Path to catalog submission document

pac catalog install

Install a catalog item to the target environment.

Required Parameters for catalog install

`--catalog-item-id` `-cid`

Catalog item to be installed on the target environment.

Optional Parameters for catalog install

`--environment` `-env`

Url or ID of the environment that has catalog installed. When not specified, the active organization selected for the current auth profile will be used.

`--poll-status` `-ps`

Poll to check status of your request

This parameter requires no value. It's a switch.

`--settings` `-s`

Runtime Package Settings for the installation framework to execute. The format of the string must be `key=value|key=value`.

Note: The format of the string must be `key=value|key=value`.

`--target-env` `-te`

Url or ID of the target environment for catalog item installation

`--target-url` `-tu`

Deprecated: Use `--target-env` instead.

`--target-version` `-tv`

Target version to install. If left empty, the published version is selected.

pac catalog list

List all published catalog items from the current Dataverse organization.

Optional Parameters for catalog list

`--catalog-item-id` `-cid`

Catalog item ID to search for. When catalog item ID is used, catalog item name is ignored.

`--catalog-item-name` `-n`

Catalog item name to search for.

`--environment` `-env`

Url or ID of the environment that has catalog installed. When not specified, the active organization selected for the current auth profile will be used.

`--include-active` `-ia`

Include active items.

This parameter requires no value. It's a switch.

`--json`

Returns the output of the command as a JSON formatted string.

pac catalog status

Get status of the catalog install/submit request.

Required Parameters for catalog status

`--tracking-id` `-id`

Request tracking ID.

`--type` `-t`

Request type

Use one of these values:

- `Install`
- `Submit`

Optional Parameters for catalog status

`--environment` `-env`

Url or ID of the environment that has catalog installed. When not specified, the active organization selected for the current auth profile will be used.

pac catalog submit

Submit catalog approval request.

Required Parameters for catalog submit

`--path` `-p`

Path to catalog submission document

Optional Parameters for catalog submit

`--environment` `-env`

Url or ID of the environment that has catalog installed. When not specified, the active organization selected for the current auth profile will be used.

`--package-zip` `-pz`

Path to package zip file.

`--poll-status` `-ps`

Poll to check status of your request

This parameter requires no value. It's a switch.

`--solution-zip` `-sz`

Path to solution zip file.

pac catalog update

Update settings for the catalog.

Required Parameters for catalog update

`--path` `-p`

Path to catalog settings document

Optional Parameters for catalog update

`--environment` `-env`

Url or ID of the environment that has catalog installed. When not specified, the active organization selected for the current auth profile will be used.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac connection

Article • 05/13/2024

Commands for working with Dataverse connection.

Commands

Expand table

Command	Description
pac connection create	Create new Dataverse connection.
pac connection delete	Delete Dataverse connection.
pac connection list	List all Connections.
pac connection update	Update Dataverse connection.

pac connection create

Create new Dataverse connection.

Required Parameters for connection create

`--application-id` `-a`

Application Id

`--client-secret` `-cs`

Client Secret

`--name` `-n`

Connection name.

`--tenant-id` `-t`

Tenant Id

Optional Parameters for connection create

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac connection delete

Delete Dataverse connection.

Required Parameters for connection delete

`--connection-id` `-id`

Connection Id.

Optional Parameters for connection delete

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac connection list

List all Connections.

Optional Parameters for connection list

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac connection update

Update Dataverse connection.

Required Parameters for connection update

`--application-id` `-a`

Application Id

`--client-secret` `-cs`

Client Secret

`--connection-id` `-id`

Connection Id.

`--tenant-id` `-t`

Tenant Id

Optional Parameters for connection update

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac connector

Article • 05/13/2024

Commands for working with Power Platform Connectors

Commands

Expand table

Command	Description
pac connector create	Creates a new row in the Connector table in Dataverse.
pac connector download	Download a Connector's OpenApiDefinition and API Properties file
pac connector init	Initializes a new API Properties file for a Connector.
pac connector list	List the Connectors registered in Dataverse.
pac connector update	Updates a Connector Entity in Dataverse.

pac connector create

Creates a new row in the Connector table in Dataverse.

Examples

The following examples show the use of the `pac connector create` command.

- Basic connector creation in current environment
- Basic connector creation in specified environment

Basic connector creation in current environment

This example creates a connector in the environment of your currently active auth profile.

PowerShell

```
pac connector create `
  --api-definition-file ./apiDefinition.json `
  --api-properties-file ./apiProperties.json
```

Basic connector creation in specified environment

This example creates a connector in the specified environment.

PowerShell

```
pac connector create `
  --api-definition-file ./apiDefinition.json `
  --api-properties-file ./apiProperties.json
  --environment 00000000-0000-0000-0000-000000000000
```

Optional Parameters for connector create

`--api-definition-file` `-df`

The filename and path to read the Connector's OpenApiDefinition.

`--api-properties-file` `-pf`

The filename and path to read the Connector's API Properties file.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--icon-file` `-if`

The filename and path to and Icon .png file.

`--script-file` `-sf`

The filename and path to a Script .csx file.

`--settings-file`

The filename and path Connector Settings file.

`--solution-unique-name` `-sol`

The unique name of the solution to add the connector to

pac connector download

Download a Connector's OpenApiDefinition and API Properties file

Examples

The following examples show the use of the `pac connector download` command.

- Basic connector download
- Basic connector download from specified environment
- Basic connector download from specified environment to the specified directory

Basic connector download

This example downloads the specified connector to the current directory.

PowerShell

```
pac connector download `
  --connector-id 00000000-0000-0000-0000-000000000000
```

Basic connector download from specified environment

This example downloads the specified connector from the specified environment to the current directory.

PowerShell

```
pac connector download `
  --connector-id 00000000-0000-0000-0000-000000000000 `
  --environment 00000000-0000-0000-0000-000000000000
```

Basic connector download from specified environment to the specified directory

This example downloads the specified connector from the specified environment to the specified directory.

PowerShell

```
pac connector download `
  --connector-id 00000000-0000-0000-0000-000000000000 `
```

```
  --environment 00000000-0000-0000-0000-000000000000 `
  --outputDirectory "contoso_Connector"
```

Required Parameters for connector download

`--connector-id` `-id`

The ID of the Connector

Note: The Connector ID must be a valid Guid.

Optional Parameters for connector download

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--outputDirectory` `-o`

Output directory

pac connector init

Initializes a new API Properties file for a Connector.

Example

The following example shows the use of the `pac connector init` command.

Connector init with output directory and connection template for Microsoft Entra ID OAuth authentication

This example initializes a connector in the current directory.

PowerShell

```
pac connector init `
  --connection-template "OAuthAAD" `
  --generate-script-file "contoso_script_file" `
```

```
    --generate-settings-file "contoso_settings_file" `
    --outputDirectory "contoso_Connector"
```

Optional Parameters for connector init

`--connection-template` `-ct`

Generate an initial Connection Parameters set with the specified template.

Use one of these values:

- `NoAuth`
- `BasicAuth`
- `ApiKey`
- `OAuthGeneric`
- `OAuthAAD`

`--generate-script-file`

Generate an initial Connector Script file

This parameter requires no value. It's a switch.

`--generate-settings-file`

Generate an initial Connector Settings file

This parameter requires no value. It's a switch.

`--outputDirectory` `-o`

Output directory

pac connector list

List the Connectors registered in Dataverse.

Examples

The following examples show the use of the `pac connector list` command.

- List connectors in current environment
- List connectors in specified environment

List connectors in current environment

This example lists all the connectors in the environment of your currently active auth profile.

PowerShell

```
pac connector list
```

List connectors in specified environment

This example lists all the connectors in the specified environment.

PowerShell

```
pac connector list `
  --environment 00000000-0000-0000-0000-000000000000
```

Optional Parameters for connector list

`--environment` **`-env`**

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--json`

Returns the output of the command as a JSON formatted string.

pac connector update

Updates a Connector Entity in Dataverse.

Examples

The following examples show the use of the `pac connector update` command.

- Basic connector update in current environment
- Basic connector update in specified environment

Basic connector update in current environment

This example updates a connector in the environment of your currently active auth profile.

PowerShell

```
pac connector update `
  --api-definition-file ./apiDefinition.json
```

Basic connector update in specified environment

This example updates a connector in the specified environment.

PowerShell

```
pac connector update `
  --api-definition-file ./apiDefinition.json `
  --environment 00000000-0000-0000-0000-000000000000
```

Optional Parameters for connector update

`--api-definition-file` `-df`

The filename and path to read the Connector's OpenApiDefinition.

`--api-properties-file` `-pf`

The filename and path to read the Connector's API Properties file.

`--connector-id` `-id`

The ID of the Connector

Note: The Connector ID must be a valid Guid.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--icon-file` `-if`

The filename and path to and Icon .png file.

`--script-file` `-sf`

The filename and path to a Script .csx file.

`--settings-file`

The filename and path Connector Settings file.

`--solution-unique-name` `-sol`

The unique name of the solution to add the connector to

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac copilot

Article • 05/13/2024

Tools and utilities for copilot management

Commands

Expand table

Command	Description
pac copilot create	Creates a new bot using an existing template file as the reference.
pac copilot extract-template	Extracts a template file from an existing bot in an environment.
pac copilot extract-translation	Extracts file containing localized content for one or more bots.
pac copilot list	List of Virtual Agents in the current or target Dataverse environment.
pac copilot merge-translation	Merge files containing localized content for one or more bots.
pac copilot model list	AI Builder models in the current environment.
pac copilot model predict	Sends text or prompt to AI Model
pac copilot model prepare-fetch	Takes the FetchXML file from the AI Large Language Model (LLM) and prepares it for execution against the current environment.
pac copilot publish	Publish a Custom Copilot
pac copilot status	Poll the deployment status of a specified Virtual Agent in the current or target Dataverse environment.

pac copilot create

Creates a new bot using an existing template file as the reference.

Required Parameters for copilot create

`--displayName`

The display name of the new bot

`--schemaName`

The schema name (unique name) of the new bot.

`--solution` `-s`

Name of the solution.

`--templateFileName`

Source yaml file containing the bot template that was extracted using the extract-template command.

Optional Parameters for copilot create

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac copilot extract-template

Extracts a template file from an existing bot in an environment.

Example

This command extracts a template file from an existing copilot in an environment.

PowerShell

```
pac copilot extract-template `
   --environment 2e250e7a-5607-4fea-aa4e-1aeb7bf79118 `
   --bot 9ee3f7aa-ab79-4cf6-a726-d85c8c18cc3e `
   --templateFileName NewTestCopilot.yaml
```

The command returns output like the following for a copilot using the English and German languages:

Text

```
Connected as user@contoso.org

Loaded 34 components for bot 'New Test Copilot ' with id 9ee3f7aa-ab79-4cf6-
a726-d85c8c18cc3e. Primary language: English, supported languages: German

New Test Copilot -> C:\Users\user\NewTestCopilot.yaml
```

Required Parameters for copilot extract-template

`--bot` `-id`

The Chatbot ID or schema name (unique name found in Bot Details or file name in solution explorer).

`--templateFileName`

Location of the yaml file to write the bot template to.

Optional Parameters for copilot extract-template

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--overwrite` `-o`

Allow overwrite of the output data file if it already exists.

This parameter requires no value. It's a switch.

pac copilot extract-translation

Extracts file containing localized content for one or more bots.

Example

This command extracts a file containing localized content for one or more copilots.

PowerShell

```
pac copilot extract-translation `
   --environment 2e250e7a-5607-4fea-aa4e-1aeb7bf79118 `
   --bot 9ee3f7aa-ab79-4cf6-a726-d85c8c18cc3e `
   --all `
   --outdir . `
   --format json
```

The command returns output like the following:

Text

```
Connected as user@contoso.org

Loaded 32 components for bot 'New Test Copilot' with id 9ee3f7aa-ab79-4cf6-
a726-d85c8c18cc3e. Primary language: English, supported languages: German
```

Optional Parameters for copilot extract-translation

`--all` `-a`

Write localization files for all supported languages. By default, only the primary language is written.

This parameter requires no value. It's a switch.

`--bot` `-id`

The Chatbot ID or schema name (unique name found in Bot Details or file name in solution explorer).

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--format`

The file format in which to write localized files, either 'resx' or 'json'. The default is 'resx'.

`--outdir`

The output directory to write to.

`--overwrite` `-o`

Allow overwrite of the output data file if it already exists.

This parameter requires no value. It's a switch.

`--sourcedir` `-src`

Source solution directory. When specified, will ignore the connected environment when looking for bots and instead look for content in the solution folder.

pac copilot list

List of Virtual Agents in the current or target Dataverse environment.

Example

This command lists all copilots in the current or target Dataverse environment.

PowerShell

```
pac copilot list --environment 2e250e7a-5607-4fea-aa4e-1aeb7bf79118
```

The command returns output like the following:

Text

```
Name                           Bot ID
Component State Is Managed Solution ID                          Status Code
State Code
Ask Me Anytyhing Copilot       584e012c-dc95-46d6-af5a-1263b6a44342
Published       Unmanaged  285af946-6383-49a0-8615-4e2afafeaf38 Active
Provisioned
New Test Copilot               9ee3f7aa-ab79-4cf6-a726-d85c8c18cc3e
Published       Unmanaged  285af946-6383-49a0-8615-4e2afafeaf38 Active
Provisioned
```

Optional Parameters for copilot list

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac copilot merge-translation

Merge files containing localized content for one or more bots.

Example

This command merges files containing localized content for one or more copilots.

PowerShell

```
pac copilot merge-translation `
   --environment 2e250e7a-5607-4fea-aa4e-1aeb7bf79118 `
   --file ms_store_newTestCopilot.de-DE.json `
   --solution SolutionName
```

The command returns output like the following when updating the German language:

Text

```
Connected as user@contoso.org

Loading language German into bot 'ms_store_newTestCopilot' (New Test
Copilot) from file 'C:\Users\user\ms_store_newTestCopilot'. 0 key(s) were
missing, 0 value(s) were not used. Use the --verbose switch to get more
details.

Updated 16 out of 16 components.
```

Learn about how to manage errors from the merge-translation command

Optional Parameters for copilot merge-translation

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--file` `-f`

The list of files that contain translations. Glob patterns are supported.

`--solution` `-s`

Name of the solution.

`--sourcedir` `-src`

Source solution directory. When specified, will ignore the connected environment when looking for bots and instead look for content in the solution folder.

`--verbose`

Output more diagnostic information during data import/export

This parameter requires no value. It's a switch.

`--whatif`

Does not execute the command, but outputs the details of what would happen.

This parameter requires no value. It's a switch.

merge-translation errors

Based on the example above, if one error exists in the file used to update the copilot, the output looks like:

Text

```
Connected as user@contoso.org

Loading language German into bot 'ms_store_newTestCopilot' (New Test
Copilot) from file 'C:\Users\user\ms_store_newTestCopilot'. 1 key(s) were
missing, 1 value(s) were not used. Use the --verbose switch to get more
details.

Updated 16 out of 16 components.
```

The `--verbose` flag adds details about the error:

Text

```
Connected as user@contoso.org

Missing translation key:
'dialog(ms_store_newTestCopilot.topic.Lesson2)'.'trigger(main)'.'action(LbWw
```

```
pD)'.Entity.Definition.'closedListItem(Redmond)'.DisplayName.

Unused translation:
'dialog(ms_store_newTestCopilot.topic.Lesson2)'.'trigger(main)'.'action(LbWw
pD)'.Entity.Definition.'closedListItem(Redmont)'.DisplayName, value: Redmond

Loading language German into bot 'ms_store_newTestCopilot' (New Test
Copilot) from file 'C:\Users\user\ms_store_newTestCopilot'. 1 key(s) were
missing, 1 value(s) were not used. Use the --verbose switch to get more
details.

Updated 16 out of 16 components.
```

pac copilot model list

AI Builder models in the current environment.

Example

This command lists all AI Builder models.

PowerShell

```
pac copilot model list --environment 2e250e7a-5607-4fea-aa4e-1aeb7bf79118
```

The command returns output like the following:

Text

```
Id                                   State    Name
32a9e265-1149-4155-af54-d2856d2b83f5 Active   Document Processing
2023/09/20, 12:21:40
2bcd7b94-50bc-4767-af4a-367c63fb5487 Inactive AI Classify
4e72b59a-17d6-451e-8657-89fbdec56d7a Inactive AI Extract
572f57a7-7a8f-49fc-adb4-331e02c509a6 Inactive AI Reply
b9b636cf-9748-47a7-b617-6df5f00f5151 Inactive AI Sentiment
c076eac8-f218-4feb-8ad1-7ee4fb039419 Inactive AI Summarize
a0440df3-2656-e911-8194-000d3a6cd5a5 Inactive BusinessCard model
62d1e848-5ca7-490a-94bf-79baabe85ef4 Inactive CategoryClassification model
4da7ec17-5c26-4fd2-9ddb-be4f7eda21a9 Inactive
DataGenieEmailAddressValidation
51f4da11-5702-401d-b53a-9638744e8ac9 Inactive Document Layout Analysis Model
d1bb8d57-24ab-3b36-9592-bd792e153b79 Inactive DVCopilotQueryModel
3fbd4e5c-32bc-40fc-acce-59c2821cf113 Inactive Empty Dynamic Prompt
8c281981-c5d6-484f-bac7-4924ddd0b8ae Inactive EntityExtraction model
a1afa5d4-7a44-4c31-9cd2-e852a78431fa Inactive GptPromptEngineering model
2c444168-f8b1-4c6a-9313-2d9c03be6fea Inactive Id Scanning Model
77365cfa-7021-4cb7-a9b2-dc9823cde772 Inactive ImageDescription model
aef1bdd2-2a74-4f74-b4eb-9dfa22e35ded Inactive Invoice Model
```

```
5ed4d0fd-e9d4-4026-b09b-71f83ea90c60 Inactive Invoice Processing Model
c8425db7-c5a7-4226-b38e-c93a044c0fe1 Inactive KeyPhraseExtraction model
17a6f893-5b0b-4867-8fac-fb2eda9080b2 Inactive LanguageDetection model
046ab801-2756-e911-8194-000d3a6cd5a5 Inactive ObjectDetectionProposal model
892d3698-ba03-3d15-8e9e-843ca4ac5e7d Inactive PowerAppsAppCopilotModel
baa44529-bebe-49e6-837a-80bee63b0d2c Inactive Receipt Scanning Model
f1c549c2-a97e-47a5-b612-c5c2bab0f163 Inactive SentimentAnalysis model
7f8a7856-003a-3662-9871-0000d7674433 Inactive SimsModel
86419a67-205a-454f-b6fc-601394f2786d Inactive TextRecognition model
6225038a-8b5a-4913-bfd2-d8236f4102ba Inactive TextTranslation model
02e1fca3-232a-4f58-8c93-bdd8c9cd6de9 Inactive TextTranslationInternal model
```

Optional Parameters for copilot model list

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac copilot model predict

Sends text or prompt to AI Model

Optional Parameters for copilot model predict

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--model-id` `-id`

AI Builder model Id

`--model-name` `-n`

Full or partial AI Model name

`--prompt` `-p`

Prompt to send to AI Model

`--text` `-t`

Text to send to AI Model

pac copilot model prepare-fetch

Takes the FetchXML file from the AI Large Language Model (LLM) and prepares it for execution against the current environment.

Required Parameters for copilot model prepare-fetch

`--inputFile` `-i`

Input FetchXML file that usually comes from AI LLM.

`--outputFile` `-o`

Output FetchXML file that is ready to execute against the current environment.

Optional Parameters for copilot model prepare-fetch

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac copilot publish

Publish a Custom Copilot

Required Parameters for copilot publish

`--bot` `-id`

The Chatbot ID or schema name (unique name found in Bot Details or file name in solution explorer).

Optional Parameters for copilot publish

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac copilot status

Poll the deployment status of a specified Virtual Agent in the current or target Dataverse environment.

Example

This command polls the deployment status of a specified copilot in the current or target Dataverse environment.

PowerShell

```
pac copilot status `
   --environment 2e250e7a-5607-4fea-aa4e-1aeb7bf79118 `
   --bot-id 9ee3f7aa-ab79-4cf6-a726-d85c8c18cc3e
```

The command returns output like the following:

Text

```
Connected as user@contoso.org
Connected to... Contoso Organization
Virtual Agent New Bot German with ID 9ee3f7aa-ab79-4cf6-a726-d85c8c18cc3e
has been provisioned.
```

Required Parameters for copilot status

`--bot-id` `-id`

The ID of a Virtual Agent (Chatbot).

Optional Parameters for copilot status

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac data

Article • 05/13/2024

Import and export data from Dataverse.

> **Note**
>
> These commands are only available for the .NET Full Framework version of the PAC CLI.

Commands

Expand table

Command	Description
pac data export	Export data
pac data import	Import data

pac data export

Export data

Required Parameters for data export

`--schemaFile` `-sf`

Schema file name. It can be created using the Configuration Migration Tool.

Optional Parameters for data export

`--dataFile` `-df`

File name for data zip file. Default data.zip

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--overwrite` `-o`

Allow overwrite of the output data file if it already exists.

This parameter requires no value. It's a switch.

`--verbose` `-v`

Output more diagnostic information during data import/export

This parameter requires no value. It's a switch.

pac data import

Import data

Required Parameters for data import

`--data` `-d`

Zip file or directory name with data for import.

Optional Parameters for data import

`--connection-count` `-c`

Number of parallel connections to use during import. Default: 5

`--dataDirectory` `-dd`

Deprecated: This parameter is ignored.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--verbose` `-v`

Output more diagnostic information during data import/export

This parameter requires no value. It's a switch.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac env

Article • 05/14/2024

Work with your Dataverse organization.

> **Note**
>
> With pac cli version 1.32, the `pac org` command was changed to `pac env`. `org` will continue to work, but we recommend using `env` going forward.

Commands

Expand table

Command	Description
pac env fetch	Performs FetchXML query against Dataverse
pac env list	Lists all of the Dataverse organizations the user has access to
pac env list-settings	List environment settings
pac env select	Select default organization for current authentication profile.
pac env update-settings	Update environment settings
pac env who	Displays information about the current Dataverse organization.

pac env fetch

Performs FetchXML query against Dataverse

Optional Parameters for env fetch

`--environment` **`-env`**

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--xml` **`-x`**

FetchXML query to run

`--xmlFile` `-xf`

File with FetchXML query to run

pac env list

Lists all of the Dataverse organizations the user has access to

Example

PowerShell

```
pac env list --filter myenv
```

The filter parameter is a regular expression based filter.

Optional Parameters for env list

`--filter` `-f`

Show only environments containing filter criteria.

pac env list-settings

List environment settings

Optional Parameters for env list-settings

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--filter` `-f`

Show only settings containing filter criteria

pac env select

Select default organization for current authentication profile.

Examples

The following examples show the use of the pac env select command.

- Select env by using environment URL
- Select env by using environment ID

Select env by using environment URL

This example shows how to select an env by using an environment URL.

PowerShell

```
pac env select --environment "https://contosotest.crm.dynamics.com"
```

Select env by using environment ID

This example shows how to select an env by using an environment ID.

PowerShell

```
pac env select --environment 00000000-0000-0000-0000-000000000000
```

Optional Parameters for env select

`--environment` **`-env`**

Default environment (ID, url, unique name, or partial name).

pac env update-settings

Update environment settings

Required Parameters for env update-settings

`--name` `-n`

Name of the setting to update

Optional Parameters for env update-settings

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--value` `-v`

Value to which setting should be updated to

pac env who

Displays information about the current Dataverse organization.

Optional Parameters for env who

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--json`

Returns the output of the command as a JSON formatted string.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac help

Article • 05/13/2024

Show help for the Microsoft Power Platform CLI.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac modelbuilder

Article • 05/13/2024

Code Generator for Dataverse APIs and Tables

Use the pac modelbuilder build command to generate early-bound .NET classes for Dataverse tables, custom messages, and a class that is derived from the OrganizationServiceContext Class. Learn more about using this command to generate early bound classes

The class derived from OrganizationServiceContext:

- Maintains state on the client to support features such as change management.
- Implements the System.Linq.IQueryable Interface and a .NET Language-Integrated Query (LINQ) query provider so you can write LINQ queries using Dataverse data.

For more information about the capabilities that this code generation tool enables:

- Late-bound and early-bound programming using the Organization service
- Use OrganizationServiceContext
- Build queries with LINQ

> **Note**
>
> The `pac modelbuilder build` command replaces the `CrmSvcUtil.exe` distributed with the Microsoft.CrmSdk.CoreTools NuGet package.

Commands

Expand table

Command	Description
pac modelbuilder build	Builds a code model for Dataverse APIs and Tables

pac modelbuilder build

Builds a code model for Dataverse APIs and Tables

> **Note**

Before you can use the `build` command you must first connect to Dataverse using the pac auth create command. If you have multiple connections, use the pac auth select to choose the Dataverse environment you want to generate code for.

Example

The following example shows how to use the `build` command with a command prompt.

dos

```
pac modelbuilder build ^
  --entitynamesfilter account;contact ^
  --generatesdkmessages ^
  --messagenamesfilter examp_* ^
  --emitfieldsclasses ^
  --emitVirtualAttributes ^
  --namespace MyApps.Model ^
  --outdirectory c:\src\MyApps\Model ^
  --writesettingsTemplateFile ^
  --serviceContextName OrgContext
```

And the same command using PowerShell:

PowerShell

```
pac modelbuilder build `
  --entitynamesfilter 'account;contact' `
  --generatesdkmessages `
  --messagenamesfilter 'examp_*' `
  --emitfieldsclasses `
  --emitVirtualAttributes `
  --namespace 'MyApps.Model' `
  --outdirectory 'c:\src\MyApps\Model' `
  --writesettingsTemplateFile `
  --serviceContextName 'OrgContext'
```

ⓘ Important

You need to surround any string parameters with single quotes when using PowerShell.

The result of this command is that the following files are written to the `c:\src\MyApps\Model` folder.

C:\src\MyApps\Model
|---**Entities**
| |--account.cs
| |--contact.cs
|---**OptionSets**
| |--addresstypes.cs
|---**Messages**
| |--examp_myapi.cs
|---**EntityOptionSetEnum.cs**
|---**builderSettings.json**
|---**OrgContext.cs**

builderSettings.json contains the parameters you specified for the command. You can use it to quickly regenerate the files as things change. The following example shows using the generated `buildersettings.json` file from the first command using the settingsTemplateFile:

PowerShell

```
pac modelbuilder build `
  --outdirectory c:\src\MyApps\Model `
  --settingsTemplateFile c:\src\MyApps\Model\builderSettings.json
```

You can also choose to create a `builderSettings.json` file and use that instead of passing all the parameters to the command. The following is an example that is equivalent to the first example above:

JSON

```
{
  "suppressINotifyPattern": false,
  "suppressGeneratedCodeAttribute": false,
  "language": "CS",
  "namespace": "MyApps.Model",
  "serviceContextName": "OrgContext",
  "generateSdkMessages": true,
  "generateGlobalOptionSets": false,
  "emitFieldsClasses": true,
  "entityTypesFolder": "Entities",
  "messagesTypesFolder": "Messages",
  "optionSetsTypesFolder": "OptionSets",
  "entityNamesFilter": [
    "account",
    "contact"
  ],
  "messageNamesFilter": [
    "examp_*"
```

```
  ],
  "emitEntityETC": false,
  "emitVirtualAttributes": true
}
```

If you pass parameters to the command while using the settingsTemplateFile parameter, the parameters passed to the command will override those set in the `builderSettings.json` file.

You can't use the settingsTemplateFile parameter and the writesettingsTemplateFile parameter at the same time.

Required Parameters for modelbuilder build

`--outdirectory` `-o`

Write directory for entity, message, and optionset files.

Optional Parameters for modelbuilder build

`--emitentityetc` `-etc`

When set, includes the entity ETC (entity type code) in the generated code.

This parameter requires no value. It's a switch.

`--emitfieldsclasses` `-efc`

Generate a constants structure that contains all of the field names by entity at the time of code generation.

This parameter requires no value. It's a switch.

`--emitvirtualattributes` `-eva`

When set, includes supporting name attributes for lookups that enable filtering on the primary name attribute values of lookup attributes.

This parameter requires no value. It's a switch.

`--entitynamesfilter` `-enf`

Filters the list of entities are retrieved when reading data from Dataverse. Passed in as a semicolon separated list. Using the form <entitylogicalname>;<entitylogicalname>

`--entitytypesfolder` `-etf`

Folder name that contains entities. The default name is 'Entities'.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--generateGlobalOptionSets` `-go`

Emit all Global OptionSets. Note: If an entity contains a reference to a global optionset, it is emitted even if this switch is not present.

`--generatesdkmessages` `-a`

When set, emits Sdk message classes as part of code generation.

This parameter requires no value. It's a switch.

`--language` `-l`

The language to use for the generated proxy code. This value can be either 'CS' or 'VB'. The default language is 'CS'.

`--logLevel` `-ll`

Log level. The default value is 'Off'.

Use one of these values:

- `Off`
- `Critical`
- `Error`
- `Warning`
- `Information`
- `Verbose`
- `ActivityTracing`

- `All`

`--messagenamesfilter` `-mnf`

Filters the list of messages that are retrieved when reading data from Dataverse. Passed in as a semicolon separated list, required messages (Create, Update, Delete, Retrieve, RetrieveMultiple, Associate and Disassociate) are always included. Use a trailing or leading asterisk (*) with the names of the messages to allow for all messages starting with or ending with a string. Using the form <messagename>;<messagename>.

`--messagestypesfolder` `-mtf`

Folder name that contains messages. The default name is 'Messages'.

`--namespace` `-n`

The namespace for the generated code. The default namespace is the global namespace.

`--optionsetstypesfolder` `-otf`

Folder name that contains option sets. The default name is 'OptionSets'.

`--serviceContextName` `-sctx`

The name for the generated service context. If a value is passed in, it's used for the Service Context. If not, no Service Context is generated.

`--settingsTemplateFile` `-stf`

Contains Settings to be used for this run of the Dataverse Model Builder, overrides any duplicate parameters on command line. Can't be set when --writesettingstemplate is used.

`--suppressGeneratedCodeAttribute` `-sgca`

When set, this suppress all generated objects being tagged with the code generation engine and version

This parameter requires no value. It's a switch.

`--suppressINotifyPattern`

When enabled, doesn't write the INotify wrappers for properties and classes.

`--writesettingsTemplateFile` `-wstf`

When set, writes a settings file out to the output directory with the current passed settings or default settings.

Remarks

The following are recommendations for using the `pac modelbuilder build` command.

Set the `entitynamesfilter` and `messagenamesfilter` parameters

> ⊗ **Caution**
>
> We **strongly recommend** that you use the **entitynamesfilter** and **messagenamesfilter** parameters to limit the generated files to those you will use in your project. Otherwise, the build command will attempt to generate code for all tables and messages from Dataverse. This will take a significant amount of time to process.

Classes for the messages found in the Microsoft.Crm.Sdk.Messages and Microsoft.Xrm.Sdk.Messages namespace are not generated using this command. You should only include messages not found there in the `messagenamesfilter` parameter, such as custom actions.

Set `suppressINotifyPattern` if you aren't building a WPF application

The `INotify` wrappers that are suppressed by this command are used for databinding scenarios with WPF applications. If you aren't building a WPF application with the generated code, you don't need them. Use the suppressINotifyPattern parameter to suppress them.

Include `serviceContextName` when generating message classes

If you are generating message classes, you should always include the serviceContextName parameter to generate a OrganizationServiceContext, even if you

aren't using it. The generated message classes require a property set in this file. Learn more about the error that occurs if you don't set this.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac package

Article • 05/13/2024

Commands for working with Dataverse package projects

Commands

Expand table

Command	Description
pac package add-external-package	Adds a package that is external to the Dataverse solution system to a Package Deployer Package project.
pac package add-reference	Adds reference to Dataverse solution project
pac package add-solution	Adds a prebuilt Dataverse solution file to a Package Deployer Package project.
pac package deploy	Deploys package to Dataverse
pac package init	Initializes a directory with a new Dataverse package project
pac package show	Shows details of Dataverse package

pac package add-external-package

Adds a package that is external to the Dataverse solution system to a Package Deployer Package project.

Required Parameters for package add-external-package

`--package-type` `-t`

The type of the package being added. For example: 'xpp' for FnO packages.

`--path` `-p`

Path to the external package

Optional Parameters for package add-external-package

`--import-order`

A whole number that indicates the order to insert this item into the final ImportConfig.xml file at build time. Negative numbers are inserted before existing elements. Positive numbers are added after existing elements.

`--skip-validation` `-sv`

Adds the item to the project file even if the file doesn't exist or appears to be invalid. Note: Using this doesn't affect any validation performed by MSBuild.

This parameter requires no value. It's a switch.

pac package add-reference

Adds reference to Dataverse solution project

Example

PowerShell

```
pac package add-reference --path c:\Users\Downloads\SampleSolution
```

Required Parameters for package add-reference

`--path` `-p`

The path to the referenced Dataverse solution project

Optional Parameters for package add-reference

`--dependency-overrides`

A semicolon delimited list of overrides. This value overrides any dependency information encoded in the solution's metadata. Each override should be in the format: `<uniquename>:<minVersion>:<maxVersion>`. Where minVersion and maxVersion are optional but should be in .NET version format syntax.

Note: Use a semicolon delimited list of dependency overrides of the format <uniquename>:<minVersion>:<maxVersion>.

`--import-mode`

Explicitly specifies the required mode when importing this solution.

Use one of these values:

- `sync`
- `async`

`--import-order`

A whole number that indicates the order to insert this item into the final ImportConfig.xml file at build time. Negative numbers are inserted before existing elements. Positive numbers are added after existing elements.

`--missing-dependency-behavior`

Specifies the behavior on import when a dependency of this solution is missing from the target environment.

Use one of these values:

- `skip`
- `fault`

`--overwrite-unmanaged-customizations`

Explicitly indicates whether to overwrite unmanaged customizations when this solution is imported.

Use one of these values:

- `true`
- `false`

`--publish-workflows-activate-plugins`

Explicitly indicates whether to publish the workflows and activate plug-ins when this solution is imported.

Use one of these values:

- `true`
- `false`

pac package add-solution

Adds a prebuilt Dataverse solution file to a Package Deployer Package project.

Required Parameters for package add-solution

`--path` `-p`

Path to the Dataverse solution file. The file must be a compressed ZIP file.

Optional Parameters for package add-solution

`--dependency-overrides`

A semicolon delimited list of overrides. This value overrides any dependency information encoded in the solution's metadata. Each override should be in the format: `<uniquename>:<minVersion>:<maxVersion>`. Where minVersion and maxVersion are optional but should be in .NET version format syntax.

Note: Use a semicolon delimited list of dependency overrides of the format <uniquename>:<minVersion>:<maxVersion>.

`--import-mode`

Explicitly specifies the required mode when importing this solution.

Use one of these values:

- `sync`
- `async`

`--import-order`

A whole number that indicates the order to insert this item into the final ImportConfig.xml file at build time. Negative numbers are inserted before existing

elements. Positive numbers are added after existing elements.

`--missing-dependency-behavior`

Specifies the behavior on import when a dependency of this solution is missing from the target environment.

Use one of these values:

- `skip`
- `fault`

`--overwrite-unmanaged-customizations`

Explicitly indicates whether to overwrite unmanaged customizations when this solution is imported.

Use one of these values:

- `true`
- `false`

`--publish-workflows-activate-plugins`

Explicitly indicates whether to publish the workflows and activate plug-ins when this solution is imported.

Use one of these values:

- `true`
- `false`

`--skip-validation` `-sv`

Adds the item to the project file even if the file doesn't exist or appears to be invalid. Note: Using this doesn't affect any validation performed by MSBuild.

This parameter requires no value. It's a switch.

pac package deploy

Deploys package to Dataverse

> ⓘ Note
>
> This command is only available for the .NET Full Framework version of the PAC CLI.

Example

PowerShell

```
pac package deploy --logFile c:\samplelogdata --package c:\samplepackage
```

Optional Parameters for package deploy

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--logConsole` `-c`

Output log to console

This parameter requires no value. It's a switch.

`--logFile` `-lf`

Log file path

`--package` `-p`

Path to a package dll or zip file with a package.

`--settings` `-s`

Runtime Package Settings that are passed to the package that is being deployed. The format of the string must be `key=value|key=value`.

Note: The format of the string must be `key=value|key=value`.

`--solution` `-sz`

Path to the Dataverse solution file. The file must be a compressed ZIP or CAB file.

`--verbose` `-vdbg`

Emit verbose logs to the log outputs.

This parameter requires no value. It's a switch.

Remarks

You can use both `logFile` and `logConsole` parameters together, or use one parameter or the other.

pac package init

Initializes a directory with a new Dataverse package project

Example

PowerShell

```
pac package init --outputdirectory c:\samplepackage
```

Optional Parameters for package init

`--outputDirectory` `-o`

Output directory

`--package-name`

Sets the default name of the package. Applies to the generation of ImportExtension.GetNameOfImport.

pac package show

Shows details of Dataverse package

> **Note**

This command is only available for the .NET Full Framework version of the PAC CLI.

Example

```
PowerShell
pac package show c:\samplepackage.dll
```

Required Parameters for package show

`--package` `-p`

Path to a package dll or zip file with a package.

Optional Parameters for package show

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--logFile` `-lf`

Log file path

`--verbose` `-vdbg`

Emit verbose logs to the log outputs.

This parameter requires no value. It's a switch.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac pages

Article • 05/14/2024

Commands for working with Power Pages website.

> **Note**
>
> With pac cli version 1.32, the `pac powerpages` command was changed to `pac pages`. With pac cli version 1.27, the `pac paportal` command was changed to `pac powerpages`. Both `powerpages` and `paportal` will continue to work, but we recommend using `pages` going forward.

See the following resources about using Power Platform CLI with Power Pages:

- Portals support for Microsoft Power Platform CLI
- Tutorial: Use Microsoft Power Platform CLI with portals
- Use the Visual Studio Code extension

Commands

Expand table

Command	Description
pac pages bootstrap-migrate	Migrates HTML code from bootstrap V3 to V5.
pac pages download	Download Power Pages website content from the current Dataverse environment.
pac pages list	List all Power Pages websites from the current Dataverse environment.
pac pages migrate-datamodel	Manage data model migration for your Power Pages website.
pac pages upload	Upload Power Pages website content to current Dataverse environment.

pac pages bootstrap-migrate

Migrates HTML code from bootstrap V3 to V5.

Required Parameters for pages bootstrap-migrate

`--path` `-p`

Path of website content.

pac pages download

Download Power Pages website content from the current Dataverse environment.

Example

PowerShell

```
pac pages download --path "C:\portals" --webSiteId f88b70cc-580b-4f1a-87c3-41debefeb902
```

Required Parameters for pages download

`--path` `-p`

Path where the Power Pages website content is downloaded

`--webSiteId` `-id`

Power Pages website ID to download

Optional Parameters for pages download

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--excludeEntities` `-xe`

Comma separated list of entity logical names to exclude downloading

`--includeEntities` `-ie`

Download only the entities specified for this argument using comma separated entity logical names.

`--modelVersion` `-mv`

Power Pages website data model version to download.

`--overwrite` `-o`

Power Pages website content to overwrite

This parameter requires no value. It's a switch.

Remarks

More information: Download portals content

pac pages list

List all Power Pages websites from the current Dataverse environment.

Optional Parameters for pages list

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--verbose` `-v`

Enables verbose mode to provide more details.

This parameter requires no value. It's a switch.

Remarks

More information: List available portals

pac pages migrate-datamodel

Manage data model migration for your Power Pages website.

Required Parameters for pages migrate-datamodel

`--webSiteId` `-id`

Power Pages website ID to update the site.

Optional Parameters for pages migrate-datamodel

`--checkMigrationStatus` `-s`

To check the status of the site with migration in progress.

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--mode` `-m`

Choose from configurationData / configurationDataRefrences / all - based on your requirement.

`--portalId` `-pid`

Portal ID for the website under migration.

`--revertToStandardDataModel` `-r`

Revert site from enhanced to standard data model.

This parameter requires no value. It's a switch.

`--siteCustomizationReportPath` `-p`

Local path to store the site customization report.

`--updateDataModelVersion` `-u`

Update data model version for the site once the data is migrated successfully.

This parameter requires no value. It's a switch.

pac pages upload

Upload Power Pages website content to current Dataverse environment.

Example

PowerShell

```
pac pages upload --path "C:\portals\starter-portal"
```

Required Parameters for pages upload

`--path` `-p`

Path from where the Power Pages website content is uploaded.

Optional Parameters for pages upload

`--deploymentProfile` `-dp`

Deployment profile name to be used. Defaults to 'default'

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--forceUploadAll` `-f`

Upload all Power Pages website content to the current Dataverse environment.

This parameter requires no value. It's a switch.

`--modelVersion` `-mv`

Power Pages website data model version to upload.

Remarks

More information:

- Upload the changes
- Use deployment profile

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac pcf

Article • 05/13/2024

Commands for working with Power Apps component framework projects

Create and build a code component explains how to use these commands.

Commands

Expand table

Command	Description
pac pcf init	Initializes a directory with a new Power Apps component framework project
pac pcf push	Import the Power Apps component framework project into the current Dataverse organization
pac pcf version	Patch version for controls

pac pcf init

Initializes a directory with a new Power Apps component framework project

Example

PowerShell

```
pac pcf init --namespace SampleNameSpace --name SampleComponent --template
field
```

Optional Parameters for pcf init

`--framework` `-fw`

The rendering framework for control. The default value is 'none', which means HTML.

Use one of these values:

- `none`

- `react`

`--name` `-n`

The name for the component.

Note: Only characters within the ranges [A - Z], [a - z] or [0 - 9] are allowed. The first character may not be a number.

`--namespace` `-ns`

The namespace for the component.

Note: Only characters within the ranges [A - Z], [a - z], [0 - 9], or '.' are allowed. The first and last character may not be the '.' character. Consecutive '.' characters are not allowed. Numbers are not allowed as the first character or immediately after a period.

`--outputDirectory` `-o`

Output directory

`--run-npm-install` `-npm`

Auto run 'npm install' after the control is created. The default value is 'false'.

This parameter requires no value. It's a switch.

`--template` `-t`

Choose a template for the component.

Use one of these values:

- `field`
- `dataset`

Remarks

Use `--framework react` with React controls & platform libraries (Preview).

pac pcf push

Import the Power Apps component framework project into the current Dataverse organization

Example

PowerShell

```
pac pcf push --publisher-prefix dev
```

Optional Parameters for pcf push

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--force-import` `-f`

Deprecated: This parameter is ignored.

`--incremental` `-inc`

Pushes only files which are different using entity updates.

This parameter requires no value. It's a switch.

`--interactive` `-i`

Indicates that actions in the build are allowed to interact with the user. Don't use this argument in an automated scenario where interactivity is not expected.

This parameter requires no value. It's a switch.

`--publisher-prefix` `-pp`

Customization prefix value for the Dataverse solution publisher

Note: The prefix must be 2 to 8 characters long, can only consist of alpha-numerics, must start with a letter, and can't start with 'mscrm'.

`--solution-unique-name`

The unique name of the solution to add the component to.

`--verbosity` `-v`

Verbosity level for MSBuild when building the temporary solution wrapper.

Use one of these values:

- `minimal`
- `normal`
- `detailed`
- `diagnostic`

pac pcf version

Patch version for controls

Example

PowerShell

```
pac pcf version --patchversion 1.0.0.0 --path
c:\Users\Downloads\SampleComponent --allmanifests
```

PowerShell

```
pac pcf version --strategy gittags
```

Optional Parameters for pcf version

`--allmanifests` `-a`

Updates patch version for all 'ControlManifest.xml' files

This parameter requires no value. It's a switch.

`--filename` `-fn`

Tracker CSV file name to be used when using filetracking as a strategy. The default value is 'ControlsStateVersionInfo.csv'.

`--patchversion` `-pv`

Patch version for controls

Note: The value must be an integer with minimum value of 0.

`--path` `-p`

Absolute/Relative path of the 'ControlManifest.xml' for updating.

`--strategy` `-s`

Updates patch version for 'ControlManifest.xml' files using specified strategy. If using gittags, set a personal access token in the following environment variable "PacCli.PAT"

Use one of these values:

- `None`
- `GitTags`
- `FileTracking`
- `Manifest`

`--updatetarget` `-ut`

Specify which target manifest needs to be updated.

Use one of these values:

- `build`
- `project`

Remarks

`--patchversion` will only take value of the third part of the version tuple: `Major.Minor.Patch`.

For `--strategy` the available values have these meanings:

Expand table

Value	Description
`gittags`	Use Git tags to decide whether a particular component's patch version needs to be updated.
`filetracking`	Use a .csv file to decide whether a particular component's patch version needs to be updated.
`manifest`	Increments the patch version by 1 for all the components.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

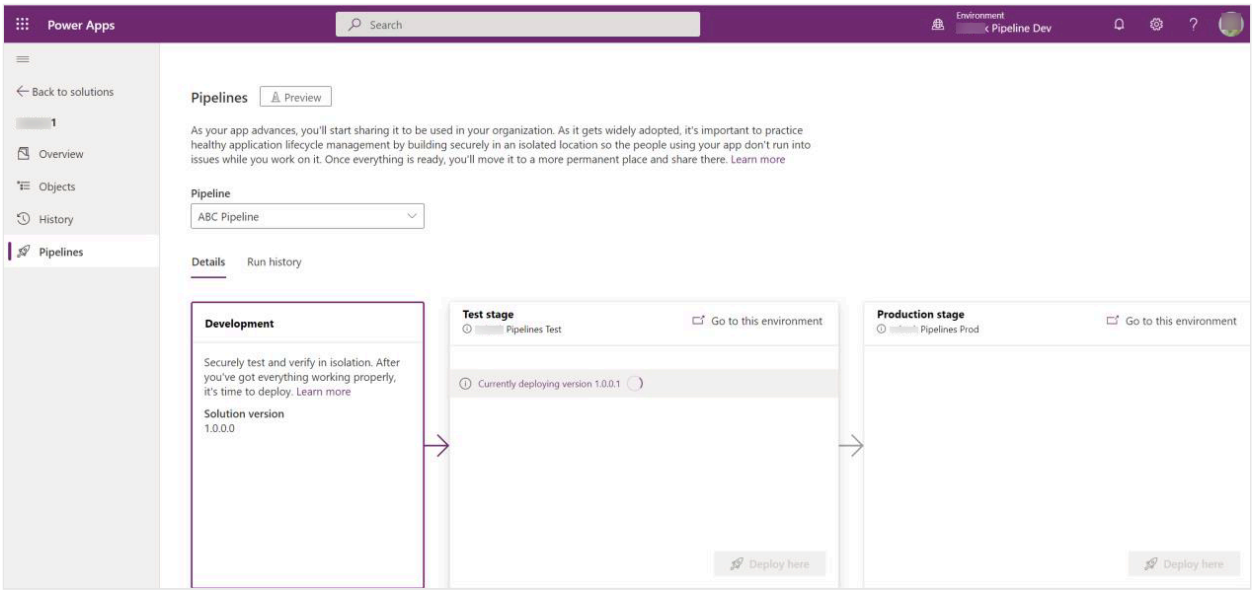
Provide product feedback

pac pipeline

Article • 05/13/2024

Work with Pipelines

Pipelines in Power Platform aim to democratize application lifecycle management (ALM) for Power Platform and Dynamics 365 customers by bringing ALM automation and continuous integration and continuous delivery (CI/CD) capabilities into the service in a manner that's more approachable for all makers, admins, and developers.



Pipelines significantly reduce the effort and domain knowledge previously required to realize the ROI from adopting healthy, automated ALM processes within your team or organization.

- Admins easily configure automated deployment pipelines in minutes rather than days or weeks.
- Makers have an intuitive user experience for easily deploying their solutions.

Learn more about pipelines in Power Platform

Commands

Expand table

Command	Description
pac pipeline deploy	Start pipeline deployment.
pac pipeline list	List Pipelines

pac pipeline deploy

Start pipeline deployment.

Use `pac pipeline deploy` to deploy/start an existing pipeline in the Power Platform environment you are connected to, or use `--environment` / `-env` to select an environment in the same tenant.

Required Parameters for pipeline deploy

`--currentVersion` `-cv`

Current solution version

`--newVersion` `-nv`

New solution version

`--solutionName` `-sn`

Solution Name

`--stageId` `-sid`

Deployment Stage ID

Optional Parameters for pipeline deploy

`--environment` `-env`

Deployment Environment

`--wait` `-w`

Wait until the deployment finishes.

This parameter requires no value. It's a switch.

pac pipeline list

List Pipelines

Use `pac pipeline list` to list existing pipelines in the Power Platform environment you are connected to, or use `--environment` / `-env` to select an environment in the same tenant.

Optional Parameters for pipeline list

`--environment` `-env`

Deployment Environment

`--pipeline` `-p`

Pipeline name or ID for which show list of details such as deployment stages

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac plugin

Article • 05/13/2024

Commands for working with Dataverse plug-in class library

Learn how to create and register a plug-in package using PAC CLI

Commands

Expand table

Command	Description
pac plugin init	Initializes a directory with a new Dataverse plug-in class library.
pac plugin push	Import plug-in into Dataverse.

pac plugin init

Initializes a directory with a new Dataverse plug-in class library.

Optional Parameters for plugin init

`--author` `-a`

One or more authors of the Dataverse Plug-in Package.

`--outputDirectory` `-o`

Output directory

`--signing-key-file-path` `-sk`

Relative path to the Dataverse plug-in assembly originator key file for signing.

`--skip-signing` `-ss`

Skip plug-in assembly signing that gives the plug-in assembly a strong name. The default value is 'false'.

This parameter requires no value. It's a switch.

pac plugin push

Import plug-in into Dataverse.

Required Parameters for plugin push

`--pluginId` `-id`

ID of plug-in assembly or plug-in package

Optional Parameters for plugin push

`--configuration` `-c`

Build configuration. The default value is: 'Debug'.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--pluginFile` `-pf`

File name of plug-in assembly or plug-in package

`--type` `-t`

Type of item if not specified explicitly through --pluginFile. The default value is: 'Nuget'.

Use one of these values:

- `Nuget`
- `Assembly`

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac power-fx

Article • 05/13/2024

(Preview) Commands for working with Power Fx

Use these commands to interact with data in Dataverse using a command line or script. Reduce the time and complexity of your Dataverse operations by using Power Fx.

You can use the formulas listed in the Formula reference - Power Platform CLI.

Commands

Expand table

Command	Description
pac power-fx repl	(Preview) Launch interactive Power Fx Read-Eval-Print Loop
pac power-fx run	(Preview) Run a file of Power Fx instructions

pac power-fx repl

(Preview) Launch interactive Power Fx Read-Eval-Print Loop

> **Note**
>
> If you installed Power Platform CLI using Windows MSI, there is a known issue with pac power-fx repl where the shell may stop responding when using this command.

Optional Parameters for power-fx repl

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

Remarks

This command provides a shell to use formulas listed in Formula reference - Power Platform CLI.

Start the Power Fx repl by running the following command:

```PowerShell
pac power-fx repl
```

After that, you can run Power Fx commands.

Examples

The following examples show the use of the `pac power-fx repl` command.

- Add rows
- Help
- Exit

Add rows

```Power Apps
Collect(Contacts, { firstname: "Patti", lastname: "Fernandez" })
```

The Power Fx repl responds with the following:

```text
{contactid:GUID("118d7b47-8f80-ee11-8179-0022482a40c7"), fullname:"Patti Fernandez", ...}
```

Help

Run the following command to get help about all the commands that are available in Power Fx repl:

```Power Apps
Help()
```

The Power Fx repl responds with the following output:

```text
  Abs             Acos            Acot            AddColumns      And
  Asin            AsType          Atan            Atan2           Average
  Blank           Boolean         Char            Clear
ClearCollect
  Coalesce        Collect         ColorFade       ColorValue      Concat
  Concatenate     Cos             Cot             Count           CountA
  CountIf         CountRows       Date            DateAdd         DateDiff
  DateTime        DateTimeValue   DateValue       Day             Dec2Hex
  Decimal         Degrees         Distinct        DropColumns     EDate
  EncodeUrl       EndsWith        EOMonth         Error           Exp
  Filter          Find            First           FirstN          Float
  ForAll          GUID            Help            Hex2Dec         Hour
  If              IfError         Index           Int             IsBlank
  IsBlankOrError  IsEmpty         IsError         IsMatch         IsNumeric
  IsToday         Language        Last            LastN           Left
  Len             Ln              Log             LookUp          Lower
  Match           MatchAll        Max             Mid             Min
  Minute          Mod             Month           Not             Notify
  Now             OptionSetInfo   Or              ParseJSON       Patch
  Pi              PlainText       Power           Proper          Radians
  Rand            RandBetween     Refresh         Remove          Replace
  RGBA            Right           Round           RoundDown       RoundUp
  Second          Sequence        Set             Shuffle         Sin
  Sort            Split           Sqrt            StartsWith      StdevP
  Substitute      Sum             Switch          Table           Tan
  Text            Time            TimeValue       TimeZoneOffset  Today
  Trim            TrimEnds        Trunc           Upper           Value
  VarP            Weekday         With            Year
>>
```

> **Note**
>
> These commands will change over time to include more commands. It should be synchronized with the documented currently available commands.

Exit

Use `Exit()` to exit the `repl` command.

pac power-fx run

(Preview) Run a file of Power Fx instructions

Examples

The following examples show the use of the `pac power-fx run` command.

- Create Dataverse records
- Query a Dataverse table
- Use PowerFx functions

> ⓘ Note
>
> In each of these examples, you need to:
>
> 1. Create a file named `test-pfx.txt` with Power Fx expressions in a folder on your computer, like `c:\test`.
> 2. Open a Windows PowerShell window, navigate to the `test` folder and use the `run` command specifying the file that contains the commands: `pac power-fx run --file test-pfx.txt`.
>
> To use Dataverse tables, you have to create and select an auth profile. See pac auth create and pac auth select.
>
> If you get an error check Troubleshooting.

Create Dataverse records

1. Add the following text to `test-pfx.txt` and save the file.

   Power Apps

   ```
   Collect(Contacts, { firstname: "Megan", lastname: "Bowen" })
   Collect(Contacts, { firstname: "Garth", lastname: "Forth" })
   Collect(Contacts, { firstname: "Adele", lastname: "Vance" })
   Collect(Contacts, { firstname: "Patti", lastname: "Fernandez" })
   ```

2. Run this command:

   PowerShell

   ```
   PS C:\test> pac power-fx run --file test-pfx.txt
   ```

   You can expect output like the following:

   PowerShell

```
{contactid:GUID("80eb0c02-9180-ee11-8179-0022482a40c7"),
fullname:"Megan Bowen", ...}
{contactid:GUID("81eb0c02-9180-ee11-8179-0022482a40c7"),
fullname:"Garth Forth", ...}
{contactid:GUID("82eb0c02-9180-ee11-8179-0022482a40c7"),
fullname:"Adele Vance", ...}
{contactid:GUID("83eb0c02-9180-ee11-8179-0022482a40c7"),
fullname:"Patti Fernandez", ...}
```

Query a Dataverse table

1. Add this text to `test-pfx.txt` and save the file.

   Power Apps

   ```
   FirstN(Contacts, 5)
   ```

   This command retrieves the first five contacts from Dataverse.

2. Run this command:

   PowerShell

   ```
   PS C:\test> pac power-fx run --file test-pfx.txt
   ```

   You can expect output like the following:

   PowerShell

   ```
   Connected to... Your Organization
   Connected as you@yourorg.onmicrosoft.com

   contactid                                    fullname           ...
   ============================================ ================== =====
   GUID("eeb27495-d127-ed11-9db1-00224804f8e2")   Yvonne McKay (sample)
   GUID("cba0ed9e-d127-ed11-9db1-00224804f8e2")   Susanna Stubberod
   (sample)
   GUID("cea0ed9e-d127-ed11-9db1-00224804f8e2")   Nancy Anderson (sample)
   GUID("6cc7483c-7a77-ee11-8179-00224809f375")   Maria Campbell (sample)
   GUID("49b0be2e-d01c-ed11-b83e-000d3a572421")   Sidney Higa (sample)
   PS C:\test>
   ```

Use PowerFx functions

1. Add the following text to `test-pfx.txt` and save the file.

```
Power Apps
```

```
Set(x, 1)
Result = If( Mid( "asdf",x,1 ) = "a", "X", "Y" )
```

This example uses the Mid, If, and Set Power Fx functions.

2. Run the following command:

```
PowerShell
```

```
PS C:\test> pac power-fx run --file test-pfx.txt
```

You can expect output like the following:

```
PowerShell
```

```
PS C:\test> pac power-fx run --file test-pfx.txt
x: 1
true
Result: "X"
PS C:\test>
```

3. Add the `--echo` parameter and you can expect this output:

```
PowerShell
```

```
PS C:\test> pac power-fx run --file test-pfx.txt --echo
Set(x, 1)

x: 1
true
Result = If( Mid( "asdf",x,1 ) = "a", "X", "Y" )

Result: "X"
PS C:\test>
```

Required Parameters for power-fx run

`--file` **`-f`**

Path of file to execute

Optional Parameters for power-fx run

`--echo` `-e`

True if input commands should echo back to output

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

Troubleshooting

The following are steps to troubleshoot errors you might encounter when using `pac power-fx` commands.

Error when no Dataverse authentication profile exists

You can get an error like this when you don't have a universal authentication profile associated with a Dataverse environment.

PowerShell

```
PS C:\test> pac power-fx run --file test-pfx.txt
Error 8-16: Name isn't valid. 'Contacts' isn't recognized.
Error 18-59: The specified column 'firstname' does not exist.
Error 0-60: The function 'Collect' has some invalid arguments.
Error 8-16: Name isn't valid. 'Contacts' isn't recognized.
Error 18-59: The specified column 'firstname' does not exist.
Error 0-60: The function 'Collect' has some invalid arguments.
Error 8-16: Name isn't valid. 'Contacts' isn't recognized.
Error 18-59: The specified column 'firstname' does not exist.
Error 0-60: The function 'Collect' has some invalid arguments.
Error 8-16: Name isn't valid. 'Contacts' isn't recognized.
Error 18-63: The specified column 'firstname' does not exist.
Error 0-64: The function 'Collect' has some invalid arguments.
```

To resolve errors like these, create a universal authentication profile associated with a Dataverse environment.

1. Run the command pac auth list.

   You might see a result like the following that includes a `UNIVERSAL` authentication profile with no Url indicating it's associated with a Dataverse environment.

```
PowerShell

Index Active Kind      Name Friendly Name Url User                Cloud
Type
[1]   *      UNIVERSAL                       you@yourcompany.com
Public OperatingSystem
```

This isn't sufficient because no Dataverse environment is specified.

2. Use pac auth create to create a `UNIVERSAL` authentication profile connected to a Dataverse environment.

> **Note**
>
> Use the `--environment` parameter using the environmentid value. Find your environment ID.
>
> Don't use the `--url` parameter because this will create a `DATAVERSE` profile rather than a `UNIVERSAL` profile.

For example:

```
PowerShell

PS C:\test> pac auth create --environment f2cef599-b112-4803-bb5d-
090077017685 --name ConnectionName
'you@yourcompany.onmicrosoft.com' authenticated successfully.
Validating connection...
Default organization: YourOrganization
Connected to... YourOrganization
Connected as you@yourcompany.onmicrosoft.com
Authentication profile created
   * UNIVERSAL ConnectionName
https://yourcompany.crm.dynamics.com/          :
you@yourcompany.onmicrosoft.com              Public

PS C:\test>
```

3. Run pac auth list again. You should see a result with `UNIVERSAL` connection profile with a URL for your Dataverse environment.

```
PowerShell

PS C:\test> pac auth list
Index Active Kind      Name           Friendly Name        Url
User                  Cloud  Type
[1]   *      UNIVERSAL
```

```
you@yourcompany.com   Public OperatingSystem
[2]   *      UNIVERSAL ConnectionName YourOrganization
https://yourcompany.crm.dynamics.com/ you@yourcompany.com   Public User

PS C:\test>
```

4. Select the `UNIVERSAL` environment to make it the default profile using the pac auth select command.

PowerShell

```
PS C:\test> pac auth select --index 2
New default profile:
   * UNIVERSAL ConnectionName
https://yourcompany.crm.dynamics.com/          : you@yourcompany.com
Public

PS C:\test>
```

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac solution

Article • 05/13/2024

Commands for working with Dataverse solution projects

Commands

Expand table

Command	Description
pac solution add-license	Add license and plan info to the solution.
pac solution add-reference	Adds a reference from the project in the current directory to the project at 'path'
pac solution add-solution-component	Add one or more solution components to the target unmanaged solution in Dataverse.
pac solution check	Upload a Dataverse solution project to run against the Power Apps Checker service.
pac solution clone	Create a solution project based on an existing solution in your organization.
pac solution create-settings	Create a settings file from solution zip or solution folder.
pac solution delete	Delete a solution from Dataverse in the current environment.
pac solution export	Export a solution from Dataverse.
pac solution import	Import the solution into Dataverse.
pac solution init	Initializes a directory with a new Dataverse solution project
pac solution list	List all Solutions from the current Dataverse organization
pac solution online-version	Sets version for solution loaded in Dataverse.
pac solution pack	Package solution components on local filesystem into solution.zip (SolutionPackager).
pac solution publish	Publishes all customizations
pac solution sync	Sync the current Dataverse solution project to the current state of the solution in your organization.

Command	Description
pac solution unpack	Extract solution components from solution.zip onto local filesystem (SolutionPackager).
pac solution upgrade	Apply solution upgrade
pac solution version	Update build or revision version for the solution.

pac solution add-license

Add license and plan info to the solution.

Example

This example shows how to add licensing info to a solution.

PowerShell

```
pac solution add-license --planDefinitionFile ../ISV_Plan_Definition.csv --
planMappingFile ../ISV_Plan_Mapping.csv
```

Plan Definition File

Below you'll find an example of the plan definition file:

csv

```
ServiceID,Display name,More info URL
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.bronzeplan,Fabr
ikam Bronze Plan,http://www.microsoft.com
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.silverplan,Fabr
ikam Silver Plan,http://www.microsoft.com
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.goldplan,Fabrik
am Gold Plan,http://www.microsoft.com
```

Plan Mapping File

Below you'll find an example of the plan mapping file:

csv

```
Service ID,Component name
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.bronzeplan,crf3
```

```
6_BronzeApp
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.silverplan,crf3
6_BronzeApp
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.silverplan,crf3
6_SilverApp
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.goldplan,crf36_
BronzeApp
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.goldplan,crf36_
SilverApp
test_isvconnect1599092224747.d365_isvconnect_prod_licensable.goldplan,crf36_
GoldApp
```

Required Parameters for solution add-license

`--planDefinitionFile` `-pd`

License plan definition file in CSV format; expected columns: Service ID, Display name, More info URL.

`--planMappingFile` `-pm`

License plan mapping file in CSV format; expected columns: Service ID, Component name

pac solution add-reference

Adds a reference from the project in the current directory to the project at 'path'

Example

PowerShell

```
pac solution add-reference --path c:\Users\Downloads\SampleComponent
```

Required Parameters for solution add-reference

`--path` `-p`

The path to the referenced project

pac solution add-solution-component

Add one or more solution components to the target unmanaged solution in Dataverse.

Example

This example shows how to add a solution component to a solution. In this case, we're adding the contact table to the solution `SampleSolution`. A table is component type 1.

PowerShell

```
pac solution add-solution-component --solutionUniqueName SampleSolution --
component contact --componentType 1
```

Required Parameters for solution add-solution-component

`--component` `-c`

The schema name or ID of the component to add to the target solution.

`--componentType` `-ct`

The value that represents the solution component that you're adding.

`--solutionUniqueName` `-sn`

Name of the solution.

Optional Parameters for solution add-solution-component

`--AddRequiredComponents` `-arc`

Indicates whether other solution components required by the solution component that you're adding should also be added to the unmanaged solution.

This parameter requires no value. It's a switch.

`--environment` `-env`

Environment URL or ID of the target environment.

pac solution check

Upload a Dataverse solution project to run against the Power Apps Checker service.

Example

PowerShell

```
pac solution check --path c:\Users\Documents\Solution.zip --outputDirectory
c:\samplepackage --geo UnitedStates
```

Optional Parameters for solution check

`--customEndpoint` `-ce`

Specify a custom URL as the Power Apps Checker endpoint.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--excludedFiles` `-ef`

Exclude Files from the Analysis. Pass as comma-separated values

`--geo` `-g`

Which geographical instance of the Power Apps Checker service to use.

Use one of these values:

- `PreviewUnitedStates`
- `UnitedStates`
- `Europe`
- `Asia`

- `Australia`
- `Japan`
- `India`
- `Canada`
- `SouthAmerica`
- `UnitedKingdom`
- `France`
- `SouthAfrica`
- `Germany`
- `UnitedArabEmirates`
- `Switzerland`
- `Norway`
- `Singapore`
- `Korea`
- `Sweden`
- `USGovernment`
- `USGovernmentL4`
- `USGovernmentL5DoD`
- `China`

`--outputDirectory` `-o`

Output directory

`--path` `-p`

Path where one or more solution files to be checked exist. The path can contain glob/wildcard characters.

`--ruleLevelOverride` `-rl`

Path to a file containing a JSON array rules and levels to override. Accepted values for OverrideLevel are: Critical, High, Medium, Low, Informational. Example: [{"Id":"meta-remove-dup-reg","OverrideLevel":"Medium"},{"Id":"il-avoid-specialized-update-ops","OverrideLevel":"Medium"}]

`--ruleSet` `-rs`

Select a rule set that is executed as part of this build. Values: A valid Guid, "AppSource Certification", "Solution Checker" (default).

`--saveResults` `-sav`

Uses current environment to store solution analysis results that can be seen in Solution Health Hub App. By default, this argument is set to false.

This parameter requires no value. It's a switch.

`--solutionUrl` `-u`

SAS Uri pointing to solution.zip to be analyzed

pac solution clone

Create a solution project based on an existing solution in your organization.

Examples

The following examples show the use of the `pac solution clone` command.

- Basic clone
- Clone with general and auto numbering settings included
- Clone with canvas app unpack

Basic clone

This example clones the solution `sampleSolution` to the current directory.

PowerShell

```
pac solution clone --name sampleSolution
```

Clone with general and auto numbering settings included

This example clones the solution `sampleSolution` to the current directory with the general and auto numbering settings included.

PowerShell

```
pac solution clone --name sampleSolution --include general,autonumbering
```

Clone with canvas app unpack

This example clones the solution `sampleSolution` and unpacks the Canvas Apps in one go.

PowerShell

```
pac solution clone --name sampleSolution --processCanvasApps
```

Required Parameters for solution clone

`--name` `-n`

The name of the solution to be exported.

Optional Parameters for solution clone

`--async` `-a`

Exports the solution asynchronously.

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--include` `-i`

Which settings should be included in the solution being exported.

Use one or more of these values separated by commas:

- `autonumbering`
- `calendar`
- `customization`

- `emailtracking`
- `externalapplications`
- `general`
- `isvconfig`
- `marketing`
- `outlooksynchronization`
- `relationshiproles`
- `sales`

`--localize` `-loc`

Extract or merge all string resources into .resx files.

This parameter requires no value. It's a switch.

`--map` `-m`

The full path to a mapping xml file from which to read component folders to pack.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--outputDirectory` `-o`

Output directory

`--packagetype` `-p`

Specifies the extraction type for the solution. Can be: 'Unmanaged', 'Managed' or 'Both'. The default value is: 'Both'.

`--processCanvasApps` `-pca`

(Preview) Pack/unpack any Canvas apps (.msapp) while processing the solution. The default value is 'false'.

This parameter requires no value. It's a switch.

`--targetversion` `-v`

Deprecated: This parameter is ignored.

pac solution create-settings

Create a settings file from solution zip or solution folder.

Example

PowerShell

```
pac solution create-settings --solution-zip C:\SampleSolution.zip --
settings-file .\SampleDeploymentSettingsDev.json
```

Optional Parameters for solution create-settings

`--settings-file` `-s`

The .json file with the deployment settings for connection references and environment variables.

`--solution-folder` `-f`

Path to the local, unpacked solution folder: either the root of the 'Other/Solution.xml' file or a folder with a .cdsproj file.

`--solution-zip` `-z`

Path to solution zip file.

pac solution delete

Delete a solution from Dataverse in the current environment.

Example

PowerShell

```
pac solution delete --solution-name Samplesolution
```

Required Parameters for solution delete

`--solution-name` `-sn`

Name of the solution.

Optional Parameters for solution delete

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac solution export

Export a solution from Dataverse.

Example

PowerShell

```
pac solution export --path c:\Users\Documents\Solution.zip --name
SampleComponentSolution --managed true --include general
```

Required Parameters for solution export

`--name` `-n`

The name of the solution to be exported.

Optional Parameters for solution export

`--async` `-a`

Exports the solution asynchronously.

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--include` `-i`

Which settings should be included in the solution being exported.

Use one or more of these values separated by commas:

- `autonumbering`
- `calendar`
- `customization`
- `emailtracking`
- `externalapplications`
- `general`
- `isvconfig`
- `marketing`
- `outlooksynchronization`
- `relationshiproles`
- `sales`

`--managed` `-m`

Whether the solution should be exported as a managed solution.

This parameter requires no value. It's a switch.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--overwrite` `-ow`

The exported solution file can overwrite the solution zip file on the local file system.

This parameter requires no value. It's a switch.

`--path` `-p`

Path where the exported solution zip file is written.

`--targetversion` `-v`

Deprecated: This parameter is ignored.

pac solution import

Import the solution into Dataverse.

Example

PowerShell

```
pac solution import --path c:\Users\Documents\Solution.zip
```

Optional Parameters for solution import

`--activate-plugins` `-ap`

Activate plug-ins and workflows on the solution.

This parameter requires no value. It's a switch.

`--async` `-a`

Imports the solution asynchronously.

This parameter requires no value. It's a switch.

`--convert-to-managed` `-cm`

Convert to a managed solution.

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--force-overwrite` `-f`

Force an overwrite of unmanaged customizations

This parameter requires no value. It's a switch.

`--import-as-holding` `-h`

Import the solution as a holding solution.

This parameter requires no value. It's a switch.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--path` `-p`

Path to solution zip file. If not specified, assumes the current folder is a cdsproj project.

`--publish-changes` `-pc`

Publish your changes upon a successful import.

This parameter requires no value. It's a switch.

`--settings-file`

The .json file with the deployment settings for connection references and environment variables.

`--skip-dependency-check` `-s`

Skip dependency check against dependencies flagged as product update

This parameter requires no value. It's a switch.

`--skip-lower-version` `-slv`

Skip solution import if same or higher version is present in current environment.

This parameter requires no value. It's a switch.

`--stage-and-upgrade` `-up`

Import and upgrade the solution.

This parameter requires no value. It's a switch.

Remarks

You be connected to an environment using the pac auth command to use `pac solution import`.

convert-to-managed

The `convert-to-managed` parameter doesn't take an unmanaged solution and import it as managed. This parameter allows a managed solution that is being imported into an environment convert unmanaged components to managed.

If this flag isn't explicitly set, the solution system will fail the import request because managed layers can't go on top of unmanaged base components. This switch allows the solution import to succeed and the net result is that the inbound managed solution will be the base layer for each of these components – the components in the destination environment are converted from being an unmanaged component to being a managed component.

pac solution init

Initializes a directory with a new Dataverse solution project

Example

PowerShell

```
pac solution init --publisher-name developer --publisher-prefix dev
```

Required Parameters for solution init

`--publisher-name` `-pn`

Name of the Dataverse solution publisher

Note: Only characters within the ranges [A - Z], [a - z], [0 - 9], or _ are allowed. The first character may only be in the ranges [A - Z], [a - z], or _.

`--publisher-prefix` `-pp`

Customization prefix value for the Dataverse solution publisher

Note: The prefix must be 2 to 8 characters long, can only consist of alpha-numerics, must start with a letter, and can't start with 'mscrm'.

Optional Parameters for solution init

`--outputDirectory` `-o`

Output directory

pac solution list

List all Solutions from the current Dataverse organization

Example

PowerShell

```
pac solution list
```

Optional Parameters for solution list

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--includeSystemSolutions`

Include system solutions like those published by Microsoft

This parameter requires no value. It's a switch.

`--json`

Returns the output of the command as a JSON formatted string.

pac solution online-version

Sets version for solution loaded in Dataverse.

Example

PowerShell

```
pac solution online-version --solution-name Samplesolution --solution-
version 1.0.0.2
```

Required Parameters for solution online-version

`--solution-name` `-sn`

Name of the solution.

`--solution-version` `-sv`

Specify the solution version number.

Optional Parameters for solution online-version

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac solution pack

Package solution components on local filesystem into solution.zip (SolutionPackager).

Example

PowerShell

```
pac solution pack --zipfile C:\SampleSolution.zip --folder
.\SampleSolutionUnpacked\.
```

Required Parameters for solution pack

`--zipfile` `-z`

The full path to the solution ZIP file

Optional Parameters for solution pack

`--allowDelete` `-ad`

Dictates if delete operations may occur. The default value is 'false'.

This parameter requires no value. It's a switch.

`--allowWrite` `-aw`

Dictates if write operations may occur. The default value is 'false'.

This parameter requires no value. It's a switch.

`--clobber` `-c`

Enables that files marked read-only can be deleted or overwritten. The default value is 'false'.

This parameter requires no value. It's a switch.

`--disablePluginRemap` `-dpm`

Disabled plug-in fully qualified type name remapping. The default value is 'false'.

This parameter requires no value. It's a switch.

`--errorlevel` `-e`

Minimum logging level for log output [Verbose|Info|Warning|Error|Off]. The default value is 'Info'.

`--folder` `-f`

The path to the root folder on the local filesystem. When unpacking or extracting, this is written to. When packing this is read from.

`--localize` `-loc`

Extract or merge all string resources into .resx files.

This parameter requires no value. It's a switch.

`--log` `-l`

The path to the log file.

`--map` `-m`

The full path to a mapping xml file from which to read component folders to pack.

`--packagetype` `-p`

When unpacking or extracting, use to specify dual Managed and Unmanaged operation. When packing, use to specify Managed or Unmanaged from a previous unpack 'Both'. Can be: 'Unmanaged', 'Managed' or 'Both'. The default value is 'Unmanaged'.

`--processCanvasApps` `-pca`

(Preview) Pack/unpack any Canvas apps (.msapp) while processing the solution. The default value is 'false'.

This parameter requires no value. It's a switch.

`--singleComponent` `-sc`

Only perform action on a single component type [WebResource|Plugin|Workflow|None]. The default value is 'None'.

`--sourceLoc` `-src`

Generates a template resource file. Valid only on extract. Possible values are 'auto', or language code of the language you wish to export. You can use Language Code

Identifier (LCID), or International Organization for Standardization (ISO) language code formats. When present, this extracts the string resources from the given locale as a neutral .resx. If 'auto' or just the long or short form of the switch is specified, the base locale for the solution is used.

`--useLcid` `-lcid`

Use Language Code Identifier (LCID) values (1033) rather than International Organization for Standardization (ISO) codes (en-US) for language files.

This parameter requires no value. It's a switch.

`--useUnmanagedFileForMissingManaged` `-same`

Use the same XML source file when packaging for Managed and only Unmanaged XML file is found; applies to AppModuleSiteMap, AppModuleMap, FormXml files.

This parameter requires no value. It's a switch.

pac solution publish

Publishes all customizations

Example

PowerShell

```
pac solution publish
```

Optional Parameters for solution publish

`--async` `-a`

Publishes all customizations asynchronously

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--max-async-wait-time` **`-wt`**

Max asynchronous wait time in minutes. The default value is 60 minutes.

pac solution sync

Sync the current Dataverse solution project to the current state of the solution in your organization.

Examples

The following examples show the use of the `pac solution sync` command.

- Basic sync
- Sync with canvas app unpack

Basic sync

This example syncs the solution to the current directory.

PowerShell

```
pac solution sync
```

Sync with canvas app unpack

This example syncs the solution to the current directory and unpacks the canvas apps in one go.

PowerShell

```
pac solution sync --processCanvasApps
```

Optional Parameters for solution sync

`--async` **`-a`**

Exports the solution asynchronously.

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--include` `-i`

Which settings should be included in the solution being exported.

Use one or more of these values separated by commas:

- `autonumbering`
- `calendar`
- `customization`
- `emailtracking`
- `externalapplications`
- `general`
- `isvconfig`
- `marketing`
- `outlooksynchronization`
- `relationshiproles`
- `sales`

`--localize` `-loc`

Extract or merge all string resources into .resx files.

This parameter requires no value. It's a switch.

`--map` `-m`

The full path to a mapping xml file from which to read component folders to pack.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

`--packagetype` `-p`

When unpacking or extracting, use to specify dual Managed and Unmanaged operation. When packing, use to specify Managed or Unmanaged from a previous unpack 'Both'. Can be: 'Unmanaged', 'Managed' or 'Both'. The default value is: 'Both'.

`--processCanvasApps` `-pca`

(Preview) Pack/unpack any Canvas apps (.msapp) while processing the solution. The default value is 'false'.

This parameter requires no value. It's a switch.

`--solution-folder` `-f`

Path to the local, unpacked solution folder: either the root of the 'Other/Solution.xml' file or a folder with a .cdsproj file.

pac solution unpack

Extract solution components from solution.zip onto local filesystem (SolutionPackager).

Example

PowerShell

```
pac solution unpack --zipfile C:\SampleSolution.zip --folder
.\SampleSolutionUnpacked\.
```

Required Parameters for solution unpack

`--zipfile` `-z`

The full path to the solution ZIP file

Optional Parameters for solution unpack

`--allowDelete` `-ad`

Dictates if delete operations may occur. The default value is 'false'.

This parameter requires no value. It's a switch.

`--allowWrite` `-aw`

Dictates if write operations may occur. The default value is 'false'.

This parameter requires no value. It's a switch.

`--clobber` `-c`

Enables that files marked read-only can be deleted or overwritten. The default value is 'false'.

This parameter requires no value. It's a switch.

`--disablePluginRemap` `-dpm`

Disabled plug-in fully qualified type name remapping. The default value is 'false'.

This parameter requires no value. It's a switch.

`--errorlevel` `-e`

Minimum logging level for log output [Verbose|Info|Warning|Error|Off]. The default value is 'Info'.

`--folder` `-f`

The path to the root folder on the local filesystem. When unpacking or extracting, this is written to. When packing this is read from.

`--localize` `-loc`

Extract or merge all string resources into .resx files.

This parameter requires no value. It's a switch.

`--log` `-l`

The path to the log file.

`--map` `-m`

The full path to a mapping xml file from which to read component folders to pack.

`--packagetype` `-p`

When unpacking or extracting, use to specify dual Managed and Unmanaged operation. When packing, use to specify Managed or Unmanaged from a previous unpack 'Both'. Can be: 'Unmanaged', 'Managed' or 'Both'. The default value is 'Unmanaged'.

`--processCanvasApps` `-pca`

(Preview) Pack/unpack any Canvas apps (.msapp) while processing the solution. The default value is 'false'.

This parameter requires no value. It's a switch.

`--singleComponent` `-sc`

Only perform action on a single component type [WebResource|Plugin|Workflow|None]. The default value is 'None'.

`--sourceLoc` `-src`

Generates a template resource file. Valid only on extract. Possible values are 'auto', or language code of the language you wish to export. You can use Language Code Identifier (LCID), or International Organization for Standardization (ISO) language code formats. When present, this extracts the string resources from the given locale as a neutral .resx. If 'auto' or just the long or short form of the switch is specified, the base locale for the solution is used.

`--useLcid` `-lcid`

Use Language Code Identifier (LCID) values (1033) rather than International Organization for Standardization (ISO) codes (en-US) for language files.

This parameter requires no value. It's a switch.

`--useUnmanagedFileForMissingManaged` `-same`

Use the same XML source file when packaging for Managed and only Unmanaged XML file is found; applies to AppModuleSiteMap, AppModuleMap, FormXml files.

This parameter requires no value. It's a switch.

pac solution upgrade

Apply solution upgrade

Example

PowerShell

```
pac solution upgrade --solution-name SampleSolution --async --max-async-
wait-time 60
```

Required Parameters for solution upgrade

`--solution-name` `-sn`

Name of the solution.

Optional Parameters for solution upgrade

`--async` `-a`

Upgrades solution asynchronously

This parameter requires no value. It's a switch.

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

`--max-async-wait-time` `-wt`

Max asynchronous wait time in minutes. The default value is 60 minutes.

pac solution version

Update build or revision version for the solution.

Example

PowerShell

```
pac solution version --patchversion 2
```

PowerShell

```
pac solution version --strategy gittags
```

Optional Parameters for solution version

`--buildversion` `-bv`

Build version for the solution.

Note: The value must be an integer with minimum value of 0.

`--filename` `-fn`

Tracker CSV file name to be used when using filetracking as a strategy. The default value is 'ControlsStateVersionInfo.csv'.

`--patchversion` `-pv`

Deprecated: This parameter is ignored.

`--revisionversion` `-rv`

Revision version for the solution.

Note: The value must be an integer with minimum value of 0.

`--solutionPath` `-sp`

Path to Dataverse solution directory or Solution.xml file.

`--strategy` `-s`

Updates build version for 'Solution.xml' file using specified strategy. If using gittags, set personal access token in the following environment variable "PacCli.PAT"

Use one of these values:

- `None`
- `GitTags`
- `FileTracking`
- `Solution`

Differences between pac solution clone and export

There are situations where you're unsure when to use `pac solution clone` or `pac solution export` command. You can use one of the commands in the following scenarios:

- Use `pac solution clone` when you need to add new components to the solution.
- Use `pac solution export` when you want to modify the existing content in a solution file but not adding any new components to the solution.

pac solution clone

The exported solution looks like a Visual Studio project when you export the solution using the `pac solution clone` command. Instead of a `.csproj` (as in Visual Studio), you'll see a `cdsproj` file. The `cdsproj` file has all the components information that is required to build the project. The build output is a solution zip file, which you can import into different environments.

```
PS C:\Users\[redacted]\Documents\GitHub\random-solutions\solutions> pac solution clone --name mytestsoln
Connected to.. [redacted]
Starting Solution Export...
Unmanaged solution export succeeded.
Starting Solution Export...
Managed solution export succeeded.
Unpacking Solution...

Extracting C:\Users\[redacted]\AppData\Local\Temp\mytestsoln.zip and C:\Users\[redacted]\AppData\Local\Temp\mytestsoln_managed.zip to C:\Users\[redacted]\Documents\GitHub\random-soluti
ons\solutions\mytestsoln\src

Skipping localization
Processing Component: Workflows
 - SubmitUserData
Processing Component: AppModuleSiteMaps
 - tst_Mytestmodelapp
Processing Component: AppModules
 - tst_Mytestmodelapp
Processing Component: CanvasApps
 - tst_app_e8d09
 - tst_mycostestimatingapp_c4c8a
Processing Component: Connectors
 - new_5Fconfapi5
 - tst_5Fcreateuserandjob

Unmanaged Extract complete.

Skipping localization
Processing Component: AppModuleSiteMaps
 - tst_Mytestmodelapp
Processing Component: AppModules
 - tst_Mytestmodelapp

Managed Extract complete.

Unpacked Solution.
Solution extract succeeded.
Dataverse solution project with name 'mytestsoln' created successfully in: 'C:\Users\[redacted]\Documents\GitHub\random-solutions\solutions'
PS C:\Users\[redacted]\Documents\GitHub\random-solutions\solutions> cd .\mytestsoln\
PS C:\Users\[redacted]\Documents\GitHub\random-solutions\solutions\mytestsoln> dir

    Directory: C:\Users\[redacted]\Documents\GitHub\random-solutions\solutions\mytestsoln

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----           9/7/2021 11:53 AM                src
-a---          8/12/2021  7:37 PM            129 .gitignore
-a---           9/7/2021 11:53 AM           2475 mytestsoln.cdsproj

PS C:\Users\[redacted]\Documents\GitHub\random-solutions\solutions\mytestsoln>
```

The developer doesn't have to unpack the cloned solution because it's rendered in an unpacked format within the src (source) folder.

```
PS C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln> pac solution unpack --zipfile .\mytestsoln.zip --folder .\.
Unpacking Solution...

Extracting C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln\mytestsoln.zip to C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln

Skipping localization
Processing Component: Workflows
 - SubmitUserData
Processing Component: AppModuleSiteMaps
 - tst_Mytestmodelapp
Processing Component: AppModules
 - tst_Mytestmodelapp
Processing Component: CanvasApps
 - tst_app_e8d09
 - tst_mycostestimatingapp_c4c8a
Processing Component: Connectors
 - new_5Fconfapi5
 - tst_5Fcreateuserandjob

Unmanaged Extract complete.
```

Now, if you want to associate a newly created plug-in with this solution, with the solution unpacked, you can use the `pac solution add-reference` command to update the `.cdsproj` file to add the new plug-in. Then, you can build the project using either `dotnet build` or `msbuild`.

It's recommended to do a build restore first before building the project. A build restore (dotnet build does a restore first automatically) will restore the required .NET libraries to generate a packed solution.

pac solution export

When you export the solution using `pac solution export` you feel like exporting the solution using the maker portal, and the resulting output is a solution zip file.

```
PS C:\Users\[redacted]\Documents\GitHub\random-solutions> pac solution export --name mytestsoln --path .\mytestsoln
Connected to.. [redacted]
Starting Solution Export...
Solution export succeeded.
PS C:\Users\[redacted]\Documents\GitHub\random-solutions> cd mytestsoln
PS C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln> dir

    Directory: C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a---           9/7/2021 12:01 PM         195479 mytestsoln.zip
```

When you unpack the solution zip file (we don't recommend that you open the zip with standard tools and use the appropriate command from CLI). The resulting directory structure is similar to the structure in `pac solution clone`. The only difference is that you can't add references to this unpacked solution, as it doesn't have the `.cdsproj` project file.

```
PS C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln> dir

    Directory: C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln

Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d----           9/7/2021 12:02 PM                AppModules
d----           9/7/2021 12:02 PM                AppModuleSiteMaps
d----           9/7/2021 12:02 PM                CanvasApps
d----           9/7/2021 12:02 PM                Connectors
d----           9/7/2021 12:02 PM                environmentvariabledefinitions
d----           9/7/2021 12:02 PM                Other
d----           9/7/2021 12:02 PM                Workflows
-a---           9/7/2021 12:01 PM         195479 mytestsoln.zip

PS C:\Users\[redacted]\Documents\GitHub\random-solutions\mytestsoln>
```

You can modify the relevant set of files that you want to update and then proceed with the solution pack, which generates the solution zip file again to facilitate importing the solution into the target environment. The result from the action is a solution zip file with updated contents and an updated timestamp.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac telemetry

Article • 05/13/2024

Manage telemetry settings.

Commands

Expand table

Command	Description
pac telemetry disable	Choose to not send usage information to help Microsoft improve this product
pac telemetry enable	Choose to send usage information to help Microsoft improve this product
pac telemetry status	Show the current status of telemetry.

pac telemetry disable

Choose to not send usage information to help Microsoft improve this product

Remarks

Set the `PP_TOOLS_TELEMETRY_OPTOUT` environment variable to `1` or `true` to completely opt out of sending telemetry.

pac telemetry enable

Choose to send usage information to help Microsoft improve this product

pac telemetry status

Show the current status of telemetry.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac test

Article • 05/13/2024

(Preview) Execution of automated tests for a Power App

[This article is pre-release documentation and is subject to change.]

> **Important**
>
> - The `run` command is in public preview.
> - Preview features aren't meant for production use and may have restricted functionality. These features are available before an official release so that customers can get early access and provide feedback.

> **Note**
>
> With pac cli version 1.27, the `pac tests` command was changed to `pac test`. `tests` will continue to work, but we recommend using `test` going forward.

Learn more about test engine

Commands

Expand table

Command	Description
pac test run	(Preview) Execute tests defined in a Test Plan file

pac test run

(Preview) Execute tests defined in a Test Plan file

Required Parameters for test run

`--environment-id` **`-env`**

Environment ID for the app to be tested.

`--tenant` `-t`

Tenant ID for the app being tested

`--test-plan-file` `-test`

Path to the Test Plan file

Optional Parameters for test run

`--domain` `-d`

The player domain for the app being tested

Use one of these values:

- `Public`
- `Preview`
- `UsGov`
- `UsGovHigh`
- `UsGovDoD`
- `Mooncake`

`--output-directory` `-o`

Path to save test output and logs

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

Provide product feedback

pac tool

Article • 05/13/2024

Power Platform tools that can be installed and launched.

> **Note**
>
> These commands are only available for the .NET Full Framework version of the PAC CLI.

Commands

Expand table

Command	Description
pac tool admin	Launch Power Platform Admin Center for the current environment.
pac tool cmt	Launch Configuration Migration Tool (CMT).
pac tool list	List the launchable tools and their local install state and version.
pac tool maker	Launch Power Apps Maker Portal for the current environment.
pac tool pd	Launch Package Deployer (PD).
pac tool prt	Launch Plug-in Registration Tool (PRT).

pac tool admin

Launch Power Platform Admin Center for the current environment.

Optional Parameters for tool admin

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac tool cmt

Launch Configuration Migration Tool (CMT).

Example

This example will download and launch the Configuration Migration tool if you run the command for the first time. If you already ran this command, it will launch it.

```
PowerShell
pac tool cmt
```

Optional Parameters for tool cmt

`--clear` `-c`

Clear tool from local file cache

This parameter requires no value. It's a switch.

`--update` `-u`

Update tool to latest available version from nuget.org.

This parameter requires no value. It's a switch.

pac tool list

List the launchable tools and their local install state and version.

Remarks

This example will return the tools you have available, whether they're installed, and if so, which version is installed.

```
PowerShell
pac tool list
```

pac tool maker

Launch Power Apps Maker Portal for the current environment.

Optional Parameters for tool maker

`--environment` `-env`

Specifies the target Dataverse. The value may be a Guid or absolute https URL. When not specified, the active organization selected for the current auth profile will be used.

pac tool pd

Launch Package Deployer (PD).

Example

This example will download and launch the Package Deployer tool if you run the command for the first time. If you already ran this command, it will launch it.

PowerShell

```
pac tool pd
```

Optional Parameters for tool pd

`--clear` `-c`

Clear tool from local file cache

This parameter requires no value. It's a switch.

`--update` `-u`

Update tool to latest available version from nuget.org.

This parameter requires no value. It's a switch.

pac tool prt

Launch Plug-in Registration Tool (PRT).

Example

This example will download and launch the Plug-in Registration tool if you run the command for the first time. If you already ran this command, it will launch it.

PowerShell

```
pac tool prt
```

Optional Parameters for tool prt

`--clear` `-c`

Clear tool from local file cache

This parameter requires no value. It's a switch.

`--update` `-u`

Update tool to latest available version from nuget.org.

This parameter requires no value. It's a switch.

See also

Microsoft Power Platform CLI Command Groups
Microsoft Power Platform CLI overview

Feedback

Was this page helpful? Yes No

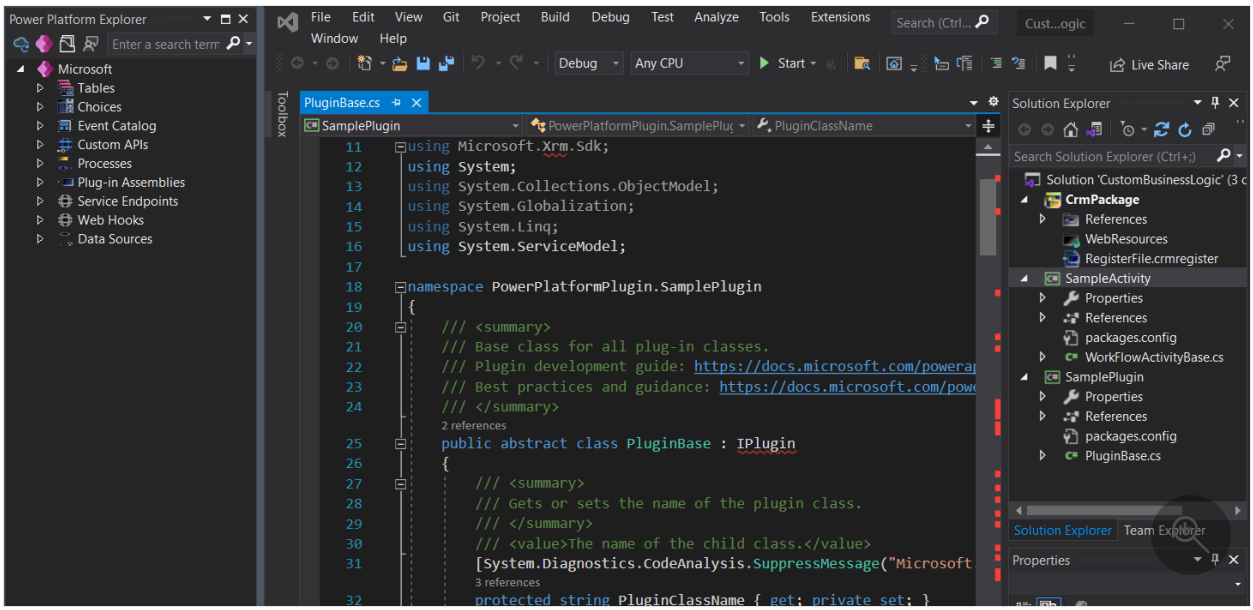
Provide product feedback

What is Power Platform Tools for Visual Studio

Article • 03/30/2023

Power Platform Tools for Visual Studio supports the rapid creation, debugging, and deployment of plug-ins. Other capabilities include development of custom workflow activities, web resources, integration technologies like Azure Service endpoints and webhooks, and more.

This topic describes how to install and uninstall the tools in Visual Studio.



> **Important**
>
> While Power Platform Tools for Visual Studio is similar in appearance and function to the Developer Toolkit for Microsoft Dynamics CRM 2013, Power Platform Tools is a new product and completely independent of the Developer Toolkit. Power Platform Tools is not directly compatible with any templates or projects from the Developer Toolkit and vice versa.

Prerequisites

Before installing Power Platform Tools into Visual Studio you must have the following applications and frameworks installed on your development computer:

- Microsoft Visual Studio 2019 or 2022.

- .NET Framework 4.6.2 (required only for plug-in and workflow activity development)
- C# programming language

In addition, you must also have access to a Microsoft Dataverse environment. A trial environment will be sufficient.

Install Power Platform Tools extension for Visual Studio

To install Power Platform Tools, follow these steps:

1. Start Visual Studio, and select **Continue without code** in the dialog. Optionally, you can open an existing project or create a new one.
2. Select **Extensions** > **Manage extensions**.
3. Expand the left navigation panel node **Online** > **Visual Studio Marketplace**. Search for "Power Platform Tools", then download and install the extension.

> **Important**
>
> After installing Power Platform Tools, you will not find any Power Platform Tools related menu items or views in the Visual Studio user interface until you create or load a Visual Studio solution that contains at least one project created from a Power Platform Tools template.

Uninstall Power Platform Tools extension

To uninstall Power Platform Tools, follow these steps:

1. Launch Visual Studio, and select **Continue without code** in the dialog. Optionally, you can open an existing project or create a new one.
2. Select **Extensions** > **Manage extensions**.
3. Expand the left navigation panel node **Installed**. Search for or scroll to the "Power Platform Tools" extension, select it, and then choose **Uninstall**.

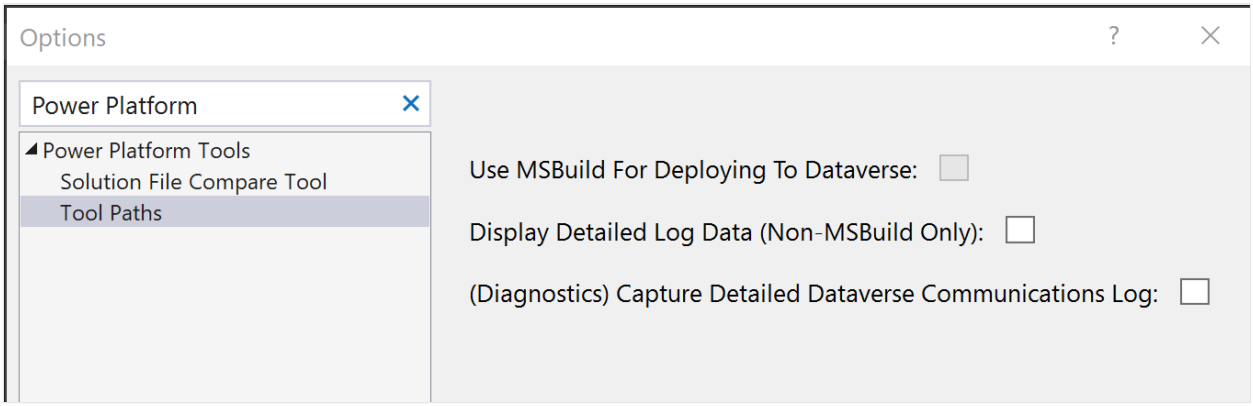
Install the Plug-in Profiler

If you are planning on developing plug-ins and would like to use the Plug-in Profiler to aid with plug-in debugging, the tool's Dataverse solution must be installed into your development environment. You can install or uninstall the tool's solution easily using Power Platform Tools for Visual Studio.

1. In Visual Studio, if you are not already connected to a Dataverse environment then do so now by choosing **Tools** > **Connect to Dataverse**.
2. Choose **View** > **Power Platform Explorer** to display its view.
3. Expand the target environment's node.
4. Right-click the **Plug-in Assemblies** node. You will see **Install Profiler** or **Uninstall Profiler** in the menu. If you see the **Uninstall Profiler** menu option, then the tool is already installed in your environment. Otherwise, choose **Install Profiler** to install the tool.

Power Platform Tools options

You can find a few Power Platform Tools options in Visual Studio by choosing **Tools** > **Options** and searching for "Power Platform" in the **Options** dialog search field.



Next steps

Quickstart: Create a Power Platform Tools project
Quickstart: Create a plug-in using Power Platform Tools

See also

Power Platform Tools extension for Visual Studio 2019
Power Platform Tools extension for Visual Studio 2022

Quickstart: Create a Power Platform Tools project

Article • 11/03/2023

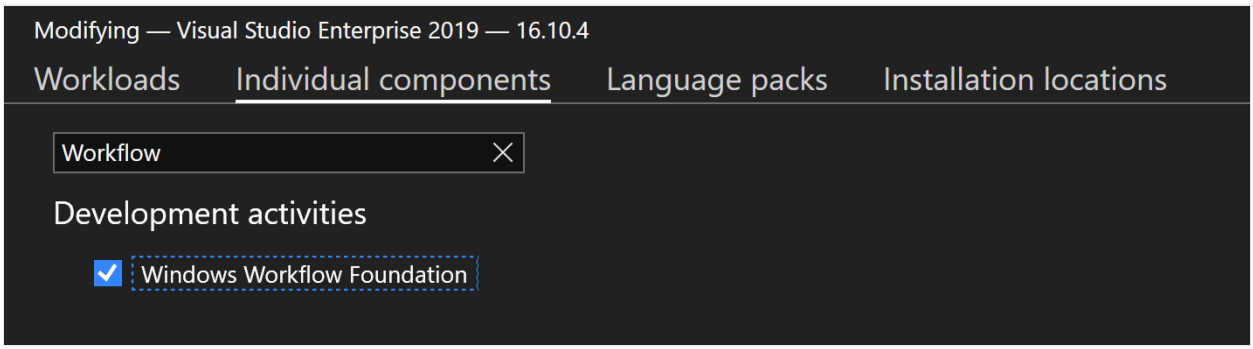
Like any Visual Studio solution, you begin by creating a new project. In the new project dialog, enter "Power Platform" in the search box. A list of available Power Platform Tools project C# templates is described in section Available project templates.

Prerequisites

- Visual Studio 2019 or 2022
- Power Platform Tools for Visual Studio
- C# language
- .NET Framework 4.6.2 or later; plug-in and custom workflow activity development requires v4.6.2
- Power Apps subscription or a trial environment
- Windows Workflow Foundation (only for custom workflow activity development)

If you haven't installed Power Platform Tools for Visual Studio, install it by following instructions here: Power Platform Tools

You can install Windows Workflow Foundation from the Visual Studio Installer application by selecting **Modify** and choosing the individual component as shown in the next figure. An alternate approach is to install a workload that contains that component.



Available project templates

The following table introduces the available Power Platform Tools project templates.

C# project template	Description
Power Platform Solution Template	Solution template for creating a Power Platform solution. This template is for a Visual Studio solution and not a Microsoft Dataverse solution.
Power Platform Plug-in Library	Project template for creating a Power Platform plug-in class library and assembly (DLL).
Power Platform Package	Project template for creating a Power Platform package (CrmPackage). The package is used to deploy the solution and custom code libraries to a Dataverse environment.
Power Platform Workflow Activity Library	Project template for creating a Power Platform custom workflow activity class library and assembly (DLL).
Customizations Project	Project template for all customizations of a Power Platform (Dataverse) solution.

Use the Power Platform solution template

The Power Platform solution template is a good starting point for any new solution. You can add and remove projects to and from the solution. However, you shouldn't remove the CrmPackage project. Doing so causes deployment of the Power Platform solution to the target environment to fail.

> **ⓘ Important**
>
> A Power Platform solution must contain one and only one CrmPackage project. Otherwise, the Dataverse solution deployment will fail.

The plug-in library and workflow library project templates are typically used for more advanced scenarios. For example, where you may want to add multiple custom code assemblies to a Power Platform solution or if you're only interested in developing that specific custom code component. Before you can deploy a solution that contains only a project of these types, you must add a CrmPackage project to the Visual Studio solution.

The easy way to create a Power Platform solution containing a CrmPackage project and optionally a plug-in and/or workflow activity project is to use the Power Platform Solution Template. To create that solution, follow these next steps.

1. In the Visual Studio new project dialog, search for and choose **Power Platform Solution Template** and then select **Next**.

2. Enter the requested project information, choose a .NET Framework version, and select **Create**.

   At this point, you should see either a Dataverse sign in dialog or a dialog to reuse your last Dataverse connection. Do whatever is appropriate to connect to your Dataverse development environment. Never develop code in a production environment.

3. At the **Configure Microsoft Power Platform Solution** dialog, choose either to use an existing Dataverse solution or create a new solution.

   Depending on what you have chosen, you're prompted to enter information about the new solution or select the existing solution from a drop-down list. **Step #3 Select Solution** is displayed but not active until you're logged in and choose **Next** in step #2.

4. Once the target Power Platform solution has been identified or created, you specify existing items or create new projects (step #1) using one of the above Power Platform project templates.

   After you choose a new item, the dialog will expand to show step #2 where you can choose (only) one of each available project to add to your solution. Choose (check) one or more projects from the list and select **Next**. Step #3 is displayed but not active until you're done with step #2 and choose **Next**.

5. In step #3 of the dialog, enter names for your chosen projects and select **Done**.

   Choose names that you want to see as project names in Visual Studio Solution Explorer.

6. You're prompted to save the solution file. Select **Save** to save the Visual Studio solution.

In **Solution Explorer**, you should now see a solution containing a single **CrmPackage** project and one or more projects based of the project templates that you chose. Each plug-in or custom workflow activity class library project builds an assembly. You can add more classes to each class library by adding a new or existing item as desired, and you can add other projects to the solution using the Power Platform project templates.

Managing projects

The following procedures describe some common operations for your Visual Studio solution.

Add a new project to a Power Platform solution

Only projects created with a Power Platform Tools template should be added to a solution created using the Power Platform Tools Solution Template. To add a new project to a solution, follow these steps.

1. Right-click the solution in **Solution Explorer**, select **Add**, and then choose **New Project**.
2. Select one of the installed Power Platform Tools templates and select **Next**.
3. Fill in any required information and select **Create**. For a plug-in or workflow activity library, be sure to choose .NET Framework 4.6.2.

Add an existing project to a Power Platform solution

Only projects created from Power Platform Tools templates should be added to a solution that was created using the Power Platform Tools Solution Template.

1. Right-click the solution in **Solution Explorer**, select **Add**, and then choose **Existing Project**.
2. Navigate to the .csprog file of the target project, select it, and choose **Open**.
3. In **Solution Explorer**, under the CrmPackage project, right-click **References** and select **Add Reference**.
4. In the **Projects** tab of the **Add Reference** dialog, select the (to be added) projects from the list and then select **Add** to add them to the list of selected projects and components.
5. Select **OK** to add the projects to the solution and close the **Add Reference** dialog box.

Remove a project from a Power Platform solution

1. In **Solution Explorer**, right-click the project and select **Remove**. The project is automatically removed from the CrmPackage references.
2. Edit the RegisterFile.crmregister XML file in the CrmPackage project and remove any XML registration code related to the removed project.
3. Remove any deployed assembly and steps of your project from the Dataverse environment and solution.

If you removed a plug-in or workflow activity library project from the Visual Studio solution, you may want to also remove any related assembly and steps from the Dataverse environment and solution. You can remove these components in **Power Platform Explorer** by expanding the Plug-in Assemblies node of your environment tree, right-click the target assembly, and choose **Delete Assembly**. doing so deletes the assembly and any registered steps.

> **Important**
>
> Your Dataverse logon account must own or have delete access to the deployed assembly in order for you to delete it from the environment.

See Also

Quickstart: Create a plug-in using Power Platform Tools

Create a plug-in using Visual Studio

Article • 03/19/2024

This article provides information on how to create a plug-in using the Power Platform Tools extension for Visual Studio.

Prerequisites

- Visual Studio 2019 or 2022
- Power Platform Tools extension for Visual Studio
- .NET Framework 4.6.2 is required for plug-in or custom workflow activity development
- Power Apps subscription or a trial environment

Create a solution with plug-in library

Follow these instructions to create a solution with plug-in library, connect to your Power Platform environment and register custom code assemblies, steps, and more.

1. Use Power Platform Tools extension for Visual Studio to create a new solution with a plug-in library. For instructions, go to the Create a Power Platform Tools project article.

   If you already have an existing solution set up, follow instructions in Add a new project to a Power Platform solution to add a Plug-in Library project to the solution using the Power Platform Tools template.

2. In the **Tools** menu, select **Connect to Dataverse**.

3. Select the desired options in the dialog and select **Login**.

4. Select an existing solution or the **Default** solution. This solution contains your registered plug-in and workflow activity assemblies for later distribution to other environments.

5. Select **Done** when finished.

The Power Platform Explorer view is displayed or you can open that view from the **View** menu. Expand the nodes to see what kinds of environment data you can view. Right-click on nodes to see what options are available.

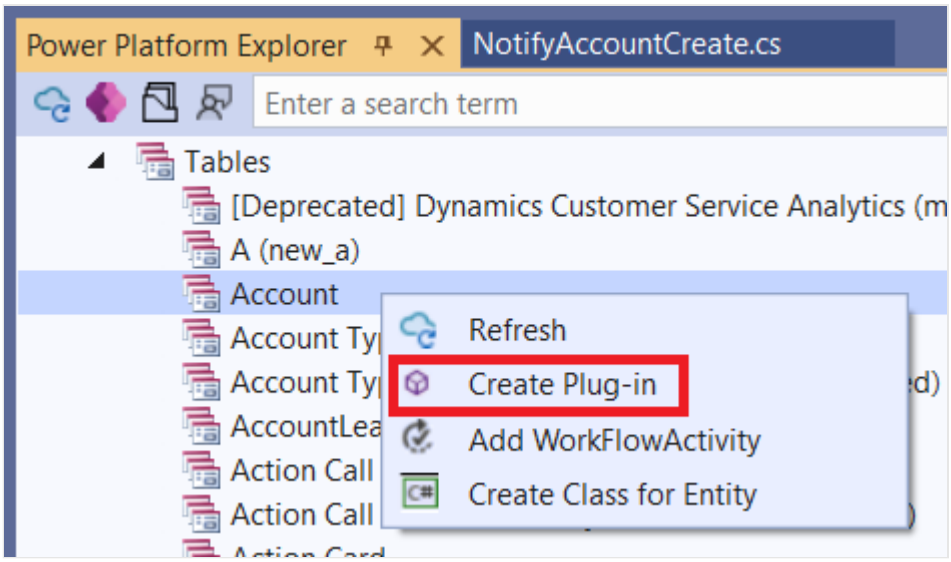
> **Note**
>
> The Power Platform Explorer view of the Power Platform Tools extension is capable of more than plug-in and workflow activity registration. Documentation for additional features will be provided in a future documentation release. In the meantime, feel free to explore.

Register a plug-in step with Dataverse

Follow these instructions to register a plug-in step (also known as an SDK message processing step). The step identifies what data table and event causes your plug-in to execute. More information: Event framework, Register plug-in step

1. Select **View** > **Power Platform Explorer**, expand your environment node and the **Tables** subnode.

2. Right-click on the table type (for example, "Account") that the step is to be registered on, then select **Create Plug-in**.



> **Note**
>
> You can also create a plug-in by expanding the **Event Catalog**, right-clicking a business event, and choose **Add Plug-in**.

3. Fill out the **Register New Step** dialog information and choose **Register New Step** when done.

The class name that you specify when filling out the step information is used to name your new plug-in class. The class is placed in the plug-in project library specified by the **Handler Assembly** dialog field. If there's only one Plug-in Library project in the solution, the **Handler Assembly** field is inactive.

A new step registration is added to the solution. However, you'll need to build and deploy your plug-in library before the plug-in assembly and step are added to the specified Dataverse environment and solution.

Add another plug-in class to the library

Let's say that you followed the steps previously described to create a plug-in class in the plug-in project library and add a step. Now you want to add another plug-in class to that same library. How do you do that? You have two options both are through context menus of the Power Platform Tools extension.

1. The first method is to do the same as you did previously by expanding the **Table** node in the **Power Platform Explorer** view, right-clicking a table type, and then selecting **Create Plug-in**. When registering the step, specify the same plug-in project library in the **Handler Assembly**, and the new class name in the **Class Name** form field.

2. For the second method, let's say that you already built and deployed the plug-in library assembly to the Dataverse environment. In this case, you'll see the deployed assembly and step you created after you refresh the explorer view and expand the **Plug-in Assemblies** node. At that point, you can simply right-click the target assembly node and select **Add Plug-in**. Doing so displays the step registration form, which you can now fill out for your new plug-in class.

Afterwards, build and deploy the plug-in library project to update the target environment and solution.

Add a plug-in step

You can add a step for an existing registered plug-in that you own. To do so, expand the target plug-in assembly node in the **Power Platform Explorer** view to display the registered plug-in. Right-click that plug-in's node and select **Add Step**. Fill out the **Register New Step** form.

You don't need to build and deploy the plug-in project for the step registration to be available in the target environment and solution.

The PluginBase abstract class

`PluginBase` is automatically generated from the Plug-in Library template. Your custom plug-in class should derive from this base class. The base class implements common plug-in code that helps to make you more productive sooner. Take a look at the `PluginBase` class code in your plug-in project to see what it does.

The generated plug-in class code

The Plug-in Library template provides the `PluginBase` abstract class. Your plug-in must derive from `PluginBase` if it's to work well with the Power Platform Tools extension. Here is the generated derived class when creating a plug-in from Power Platform Explorer. You typically would add your code where the TODO comments are. Notice that the standard IPlugin interface `Execute` method is now `ExecuteCdsPlugin` in the plug-in code.

C#

```
using System;
using System.ServiceModel;
using Microsoft.Xrm.Sdk;

namespace PPTools_Sample_Solution.NotifyPlugin
{
    public class NotifyAccountCreate: PluginBase
    {
        public NotifyAccountCreate(string unsecure, string secure)
            : base(typeof(NotifyAccountCreate))
        {
           // TODO: Implement your custom configuration handling.
        }

        protected override void ExecuteCdsPlugin(LocalPluginContext
localContext)
        {
            if (localContext == null)
            {
                throw new
InvalidPluginExecutionException(nameof(localContext));
            }
            ITracingService tracingService = localContext.TracingService;

            try
            {
                IPluginExecutionContext context =
(IPluginExecutionContext)localContext.PluginExecutionContext;

                IOrganizationService service =
localContext.OrganizationService;
```

```
                // TODO: Implement your custom Plug-in business logic.

            }
            catch (Exception ex)
            {
                tracingService?.Trace("An error occurred executing Plugin
PPTools_Sample_Solution.NotifyPlugin.NotifyAccountCreate : {0}",
ex.ToString());
                throw new InvalidPluginExecutionException("An error occurred
executing Plugin PPTools_Sample_Solution.NotifyPlugin.NotifyAccountCreate
.", ex);
            }
        }
    }
}
```

At this point, you would add your custom plug-in code where indicated by the TODO code comments. For more information, read some of these related articles: Pass configuration data to your plug-in, Understand the execution context, and Tracing and logging.

> **ⓘ Important**
>
> We encourage you to use the Power Platform Tools context menus. When you use the Power Platform Explorer context menus to add plug-ins, classes are generated using the provided information and deployment entries in the CrmPackage project's RegisterFile.crmregister file are maintained. It is this information that is used for deployment of assemblies, plug-ins, and steps.

Sign the assembly

All plug-in and custom workflow assemblies must be digitally signed before they're uploaded to the Dataverse server. To sign the assembly, follow these steps.

1. Select the plug-in or workflow activity project in **Solution Explorer**.
2. Choose **Project** > <project name> **Properties** to edit the project's properties.
3. On the **Signing** tab, check **Sign the assembly**, and then specify a strong name key file.

Deploy the plug-in to the environment solution

After you're done modifying code and are ready to deploy the plug-in assembly and steps to your environment, follow these steps.

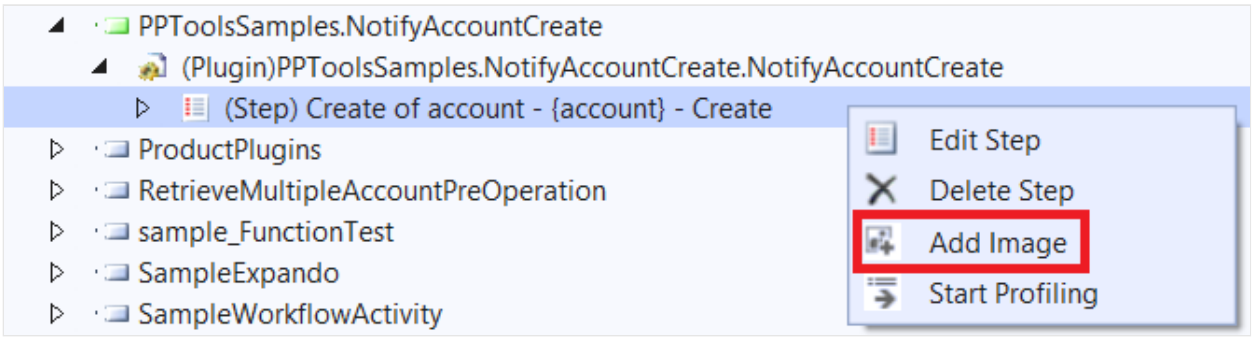
1. Build the plug-in library.
2. Right-click the plug-in library project in **Solution Explorer**.
3. Select **Deploy** in the context menu.

> **Tip**
>
> You can deploy all projects in the Visual Studio solution by right-clicking the CrmPackage project and choosing **Deploy**.

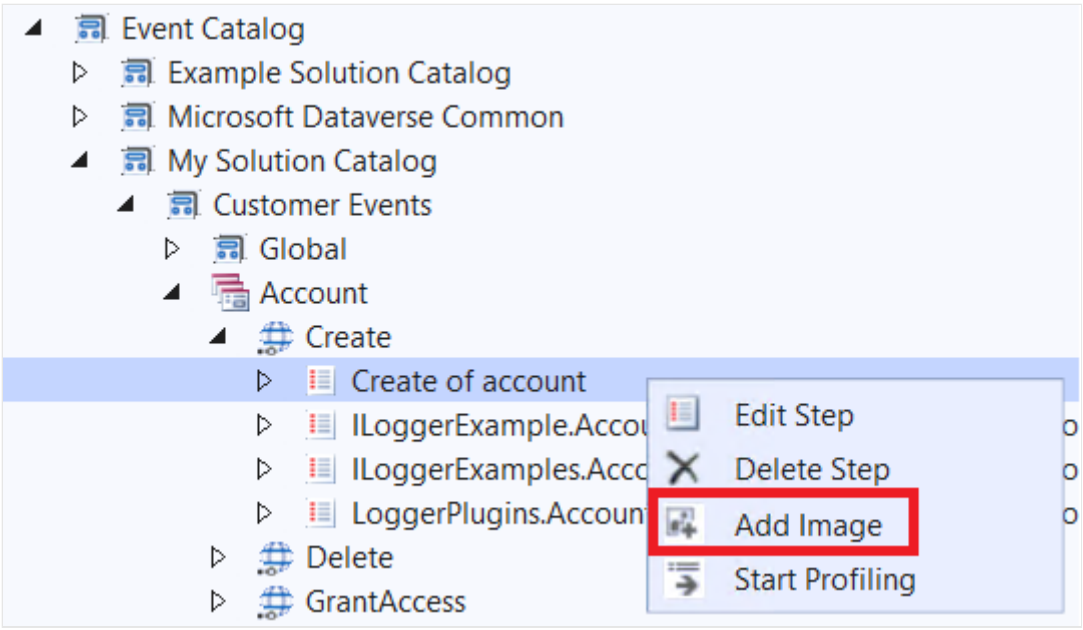
After deployment completes, select the refresh icon in **Power Platform Explorer**. Expand the **Plug-in Assemblies** subnode of your environment node to see your registered assembly. Right-click on the plug-in assembly and step in **Power Platform Explorer** to see what operations are supported. Selecting **Delete Assembly** unregisters the assembly and its related steps.

Add an entity image

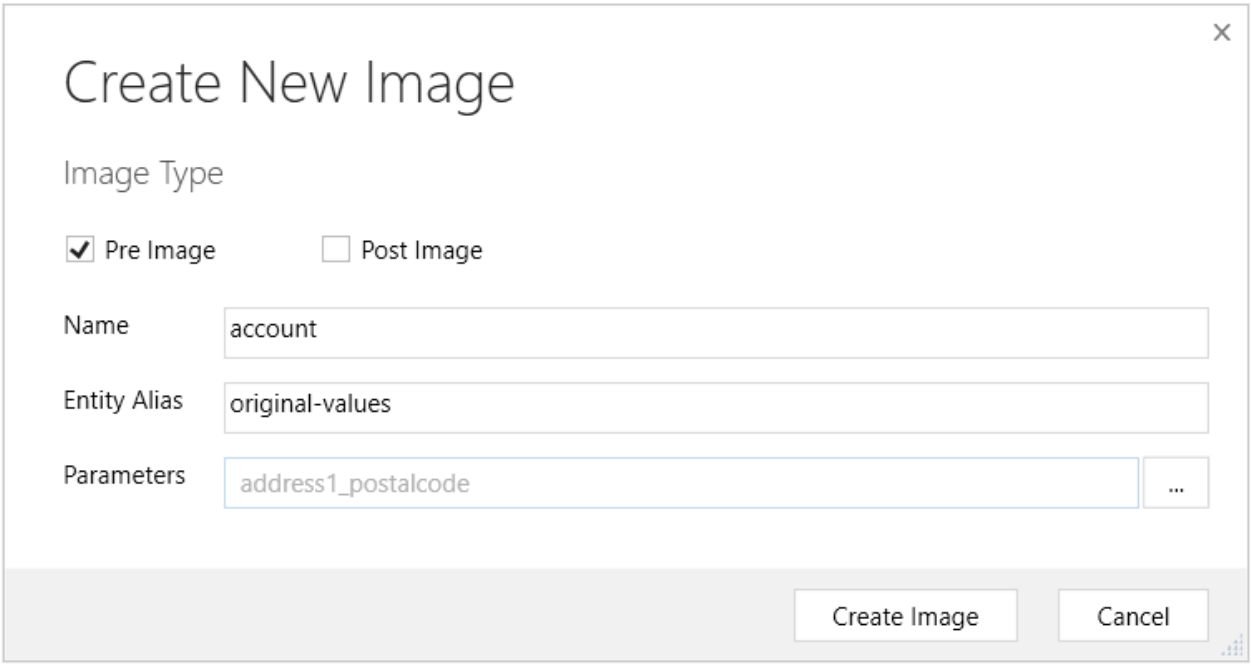
Entity images are snapshots of entity data before or after the core operation (for example: a create or update). You may (optionally) add entity images on plug-in steps using the **Power Platform Explorer** view. To add an image to a step in the **Power Platform Explorer** view, expand the **Plug-in Assemblies** node tree to display the target plug-in step, and then right-click on the step to display the context menu from which you can choose **Add Image**.



You can also add an image in the **Power Platform Explorer** view under **Event Catalog** on a step of an event plug-in.



After you select **Add Image** in the context menu, fill out the form that appears.



In the form, **Pre Image/Post Image** specifies the entity data as it exists before (Pre) or after (Post) the core operation. The **Name** field defines the logical name of the entity that you want data for. **Entity Alias** is the named index that you'll use in your code to identify the row in the image table that contains the target entity data. **Parameters** is the list of entity data columns that you want. Specify only the columns that you need since plug-in performance is reduced the more columns you specify.

More information: Define entity images.

Providing tool feedback

You can send tool feedback to Microsoft using the feedback icon in the Power Platform Explorer view.



See Also

Tutorial: Debug a plug-in
Event framework
Use plug-ins to extend business processes

Create and register a plug-in package using Visual Studio

Article • 11/03/2023

In this article you'll learn how to create a plug-in package, and then upload and register it on the Dataverse server. You'll be using Visual Studio with the Power Platform Tools extension to create the plug-in package, upload and register the package, and perform some other package management tasks.

This article helps you:

- ✓ Create a Visual Studio plug-in project
- ✓ Add a dependent assembly to the project and build
- ✓ Use Visual Studio to upload and register your package
- ✓ Use Visual Studio to update and delete a package

More information: Build and package plug-in code.

Prerequisites

- Access to a Dataverse environment
- A system user account, with System Administrator or System Customizer role, in the target environment
- Visual Studio 2019 with Power Platform Tools extension installed

Review these articles related to installing and using Power Platform Tools for Visual Studio.

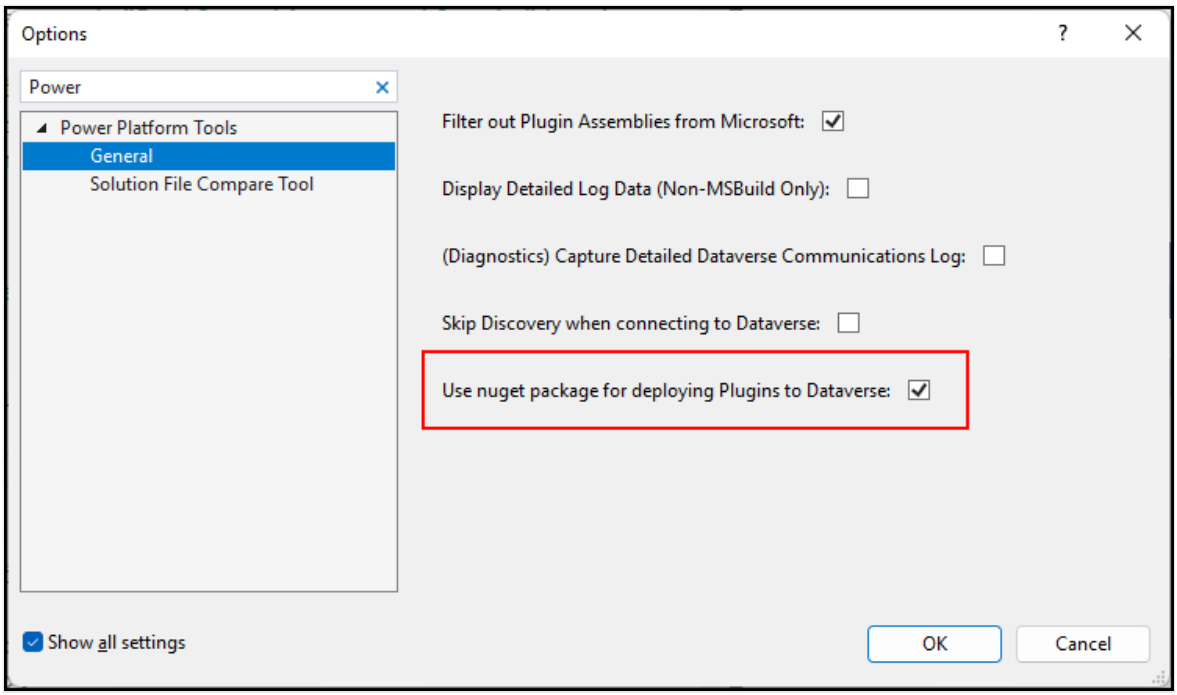
- What is Power Platform Tools for Visual Studio
- Create a Power Platform Tools project
- Create a plug-in using Power Platform Tools

Enable plug-in packages for Power Platform Tools

Power Platform Tools for Visual Studio provides several configuration options as described in Power Platform Tools options.

1. In Visual Studio, go to **Tools** > **Options** and search for **Power Platform tools**.

2. Select **Use NuGet package for deploying plug-ins to Dataverse**.



> **Note**
>
> When this option is selected, all your plug-in projects will be deployed as plug-in packages.

Add a dependent assembly to your project

You can add a NuGet package for satisfying required references to your Visual Studio project as you normally do. After you build the project, you should find your plug-in assembly in the generated NuGet package located in the `bin\outputPackages` folder.

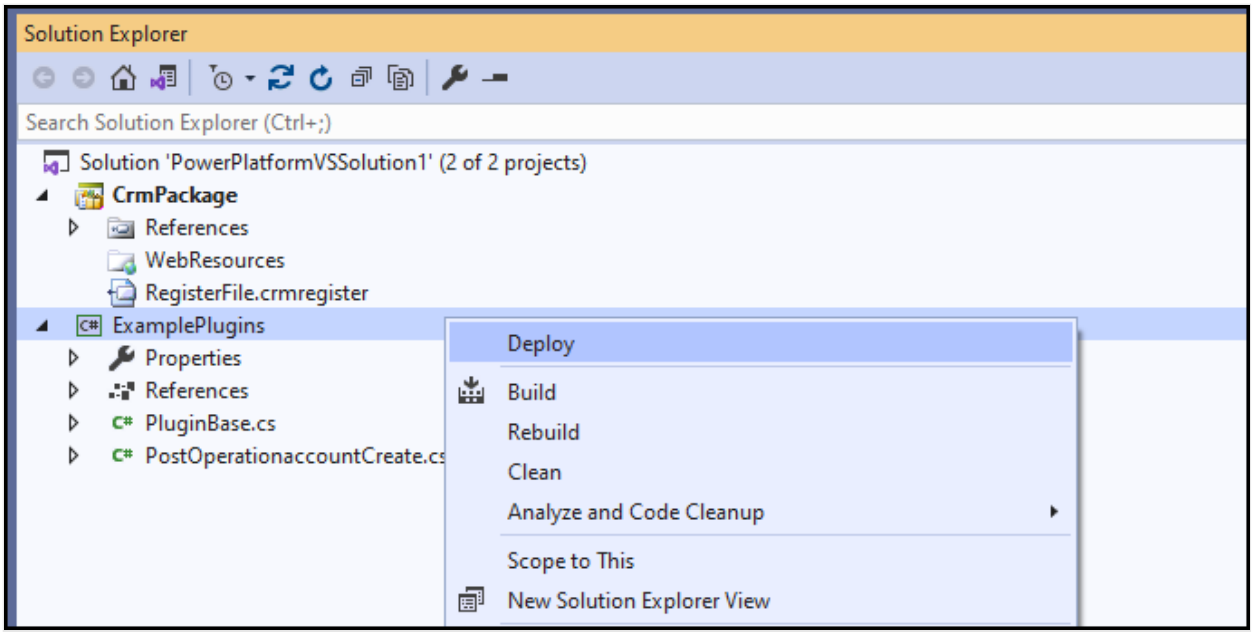
You can use NuGet Package Explorer to examine the generated NuGet package.

Add a dependent assembly without using NuGet

If you have an assembly that isn't distributed as a NuGet package, you can add it to your project as you normally do. In **Solution Explorer**, right-click **Dependencies** and choose **Add Assembly Reference...**. Select the assembly you want to add.

Deploy plug-in packages

To deploy (upload and register) your plug-in package, in **Solution Explorer** right-click the plug-in project and select **Deploy** from the context menu.



When you deploy for the first time, you should see a message in the output window that informs you that the plug-in package was created:

```
6/22/2022 3:03:17 PM : registration of Plugin Package
sample_PowerPlatformVSSolution.ExamplePlugins,
at
E:\projects\PowerPlatformVSSolution\ExamplePlugins\bin\outputPackages\PowerP
latformVSSolution.ExamplePlugins.1.0.0.nupkg was successful.
ID allotted was 06a20e15-77f2-ec11-bb3c-000d3a892245.
```

Each time you deploy after that, you'll see a message in the output window that informs you that the plug-in package was updated.

```
6/22/2022 3:20:14 PM : update of Plugin Package
sample_PowerPlatformVSSolution.ExamplePlugins, at
E:\projects\PowerPlatformVSSolution\ExamplePlugins\bin\outputPackages\PowerP
latformVSSolution.ExamplePlugins.1.0.0.nupkg was successful.
```

> **ⓘ Important**
>
> If your update removes any plug-in assemblies, or types which are used in plug-in step registrations, the update will be rejected. You must manually remove any step registrations that use plug-in assemblies or plug-in types that you want to remove with your update.

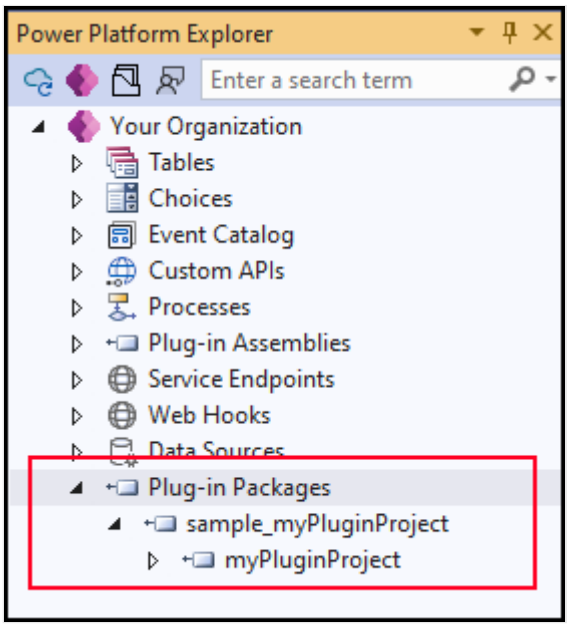
The version of the plug-in package or plug-in assembly is not a factor in any upgrade behaviors. You can update the versions of these items as you need.

Plug-in package management

Within Visual Studio, you can perform additional operations on plug-in packages as described in the next sections.

View plug-in packages with Power Platform Tools

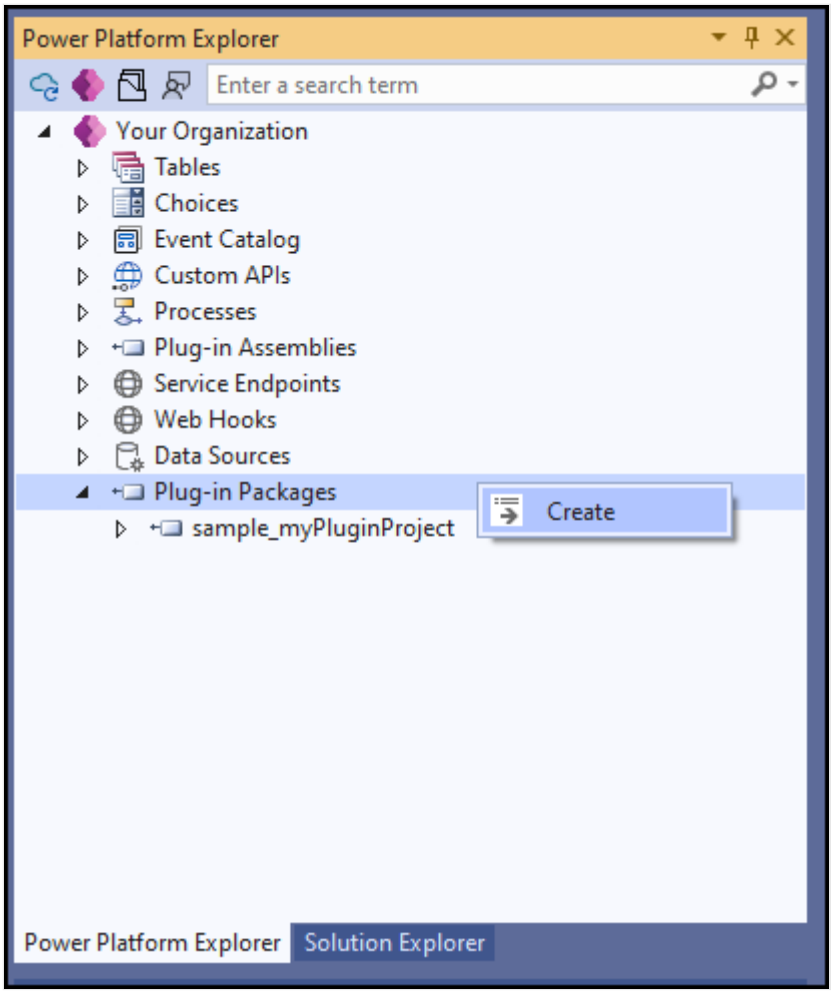
Within the Power Platform Explorer, you can view available plug-in packages.



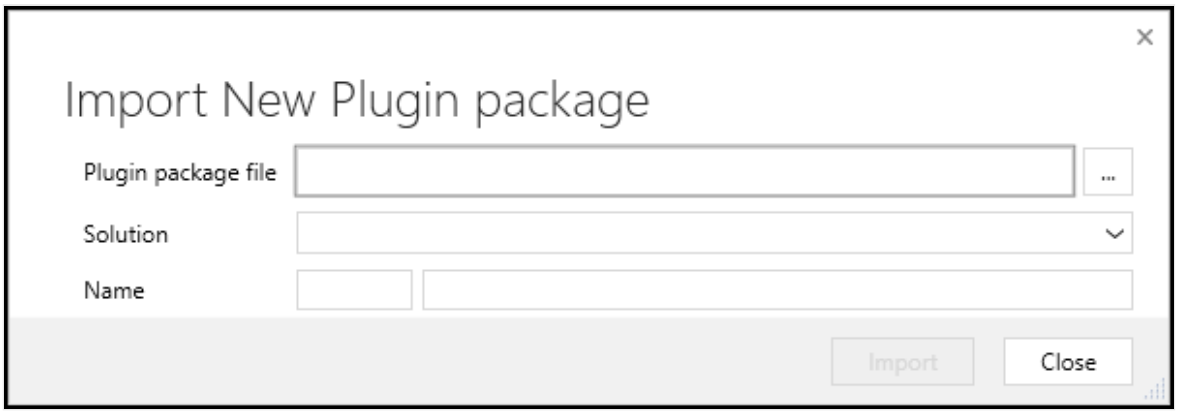
Create a plug-in package, alternate method

As mentioned in Deploy plug-in packages, you will not typically have to create a plug-in package in the usual flow of creating a plug-in. It will be created automatically the first time you deploy the plug-in while the **Use NuGet package for deploying plug-ins to Dataverse** option is set in Visual Studio. However, you can create a new plug-in package from a NuGet package using the following steps.

1. Within the Power Platform Explorer, select **Plug-in Packages** and select **Create** from the context menu.



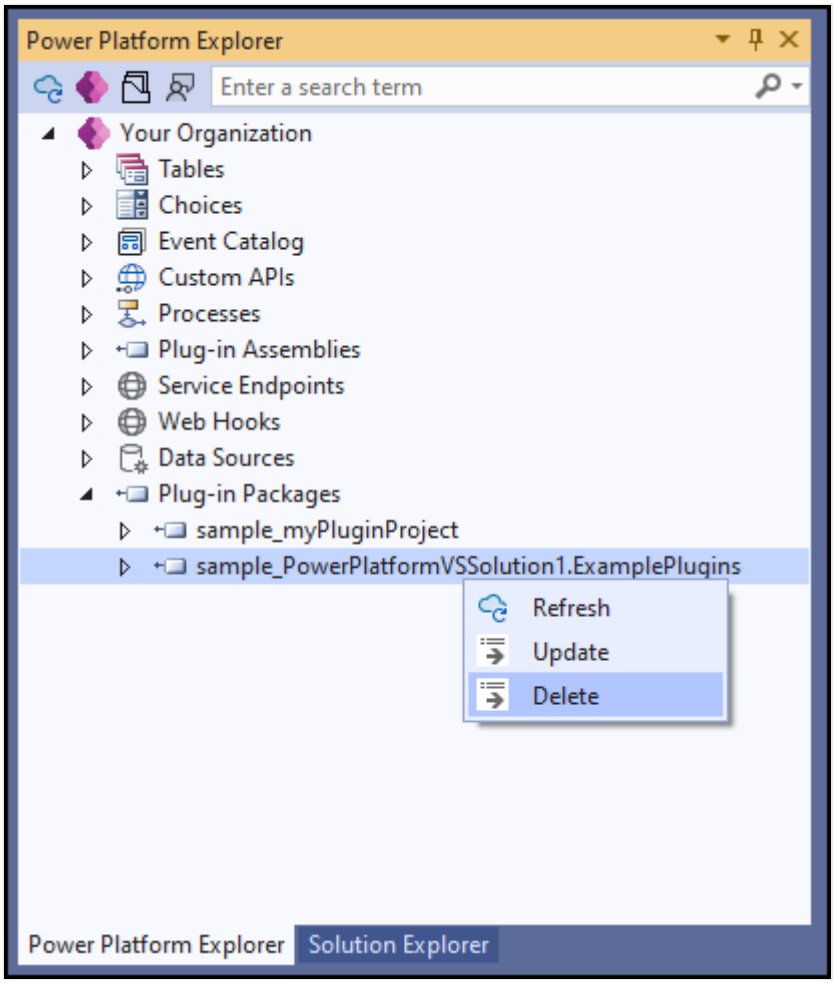
2. This command opens the **Import new plug-in package** dialog.



3. Select the NuGet package and choose which solution it should be added to, or create a new solution.

Delete a plug-in package

Within the Power Platform Explorer, select a plug-in package and then select **Delete** from the context menu.

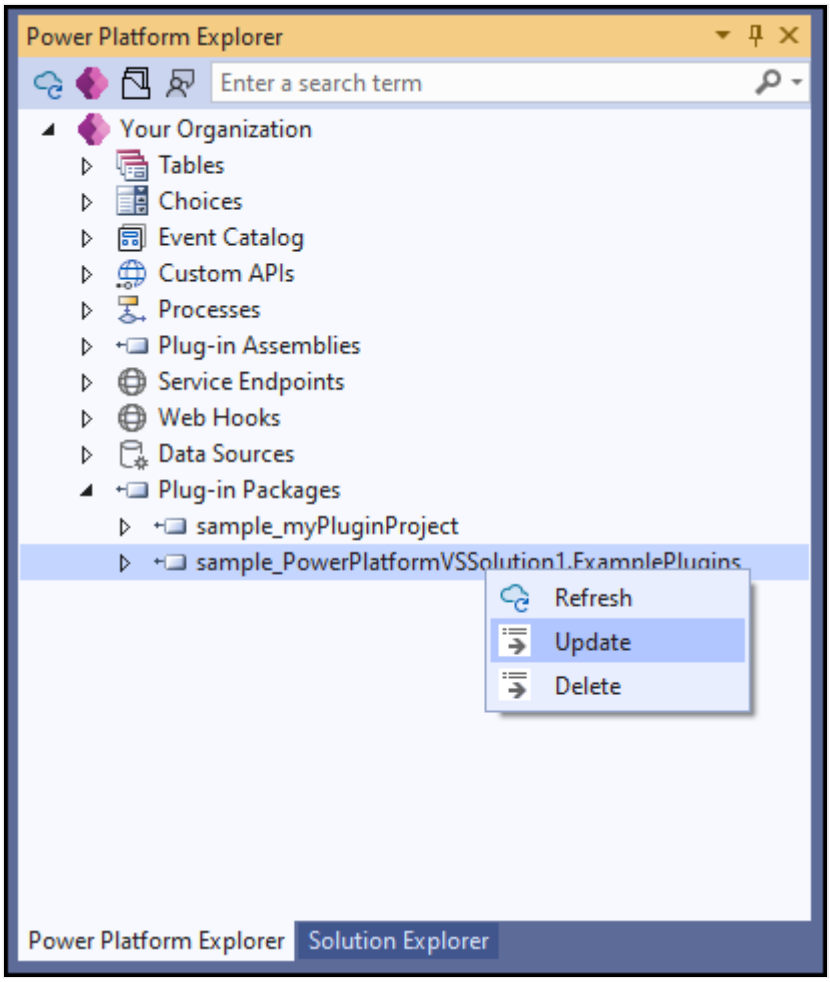


> **Important**
>
> Deleting a package will delete the package, all assemblies within it, all plug-ins within the assembly, and any plug-in step registrations for the plug-ins.

Update a plug-in package, alternate method

As mentioned in Deploy plug-in packages, you will not typically have to update a plug-in package in the usual flow of creating a plug-in. It will be updated automatically each time you deploy the plug-in while the **Use nuget package for deploying plug-ins to Dataverse** option is set in Visual Studio. However, you can update a new plug-in package from a Nuget package using the steps below.

Within the **Power Platform Explorer**, select a plug-in package and select **Update** from the context menu.



This command opens a dialog so you can select a NuGet package to update the plug-in package.

Visual Studio connected service

Article • 02/16/2024

Developers can use Connected Services within Visual Studio to create custom connectors. Connected Services is a collection of tools in Visual Studio that help you connect to services. With the existing capabilities developers have to build Microsoft Azure Functions or APIs in Visual Studio, the Connected Services integration offers a complete end-to-end experience in custom connector development.

For Visual Studio 2022 17.7, we added Connected Services support for the Microsoft Power Platform for ASP.NET Core web API projects. This integration allows you to:

- Connect to a Power Platform environment.
- Automatically generate a custom connector for your Power Platform environment.
- Configure a dev tunnel to locally connect to your custom connector.

Dev tunnels create an ad-hoc connection between your web API running locally and your Power Apps. You can use them to locally test and debug your web API in the context of the Power Platform in real-time (inner loop).

Prerequisites

- Visual Studio 2022 version 17.7 with the ASP.NET and web development workload installed. You need to be signed in to Visual Studio to create and use dev tunnels. The feature isn't available in Visual Studio for Mac.
- An ASP.NET Core web API project
- A Power Platform developer environment

Limitations

The current integration works only for the following configurations of dev tunnels:

- Tunnel type: Persistent
- Access type: Public

Next steps

To get started, right-click the **Connected Services** node in **Solution Explorer** and select **Manage Connected Services**. Search for and select **Power Platform**, and then set up

your connection.

Create a canvas app front end for your ASP.NET web API

Create a canvas app front end for your ASP.NET web API (preview)

Article • 06/15/2023

[This article is pre-release documentation and is subject to change.]

Power Platform enables web API developers to quickly build mobile or web applications using a Power Apps canvas app as a front end. With the Power Platform connected service in Visual Studio 2022, ASP.NET web API developers can easily create a custom connector for use with Power Apps and Power Automate.

You can use Visual Studio *dev tunnels* to create a canvas app before you publish your API to a public endpoint. Dev tunnels create an ad-hoc connection between your web API running locally and your Power Apps. You can use them to locally test and debug your web API in the context of the Power Platform in real-time (inner loop).

This tutorial uses the sample ASP.NET Weather web API to add a connection to a canvas app. It's based on the Create a web API with ASP.NET Core tutorial.

Prerequisites

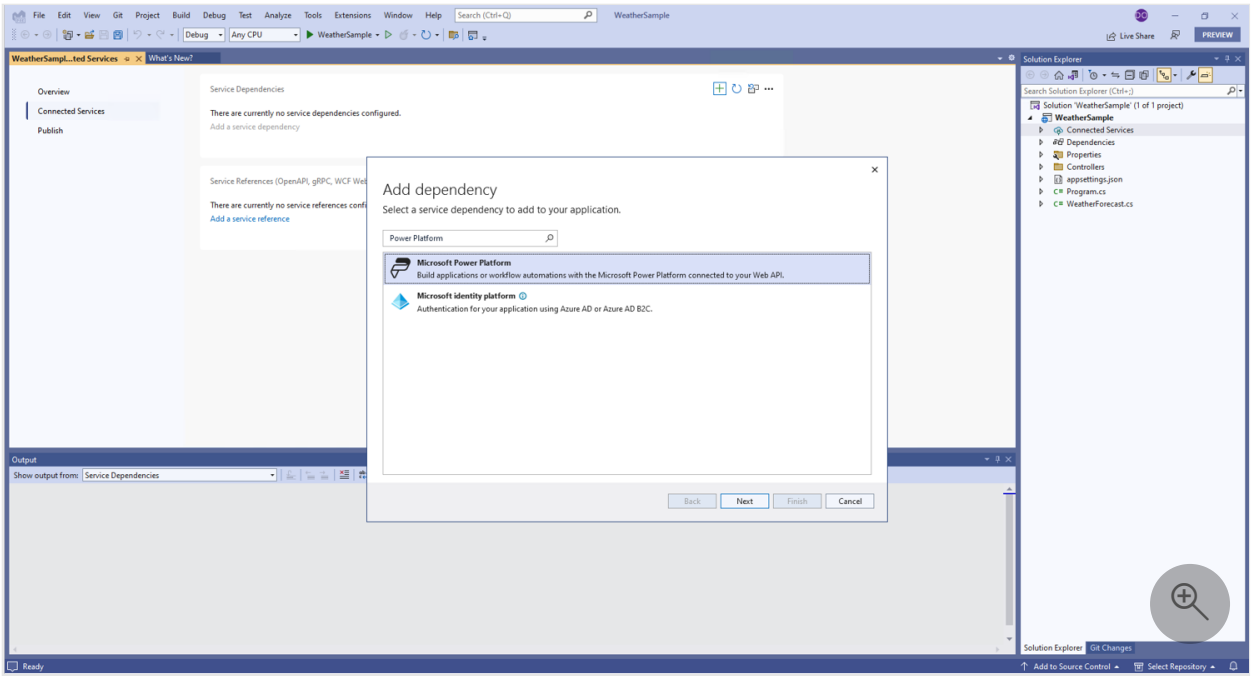
- Visual Studio 2022 version 17.6 Preview 2 with the ASP.NET and web development workload installed. You need to be signed in to Visual Studio to create and use dev tunnels. The feature isn't available in Visual Studio for Mac.
- A Power Platform developer environment with permission to create custom connectors

Create an ASP.NET project in Visual Studio

1. Create an ASP.NET Core web API project.
2. Select **Next**.
3. Enter *WeatherSample* for the **Project name**, and then select **Next**.
4. Select **Create**.

The `WeatherSample` project contains an example controller for a RESTful HTTP service.

Add Power Platform as a connected service



1. In **Solution Explorer**, right-click the **Connected Services** node and select **Manage Connected Services** from the context menu.

2. In the **Connected Services** tab, select the + icon for **Service Dependencies**.

3. On the **Add dependency** dialog, type `Power Platform` into the search box.

4. Select **Microsoft Power Platform**, and then select **Next**.

   If you aren't signed in already, sign into your Microsoft Power Platform account. If you don't have a Power Platform account, Create a Developer Environment.

5. In the **Connect to Microsoft Power Platform** screen: select your developer environment.

6. In **Custom connectors name**, the value `WeatherSample_Connector` should already be set.

7. In **Select a public dev tunnel**, select the + icon.
   a. In the field **Name**, type `SampleTunnel`.
   b. Select **Tunnel Type**: **Persistent**.
   c. Select **Access** : **public**.
   d. Select **OK**.

8. Select **Finish**

9. Once the connected service is configured, select **Close**.

Create a canvas app with the custom connector

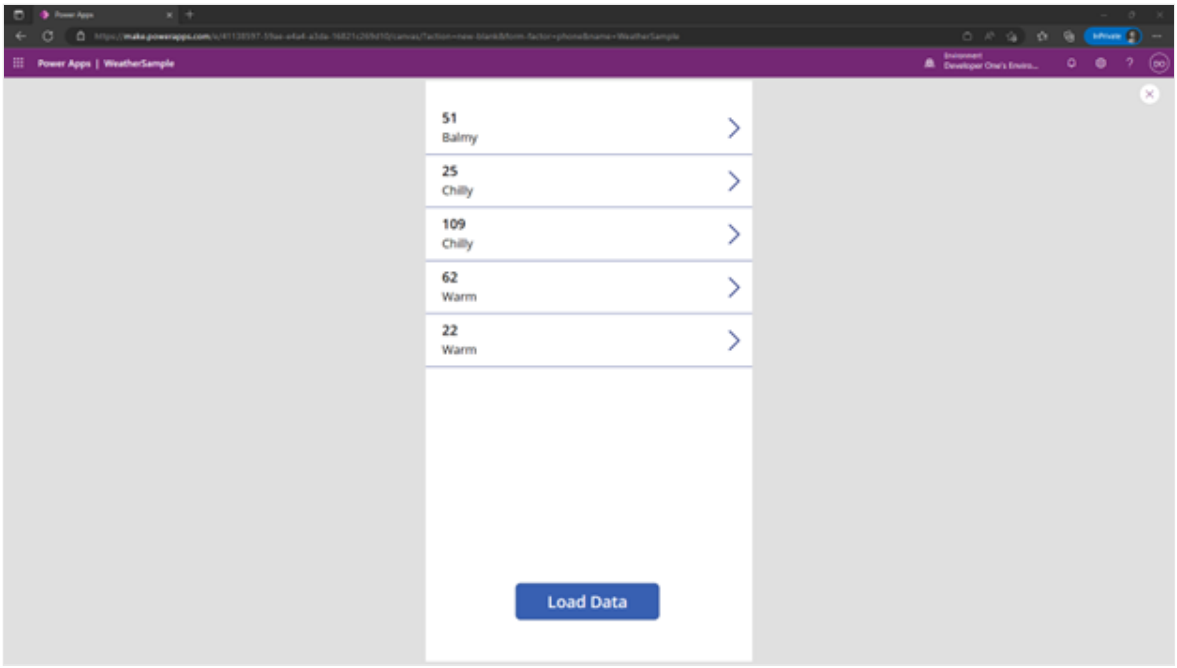
When Visual Studio runs a web app and a tunnel is active, the web browser opens to a tunnel URL instead of a localhost URL.

1. Run your Visual Studio solution and start debugging.
2. When the browser opens, a warning page opens with the first request sent to the tunnel URL. Select **Continue**.
3. With the web API running, open Power Apps in a new browser tab.
4. Select your developer environment in the upper-right corner.
5. Create a blank canvas app with **Phone** as the format.
6. In the top menu, select **Add data**.
7. Search for and select the **WeatherSample_Connector**, and then select **Connect**. More information: Add connections to your canvas app
8. Insert a button and drag it to the bottom of the form.
9. Change the button **Text** to *Load Data*.
10. Enter the following formula in the button's **OnSelect** property:

    Power Apps

    ```
    ClearCollect(weatherCollection,
    WeatherSample_Connector.GetWeatherForecast())
    ```

11. Insert a vertical gallery
12. Select the **weatherCollection** as the data source, and then change the layout to **Title and subtitle**.
13. Run the app. It should look like the following example:



Debug your API

With your canvas app running, set a breakpoint and debug it. You can also use Hot Reload.

See also

What are canvas apps
How to use dev tunnels in Visual Studio 2022 with ASP.NET Core apps

Power Platform ALM developer tools

Article • 05/14/2024

Use the following application lifecycle management (ALM) developer tools to create and manage projects for testing, deployment, maintenance, and governance of solutions.

Power Platform Build Tools for Azure DevOps

Use Microsoft Power Platform Build Tools for Azure DevOps to automate common build and deployment tasks related to apps built on Microsoft Power Platform.

More information: Microsoft Power Platform Build Tools for Azure DevOps

GitHub actions for Power Platform

With GitHub actions for Microsoft Power Platform, you can:

- Create workflows in your repository to build, test, package, release, and deploy apps
- Perform automation
- Manage bots and other components built on Power Platform

More information: GitHub Actions for Power Platform

See also

Power Platform Build Tools for Azure DevOps release notes
GitHub Actions for Power Platform release notes
Power Platform ALM guide
Power Platform ALM for developers

Feedback

Was this page helpful? Yes No

Provide product feedback

Dataverse developer tools

Article • 03/30/2023

Microsoft Dataverse is the underlying data platform for Power Platform that stores business data for building apps, workflows, websites, intelligent virtual agents, reports, and analytics. More info:

- What is Dataverse
- Developers: Get started with Microsoft Dataverse

Dataverse provides various tools for developers to extend the data platform capabilities, such as Configuration Migration tool, Package Deployer tool, Plug-in Registration tool, SolutionPackager tool, and Code Generation tool.

Go to this article in the Dataverse developer guide to find detailed information about how to download, launch, and use these tools: Dataverse development tools

See also

Dataverse developer guide